19.3.6 Deep generative models *

Generative models provide a natural way of making use of unlabeled data through learning a model of the marginal distribution by minimizing $\mathcal{L}_U = -\sum_n \log p_{\boldsymbol{\theta}}(\boldsymbol{x}_n)$. Various approaches have leveraged generative models for semi-supervised learning by developing ways to use the model of $p_{\boldsymbol{\theta}}(\boldsymbol{x}_n)$ to help produce a better supervised model.

19.3.6.1 Variational autoencoders

In Section 20.3.5, we describe the variational autoencoder (VAE), which defines a probabilistic model of the joint distribution of data $\boldsymbol{x}$ and latent variables $\boldsymbol{z}$. Data is assumed to be generated by first sampling $\boldsymbol{z} \sim p(\boldsymbol{z})$ and then sampling $\boldsymbol{x} \sim p(\boldsymbol{x}|\boldsymbol{z})$. For learning, the VAE uses an encoder $\boldsymbol{q}_\lambda(\boldsymbol{z}|\boldsymbol{x})$ to approximate the posterior and a decoder $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ to approximate the likelihood. The encoder and decoder are typically deep neural networks. The parameters of the encoder and decoder can be jointly trained by maximizing the evidence lower bound (ELBO) of data.

The marginal distribution of latent variables $p(\boldsymbol{z})$ is often chosen to be a simple distribution like a diagonal-covariance Gaussian. In practice, this can make the latent variables $\boldsymbol{z}$ more amenable to downstream classification thanks to the facts that $\boldsymbol{z}$ is typically lower-dimensional than $\boldsymbol{x}$, that $\boldsymbol{z}$ is constructed via cascaded nonlinear transformations, and that the dimensions of the latent variables are designed to be independent. In other words, the latent variables can provide a (learned) representation where data may be more easily separable. In [Kin+14], this approach is called **M1** and it is indeed shown that the latent variables can be used to train stronger models when labels are scarce. (The general idea of unsupervised learning of representations to help with downstream classification tasks is described further in Section 19.2.4.)

An alternative approach to leveraging VAEs, also proposed in [Kin+14] and called **M2**, has the form

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}, y) = p_{\boldsymbol{\theta}}(y)p_{\boldsymbol{\theta}}(\boldsymbol{x}|y) = p_{\boldsymbol{\theta}}(y) \int p_{\boldsymbol{\theta}}(\boldsymbol{x}|y, \boldsymbol{z})p_{\boldsymbol{\theta}}(\boldsymbol{z})d\boldsymbol{z} \tag{19.30}$$

where $\boldsymbol{z}$ is a latent variable, $p_{\boldsymbol{\theta}}(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{\mu}_{\boldsymbol{\theta}}, \boldsymbol{\Sigma}_{\boldsymbol{\theta}})$ is the latent prior (typically we fix $\boldsymbol{\mu}_{\boldsymbol{\theta}} = \mathbf{0}$ and $\boldsymbol{\Sigma}_{\boldsymbol{\theta}} = \mathbf{I}$), $p_{\boldsymbol{\theta}}(y) = \text{Cat}(y|\boldsymbol{\pi}_{\boldsymbol{\theta}})$ the label prior, and $p_{\boldsymbol{\theta}}(\boldsymbol{x}|y, \boldsymbol{z}) = p(\boldsymbol{x}|f_{\boldsymbol{\theta}}(y, \boldsymbol{z}))$ is the likelihood, such as a Gaussian, with parameters computed by f (a deep neural network). The main innovation of this approach is to assume that data is generated according to both a latent class variable y as well as the continuous latent variable $\boldsymbol{z}$. The class variable y is observed for labeled data and unobserved for unlabled data.

To compute the likelihood for the *labeled data*, $p_{\boldsymbol{\theta}}(\boldsymbol{x}, y)$, we need to marginalize over $\boldsymbol{z}$, which we can do by using an inference network of the form

$$q_{\boldsymbol{\phi}}(\boldsymbol{z}|y, \boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{\mu}_{\boldsymbol{\phi}}(y, \boldsymbol{x}), \text{diag}(\sigma^2_{\boldsymbol{\phi}}(\boldsymbol{x})) \tag{19.31}$$

We then use the following variational lower bound

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}, y) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x},y)} \left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|y, \boldsymbol{z}) + \log p_{\boldsymbol{\theta}}(y) + \log p_{\boldsymbol{\theta}}(\boldsymbol{z}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}, y)\right] = -\mathcal{L}(\boldsymbol{x}, y) \tag{19.32}$$

as is standard for VAEs (see Section 20.3.5). The only difference is that we observe two kinds of data: $\boldsymbol{x}$ and y.

To compute the likelihood for the *unlabeled* data, $p_{\boldsymbol{\theta}}(\boldsymbol{x})$, we need to marginalize over $\boldsymbol{z}$ *and* y, which we can do by using an inference network of the form

$$q_{\boldsymbol{\phi}}(\boldsymbol{z}, y|\boldsymbol{x}) = q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})q_{\boldsymbol{\phi}}(y|\boldsymbol{x}) \tag{19.33}$$

$$q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}), \text{diag}(\sigma^2_{\boldsymbol{\phi}}(\boldsymbol{x})) \tag{19.34}$$

$$q_{\boldsymbol{\phi}}(y|\boldsymbol{x}) = \text{Cat}(y|\boldsymbol{\pi}_{\boldsymbol{\phi}}(\boldsymbol{x})) \tag{19.35}$$

Note that $q_{\boldsymbol{\phi}}(y|\boldsymbol{x})$ acts like a discriminative classifier, that imputes the missing labels. We then use the following variational lower bound:

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z},y|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|y,\boldsymbol{z}) + \log p_{\boldsymbol{\theta}}(y) + \log p_{\boldsymbol{\theta}}(\boldsymbol{z}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}, y|\boldsymbol{x})\right] \tag{19.36}$$

$$= -\sum_y q_{\boldsymbol{\phi}}(y|\boldsymbol{x})\mathcal{L}(\boldsymbol{x}, y) + \mathbb{H}\left(q_{\boldsymbol{\phi}}(y|\boldsymbol{x})\right) = -\mathcal{U}(\boldsymbol{x}) \tag{19.37}$$

Note that the discriminative classifier $q_{\boldsymbol{\phi}}(y|\boldsymbol{x})$ is only used to compute the log-likelihood of the unlabeled data, which is undesirable. We can therefore add an extra classification loss on the supervised data, to get the following overall objective function:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{D}_L}\left[\mathcal{L}(\boldsymbol{x}, y)\right] + \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}_U}\left[\mathcal{U}(\boldsymbol{x})\right] + \alpha\mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{D}_L}\left[-\log q_{\boldsymbol{\phi}}(y|\boldsymbol{x})\right] \tag{19.38}$$

where α is a hyperparameter that controls the relative weight of generative and discriminative learning.

Of course, the probablistic model used in M2 is just one of many ways to decompose the dependencies between the observed data, the class labels, and the continuous latent variables. There are also many ways other than variational inference to perform approximate inference. The best technique will be problem dependent, but overall the main advantage of the generative approach is that we can incorporate domain knowledge. For example, we can model the missing data mechanism, since the absence of a label may be informative about the underlying data (e.g., people may be reluctant to answer a survey question about their health if they are unwell).

19.3.6.2 Generative adversarial networks

Generative adversarial networks (GANs) (described in more detail in the sequel to this book, [Mur23]) are a popular class of generative models that learn an implicit model of the data distribution. They consist of a generator network, which maps samples from a simple latent distribution to the data space, and a critic network, which attempts to distinguish between the outputs of the generator and samples from the true data distribution. The generator is trained to generate samples that the critic classifies as "real".

Since standard GANs do not produce a learned latent representation of a given datapoint and do not learn an explicit model of the data distribution, we cannot use the same approaches as were used for VAEs. Instead, semi-supervised learning with GANs is typically done by modifying the critic so that it outputs either a class label or "fake" instead of simply classifying real vs. fake [Sal+16; Ode16]. For labeled real data, the critic is trained to output the appropriate class label, and for unlabeled real data, it is trained to raise the probability of any of the class labels. As with standard GAN training, the critic is trained to classify outputs from the generator as fake and the generator is trained to fool the critic.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

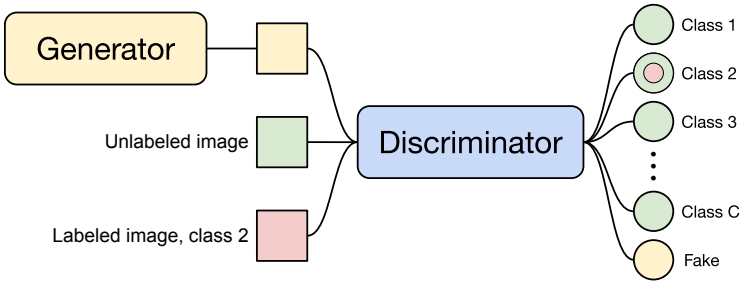


Figure 19.12: Diagram of the semi-supervised GAN framework. The discriminator is trained to output the class of labeled datapoints (red), a "fake" label for outputs from the generator (yellow), and any label for unlabeled data (green).

In more detail, let $p_\theta(y|\boldsymbol{x})$ denote the critic with $C+1$ outputs corresponding to C classes plus a "fake" class, and let $G(\boldsymbol{z})$ denote the generator which takes as input samples from the prior distribution $p(\boldsymbol{z})$. Let us assume that we are using the standard cross-entropy GAN loss as originally proposed in [Goo+14]. Then the critic's loss is

$$-\mathbb{E}_{\boldsymbol{x},y\sim p(\boldsymbol{x},y)} \log p_\theta(y|\boldsymbol{x}) - \mathbb{E}_{\boldsymbol{x}\sim p(\boldsymbol{x})} \log[1-p_\theta(y=C+1|\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{z}\sim p(\boldsymbol{z})} \log p_\theta(y=C+1|G(\boldsymbol{z})) \quad (19.39)$$

This tries to maximize the probability of the correct class for the labeled examples, to minimize the probability of the fake class for real unlabeled examples, and to maximize the probability of the fake class for generated examples. The generator's loss is simpler, namely

$$\mathbb{E}_{z\sim p(\boldsymbol{z})} \log p_\theta(y=C+1|G(\boldsymbol{z})) \quad (19.40)$$

A diagram visualizing the semi-supervised GAN framework is shown in Figure 19.12.

19.3.6.3 Normalizing flows

Normalizing flows (described in more detail in the sequel to this book, [Mur23]) are a tractable way to define deep generative models. More precisely, they define an invertible mapping $f_\theta : \mathcal{X} \to \mathcal{Z}$, with parameters θ, from the data space $\mathcal{X}$ to the latent space $\mathcal{Z}$. The density in data space can be written starting from the density in the latent space using the change of variables formula:

$$p(x) = p(f(x)) \cdot \left| \det \left(\frac{\partial f}{\partial x} \right) \right|. \quad (19.41)$$

We can extend this to semi-supervised learning, as proposed in [Izm+20]. For class labels $y \in \{1 \dots \mathcal{C}\}$, we can specify the latent distribution, conditioned on a label k, as Gaussian with mean μ_k and covariance Σ_k: $p(z|y=k) = \mathcal{N}(z|\mu_k, \Sigma_k)$. The marginal distribution of z is then a Gaussian mixture. The likelihood for labeled data is then

$$p_{\mathcal{X}}(x|y=k) = \mathcal{N}\left(f(x)|\mu_k, \Sigma_k\right) \cdot \left| \det \left(\frac{\partial f}{\partial x} \right) \right|, \quad (19.42)$$

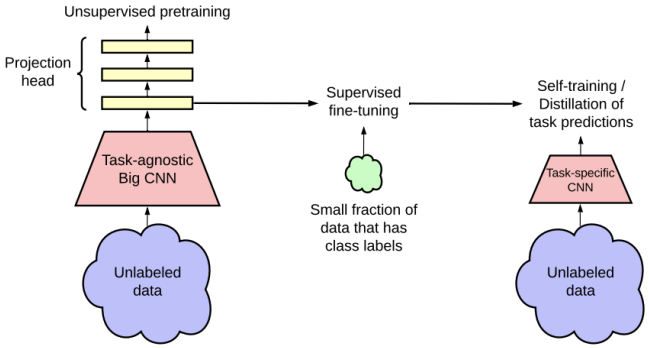


Figure 19.13: Combining self-supervised learning on unlabeled data (left), supervised fine-tuning (middle), and self-training on pseudo-labeled data (right). From Figure 3 of [Che+20c]. Used with kind permission of Ting Chen.

and the likelihood for data with unknown label is $p(x) = \sum_k p(x|y=k)p(y=k)$.

For semi-supervised learning we can then maximize the joint likelihood of the labeled $\mathcal{D}_\ell$ and unlabeled data $\mathcal{D}_u$:

$$p(\mathcal{D}_\ell, \mathcal{D}_u|\theta) = \prod_{(x_i,y_i)\in\mathcal{D}_\ell} p(x_i, y_i) \prod_{x_j\in\mathcal{D}_u} p(x_j), \tag{19.43}$$

over the parameters θ of the bijective function f, which learns a density model for a Bayes classifier.

Given a test point x, the model predictive distribution is given by

$$p_{\mathcal{X}}(y=c|x) = \frac{p(x|y=c)p(y=c)}{p(x)} = \frac{p(x|y=c)p(y=c)}{\sum_{k=1}^C p(x|y=k)p(y=k)} = \frac{\mathcal{N}(f(x)|\mu_c, \Sigma_c)}{\sum_{k=1}^C \mathcal{N}(f(x)|\mu_k, \Sigma_k)}, \tag{19.44}$$

where we have assumed $p(y=c) = 1/C$. We can make predictions for a test point x with the Bayes decision rule $y = \arg\max_{c\in\{1,\ldots,C\}} p(y=c|x)$.

19.3.7 Combining self-supervised and semi-supervised learning

It is possible to combine self-supervised and semi-supervised learning. For example, [Che+20c] use SimCLR (Section 19.2.4.4) to perform self-supervised representation learning on the unlabeled data, they then fine-tune this representation on a small labeled dataset (as in transfer learning, Section 19.2), and finally, they apply the trained model back to the original unlabeled dataset, and distill the predictions from this teacher model T into a student model S. (**Knowledge distillation** is the name given to the approach of training one model on the predictions of another, as originally proposed in [HVD14].) That is, after fine-tuning T, they train S by minimizing

$$\mathcal{L}(T) = -\sum_{\boldsymbol{x}_i\in\mathcal{D}} \left[\sum_y p^T(y|\boldsymbol{x}_i;\tau)\log p^S(y|\boldsymbol{x}_i;\tau)\right] \tag{19.45}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\tau > 0$ is a temperature parameter applied to the softmax output, which is used to perform **label smoothing**. If S has the same form as T, this is known as **self-training**, as discussed in Section 19.3.1. However, normally the student S is smaller than the teacher T. (For example, T might be a high capacity model, and S is a lightweight version that runs on a phone.) See Figure 19.13 for an illustration of the overall approach.

19.4 Active learning

In **active learning**, the goal is to identify the true predictive mapping $y = f(\boldsymbol{x})$ by querying as few $(\boldsymbol{x}, y)$ points as possible. There are three main variants. In **query synthesis**, the algorithm gets to choose any input $\boldsymbol{x}$, and can ask for its corresponding output $y = f(\boldsymbol{x})$. In **pool-based active learning**, there is a large, but fixed, set of unlabeled data points, and the algorithm gets to ask for a label for one or more of these points. Finally, in **stream-based active learning**, the incoming data is arriving continuously, and the algorithm must choose whether it wants to request a label for the current input or not.

There are various closely related problems. In **Bayesian optimization** the goal is to estimate the location of the global optimum $\boldsymbol{x}^* = \text{argmin}_{\boldsymbol{x}} f(\boldsymbol{x})$ in as few queries as possible; typically we fit a surrogate (response surface) model to the intermediate $(\boldsymbol{x}, y)$ queries, to decide which question to ask next. In **experiment design**, the goal is to infer a parameter vector of some model, using carefully chosen data samples $\mathcal{D} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$, i.e. we want to estimate $p(\boldsymbol{\theta}|\mathcal{D})$ using as little data as possible. (This can be thought of as an unsupervised, or generalized, form of active learning.)

In this section, we give a brief review of the pool based approach to active learning. For more details, see e.g., [Set12] for a review.

19.4.1 Decision-theoretic approach

In the decision theoretic approach to active learning, proposed in [KHB07; RM01], we define the utility of querying $\boldsymbol{x}$ in terms of the **value of information**. In particular, we define the utility of issuing query $\boldsymbol{x}$ as

$$U(\boldsymbol{x}) \triangleq \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}\left[\min_a \left(\rho(a|\mathcal{D}) - \rho(a|\mathcal{D}, (\boldsymbol{x}, y))\right)\right] \tag{19.46}$$

where $\rho(a|\mathcal{D}) = \mathbb{E}_{p(\theta|\mathcal{D})}\left[\ell(\theta, a)\right]$ is the posterior expected loss of taking some future action a given the data $\mathcal{D}$ observed so far. Unfortunately, evaluating $U(\boldsymbol{x})$ for each $\boldsymbol{x}$ is quite expensive, since for each possible response y we might observe, we have to update our beliefs given $(\boldsymbol{x}, y)$ to see what effect it might have on our future decisions (similar to look ahead search technique applied to belief states).

19.4.2 Information-theoretic approach

In the information theoretic approach to active supervised learning, we avoid using task-specific loss functions, and instead focus on learning our model as well as we can. In particular, [Lin56] proposed to define the utility of querying $\boldsymbol{x}$ in terms of **information gain** about the parameters $\boldsymbol{\theta}$, i.e., the reduction in entropy:

$$U(\boldsymbol{x}) \triangleq \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}\left[\mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D}, \boldsymbol{x}, y)\right)\right] \tag{19.47}$$

(Note that the first term is a constant wrt $\boldsymbol{x}$, but we include it for later convenience.) Exercise 19.1 asks you to show that this objective is identical to the expected change in the posterior over the parameters which is given by

$$U'(\boldsymbol{x}) \triangleq \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}\left[D_{\mathbb{KL}}\left(p(\boldsymbol{\theta}|\mathcal{D},\boldsymbol{x},y) \parallel p(\boldsymbol{\theta}|\mathcal{D})\right)\right] \tag{19.48}$$

Using symmetry of the mutual information, we can rewrite Equation (19.47) as follows:

$$U(\boldsymbol{x}) = \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}\left[\mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D},\boldsymbol{x},y)\right)\right] \tag{19.49}$$

$$= \mathbb{I}(\boldsymbol{\theta}, y|\mathcal{D},\boldsymbol{x}) \tag{19.50}$$

$$= \mathbb{H}\left(p(y|\boldsymbol{x},\mathcal{D})\right) - \mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}\left[\mathbb{H}\left(p(y|\boldsymbol{x},\boldsymbol{\theta})\right)\right] \tag{19.51}$$

The advantage of this approach is that we now only have to reason about the uncertainty of the predictive distribution over outputs y, not over the parameters $\boldsymbol{\theta}$.

Equation (19.51) has an interesting interpretation. The first term prefers examples $\boldsymbol{x}$ for which there is uncertainty in the predicted label. Just using this as a selection criterion is called **maximum entropy sampling** [SW87]. However, this can have problems with examples which are inherently ambiguous or mislabeled. The second term in Equation (19.51) will discourage such behavior, since it prefers examples $\boldsymbol{x}$ for which the predicted label is fairly certain once we know $\boldsymbol{\theta}$; this will avoid picking inherently hard-to-predict examples. In other words, Equation (19.51) will select examples $\boldsymbol{x}$ for which the model makes confident predictions which are highly diverse. This approach has therefore been called **Bayesian active learning by disagreement** or **BALD** [Hou+12].

This method can be used to train classifiers for other domains where expert labels are hard to acquire, such as medical images or astronomical images [Wal+20].

19.4.3 Batch active learning

So far, we have assumed a greedy or **myopic** strategy, in which we select a single example $\boldsymbol{x}$, as if it were the last datapoint to be selected. But sometimes we have a budget to collect a set of B samples, call them $(\mathbf{X}, \mathbf{Y})$. In this case, the information gain criterion becomes $U(\boldsymbol{x}) = \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{E}_{p(\mathbf{Y}|\boldsymbol{x},\mathcal{D})}\left[\mathbb{H}\left(p(\boldsymbol{\theta}|\mathbf{Y},\boldsymbol{x},\mathcal{D})\right)\right]$. Unfortunately, optimizing this is NP-hard in the horizon length B [KLQ95; KG05].

Fortunately, under certain conditions, the greedy strategy is near-optimal, as we now explain. Let us fix query $\boldsymbol{x}$ and define $f(\boldsymbol{y}) \triangleq \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{H}\left(p(\boldsymbol{\theta}|\mathbf{Y},\boldsymbol{x},\mathcal{D})\right)$ as the information gain function, so $U(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{y}}\left[f(\boldsymbol{y},\boldsymbol{x})\right]$. It is clear that $f(\emptyset) = 0$, and that f is non-decreasing, meaning $f(Y^{\text{large}}) \geq f(Y^{\text{small}})$, due to the "more information never hurts" principle. Furthermore, [KG05] proved that f is **submodular**. As a consequence, a sequential greedy approach is within a constant factor of optimal. If we combine this greedy technique with the BALD objective, we get a method called **BatchBALD** [KAG19].

19.5 Meta-learning

We can think of a learning algorithm as a function A that maps data to a parameter estimate, $\theta = A(\mathcal{D})$. The function A usually has its own parameter — call them ϕ — such as the initial values for θ, or the learning rate, etc. We denote this by $\theta = A(\mathcal{D}; \phi)$. We can imagine learning ϕ itself,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

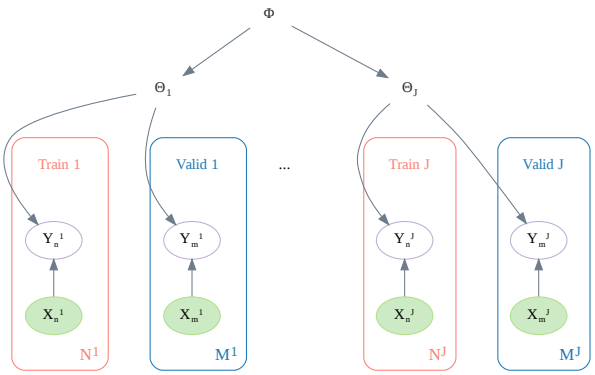


Figure 19.14: Illustration of a hierarchical Bayesian model for meta-learning. Generated by hbayes_maml.ipynb.

given a collection of datasets $\mathcal{D}_{1:J}$ and some **meta-learning** algorithm M, i.e., $\phi = M(\mathcal{D}_{1:J})$. We can then apply $A(\cdot;\phi)$ to learn the parameters θ_{J+1} on some new dataset $\mathcal{D}_{J+1}$. There are many techniques for meta-learning — see e.g., [Van18; HRP21] for recent reviews. Below we discuss one particularly popular method. (Note that meta-learning is also called **learning to learn** [TP97].)

19.5.1 Model-agnostic meta-learning (MAML)

A natural approach to meta learning is to use a hierarchical Bayesian model, as illustrated in Figure 19.14. The parameters for each task $\boldsymbol{\theta}_j$ are assumed to come from a common prior, $p(\boldsymbol{\theta}_j|\boldsymbol{\xi})$, which can be used to help pool statistical strength from multiple data-poor problems. Meta-learning becomes equivalent to learning the prior ϕ. Rather than performing full Bayesian inference in this model, a more efficient approach is to use the following empirical Bayes (Section 4.6.5.3) approximation:

$$\boldsymbol{\xi}^* = \underset{\boldsymbol{\xi}}{\operatorname{argmax}} \frac{1}{J} \sum_{j=1}^{J} \log p(\mathcal{D}^j_{\text{valid}}|\hat{\boldsymbol{\theta}}_j(\boldsymbol{\xi}, \mathcal{D}^j_{\text{train}})) \tag{19.52}$$

where $\hat{\boldsymbol{\theta}}_j = \hat{\boldsymbol{\theta}}(\boldsymbol{\xi}, \mathcal{D}^j_{\text{train}})$ is a point estimate of the parameters for task j based on $\mathcal{D}^j_{\text{train}}$ and prior $\boldsymbol{\xi}$, and where we use a cross-validation approximation to the marginal likelihood (Section 5.2.4).

To compute the point estimate of the parameters for the target task $\hat{\boldsymbol{\theta}}_{J+1}$, we use K steps of a gradient ascent procedure starting at $\boldsymbol{\xi}$ with a learning rate of η. This is known as **model-agnostic meta-learning** or **MAML** [FAL17]. This can be shown to be equivalent to an approximate MAP estimate using a Gaussian prior centered at $\boldsymbol{\xi}$, where the strength of the prior is controlled by the number of gradient steps [San96; Gra+18]. (This is an example of **fast adapation** of the task specific weights starting from the shared prior $\boldsymbol{\xi}$.)

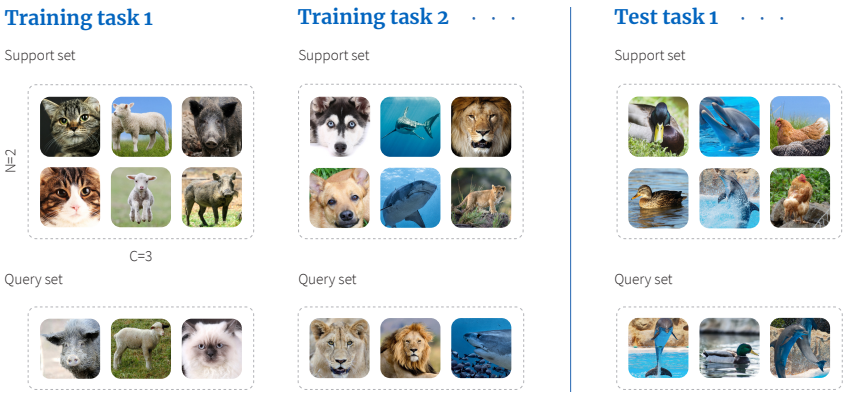


Figure 19.15: Illustration of meta-learning for few-shot learning. Here, each task is a 3-way-2-shot classification problem because each training task contains a support set with three classes, each with two examples. From https://bit.ly/3rrvSjw. Copyright (2019) Borealis AI. Used with kind permission of Simon Prince and April Cooper.

19.6 Few-shot learning

People can learn to predict from very few labeled examples. This is called **few-shot learning**. In the extreme in which the person or system learns from a single example of each class, this is called **one-shot learning**, and if no labeled examples are given, it is called **zero-shot learning**.

A common way to evaluate methods for FSL is to use **C-way N-shot classification**, in which the system is expected to learn to classify C classes using just N training examples of each class. Typically N and C are very small, e.g., Figure 19.15 illustrates the case where we have $C = 3$ classes, each with $N = 2$ examples. Since the amount of data from the new domain (here, ducks, dolphins and hens) is so small, we cannot expect to learn from scratch. Therefore we turn to meta-learning.

During training, the meta-algorithm M trains on a labeled support set from group j, returns a predictor f^j, which is then evaluated on a disjoint query set also from group j. We optimize M over all J groups. Finally we can apply M to our new labeled support set to get f^{test}, which is applied to the query set from the test domain. This is illustrated in Figure 19.15. We see that there is no overlap between the classes in the two training tasks ({cat, lamb, pig} and {dog, shark, lion}) and those in the test task ({duck, dolphin, hen}). Thus the algorithm M must learn to predict image classes in general rather than any particular set of labels.

There are many approaches to few-shot learning. We discuss one such method in Section 19.6.1. For more methods, see e.g., [Wan+20b].

19.6.1 Matching networks

One approach to few shot learning is to learn a distance metric on some other dataset, and then to use $d_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{x}')$ inside of a nearest neighbor classifier. Essentially this defines a semi-parametric model of the form $p_{\boldsymbol{\theta}}(y|\boldsymbol{x}, \mathcal{S})$, where $\mathcal{S}$ is the small labeled dataset (known as the support set), and $\boldsymbol{\theta}$ are the parameters of the distance function. This approach is widely used for **fine-grained classification**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

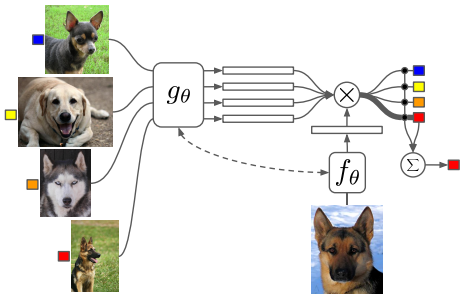


Figure 19.16: Illustration of a matching network for one-shot learning. From Figure 1 of [Vin+16]. Used with kind permission of Oriol Vinyals.

tasks, where there are many different visually similar categories, such as face images from a gallery, or product images from a catalog.

An extension of this approach is to learn a function of the form

$$p_{\boldsymbol{\theta}}(y|\boldsymbol{x}, \mathcal{S}) = \mathbb{I}\left(y = \sum_{n \in \mathcal{S}} a_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{x}_n; \mathcal{S}) y_n\right) \tag{19.53}$$

where $a_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{x}_n; \mathcal{S}) \in \mathbb{R}^+$ is some kind of adaptive similarity kernel. For example, we can use an **attention kernel** of the form

$$a(\boldsymbol{x}, \boldsymbol{x}_n; \mathcal{S}) = \frac{\exp(c(f(\boldsymbol{x}), g(\boldsymbol{x}_n)))}{\sum_{n'=1}^{N} \exp(c(f(\boldsymbol{x}), g(\boldsymbol{x}_{n'})))} \tag{19.54}$$

where $c(\boldsymbol{u}, \boldsymbol{v})$ is the cosine distance. (We can make f and g be the same function if we want.) Intuitively, the attention kernel will compare $\boldsymbol{x}$ to $\boldsymbol{x}_n$ in the context of all the labeled examples, which provides an implicit signal about which feature dimensions are relevant. (We discuss attention mechanisms in more detail in Section 15.4.) This is called a **matching network** [Vin+16]. See Figure 19.16 for an illustration.

We can train the f and g functions using multiple small datasets, as in meta-learning (Section 19.5). More precisely, let $\mathcal{D}$ be a large labeled dataset (e.g., ImageNet), and let $p(\mathcal{L})$ be a distribution over its labels. We create a task by sampling a small set of labels (say 25), $\mathcal{L} \sim p(\mathcal{L})$, and then sampling a small support set of examples from $\mathcal{D}$ with those labels, $\mathcal{S} \sim \mathcal{L}$, and finally sampling a small test set with those same labels, $\mathcal{T} \sim \mathcal{L}$. We then train the model to predict the test labels given the support set, i.e., we optimize the following objective:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \mathbb{E}_{\mathcal{L} \sim p(\mathcal{L})}\left[\mathbb{E}_{\mathcal{S} \sim \mathcal{L}, \mathcal{T} \sim \mathcal{L}}\left[\sum_{(\boldsymbol{x}, y) \in \mathcal{T}} \log p_{\boldsymbol{\theta}}(y|\boldsymbol{x}, \mathcal{S})\right]\right] \tag{19.55}$$

After training, we freeze $\boldsymbol{\theta}$, and apply Equation (19.53) to a test support set $\mathcal{S}$.

19.7 Weakly supervised learning

The term **weakly supervised learning** refers to scenarios where we do not have an exact label associated with every feature vector in the training set.

One scenario is when we have a *distribution* over labels for each case, rather than a single label. Fortunately, we can still do maximum likelihood training: we just have to minimize the cross entropy,

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_n \sum_y p(y|\boldsymbol{x}_n) \log q_{\boldsymbol{\theta}}(y|\boldsymbol{x}_n) \tag{19.56}$$

where $p(y|\boldsymbol{x}_n)$ is the label distribution for case n, and $q_{\boldsymbol{\theta}}(y|\boldsymbol{x}_n)$ is the predicted distribution. Indeed, it is often useful to artificially replace exact labels with a "soft" version, in which we replace the delta function with a distribution that puts, say, 90% of its mass on the observed label, and spreads the remaining mass uniformly over the other choices. This is called **label smoothing**, and is a useful form of regularization (see e.g., [MKH19]).

Another scenario is when we have a set, or **bag**, of instances, $\boldsymbol{x}_n = \{\boldsymbol{x}_{n,1}, \ldots, \boldsymbol{x}_{n,B}\}$, but we only have a label for the entire bag, y_n, not for the members of the bag, y_{nb}. We often assume that if any member of the bag is positive, the whole bag is labeled positive, so $y_n = \vee_{b=1}^{B} y_{nb}$, but we do not know which member "caused" the positive outcome. However, if all the members are negative, the entire bag is negative. This is known as **multi-instance learning** [DLLP97]. (For a recent example of this in the context of COVID-19 risk score learning, see [MKS21].) Various algorithms can be used to solve the MIL problem, depending on what assumptions we make about the correlation between the labels in each bag, and the fraction of positive members we expect to see (see e.g., [KF05]).

Yet another scenario is known as **distant supervision** [Min+09], which is often used to train information extraction systems. The idea is that we have some fact, such as "Married(B,M)", that we know to be true (since it is stored in a database). We use this to label every sentence (in our unlabeled training corpus) in which the entities B and M are mentioned as being a positive example of the "Married" relation. For example, the sentence "B and M invited 100 people to their wedding" will be labeled positive. But this heuristic may include false positives, for example "B and M went out to dinner" will also be labeled positive. Thus the resulting labels will be noisy. We discuss some ways to handle label noise in Section 10.4.

19.8 Exercises

Exercise 19.1 [Information gain equations]

Consider the following two objectives for evaluating the utility of querying a datapoint $\boldsymbol{x}$ in an active learning setting:

$$U(\boldsymbol{x}) \triangleq \mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D})\right) - \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}\left[\mathbb{H}\left(p(\boldsymbol{\theta}|\mathcal{D},\boldsymbol{x},y)\right)\right] \tag{19.57}$$

$$U'(\boldsymbol{x}) \triangleq \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}\left[D_{\mathbb{KL}}\left(p(\boldsymbol{\theta}|\mathcal{D},\boldsymbol{x},y) \parallel p(\boldsymbol{\theta}|\mathcal{D})\right)\right] \tag{19.58}$$

Prove that these are equal.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20 Dimensionality Reduction

A common form of unsupervised learning is **dimensionality reduction**, in which we learn a mapping from the high-dimensional visible space, $\boldsymbol{x} \in \mathbb{R}^D$, to a low-dimensional latent space, $\boldsymbol{z} \in \mathbb{R}^L$. This mapping can either be a parametric model $\boldsymbol{z} = f(\boldsymbol{x}; \boldsymbol{\theta})$ which can be applied to any input, or it can be a nonparametric mapping where we compute an **embedding** $\boldsymbol{z}_n$ for each input $\boldsymbol{x}_n$ in the data set, but not for any other points. This latter approach is mostly used for data visualization, whereas the former approach can also be used as a preprocessing step for other kinds of learning algorithms. For example, we might first reduce the dimensionality by learning a mapping from $\boldsymbol{x}$ to $\boldsymbol{z}$, and then learn a simple linear classifier on this embedding, by mapping $\boldsymbol{z}$ to y.

20.1 Principal components analysis (PCA)

The simplest and most widely used form of dimensionality reduction is **principal components analysis** or **PCA**. The basic idea is to find a linear and orthogonal projection of the high dimensional data $\boldsymbol{x} \in \mathbb{R}^D$ to a low dimensional subspace $\boldsymbol{z} \in \mathbb{R}^L$, such that the low dimensional representation is a "good approximation" to the original data, in the following sense: if we project or **encode** $\boldsymbol{x}$ to get $\boldsymbol{z} = \mathbf{W}^\mathsf{T}\boldsymbol{x}$, and then unproject or **decode** $\boldsymbol{z}$ to get $\hat{\boldsymbol{x}} = \mathbf{W}\boldsymbol{z}$, then we want $\hat{\boldsymbol{x}}$ to be close to $\boldsymbol{x}$ in ℓ_2 distance. In particular, we can define the following **reconstruction error** or **distortion**:

$$\mathcal{L}(\mathbf{W}) \triangleq \frac{1}{N}\sum_{n=1}^{N} ||\boldsymbol{x}_n - \text{decode}(\text{encode}(\boldsymbol{x}_n; \mathbf{W}); \mathbf{W})||_2^2 \tag{20.1}$$

where the encode and decoding stages are both linear maps, as we explain below.

In Section 20.1.2, we show that we can minimize this objective by setting $\hat{\mathbf{W}} = \mathbf{U}_L$, where $\mathbf{U}_L$ contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n - \overline{\boldsymbol{x}})(\boldsymbol{x}_n - \overline{\boldsymbol{x}})^\mathsf{T} = \frac{1}{N}\mathbf{X}_c^\mathsf{T}\mathbf{X}_c \tag{20.2}$$

where $\mathbf{X}_c$ is a centered version of the $N \times D$ design matrix. In Section 20.2.2, we show that this is equivalent to maximizing the likelihood of a latent linear Gaussian model known as probabilistic PCA.

20.1.1 Examples

Before giving the details, we start by showing some examples.

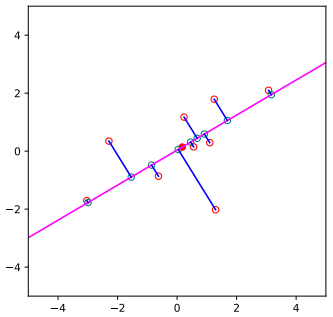


Figure 20.1: An illustration of PCA where we project from 2d to 1d. Red circles are the original data points, blue circles are the reconstructions. The red dot is the data mean. Generated by pcaDemo2d.ipynb.

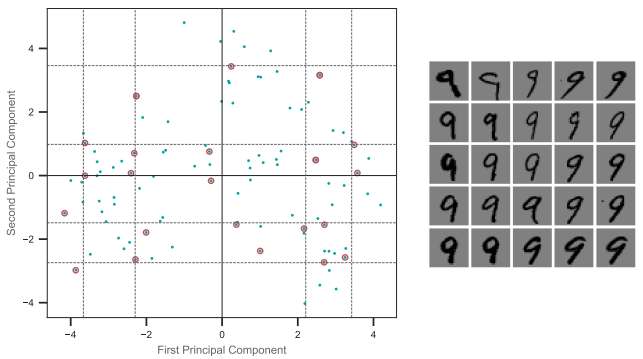


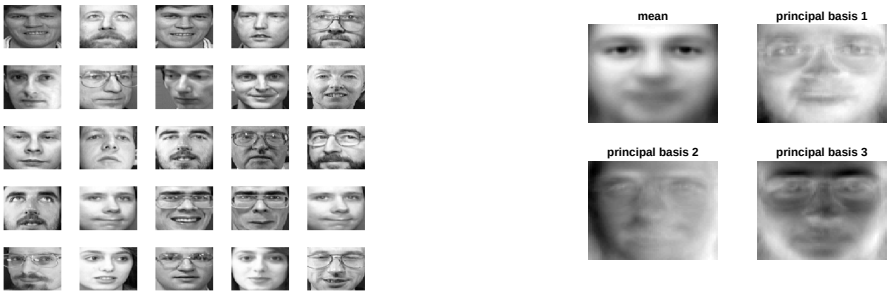
Figure 20.2: An illustration of PCA applied to MNIST digits from class 9. Grid points are at the 5, 25, 50, 75, 95 % quantiles of the data distribution along each dimension. The circled points are the closest projected images to the vertices of the grid. Adapted from Figure 14.23 of [HTF09]. Generated by pca_ digits.ipynb.

Figure 20.1 shows a very simple example, where we project 2d data to a 1d line. This direction captures most of the variation in the data.

In Figure 20.2, we show what happens when we project some MNIST images of the digit 9 down to 2d. Although the inputs are high dimensional (specifically $28 \times 28 = 784$ dimensional), the number of "effective degrees of freedom" is much less, since the pixels are correlated, and many digits look similar. Therefore we can represent each image as a point in a low dimensional linear space.

In general, it can be hard to interpret the latent dimensions to which the data is projected. However, by looking at several projected points along a given direction, and the examples from which they are derived, we see that the first principal component (horizontal direction) seems to capture the orientation of the digit, and the second component (vertical direction) seems to capture line thickness.

In Figure 20.3, we show PCA applied to another image dataset, known as the Olivetti face dataset,



(a) (b)

Figure 20.3: a) Some randomly chosen 64×64 *pixel images from the Olivetti face database. (b) The mean and the first three PCA components represented as images. Generated by pcaImageDemo.ipynb.*

which is a set of 64×64 grayscale images. We project these to a 3d subspace. The resulting basis vectors (columns of the projection matrix $\mathbf{W}$) are shown as images in in Figure 20.3b; these are known as **eigenfaces** [Tur13], for reasons that will be explained in Section 20.1.2. We see that the main modes of variation in the data are related to overall lighting, and then differences in the eyebrow region of the face. If we use enough dimensions (but fewer than the 4096 we started with), we can use the representation $\boldsymbol{z} = \mathbf{W}^\mathsf{T}\boldsymbol{x}$ as input to a nearest-neighbor classifier to perform face recognition; this is faster and more reliable than working in pixel space [MWP98].

20.1.2 Derivation of the algorithm

Suppose we have an (unlabeled) dataset $\mathcal{D} = \{\boldsymbol{x}_n : n = 1 : N\}$, where $\boldsymbol{x}_n \in \mathbb{R}^D$. We can represent this as an $N \times D$ data matrix $\mathbf{X}$. We will assume $\overline{\boldsymbol{x}} = \frac{1}{N}\sum_{n=1}^N \boldsymbol{x}_n = \mathbf{0}$, which we can ensure by centering the data.

We would like to approximate each $\boldsymbol{x}_n$ by a low dimensional representation, $\boldsymbol{z}_n \in \mathbb{R}^L$. We assume that each $\boldsymbol{x}_n$ can be "explained" in terms of a weighted combination of basis functions $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_L$, where each $\boldsymbol{w}_k \in \mathbb{R}^D$, and where the weights are given by $\boldsymbol{z}_n \in \mathbb{R}^L$, i.e., we assume $\boldsymbol{x}_n \approx \sum_{k=1}^L z_{nk}\boldsymbol{w}_k$. The vector $\boldsymbol{z}_n$ is the low dimensional representation of $\boldsymbol{x}_n$, and is known as the **latent vector**, since it consists of latent or "hidden" values that are not observed in the data. The collection of these latent variables are called the **latent factors**.

We can measure the error produced by this approximation as follows:

$$\mathcal{L}(\mathbf{W}, \mathbf{Z}) = \frac{1}{N}||\mathbf{X} - \mathbf{Z}\mathbf{W}^\mathsf{T}||_F^2 = \frac{1}{N}||\mathbf{X}^\mathsf{T} - \mathbf{W}\mathbf{Z}^\mathsf{T}||_F^2 = \frac{1}{N}\sum_{n=1}^N ||\boldsymbol{x}_n - \mathbf{W}\boldsymbol{z}_n||^2 \tag{20.3}$$

where the rows of $\mathbf{Z}$ contain the low dimension versions of the rows of $\mathbf{X}$. This is known as the (average) **reconstruction error**, since we are approximating each $\boldsymbol{x}_n$ by $\hat{\boldsymbol{x}}_n = \mathbf{W}\boldsymbol{z}_n$.

We want to minimize this subject to the constraint that $\mathbf{W}$ is an orthogonal matrix. Below we show that the optimal solution is obtained by setting $\hat{\mathbf{W}} = \mathbf{U}_L$, where $\mathbf{U}_L$ contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.1.2.1 Base case

Let us start by estimating the best 1d solution, $\boldsymbol{w}_1 \in \mathbb{R}^D$. We will find the remaining basis vectors $\boldsymbol{w}_2$, $\boldsymbol{w}_3$, etc. later.

Let the coefficients for each of the data points associated with the first basis vector be denoted by $\tilde{\mathbf{z}}_1 = [z_{11}, \ldots, z_{N1}] \in \mathbb{R}^N$. The reconstruction error is given by

$$\mathcal{L}(\boldsymbol{w}_1, \tilde{\mathbf{z}}_1) = \frac{1}{N}\sum_{n=1}^{N} ||\boldsymbol{x}_n - z_{n1}\boldsymbol{w}_1||^2 = \frac{1}{N}\sum_{n=1}^{N} (\boldsymbol{x}_n - z_{n1}\boldsymbol{w}_1)^\mathsf{T}(\boldsymbol{x}_n - z_{n1}\boldsymbol{w}_1) \tag{20.4}$$

$$= \frac{1}{N}\sum_{n=1}^{N} [\boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n - 2z_{n1}\boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n + z_{n1}^2\boldsymbol{w}_1^\mathsf{T}\boldsymbol{w}_1] \tag{20.5}$$

$$= \frac{1}{N}\sum_{n=1}^{N} [\boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n - 2z_{n1}\boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n + z_{n1}^2] \tag{20.6}$$

since $\boldsymbol{w}_1^\mathsf{T}\boldsymbol{w}_1 = 1$ (by the orthonormality assumption). Taking derivatives wrt z_{n1} and equating to zero gives

$$\frac{\partial}{\partial z_{n1}}\mathcal{L}(\boldsymbol{w}_1, \tilde{\mathbf{z}}_1) = \frac{1}{N}[-2\boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n + 2z_{n1}] = 0 \Rightarrow z_{n1} = \boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n \tag{20.7}$$

So the optimal embedding is obtained by orthogonally projecting the data onto $\boldsymbol{w}_1$ (see Figure 20.1). Plugging this back in gives the loss for the weights:

$$\mathcal{L}(\boldsymbol{w}_1) = \mathcal{L}(\boldsymbol{w}_1, \tilde{\mathbf{z}}_1^*(\boldsymbol{w}_1)) = \frac{1}{N}\sum_{n=1}^{N} [\boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_n - z_{n1}^2] = \text{const} - \frac{1}{N}\sum_{n=1}^{N} z_{n1}^2 \tag{20.8}$$

To solve for $\boldsymbol{w}_1$, note that

$$\mathcal{L}(\boldsymbol{w}_1) = -\frac{1}{N}\sum_{n=1}^{N} z_{n1}^2 = -\frac{1}{N}\sum_{n=1}^{N} \boldsymbol{w}_1^\mathsf{T}\boldsymbol{x}_n\boldsymbol{x}_n^\mathsf{T}\boldsymbol{w}_1 = -\boldsymbol{w}_1^\mathsf{T}\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 \tag{20.9}$$

where $\boldsymbol{\Sigma}$ is the empirical covariance matrix (since we assumed the data is centered). We can trivially optimize this by letting $||\boldsymbol{w}_1|| \to \infty$, so we impose the constraint $||\boldsymbol{w}_1|| = 1$ and instead optimize

$$\tilde{\mathcal{L}}(\boldsymbol{w}_1) = \boldsymbol{w}_1^\mathsf{T}\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 - \lambda_1(\boldsymbol{w}_1^\mathsf{T}\boldsymbol{w}_1 - 1) \tag{20.10}$$

where λ_1 is a Lagrange multiplier (see Section 8.5.1). Taking derivatives and equating to zero we have

$$\frac{\partial}{\partial \boldsymbol{w}_1}\tilde{\mathcal{L}}(\boldsymbol{w}_1) = 2\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 - 2\lambda_1\boldsymbol{w}_1 = 0 \tag{20.11}$$

$$\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 = \lambda_1\boldsymbol{w}_1 \tag{20.12}$$

Hence the optimal direction onto which we should project the data is an eigenvector of the covariance matrix. Left multiplying by $\boldsymbol{w}_1^\mathsf{T}$ (and using $\boldsymbol{w}_1^\mathsf{T}\boldsymbol{w}_1 = 1$) we find

$$\boldsymbol{w}_1^\mathsf{T}\hat{\boldsymbol{\Sigma}}\boldsymbol{w}_1 = \lambda_1 \tag{20.13}$$

Since we want to maximize this quantity (minimize the loss), we pick the eigenvector which corresponds to the largest eigenvalue.

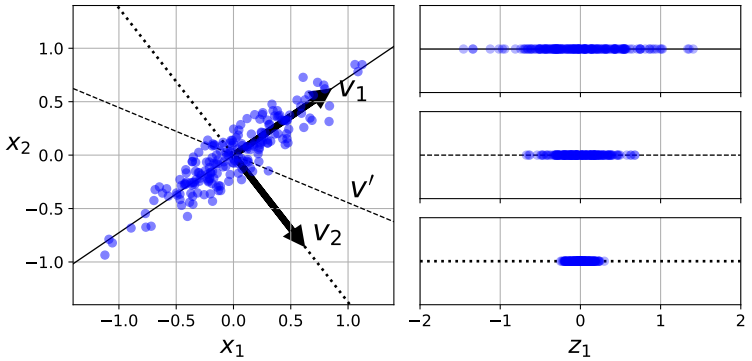


Figure 20.4: Illustration of the variance of the points projected onto different 1d vectors. v_1 is the first principal component, which maximizes the variance of the projection. v_2 is the second principal component which is direction orthogonal to v_1. Finally v' is some other vector in between v_1 and v_2. Adapted from Figure 8.7 of [Gér19]. Generated by pca_projected_variance.ipynb

20.1.2.2 Optimal weight vector maximizes the variance of the projected data

Before continuing, we make an interesting observation. Since the data has been centered, we have

$$\mathbb{E}\left[z_{n1}\right] = \mathbb{E}\left[\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{w}_1\right] = \mathbb{E}\left[\boldsymbol{x}_n\right]^{\mathsf{T}}\boldsymbol{w}_1 = 0 \tag{20.14}$$

Hence variance of the projected data is given by

$$\mathbb{V}\left[\tilde{\mathbf{z}}_1\right] = \mathbb{E}\left[\tilde{\mathbf{z}}_1^2\right] - \left(\mathbb{E}\left[\tilde{\mathbf{z}}_1\right]\right)^2 = \frac{1}{N}\sum_{n=1}^{N} z_{n1}^2 - 0 = -\mathcal{L}(\boldsymbol{w}_1) + \text{const} \tag{20.15}$$

From this, we see that *minimizing* the reconstruction error is equivalent to *maximizing* the variance of the projected data:

$$\arg\min_{\boldsymbol{w}_1} \mathcal{L}(\boldsymbol{w}_1) = \arg\max_{\boldsymbol{w}_1} \mathbb{V}\left[\tilde{\mathbf{z}}_1(\boldsymbol{w}_1)\right] \tag{20.16}$$

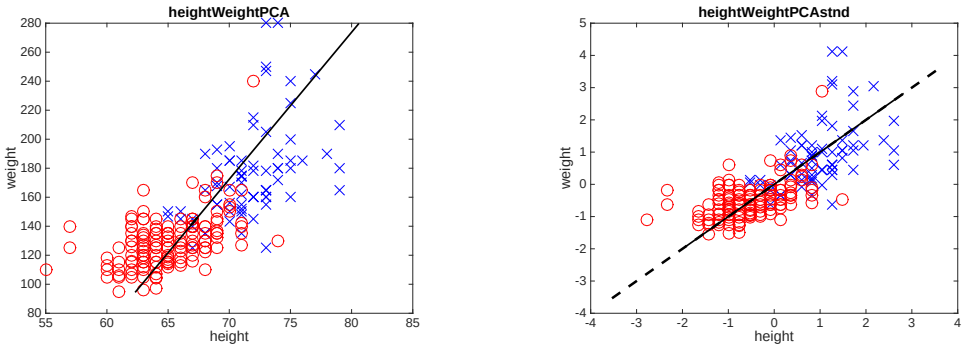
This is why it is often said that PCA finds the directions of maximal variance. (See Figure 20.4 for an illustration.) However, the minimum error formulation is easier to understand and is more general.

20.1.2.3 Induction step

Now let us find another direction $\boldsymbol{w}_2$ to further minimize the reconstruction error, subject to $\boldsymbol{w}_1^{\mathsf{T}}\boldsymbol{w}_2 = 0$ and $\boldsymbol{w}_2^{\mathsf{T}}\boldsymbol{w}_2 = 1$. The error is

$$\mathcal{L}(\boldsymbol{w}_1, \tilde{\mathbf{z}}_1, \boldsymbol{w}_2, \tilde{\mathbf{z}}_2) = \frac{1}{N}\sum_{n=1}^{N} ||\boldsymbol{x}_n - z_{n1}\boldsymbol{w}_1 - z_{n2}\boldsymbol{w}_2||^2 \tag{20.17}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 20.5: Effect of standardization on PCA applied to the height/weight dataset. (Red=female, blue=male.) Left: PCA of raw data. Right: PCA of standardized data. Generated by pcaStandardization.ipynb.

Optimizing wrt $\boldsymbol{w}_1$ and $\boldsymbol{z}_1$ gives the same solution as before. Exercise 20.3 asks you to show that $\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}_2} = 0$ yields $z_{n2} = \boldsymbol{w}_2^\mathsf{T} \boldsymbol{x}_n$. Substituting in yields

$$\mathcal{L}(\boldsymbol{w}_2) = \frac{1}{N} \sum_{n=1}^{N} [\boldsymbol{x}_n^\mathsf{T} \boldsymbol{x}_n - \boldsymbol{w}_1^\mathsf{T} \boldsymbol{x}_n \boldsymbol{x}_n^\mathsf{T} \boldsymbol{w}_1 - \boldsymbol{w}_2^\mathsf{T} \boldsymbol{x}_n \boldsymbol{x}_n^\mathsf{T} \boldsymbol{w}_2] = \text{const} - \boldsymbol{w}_2^\mathsf{T} \hat{\boldsymbol{\Sigma}} \boldsymbol{w}_2 \tag{20.18}$$

Dropping the constant term, plugging in the optimal $\boldsymbol{w}_1$ and adding the constraints yields

$$\tilde{\mathcal{L}}(\boldsymbol{w}_2) = -\boldsymbol{w}_2^\mathsf{T} \hat{\boldsymbol{\Sigma}} \boldsymbol{w}_2 + \lambda_2 (\boldsymbol{w}_2^\mathsf{T} \boldsymbol{w}_2 - 1) + \lambda_{12} (\boldsymbol{w}_2^\mathsf{T} \boldsymbol{w}_1 - 0) \tag{20.19}$$

Exercise 20.3 asks you to show that the solution is given by the eigenvector with the second largest eigenvalue:

$$\hat{\boldsymbol{\Sigma}} \boldsymbol{w}_2 = \lambda_2 \boldsymbol{w}_2 \tag{20.20}$$

The proof continues in this way to show that $\hat{\mathbf{W}} = \mathbf{U}_L$.

20.1.3 Computational issues

In this section, we discuss various practical issues related to using PCA.

20.1.3.1 Covariance matrix vs correlation matrix

We have been working with the eigendecomposition of the covariance matrix. However, it is better to use the correlation matrix instead. The reason is that otherwise PCA can be "misled" by directions in which the variance is high merely because of the measurement scale. Figure 20.5 shows an example of this. On the left, we see that the vertical axis uses a larger range than the horizontal axis. This results in a first principal component that looks somewhat "unnatural". On the right, we show the results of PCA after standardizing the data (which is equivalent to using the correlation matrix instead of the covariance matrix); the results look much better.

20.1.3.2 Dealing with high-dimensional data

We have presented PCA as the problem of finding the eigenvectors of the $D \times D$ covariance matrix $\mathbf{X}^\mathsf{T}\mathbf{X}$. If $D > N$, it is faster to work with the $N \times N$ Gram matrix $\mathbf{X}\mathbf{X}^\mathsf{T}$. We now show how to do this.

First, let $\mathbf{U}$ be an orthonormal matrix containing the eigenvectors of $\mathbf{X}\mathbf{X}^\mathsf{T}$ with corresponding eigenvalues in $\mathbf{\Lambda}$. By definition we have $(\mathbf{X}\mathbf{X}^\mathsf{T})\mathbf{U} = \mathbf{U}\mathbf{\Lambda}$. Pre-multiplying by $\mathbf{X}^\mathsf{T}$ gives

$$(\mathbf{X}^\mathsf{T}\mathbf{X})(\mathbf{X}^\mathsf{T}\mathbf{U}) = (\mathbf{X}^\mathsf{T}\mathbf{U})\mathbf{\Lambda} \tag{20.21}$$

from which we see that the eigenvectors of $\mathbf{X}^\mathsf{T}\mathbf{X}$ are $\mathbf{V} = \mathbf{X}^\mathsf{T}\mathbf{U}$, with eigenvalues given by $\mathbf{\Lambda}$ as before. However, these eigenvectors are not normalized, since $||\boldsymbol{v}_j||^2 = \boldsymbol{u}_j^\mathsf{T}\mathbf{X}\mathbf{X}^\mathsf{T}\boldsymbol{u}_j = \lambda_j \boldsymbol{u}_j^\mathsf{T}\boldsymbol{u}_j = \lambda_j$. The normalized eigenvectors are given by

$$\mathbf{V} = \mathbf{X}^\mathsf{T}\mathbf{U}\mathbf{\Lambda}^{-\frac{1}{2}} \tag{20.22}$$

This provides an alternative way to compute the PCA basis. It also allows us to use the kernel trick, as we discuss in Section 20.4.6.

20.1.3.3 Computing PCA using SVD

In this section, we show the equivalence between PCA as computed using eigenvector methods (Section 20.1) and the truncated SVD.[1]

Let $\mathbf{U}_\Sigma \mathbf{\Lambda}_\Sigma \mathbf{U}_\Sigma^\mathsf{T}$ be the top L eigendecomposition of the covariance matrix $\mathbf{\Sigma} \propto \mathbf{X}^\mathsf{T}\mathbf{X}$ (we assume $\mathbf{X}$ is centered). Recall from Section 20.1.2 that the optimal estimate of the projection weights $\mathbf{W}$ is given by the top L eigenvalues, so $\mathbf{W} = \mathbf{U}_\Sigma$.

Now let $\mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\mathsf{T} \approx \mathbf{X}$ be the L-truncated SVD approximation to the data matrix $\mathbf{X}$. From Equation (7.184), we know that the right singular vectors of $\mathbf{X}$ are the eigenvectors of $\mathbf{X}^\mathsf{T}\mathbf{X}$, so $\mathbf{V}_X = \mathbf{U}_\Sigma = \mathbf{W}$. (In addition, the eigenvalues of the covariance matrix are related to the singular values of the data matrix via $\lambda_k = s_k^2/N$.)

Now suppose we are interested in the projected points (also called the principal components or PC scores), rather than the projection matrix. We have

$$\mathbf{Z} = \mathbf{X}\mathbf{W} = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\mathsf{T} \mathbf{V}_X = \mathbf{U}_X \mathbf{S}_X \tag{20.23}$$

Finally, if we want to approximately reconstruct the data, we have

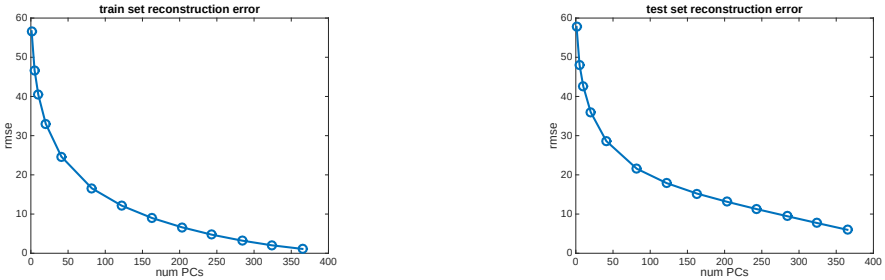
$$\hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}^\mathsf{T} = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\mathsf{T} \tag{20.24}$$

This is precisely the same as a truncated SVD approximation (Section 7.5.5).

Thus we see that we can perform PCA either using an eigendecomposition of $\mathbf{\Sigma}$ or an SVD decomposition of $\mathbf{X}$. The latter is often preferable, for computational reasons. For very high dimensional problems, we can use a randomized SVD algorithm, see e.g., [HMT11; SKT14; DM16]. For example, the randomized solver used by sklearn takes $O(NL^2) + O(L^3)$ time for N examples and L principal components, whereas exact SVD takes $O(ND^2) + O(D^3)$ time.

1. A more detailed explanation can be found at https://bit.ly/2I566OK.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 20.6: Reconstruction error on MNIST vs number of latent dimensions used by PCA. (a) Training set. (b) Test set. Generated by pcaOverfitDemo.ipynb.

20.1.4 Choosing the number of latent dimensions

In this section, we discuss how to choose the number of latent dimensions L for PCA.

20.1.4.1 Reconstruction error

Let us define the reconstruction error on some dataset $\mathcal{D}$ incurred by the model when using L dimensions:

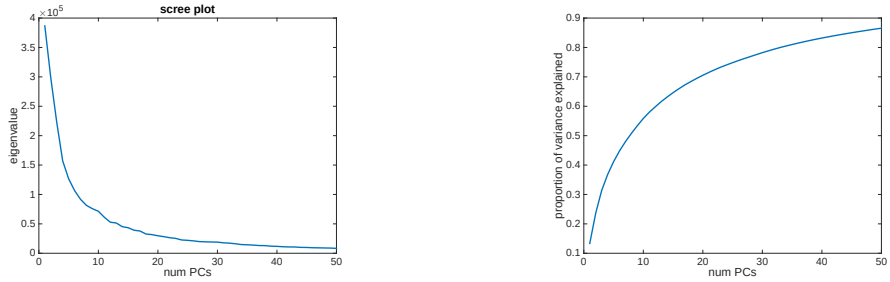
$$\mathcal{L}_L = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}} ||\boldsymbol{x}_n - \hat{\boldsymbol{x}}_n||^2 \tag{20.25}$$

where the reconstruction is given by $\hat{\boldsymbol{x}}_n = \mathbf{W}\boldsymbol{z}_n + \boldsymbol{\mu}$, where $\boldsymbol{z}_n = \mathbf{W}^\mathsf{T}(\boldsymbol{x}_n - \boldsymbol{\mu})$ and $\boldsymbol{\mu}$ is the empirical mean, and $\mathbf{W}$ is estimated as above. Figure 20.6(a) plots $\mathcal{L}_L$ vs L on the MNIST training data. We see that it drops off quite quickly, indicating that we can capture most of the empirical correlation of the pixels with a small number of factors.

Of course, if we use $L = \text{rank}(\mathbf{X})$, we get zero reconstruction error on the training set. To avoid overfitting, it is natural to plot reconstruction error on the test set. This is shown in Figure 20.6(b). Here we see that the error continues to go down even as the model becomes more complex! Thus we do not get the usual U-shaped curve that we typically expect to see in supervised learning. The problem is that PCA is not a proper generative model of the data: If you give it more latent dimensions, it will be able to approximate the test data more accurately. (A similar problem arises if we plot reconstruction error on the test set using K-means clustering, as discussed in Section 21.3.7.) We discuss some solutions to this below.

20.1.4.2 Scree plots

A common alternative to plotting reconstruction error vs L is to use something called a **scree plot**, which is a plot of the eigenvalues λ_j vs j in order of decreasing magnitude. One can show



(a) (b)

Figure 20.7: (a) Scree plot for training set, corresponding to Figure 20.6(a). (b) Fraction of variance explained. Generated by pcaOverfitDemo.ipynb.

(Exercise 20.4) that

$$\mathcal{L}_L = \sum_{j=L+1}^{D} \lambda_j \tag{20.26}$$

Thus as the number of dimensions increases, the eigenvalues get smaller, and so does the reconstruction error, as shown in Figure 20.7a.[2] A related quantity is the **fraction of variance explained**, defined as

$$F_L = \frac{\sum_{j=1}^{L} \lambda_j}{\sum_{j'=1}^{L^{\max}} \lambda_{j'}} \tag{20.27}$$

This captures the same information as the scree plot, but goes up with L (see Figure 20.7b).

20.1.4.3 Profile likelihood

Although there is no U-shape in the reconstruction error plot, there is sometimes a "knee" or "elbow" in the curve, where the error suddenly changes from relatively large errors to relatively small. The idea is that for $L < L^*$, where L^* is the "true" latent dimensionality (or number of clusters), the rate of decrease in the error function will be high, whereas for $L > L^*$, the gains will be smaller, since the model is already sufficiently complex to capture the true distribution.

One way to automate the detection of this change in the gradient of the curve is to compute the **profile likelihood**, as proposed in [ZG06]. The idea is this. Let λ_L be some measure of the error incurred by a model of size L, such that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{L^{\max}}$. In PCA, these are the eigenvalues, but the method can also be applied to the reoconstruction error from K-means clustering (see Section 21.3.7). Now consider partitioning these values into two groups, depending on whether $k < L$ or $k > L$, where L is some threshold which we will determine. To measure the quality of L,

2. The reason for the term "scree plot" is that "the plot looks like the side of a mountain, and 'scree' refers to the debris fallen from a mountain and lying at its base". (Quotation from Kenneth Janda, https://bit.ly/2kqG1yW.)

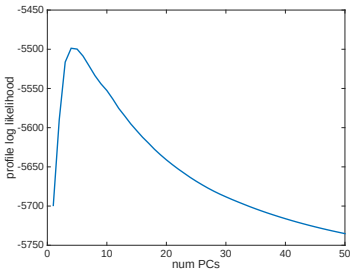


Figure 20.8: Profile likelihood corresponding to PCA model in Figure 20.6(a). Generated by pcaOverfitDemo.ipynb.

we will use a simple change-point model, where $\lambda_k \sim \mathcal{N}(\mu_1, \sigma^2)$ if $k \leq L$, and $\lambda_k \sim \mathcal{N}(\mu_2, \sigma^2)$ if $k > L$. (It is important that σ^2 be the same in both models, to prevent overfitting in the case where one regime has less data than the other.) Within each of the two regimes, we assume the λ_k are iid, which is obviously incorrect, but is adequate for our present purposes. We can fit this model for each $L = 1 : L^{\max}$ by partitioning the data and computing the MLEs, using a pooled estimate of the variance:

$$\mu_1(L) = \frac{\sum_{k \leq L} \lambda_k}{L} \tag{20.28}$$

$$\mu_2(L) = \frac{\sum_{k > L} \lambda_k}{L^{\max} - L} \tag{20.29}$$

$$\sigma^2(L) = \frac{\sum_{k \leq L} (\lambda_k - \mu_1(L))^2 + \sum_{k > L} (\lambda_k - \mu_2(L))^2}{L^{\max}} \tag{20.30}$$

We can then evaluate the profile log likelihood

$$\ell(L) = \sum_{k=1}^{L} \log \mathcal{N}(\lambda_k | \mu_1(L), \sigma^2(L)) + \sum_{k=L+1}^{L^{\max}} \log \mathcal{N}(\lambda_k | \mu_2(L), \sigma^2(L)) \tag{20.31}$$

This is illustrated in Figure 20.8. We see that the peak $L^* = \operatorname{argmax} \ell(L)$ is well determined.

20.2 Factor analysis *

PCA is a simple method for computing a linear low-dimensional representation of data. In this section, we present a generalization of PCA known as **factor analysis**. This is based on a probabilistic model, which means we can treat it as a building block for more complex models, such as the mixture of FA models in Section 20.2.6, or the nonlinear FA model in Section 20.3.5. We can recover PCA as a special limiting case of FA, as we discuss in Section 20.2.2.

20.2.1 Generative model

Factor analysis corresponds to the following linear-Gaussian latent variable generative model:

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0) \tag{20.32}$$

$$p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}|\mathbf{W}\boldsymbol{z} + \boldsymbol{\mu}, \boldsymbol{\Psi}) \tag{20.33}$$

where $\mathbf{W}$ is a $D \times L$ matrix, known as the **factor loading matrix**, and $\boldsymbol{\Psi}$ is a diagonal $D \times D$ covariance matrix.

FA can be thought of as a low-rank version of a Gaussian distribution. To see this, note that the induced marginal distribution $p(\boldsymbol{x}|\boldsymbol{\theta})$ is a Gaussian (see Equation (3.38) for the derivation):

$$p(\boldsymbol{x}|\boldsymbol{\theta}) = \int \mathcal{N}(\boldsymbol{x}|\mathbf{W}\boldsymbol{z} + \boldsymbol{\mu}, \boldsymbol{\Psi})\mathcal{N}(\boldsymbol{z}|\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)d\boldsymbol{z} \tag{20.34}$$

$$= \mathcal{N}(\boldsymbol{x}|\mathbf{W}\boldsymbol{\mu}_0 + \boldsymbol{\mu}, \boldsymbol{\Psi} + \mathbf{W}\boldsymbol{\Sigma}_0\mathbf{W}^\mathsf{T}) \tag{20.35}$$

Hence $\mathbb{E}[\boldsymbol{x}] = \mathbf{W}\boldsymbol{\mu}_0 + \boldsymbol{\mu}$ and $\text{Cov}[\boldsymbol{x}] = \mathbf{W}\text{Cov}[\boldsymbol{z}]\mathbf{W}^\mathsf{T} + \boldsymbol{\Psi} = \mathbf{W}\boldsymbol{\Sigma}_0\mathbf{W}^\mathsf{T} + \boldsymbol{\Psi}$. From this, we see that we can set $\boldsymbol{\mu}_0 = \mathbf{0}$ without loss of generality, since we can always absorb $\mathbf{W}\boldsymbol{\mu}_0$ into $\boldsymbol{\mu}$. Similarly, we can set $\boldsymbol{\Sigma}_0 = \mathbf{I}$ without loss of generality, since we can always absorb a correlated prior by using a new weight matrix, $\tilde{\mathbf{W}} = \mathbf{W}\boldsymbol{\Sigma}_0^{-\frac{1}{2}}$. After these simplifications we have

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\mathbf{0}, \mathbf{I}) \tag{20.36}$$

$$p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\mathbf{W}\boldsymbol{z} + \boldsymbol{\mu}, \boldsymbol{\Psi}) \tag{20.37}$$

$$p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^\mathsf{T} + \boldsymbol{\Psi}) \tag{20.38}$$

For example, suppose where $L = 1$, $D = 2$ and $\boldsymbol{\Psi} = \sigma^2\mathbf{I}$. We illustrate the generative process in this case in Figure 20.9. We can think of this as taking an isotropic Gaussian "spray can", representing the likelihood $p(\boldsymbol{x}|\boldsymbol{z})$, and "sliding it along" the 1d line defined by $\boldsymbol{w}z + \boldsymbol{\mu}$ as we vary the 1d latent prior z. This induces an elongated (and hence correlated) Gaussian in 2d. That is, the induced distribution has the form $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{w}\boldsymbol{w}^\mathsf{T} + \sigma^2\mathbf{I})$.

In general, FA approximates the covariance matrix of the visible vector using a low-rank decomposition:

$$\mathbf{C} = \text{Cov}[\boldsymbol{x}] = \mathbf{W}\mathbf{W}^\mathsf{T} + \boldsymbol{\Psi} \tag{20.39}$$

This only uses $O(LD)$ parameters, which allows a flexible compromise between a full covariance Gaussian, with $O(D^2)$ parameters, and a diagonal covariance, with $O(D)$ parameters.

From Equation (20.39), we see that we should restrict $\boldsymbol{\Psi}$ to be diagonal, otherwise we could set $\mathbf{W} = \mathbf{0}$, thus ignoring the latent factors, while still being able to model any covariance. The marginal variance of each visible variable is given by $\mathbb{V}[x_d] = \sum_{k=1}^{L} w_{dk}^2 + \psi_d$, where the first term is the variance due to the common factors, and the second ψ_d term is called the **uniqueness**, and is the variance term that is specific to that dimension.

We can estimate the parameters of an FA model using EM (see Section 20.2.3). Once we have fit the model, we can compute probabilistic latent embeddings using $p(\boldsymbol{z}|\boldsymbol{x})$. Using Bayes rule for Gaussians we have

$$p(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}|\mathbf{W}^\mathsf{T}\mathbf{C}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}), \mathbf{I} - \mathbf{W}^\mathsf{T}\mathbf{C}^{-1}\mathbf{W}) \tag{20.40}$$

where $\mathbf{C}$ is defined in Equation (20.39).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

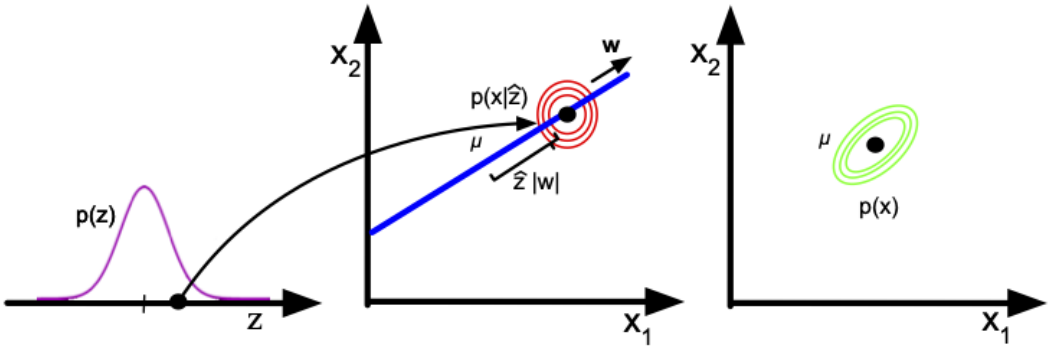


Figure 20.9: Illustration of the FA generative process, where we have $L = 1$ latent dimension generating $D = 2$ observed dimensions; we assume $\boldsymbol{\Psi} = \sigma^2\mathbf{I}$. The latent factor has value $z \in \mathbb{R}$, sampled from $p(z)$; this gets mapped to a 2d offset $\boldsymbol{\delta} = z\boldsymbol{w}$, where $\boldsymbol{w} \in \mathbb{R}^2$, which gets added to $\boldsymbol{\mu}$ to define a Gaussian $p(\boldsymbol{x}|z) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu} + \boldsymbol{\delta}, \sigma^2\mathbf{I})$. By integrating over z, we "slide" this circular Gaussian "spray can" along the principal component axis $\boldsymbol{w}$, which induces elliptical Gaussian contours in $\boldsymbol{x}$ space centered on $\boldsymbol{\mu}$. Adapted from Figure 12.9 of [Bis06].

20.2.2 Probabilistic PCA

In this section, we consider a special case of the factor analysis model in which $\mathbf{W}$ has orthonormal columns, and $\boldsymbol{\Psi} = \sigma^2\mathbf{I}$. This model is called **probabilistic principal components analysis** (**PPCA**) [TB99], or **sensible PCA** [Row97]. The marginal distribution on the visible variables has the form

$$p(\boldsymbol{x}|\boldsymbol{\theta}) = \int \mathcal{N}(\boldsymbol{x}|\mathbf{W}\boldsymbol{z}, \sigma^2\mathbf{I})\mathcal{N}(\boldsymbol{z}|\mathbf{0}, \mathbf{I})d\boldsymbol{z} = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \mathbf{C}) \tag{20.41}$$

where

$$\mathbf{C} = \mathbf{W}\mathbf{W}^\mathsf{T} + \sigma^2\mathbf{I} \tag{20.42}$$

The log likelihood for PPCA is given by

$$\log p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2) = -\frac{ND}{2}\log(2\pi) - \frac{N}{2}\log|\mathbf{C}| - \frac{1}{2}\sum_{n=1}^{N}(\boldsymbol{x}_n - \boldsymbol{\mu})^\mathsf{T}\mathbf{C}^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}) \tag{20.43}$$

The MLE for $\boldsymbol{\mu}$ is $\overline{\boldsymbol{x}}$. Plugging in gives

$$\log p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2) = -\frac{N}{2}\left[D\log(2\pi) + \log|\mathbf{C}| + \mathrm{tr}(\mathbf{C}^{-1}\mathbf{S})\right] \tag{20.44}$$

where $\mathbf{S} = \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n - \overline{\boldsymbol{x}})(\boldsymbol{x}_n - \overline{\boldsymbol{x}})^\mathsf{T}$ is the empirical covariance matrix.

In [TB99; Row97] they show that the maximum of this objective must satisfy

$$\mathbf{W} = \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})^{\frac{1}{2}}\mathbf{R} \tag{20.45}$$

where $\mathbf{U}_L$ is a $D \times L$ matrix whose columns are given by the L eigenvectors of $\mathbf{S}$ with largest eigenvalues, $\mathbf{L}_L$ is the $L \times L$ diagonal matrix of eigenvalues, and $\mathbf{R}$ is an arbitrary $L \times L$ orthogonal matrix, which (WLOG) we can take to be $\mathbf{R} = \mathbf{I}$. In the noise-free limit, where $\sigma^2 = 0$, we see that $\mathbf{W}_{\text{mle}} = \mathbf{U}_L \mathbf{L}_L^{\frac{1}{2}}$, which is proportional to the PCA solution.

The MLE for the observation variance is

$$\sigma^2 = \frac{1}{D-L} \sum_{i=L+1}^{D} \lambda_i \tag{20.46}$$

which is the average distortion associated with the discarded dimensions. If $L = D$, then the estimated noise is 0, since the model collapses to $\boldsymbol{z} = \boldsymbol{x}$.

To compute the likelihood $p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2)$, we need to evaluate $\mathbf{C}^{-1}$ and $\log|\mathbf{C}|$, where $\mathbf{C}$ is a $D \times D$ matrix. To do this efficiently, we can use the matrix inversion lemma to write

$$\mathbf{C}^{-1} = \sigma^{-2}\left[\mathbf{I} - \mathbf{W}\mathbf{M}^{-1}\mathbf{W}^{\mathsf{T}}\right] \tag{20.47}$$

where the $L \times L$ dimensional matrix $\mathbf{M}$ is given by

$$\mathbf{M} = \mathbf{W}^{\mathsf{T}}\mathbf{W} + \sigma^2\mathbf{I} \tag{20.48}$$

When we plug in the MLE for $\mathbf{W}$ from Equation (20.45) (using $\mathbf{R} = \mathbf{I}$) we find

$$\mathbf{M} = \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})\mathbf{U}_L^{\mathsf{T}} + \sigma^2\mathbf{I} \tag{20.49}$$

and hence

$$\mathbf{C}^{-1} = \sigma^{-2}\left[\mathbf{I} - \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})\boldsymbol{\Lambda}_L^{-1}\mathbf{U}_L^{\mathsf{T}}\right] \tag{20.50}$$

$$\log|\mathbf{C}| = (D-L)\log\sigma^2 + \sum_{j=1}^{L}\log\lambda_j \tag{20.51}$$

Thus we can avoid all matrix inversions (since $\boldsymbol{\Lambda}_L^{-1} = \text{diag}(1/\lambda_j)$).

To use PPCA as an alternative to PCA, we need to compute the posterior mean $\mathbb{E}[\boldsymbol{z}|\boldsymbol{x}]$, which is the equivalent of the encoder model. Using Bayes rule for Gaussians we have

$$p(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}|\mathbf{M}^{-1}\mathbf{W}^{\mathsf{T}}(\boldsymbol{x} - \boldsymbol{\mu}), \sigma^2\mathbf{M}^{-1}) \tag{20.52}$$

where $\mathbf{M}$ is defined in Equation (20.48). In the $\sigma^2 = 0$ limit, the posterior mean using the MLE parameters becomes

$$\mathbb{E}[\boldsymbol{z}|\boldsymbol{x}] = (\mathbf{W}^{\mathsf{T}}\mathbf{W})^{-1}\mathbf{W}^{\mathsf{T}}(\boldsymbol{x} - \overline{\boldsymbol{x}}) \tag{20.53}$$

which is the orthogonal projection of the data into the latent space, as in standard PCA.

20.2.3 EM algorithm for FA/PPCA

In this section, we describe one method for computing the MLE for the FA model using the EM algorithm, based on [RT82; GH96].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.2.3.1 EM for FA

In the E step, we compute the posterior embeddings

$$p(\boldsymbol{z}_i|\boldsymbol{x}_i,\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{z}_i|\boldsymbol{m}_i,\boldsymbol{\Sigma}_i) \tag{20.54}$$

$$\boldsymbol{\Sigma}_i \triangleq (\mathbf{I}_L + \mathbf{W}^\mathsf{T}\boldsymbol{\Psi}^{-1}\mathbf{W})^{-1} \tag{20.55}$$

$$\boldsymbol{m}_i \triangleq \boldsymbol{\Sigma}_i(\mathbf{W}^\mathsf{T}\boldsymbol{\Psi}^{-1}(\boldsymbol{x}_i - \boldsymbol{\mu})) \tag{20.56}$$

In the M step, it is easiest to estimate $\boldsymbol{\mu}$ and $\mathbf{W}$ at the same time, by defining $\tilde{\mathbf{W}} = (\mathbf{W}, \boldsymbol{\mu})$, $\tilde{\boldsymbol{z}} = (\boldsymbol{z}, 1)$, Also, define

$$\boldsymbol{b}_i \triangleq \mathbb{E}\left[\tilde{\mathbf{z}}|\boldsymbol{x}_i\right] = [\boldsymbol{m}_i; 1] \tag{20.57}$$

$$\mathbf{C}_i \triangleq \mathbb{E}\left[\tilde{\mathbf{z}}\tilde{\mathbf{z}}^T|\boldsymbol{x}_i,\right] = \begin{pmatrix} \mathbb{E}\left[\boldsymbol{z}\boldsymbol{z}^T|\boldsymbol{x}_i\right] & \mathbb{E}\left[\boldsymbol{z}|\boldsymbol{x}_i\right] \\ \mathbb{E}\left[\boldsymbol{z}|\boldsymbol{x}_i\right]^T & 1 \end{pmatrix} \tag{20.58}$$

Then the M step is as follows:

$$\hat{\tilde{\mathbf{W}}} = \left[\sum_i \boldsymbol{x}_i \boldsymbol{b}_i^\mathsf{T}\right]\left[\sum_i \mathbf{C}_i\right]^{-1} \tag{20.59}$$

$$\hat{\boldsymbol{\Psi}} = \frac{1}{N}\text{diag}\left\{\sum_i \left(\boldsymbol{x}_i - \hat{\tilde{\mathbf{W}}}\boldsymbol{b}_i\right)\boldsymbol{x}_i^T\right\} \tag{20.60}$$

Note that these updates are for "vanilla" EM. A much faster version of this algorithm, based on ECM, is described in [ZY08].

20.2.3.2 EM for (P)PCA

We can also use EM to fit the PPCA model, which provides a useful alternative to eigenvector methods. This relies on the probabilistic formulation of PCA. However the algorithm continues to work in the zero noise limit, $\sigma^2 = 0$, as shown by [Row97].

In particular, let $\tilde{\mathbf{Z}} = \mathbf{Z}^\mathsf{T}$ be a $L \times N$ matrix storing the posterior means (low-dimensional representations) along its columns. Similarly, let $\tilde{\mathbf{X}} = \mathbf{X}^\mathsf{T}$ store the original data along its columns. From Equation (20.52), when $\sigma^2 = 0$, we have

$$\tilde{\mathbf{Z}} = (\mathbf{W}^T\mathbf{W})^{-1}\mathbf{W}^T\tilde{\mathbf{X}} \tag{20.61}$$

This constitutes the E step. Notice that this is just an orthogonal projection of the data.

From Equation 20.59, the M step is given by

$$\hat{\mathbf{W}} = \left[\sum_i \boldsymbol{x}_i \mathbb{E}\left[\boldsymbol{z}_i\right]^T\right]\left[\sum_i \mathbb{E}\left[\boldsymbol{z}_i\right]\mathbb{E}\left[\boldsymbol{z}_i\right]^T\right]^{-1} \tag{20.62}$$

where we exploited the fact that $\boldsymbol{\Sigma} = \text{Cov}\left[\boldsymbol{z}_i|\boldsymbol{x}_i,\boldsymbol{\theta}\right] = 0\mathbf{I}$ when $\sigma^2 = 0$.

It is worth comparing this expression to the MLE for multi-output linear regression (Equation 11.2), which has the form $\mathbf{W} = (\sum_i \boldsymbol{y}_i\boldsymbol{x}_i^T)(\sum_i \boldsymbol{x}_i\boldsymbol{x}_i^T)^{-1}$. Thus we see that the M step is like linear regression where we replace the observed inputs by the expected values of the latent variables.

In summary, here is the entire algorithm:

$$\tilde{\mathbf{Z}} = (\mathbf{W}^T\mathbf{W})^{-1}\mathbf{W}^T\tilde{\mathbf{X}} \text{ (E step)} \tag{20.63}$$

$$\mathbf{W} = \tilde{\mathbf{X}}\tilde{\mathbf{Z}}^T(\tilde{\mathbf{Z}}\tilde{\mathbf{Z}}^T)^{-1} \text{ (M step)} \tag{20.64}$$

[TB99] showed that the only stable fixed point of the EM algorithm is the globally optimal solution. That is, the EM algorithm converges to a solution where $\mathbf{W}$ spans the same linear subspace as that defined by the first L eigenvectors. However, if we want $\mathbf{W}$ to be orthogonal, and to contain the eigenvectors in descending order of eigenvalue, we have to orthogonalize the resulting matrix (which can be done quite cheaply). Alternatively, we can modify EM to give the principal basis directly [AO03].

This algorithm has a simple physical analogy in the case $D = 2$ and $L = 1$ [Row97]. Consider some points in $\mathbb{R}^2$ attached by springs to a rigid rod, whose orientation is defined by a vector $\boldsymbol{w}$. Let z_i be the location where the i'th spring attaches to the rod. In the E step, we hold the rod fixed, and let the attachment points slide around so as to minimize the spring energy (which is proportional to the sum of squared residuals). In the M step, we hold the attachment points fixed and let the rod rotate so as to minimize the spring energy. See Figure 20.10 for an illustration.

20.2.3.3 Advantages

EM for PCA has the following advantages over eigenvector methods:

- EM can be faster. In particular, assuming $N, D \gg L$, the dominant cost of EM is the projection operation in the E step, so the overall time is $O(TLND)$, where T is the number of iterations. [Row97] showed experimentally that the number of iterations is usually very small (the mean was 3.6), regardless of N or D. (This result depends on the ratio of eigenvalues of the empirical covariance matrix.) This is much faster than the $O(\min(ND^2, DN^2))$ time required by straightforward eigenvector methods, although more sophisticated eigenvector methods, such as the Lanczos algorithm, have running times comparable to EM.
- EM can be implemented in an online fashion, i.e., we can update our estimate of $\mathbf{W}$ as the data streams in.
- EM can handle missing data in a simple way (see e.g., [IR10; DJ15]).
- EM can be extended to handle mixtures of PPCA/ FA models (see Section 20.2.6).
- EM can be modified to variational EM or to variational Bayes EM to fit more complex models (see e.g., Section 20.2.7).

20.2.4 Unidentifiability of the parameters

The parameters of a FA model are unidentifiable. To see this, consider a model with weights $\mathbf{W}$ and observation covariance $\boldsymbol{\Psi}$. We have

$$\text{Cov}[\boldsymbol{x}] = \mathbf{W}\mathbb{E}\left[\boldsymbol{z}\boldsymbol{z}^\mathsf{T}\right]\mathbf{W}^\mathsf{T} + \mathbb{E}\left[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\mathsf{T}\right] = \mathbf{W}\mathbf{W}^\mathsf{T} + \boldsymbol{\Psi} \tag{20.65}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

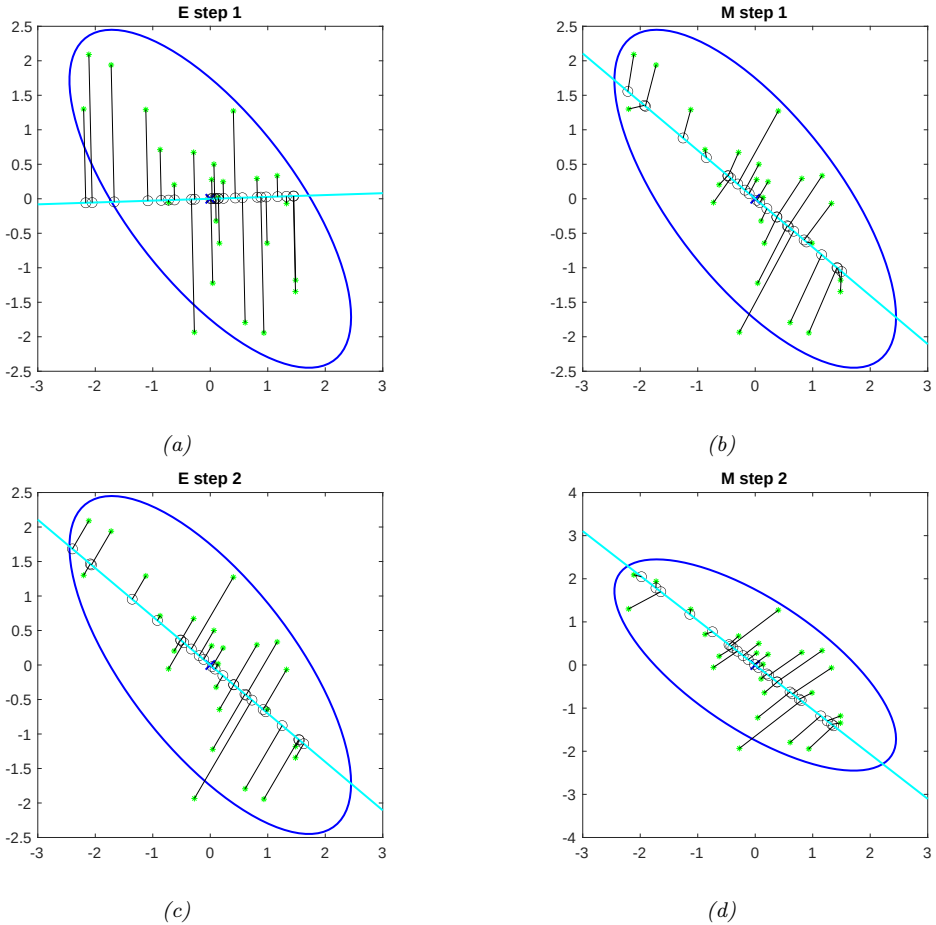


Figure 20.10: Illustration of EM for PCA when $D = 2$ and $L = 1$. Green stars are the original data points, black circles are their reconstructions. The weight vector $\boldsymbol{w}$ is represented by blue line. (a) We start with a random initial guess of $\boldsymbol{w}$. The E step is represented by the orthogonal projections. (b) We update the rod $\boldsymbol{w}$ in the M step, keeping the projections onto the rod (black circles) fixed. (c) Another E step. The black circles can 'slide' along the rod, but the rod stays fixed. (d) Another M step. Adapted from Figure 12.12 of [Bis06]. Generated by pcaEmStepByStep.ipynb.

Now consider a different model with weights $\tilde{\mathbf{W}} = \mathbf{WR}$, where $\mathbf{R}$ is an arbitrary orthogonal rotation matrix, satisfying $\mathbf{RR}^\mathsf{T} = \mathbf{I}$. This has the same likelihood, since

$$\text{Cov}\left[\boldsymbol{x}\right] = \tilde{\mathbf{W}}\mathbb{E}\left[\boldsymbol{z}\boldsymbol{z}^\mathsf{T}\right]\tilde{\mathbf{W}}^\mathsf{T} + \mathbb{E}\left[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\mathsf{T}\right] = \mathbf{WRR}^\mathsf{T}\mathbf{W}^\mathsf{T} + \boldsymbol{\Psi} = \mathbf{WW}^\mathsf{T} + \boldsymbol{\Psi} \tag{20.66}$$

Geometrically, multiplying $\mathbf{W}$ by an orthogonal matrix is like rotating $\boldsymbol{z}$ before generating $\boldsymbol{x}$; but since $\boldsymbol{z}$ is drawn from an isotropic Gaussian, this makes no difference to the likelihood. Consequently, we cannot uniquely identify $\mathbf{W}$, and therefore cannot uniquely identify the latent factors, either.

To break this symmetry, several solutions can be used, as we discuss below.

- **Forcing W to have orthonormal columns**. Perhaps the simplest solution to the identifiability problem is to force $\mathbf{W}$ to have orthonormal columns. This is the approach adopted by PCA. The resulting posterior estimate will then be unique, up to permutation of the latent dimensions. (In PCA, this ordering ambiguity is resolved by sorting the dimensions in order of decreasing eigenvalues of $\mathbf{W}$.)
- **Forcing W to be lower triangular**. One way to resolve permutation unidentifiability, which is popular in the Bayesian community (e.g., [LW04c]), is to ensure that the first visible feature is only generated by the first latent factor, the second visible feature is only generated by the first two latent factors, and so on. For example, if $L = 3$ and $D = 4$, the corresponding factor loading matrix is given by

$$\mathbf{W} = \begin{pmatrix} w_{11} & 0 & 0 \\ w_{21} & w_{22} & 0 \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{pmatrix} \tag{20.67}$$

  We also require that $w_{kk} > 0$ for $k = 1 : L$. The total number of parameters in this constrained matrix is $D + DL - L(L-1)/2$, which is equal to the number of uniquely identifiable parameters in FA.[3] The disadvantage of this method is that the first L visible variables, known as the **founder variables**, affect the interpretation of the latent factors, and so must be chosen carefully.
- **Sparsity promoting priors on the weights**. Instead of pre-specifying which entries in $\mathbf{W}$ are zero, we can encourage the entries to be zero, using ℓ_1 regularization [ZHT06], ARD [Bis99; AB08], or spike-and-slab priors [Rat+09]. This is called sparse factor analysis. This does not necessarily ensure a unique MAP estimate, but it does encourage interpretable solutions.
- **Choosing an informative rotation matrix**. There are a variety of heuristic methods that try to find rotation matrices $\mathbf{R}$ which can be used to modify $\mathbf{W}$ (and hence the latent factors) so as to try to increase the interpretability, typically by encouraging them to be (approximately) sparse. One popular method is known as **varimax** [Kai58].
- **Use of non-Gaussian priors for the latent factors**. If we replace the prior on the latent variables, $p(\boldsymbol{z})$, with a non-Gaussian distribution, we can sometimes uniquely identify $\mathbf{W}$, as well as the latent factors. See e.g., [KKH20] for details.

20.2.5 Nonlinear factor analysis

The FA model assumes the observed data can be modeled as arising from a linear mapping from a low-dimensional set of Gaussian factors. One way to relax this assumption is to let the mapping from $\boldsymbol{z}$ to $\boldsymbol{x}$ be a nonlinear model, such as a neural network. That is, the model becomes

$$p(\boldsymbol{x}) = \int \mathcal{N}(\boldsymbol{x}|f(\boldsymbol{z};\boldsymbol{\theta}), \boldsymbol{\Psi})\mathcal{N}(\boldsymbol{z}|\mathbf{0}, \mathbf{I})d\boldsymbol{z} \tag{20.68}$$

3. We get D parameters for $\boldsymbol{\Psi}$ and DL for $\mathbf{W}$, but we need to remove $L(L-1)/2$ degrees of freedom coming from $\mathbf{R}$, since that is the dimensionality of the space of orthogonal matrices of size $L \times L$. To see this, note that there are $L-1$ free parameters in $\mathbf{R}$ in the first column (since the column vector must be normalized to unit length), there are $L-2$ free parameters in the second column (which must be orthogonal to the first), and so on.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

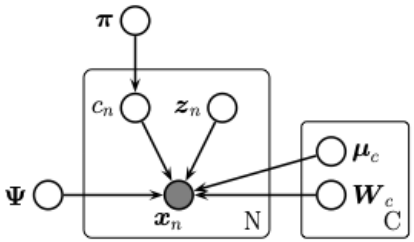


Figure 20.11: Mixture of factor analyzers as a PGM.

This is called **nonlinear factor analysis**. Unfortunately we can no longer compute the posterior or the MLE exactly, so we need to use approximate methods. In Section 20.3.5, we discuss variational autoencoders, which is the most common way to approximate a nonlinear FA model.

20.2.6 Mixtures of factor analyzers

The factor analysis model (Section 20.2) assumes the observed data can be modeled as arising from a linear mapping from a low-dimensional set of Gaussian factors. One way to relax this assumption is to assume the model is only locally linear, so the overall model becomes a (weighted) combination of FA models; this is called a **mixture of factor analyzers**. The overall model for the data is a mixture of linear manifolds, which can be used to approximate an overall curved manifold.

More precisely, let the latent indicator $m_n \in \{1, \ldots, K\}$ specify which subspace (cluster) we should use to generate data point n. If $m_n = k$, we sample $\boldsymbol{z}_n$ from a Gaussian prior and pass it through the $\mathbf{W}_k$ matrix and add noise, where $\mathbf{W}_k$ maps from the L-dimensional subspace to the D-dimensional visible space.[4] More precisely, the model is as follows:

$$p(\boldsymbol{x}_n|\boldsymbol{z}_n, m_n = k, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_k + \mathbf{W}_k\boldsymbol{z}_n, \boldsymbol{\Psi}_k) \tag{20.69}$$

$$p(\boldsymbol{z}_n|\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{z}_n|\mathbf{0}, \mathbf{I}) \tag{20.70}$$

$$p(m_n|\boldsymbol{\theta}) = \text{Cat}(m_n|\boldsymbol{\pi}) \tag{20.71}$$

This is called a **mixture of factor analyzers** (MFA) [GH96]. The corresponding distribution in the visible space is given by

$$p(\boldsymbol{x}|\boldsymbol{\theta}) = \sum_k p(c = k) \int d\boldsymbol{z}\; p(\boldsymbol{z}|c)p(\boldsymbol{x}|\boldsymbol{z}, c) = \sum_k \pi_k \int d\boldsymbol{z}\; \mathcal{N}(\boldsymbol{z}|\boldsymbol{\mu}_k, \mathbf{I})\mathcal{N}(\boldsymbol{x}|\mathbf{W}\boldsymbol{z}, \sigma^2\mathbf{I}) \tag{20.72}$$

In the special case that $\boldsymbol{\Psi}_k = \sigma^2\mathbf{I}$, we get a mixture of PPCA models (although it is difficult to ensure orthogonality of the $\mathbf{W}_k$ in this case). See Figure 20.12 for an example of the method applied to some 2d data.

We can think of this as a low-rank version of a mixture of Gaussians. In particular, this model needs $O(KLD)$ parameters instead of the $O(KD^2)$ parameters needed for a mixture of full covariance Gaussians. This can reduce overfitting.

4. If we allow $\boldsymbol{z}_n$ to depend on m_n, we can let each subspace have a different dimensionality, as suggested in [KS15].

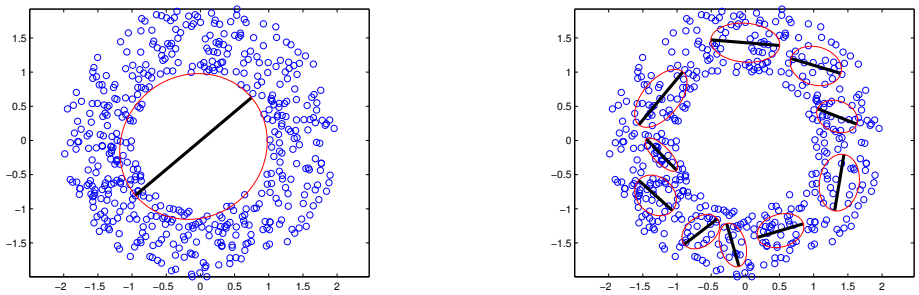


Figure 20.12: Mixture of PPCA models fit to a 2d dataset, using $L = 1$ latent dimensions. (a) $K = 1$ mixture components. (b) $K = 10$ mixture components. Generated by mixPpcaDemo.ipynb.

20.2.7 Exponential family factor analysis

So far we have assumed the observed data is real-valued, so $\boldsymbol{x}_n \in \mathbb{R}^D$. If we want to model other kinds of data (e.g., binary or categorical), we can simply replace the Gaussian output distribution with a suitable member of the exponential family, where the natural parameters are given by a linear function of $\boldsymbol{z}_n$. That is, we use

$$p(\boldsymbol{x}_n|\boldsymbol{z}_n) = \exp(\mathcal{T}(\boldsymbol{x})^\mathsf{T}\boldsymbol{\theta} + h(\boldsymbol{x}) - g(\boldsymbol{\theta})) \tag{20.73}$$

where the $N \times D$ matrix of natural parameters is assumed to be given by the low rank decomposition $\boldsymbol{\Theta} = \mathbf{ZW}$, where $\mathbf{Z}$ is $N \times L$ and $\mathbf{W}$ is $L \times D$. The resulting model is called **exponential family factor analysis**.

Unlike the linear-Gaussian FA, we cannot compute the exact posterior $p(\boldsymbol{z}_n|\boldsymbol{x}_n, \mathbf{W})$ due to the lack of conjugacy between the expfam likelihood and the Gaussian prior. Furthermore, we cannot compute the exact marginal likelihood either, which prevents us from finding the optimal MLE.

[CDS02] proposed a coordinate ascent method for a deterministic variant of this model, known as **exponential family PCA**. This alternates between computing a point estimate of $\boldsymbol{z}_n$ and $\mathbf{W}$. This can be regarded as a degenerate version of variational EM, where the E step uses a delta function posterior for $\boldsymbol{z}_n$. [GS08] present an improved algorithm that finds the global optimum, and [Ude+16] presents an extension called **generalized low rank models**, that covers many different kinds of loss function.

However, it is often preferable to use a probabilistic version of the model, rather than computing point estimates of the latent factors. In this case, we must represent the posterior using a non-degenerate distribution to avoid overfitting, since the number of latent variables is proportional to the number of datacases [WCS08]. Fortunately, we can use a non-degenerate posterior, such as a Gaussian, by optimizing the variational lower bound. We give some examples of this below.

20.2.7.1 Example: binary PCA

Consider a factored Bernoulli likelihood:

$$p(\boldsymbol{x}|\boldsymbol{z}) = \prod_d \text{Ber}(x_d|\sigma(\boldsymbol{w}_d^\mathsf{T}\boldsymbol{z})) \tag{20.74}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

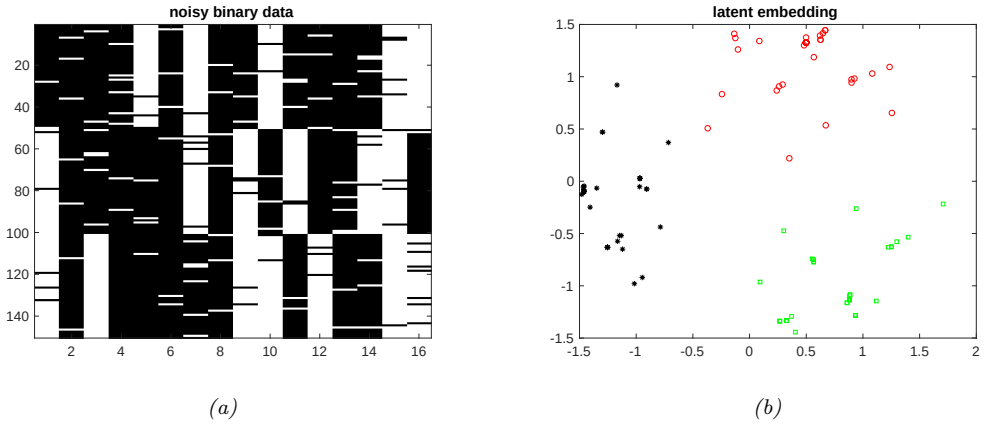


Figure 20.13: (a) 150 synthetic 16 dimensional bit vectors. (b) The 2d embedding learned by binary PCA, fit using variational EM. We have color coded points by the identity of the true "prototype" that generated them. (c) Predicted probability of being on. (d) Thresholded predictions. Generated by binary_fa_demo.ipynb.

Suppose we observe $N = 150$ bit vectors of length $D = 16$. Each example is generated by choosing one of three binary prototype vectors, and then by flipping bits at random. See Figure 20.13(a) for the data. We can fit this using the variational EM algorithm (see [Tip98] for details). We use $L = 2$ latent dimensions to allow us to visualize the latent space. In Figure 20.13(b), we plot $\mathbb{E}\left[\boldsymbol{z}_n|\boldsymbol{x}_n, \hat{\mathbf{W}}\right]$. We see that the projected points group into three distinct clusters, as is to be expected. In Figure 20.13(c), we plot the reconstructed version of the data, which is computed as follows:

$$p(\hat{x}_{nd} = 1|\boldsymbol{x}_n) = \int d\boldsymbol{z}_n \; p(\boldsymbol{z}_n|\boldsymbol{x}_n) p(\hat{x}_{nd}|\boldsymbol{z}_n) \tag{20.75}$$

If we threshold these probabilities at 0.5 (corresponding to a MAP estimate), we get the "denoised" version of the data in Figure 20.13(d).

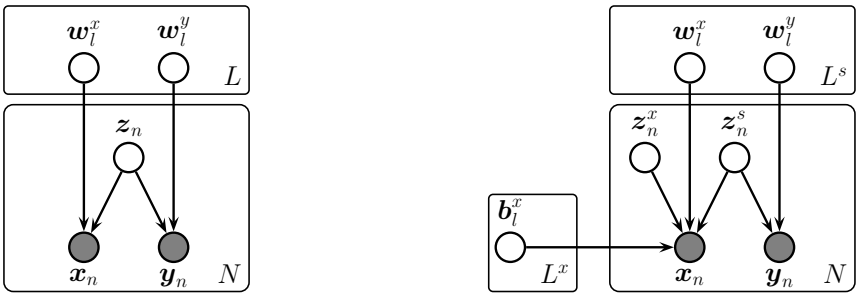


Figure 20.14: Gaussian latent factor models for paired data. (a) Supervised PCA. (b) Partial least squares.

20.2.7.2 Example: categorical PCA

We can generalize the model in Section 20.2.7.1 to handle categorical data by using the following likelihood:

$$p(\boldsymbol{x}|\boldsymbol{z}) = \prod_d \text{Cat}(x_d|\text{softmax}(\mathbf{W}_d\boldsymbol{z})) \tag{20.76}$$

We call this **categorical PCA** (**CatPCA**). A variational EM algorithm for fitting this is described in [Kha+10].

20.2.8 Factor analysis models for paired data

In this section, we discuss linear-Gaussian factor analysis models when we have two kinds of observed variables, $\boldsymbol{x} \in \mathbb{R}^{D_x}$ and $\boldsymbol{y} \in \mathbb{R}^{D_y}$, which are paired. These often correspond to different sensors or modalities (e.g., images and sound). We follow the presentation of [Vir10].

20.2.8.1 Supervised PCA

In **supervised PCA** [Yu+06], we model the joint $p(\boldsymbol{x}, \boldsymbol{y})$ using a shared low-dimensional representation using the following linear Gaussian model:

$$p(\boldsymbol{z}_n) = \mathcal{N}(\boldsymbol{z}_n|\mathbf{0}, \mathbf{I}_L) \tag{20.77}$$
$$p(\boldsymbol{x}_n|\boldsymbol{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}_n|\mathbf{W}_x\boldsymbol{z}_n, \sigma_x^2\mathbf{I}_{D_x}) \tag{20.78}$$
$$p(\boldsymbol{y}_n|\boldsymbol{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{y}_n|\mathbf{W}_y\boldsymbol{z}_n, \sigma_y^2\mathbf{I}_{D_y}) \tag{20.79}$$

This is illustrated as a graphical model in Figure 20.14a. The intuition is that $\boldsymbol{z}_n$ is a shared latent subspace, that captures features that $\boldsymbol{x}_n$ and $\boldsymbol{y}_n$ have in common. The variance terms σ_x and σ_y control how much emphasis the model puts on the two different signals. If we put a prior on the parameters $\boldsymbol{\theta} = (\mathbf{W}_x, \mathbf{W}_y, \sigma_x, \sigma_y)$, we recover the **Bayesian factor regression** model of [Wes03].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can marginalize out $\boldsymbol{z}_n$ to get $p(\boldsymbol{y}_n|\boldsymbol{x}_n)$. If $\boldsymbol{y}_n$ is a scalar, this becomes

$$p(y_n|\boldsymbol{x}_n, \boldsymbol{\theta}) = \mathcal{N}(y_n|\boldsymbol{x}_n^\mathsf{T}\boldsymbol{v}, \boldsymbol{w}_y^\mathsf{T}\mathbf{C}\boldsymbol{w}_y + \sigma_y^2) \tag{20.80}$$

$$\mathbf{C} = (\mathbf{I} + \sigma_x^{-2}\mathbf{W}_x^\mathsf{T}\mathbf{W}_x)^{-1} \tag{20.81}$$

$$\boldsymbol{v} = \sigma_x^{-2}\mathbf{W}_x\mathbf{C}\boldsymbol{w}_y \tag{20.82}$$

To apply this to the classification setting, we can use supervised ePCA [Guo09], in which we replace the Gaussian $p(\boldsymbol{y}|\boldsymbol{z})$ with a logistic regression model.

This model is completely symmetric in $\boldsymbol{x}$ and $\boldsymbol{y}$. If our goal is to predict $\boldsymbol{y}$ from $\boldsymbol{x}$ via the latent bottleneck $\boldsymbol{z}$, then we might want to upweight the likelihood term for $\boldsymbol{y}$, as proposed in [Ris+08]. This gives

$$p(\mathbf{X}, \mathbf{Y}, \mathbf{Z}|\boldsymbol{\theta}) = p(\mathbf{Y}|\mathbf{Z}, \mathbf{W}_y)p(\mathbf{X}|\mathbf{Z}, \mathbf{W}_x)^\alpha p(\mathbf{Z}) \tag{20.83}$$

where $\alpha \leq 1$ controls the relative importance of modeling the two sources. The value of α can be chosen by cross-validation.

20.2.8.2 Partial least squares

Another way to improve the predictive performance in supervised tasks is to allow the inputs $\boldsymbol{x}$ to have their own "private" noise source that is independent on the target variable, since not all variation in $\boldsymbol{x}$ is relevant for predictive purposes. We can do this by introducing an extra latent variable $\boldsymbol{z}_n^x$ just for the inputs, that is different from $\boldsymbol{z}_n^s$ that is the shared bottleneck between $\boldsymbol{x}_n$ and $\boldsymbol{y}_n$. In the Gaussian case, the overall model has the form

$$p(\boldsymbol{z}_n) = \mathcal{N}(\boldsymbol{z}_n^s|\mathbf{0}, \mathbf{I})\mathcal{N}(\boldsymbol{z}_n^x|\mathbf{0}, \mathbf{I}) \tag{20.84}$$

$$p(\boldsymbol{x}_n|\boldsymbol{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}_n|\mathbf{W}_x\boldsymbol{z}_n^s + \mathbf{B}_x\boldsymbol{z}_n^x, \sigma_x^2\mathbf{I}) \tag{20.85}$$

$$p(\boldsymbol{y}_n|\boldsymbol{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{y}_n|\mathbf{W}_y\boldsymbol{z}_n^s, \sigma_y^2\mathbf{I}) \tag{20.86}$$

See Figure 20.14b. MLE for $\boldsymbol{\theta}$ in this model is equivalent to the technique of **partial least squares** (**PLS**) [Gus01; Nou+02; Sun+09].

20.2.8.3 Canonical correlation analysis

In some cases, we want to use a fully symmetric model, so we can capture the dependence between $\boldsymbol{x}$ and $\boldsymbol{y}$, while allowing for domain-specific or "private" noise sources. We can do this by introducing a latent variable $\boldsymbol{z}_n^x$ just for $\boldsymbol{x}_n$, a latent variable $\boldsymbol{z}_n^y$ just for $\boldsymbol{y}_n$, and a shared latent variable $\boldsymbol{z}_n^s$. In the Gaussian case, the overall model has the form

$$p(\boldsymbol{z}_n) = \mathcal{N}(\boldsymbol{z}_n^s|\mathbf{0}, \mathbf{I})\mathcal{N}(\boldsymbol{z}_n^x|\mathbf{0}, \mathbf{I})\mathcal{N}(\boldsymbol{z}_n^y|\mathbf{0}, \mathbf{I}) \tag{20.87}$$

$$\tag{20.88}$$

$$p(\boldsymbol{x}_n|\boldsymbol{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}_n|\mathbf{W}_x\boldsymbol{z}_n^s + \mathbf{B}_x\boldsymbol{z}_n^x, \sigma_x^2\mathbf{I}) \tag{20.89}$$

$$p(\boldsymbol{y}_n|\boldsymbol{z}_n, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{y}_n|\mathbf{W}_y\boldsymbol{z}_n^s + \mathbf{B}_y\boldsymbol{z}_n^y, \sigma_y^2\mathbf{I}) \tag{20.90}$$

where $\mathbf{W}_x$ and $\mathbf{W}_y$ are $L^s \times D$ dimensional, $\mathbf{B}_x$ is $L^x \times D$ dimensional, and $\mathbf{B}_y$ is $L^y \times D$ dimensional. See Figure 20.15 for the PGM.

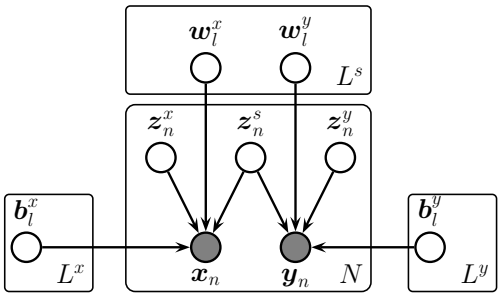


Figure 20.15: Canonical correlation analysis as a PGM.

If we marginalize out all the latent variables, we get the following distribution on the visibles (where we assume $\sigma_x = \sigma_y = \sigma$):

$$p(\boldsymbol{x}_n, \boldsymbol{y}_n) = \int d\boldsymbol{z}_n p(\boldsymbol{z}_n) p(\boldsymbol{x}_n, \boldsymbol{y}_n | \boldsymbol{z}_n) = \mathcal{N}(\boldsymbol{x}_n, \boldsymbol{y}_n | \boldsymbol{\mu}, \mathbf{W}\mathbf{W}^\mathsf{T} + \sigma^2 \mathbf{I}) \tag{20.91}$$

where $\boldsymbol{\mu} = (\boldsymbol{\mu}_x; \boldsymbol{\mu}_y)$, and $\mathbf{W} = [\mathbf{W}_x; \mathbf{W}_y]$. Thus the induced covariance is the following low rank matrix:

$$\mathbf{W}\mathbf{W}^\mathsf{T} = \begin{pmatrix} \mathbf{W}_x \mathbf{W}_x^\mathsf{T} + \mathbf{B}_x \mathbf{B}_x^\mathsf{T} & \mathbf{W}_x \mathbf{W}_y^\mathsf{T} \\ \mathbf{W}_y \mathbf{W}_x^\mathsf{T} & \mathbf{W}_y \mathbf{W}_y^\mathsf{T} + \mathbf{B}_y \mathbf{B}_y^\mathsf{T} \end{pmatrix} \tag{20.92}$$

[BJ05] showed that MLE for this model is equivalent to a classical statistical method known as **canonical correlation analysis** or **CCA** [Hot36]. However, the PGM perspective allows us to easily generalize to multiple kinds of observations (this is known as **generalized CCA** [Hor61]) or to nonlinear models (this is known as **deep CCA** [WLL16; SNM16]), or exponential family CCA [KVK10]. See [Uur+17] for further discussion of CCA and its extensions.

20.3 Autoencoders

We can think of PCA (Section 20.1) and factor analysis (Section 20.2) as learning a (linear) mapping from $\boldsymbol{x} \to \boldsymbol{z}$, called the **encoder**, f_e, and learning another (linear) mapping $\boldsymbol{z} \to \boldsymbol{x}$, called the **decoder**, f_d. The overall reconstruction function has the form $r(\boldsymbol{x}) = f_d(f_e(\boldsymbol{x}))$. The model is trained to minimize $\mathcal{L}(\boldsymbol{\theta}) = ||r(\boldsymbol{x}) - \boldsymbol{x}||_2^2$. More generally, we can use $\mathcal{L}(\boldsymbol{\theta}) = -\log p(\boldsymbol{x}|r(\boldsymbol{x}))$.

In this section, we consider the case where the encoder and decoder are nonlinear mappings implemented by neural networks. This is called an **autoencoder**. If we use an MLP with one hidden layer, we get the model shown Figure 20.16. We can think of the hidden units in the middle as a low-dimensional **bottleneck** between the input and its reconstruction.

Of course, if the hidden layer is wide enough, there is nothing to stop this model from learning the identity function. To prevent this degenerate solution, we have to restrict the model in some way. The simplest approach is to use a narrow bottleneck layer, with $L \ll D$; this is called an **undercomplete representation**. The other approach is to use $L \gg D$, known as an **overcomplete representation**, but to impose some other kind of regularization, such as adding noise to the inputs,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

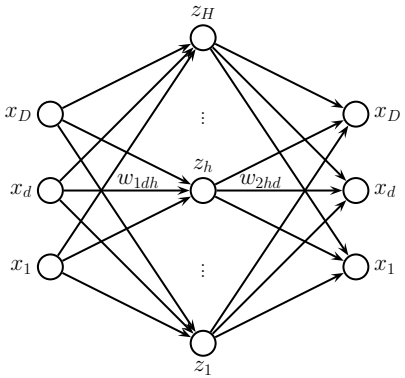


Figure 20.16: An autoencoder with one hidden layer.

forcing the activations of the hidden units to be sparse, or imposing a penalty on the derivatives of the hidden units. We discuss these options in more detail below.

20.3.1 Bottleneck autoencoders

We start by considering the special case of a **linear autoencoder**, in which there is one hidden layer, the hidden units are computed using $\boldsymbol{z} = \mathbf{W}_1\boldsymbol{x}$, and the output is reconstructed using $\hat{\boldsymbol{x}} = \mathbf{W}_2\boldsymbol{z}$, where $\mathbf{W}_1$ is a $L \times D$ matrix, $\mathbf{W}_2$ is a $D \times L$ matrix, and $L < D$. Hence $\hat{\boldsymbol{x}} = \mathbf{W}_2\mathbf{W}_1\boldsymbol{x} = \mathbf{W}\boldsymbol{x}$ is the output of the model. If we train this model to minimize the squared reconstruction error, $\mathcal{L}(\mathbf{W}) = \sum_{n=1}^{N} ||\boldsymbol{x}_n - \mathbf{W}\boldsymbol{x}_n||_2^2$, one can show [BH89; KJ95] that $\hat{\mathbf{W}}$ is an orthogonal projection onto the first L eigenvectors of the empirical covariance matrix of the data. This is therefore equivalent to PCA.

If we introduce nonlinearities into the autoencoder, we get a model that is strictly more powerful than PCA, as proved in [JHG00]. Such methods can learn very useful low dimensional representations of data.

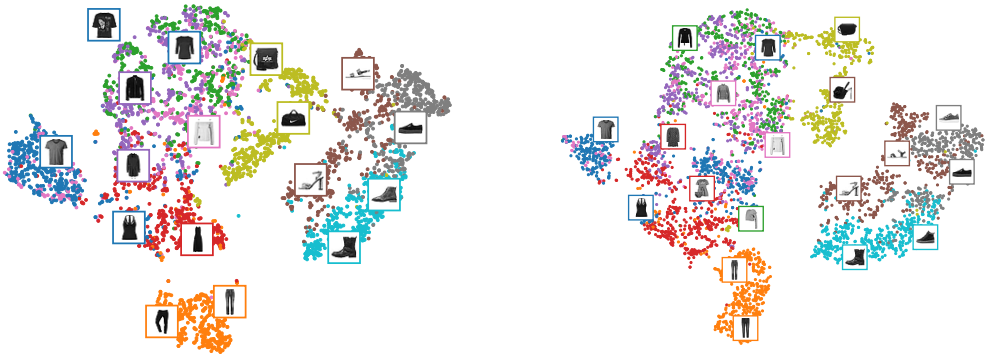
Consider fitting an autoencoder to the Fashion MNIST dataset. We consider both an MLP architecture (with 2 layers and a bottleneck of size 30), and a CNN based architecture (with 3 layers and a 3d bottleneck with 64 channels). We use a Bernoulli likelihood model and binary cross entropy as the loss. Figure 20.17 shows some test images and their reconstructions. We see that the CNN model reconstructs the images more accurately than the MLP model. However, both models are small, and were only trained for 5 epochs; results can be improved by using larger models, and training for longer.

Figure 20.18 visualizes the first 2 (of 30) latent dimensions produced by the MLP-AE. More precisely, we plot the tSNE embeddings (see Section 20.4.10), color coded by class label. We also show some corresponding images from the dataset, from which the embeddings were derived. We see that the method has done a good job of separating the classes in a fully unsupervised way. We also see that the latent space of the MLP and CNN models is very similar (at least when viewed through this 2d projection).



(a) (b)

Figure 20.17: Results of applying an autoencoder to the Fashion MNIST data. Top row are first 5 images from validation set. Bottom row are reconstructions. (a) MLP model (trained for 20 epochs). The encoder is an MLP with architecture 784-100-30. The decoder is the mirror image of this. (b) CNN model (trained for 5 epochs). The encoder is a CNN model with architecture Conv2D(16, 3×3, same, selu), MaxPool2D(2x2), Conv2D(32, 3×3, same, selu), MaxPool2D(2×2), Conv2D(64, 3×3, same, selu), MaxPool2D(2×2). The decoder is the mirror image of this, using transposed convolution and without the max pooling layers. Adapted from Figure 17.4 of [Gér19]. Generated by ae_ mnist_ tf.ipynb.



(a) (b)

Figure 20.18: tSNE plot of the first 2 latent dimensions of the Fashion MNIST validation set using an autoencoder. (a) MLP. (b) CNN. Adapted from Figure 17.5 of [Gér19]. Generated by ae_ mnist_ tf.ipynb.

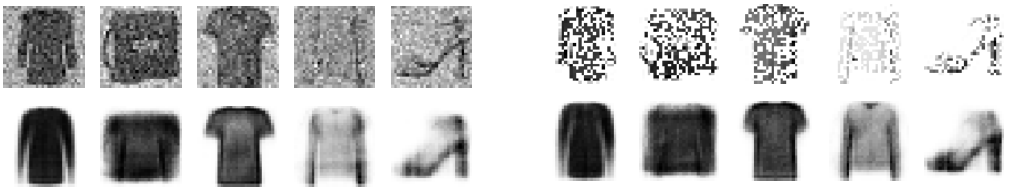
20.3.2 Denoising autoencoders

One useful way to control the capacity of an autoencoder is to add noise to its input, and then train the model to reconstruct a clean (uncorrupted) version of the original input. This is called a **denoising autoencoder** [Vin+10a].

We can implement this by adding Gaussian noise, or using Bernoulli dropout. Figure 20.19 shows some reconstructions of corrupted images computed using a DAE. We see that the model is able to "hallucinate" details that are missing in the input, since it has seen similar images before, and can store this information in the parameters of the model.

Suppose we train a DAE using Gaussian corruption and squared error reconstruction, i.e., we use $p_c(\tilde{\boldsymbol{x}}|\boldsymbol{x}) = \mathcal{N}(\tilde{\boldsymbol{x}}|\boldsymbol{x}, \sigma^2\mathbf{I})$ and $\ell(\boldsymbol{x}, r(\tilde{\boldsymbol{x}})) = ||\boldsymbol{e}||_2^2$, where $\boldsymbol{e}(\boldsymbol{x}) = r(\tilde{\boldsymbol{x}}) - \boldsymbol{x}$ is the residual error for example

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 20.19: Denoising autoencoder (MLP architecture) applied to some noisy Fashion MNIST images from the validation set. (a) Gaussian noise. (b) Bernoulli dropout noise. Top row: input. Bottom row: output. Adapted from Figure 17.9 of [Gér19]. Generated by ae_mnist_tf.ipynb.

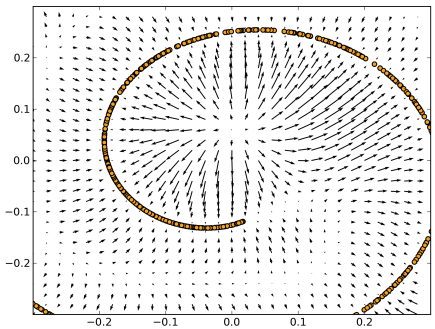


Figure 20.20: The residual error from a DAE, $\boldsymbol{e}(\boldsymbol{x}) = r(\tilde{\boldsymbol{x}}) - \boldsymbol{x}$, can learn a vector field corresponding to the score function. Arrows point towards higher probability regions. The length of the arrow is proportional to $||\boldsymbol{e}(\boldsymbol{x})||$, so points near the 1d data manifold (represented by the curved line) have smaller arrows. From Figure 5 of [AB14]. Used with kind permission of Guillaume Alain.

$\boldsymbol{x}$. Then one can show [AB14] the remarkable result that, as $\sigma \to 0$ (and with a sufficiently powerful model and enough data), the residuals approximate the **score function**, which is the log probability of the data, i.e., $\boldsymbol{e}(\boldsymbol{x}) \approx \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$. That is, the DAE learns a **vector field**, corresponding to the gradient of the log data density. Thus points that are close to the data manifold will be projected onto it via the sampling process. See Figure 20.20 for an illustration.

20.3.3 Contractive autoencoders

A different way to regularize autoencoders is by adding the penalty term

$$\Omega(\boldsymbol{z}, \boldsymbol{x}) = \lambda ||\frac{\partial f_e(\boldsymbol{x})}{\partial \boldsymbol{x}}||_F^2 = \lambda \sum_k ||\nabla_{\boldsymbol{x}} \, h_k(\boldsymbol{x})||_2^2 \tag{20.93}$$

to the reconstruction loss, where h_k is the value of the k'th hidden embedding unit. That is, we penalize the Frobenius norm of the encoder's Jacobian. This is called a **contractive autoencoder**

[Rif+11]. (A linear operator with Jacobian $\mathbf{J}$ is called a **contraction** if $||\mathbf{J}\boldsymbol{x}|| \leq 1$ for all unit-norm inputs $\boldsymbol{x}$.)

To understand why this is useful, consider Figure 20.20. We can approximate the curved low-dimensional manifold by a series of locally linear manifolds. These linear approximations can be computed using the Jacobian of the encoder at each point. By encouraging these to be contractive, we ensure the model "pushes" inputs that are off the manifold to move back towards it.

Another way to think about CAEs is as follows. To minimize the penalty term, the model would like to ensure the encoder is a constant function. However, if it was completely constant, it would ignore its input, and hence incur high reconstruction cost. Thus the two terms together encourage the model to learn a representation where only a few units change in response to the most significant variations in the input.

One possible degenerate solution is that the encoder simply learns to multiply the input by a small constant ϵ (which scales down the Jacobian), followed by a decoder that divides by ϵ (which reconstructs perfectly). To avoid this, we can tie the weights of the encoder and decoder, by setting the weight matrix for layer ℓ of f_d to be the transpose of the weight matrix for layer ℓ of f_e, but using untied bias terms. Unfortunately CAEs are slow to train, because of the expense of computing the Jacobian.

20.3.4 Sparse autoencoders

Yet another way to regularize autoencoders is to add a sparsity penalty to the latent activations of the form $\Omega(\boldsymbol{z}) = \lambda||\boldsymbol{z}||_1$. (This is called **activity regularization**.)

An alternative way to implement sparsity, that often gives better results, is to use logistic units, and then to compute the expected fraction of time each unit k is on within a minibatch (call this q_k), and ensure that this is close to a desired target value p, as proposed in [GBB11]. In particular, we use the regularizer $\Omega(\boldsymbol{z}_{1:L,1:N}) = \lambda \sum_k D_{\mathbb{KL}}\left(\boldsymbol{p} \parallel \boldsymbol{q}_k\right)$ for latent dimensions $1:L$ and examples $1:N$, where $\boldsymbol{p} = (p, 1-p)$ is the desired target distribution, and $\boldsymbol{q}_k = (q_k, 1-q_k)$ is the empirical distribution for unit k, computed using $q_k = \frac{1}{N}\sum_{n=1}^{N} \mathbb{I}\left(z_{n,k} = 1\right)$.

Figure 20.21 shows the results when fitting an AE-MLP (with 300 hidden units) to Fashion MNIST. If we set $\lambda = 0$ (i.e., if we don't impose a sparsity penalty), we see that the average activation value is about 0.4, with most neurons being partially activated most of the time. With the ℓ_1 penalty, we see that most units are off all the time, which means they are not being used at all. With the KL penalty, we see that about 70% of neurons are off on average, but unlike the ℓ_1 case, we don't see units being permanently turned off (the average activation level is 0.1). This latter kind of sparse firing pattern is similar to that observed in biological brains (see e.g., [Bey+19]).

20.3.5 Variational autoencoders

In this section, we discuss the **variational autoencoder** or **VAE** [KW14; RMW14; KW19a], which can be thought of as a probabilistic version of a deterministic autoencoder (Section 20.3) The principal advantage is that a VAE is a generative model that can create new samples, whereas an autoencoder just computes embeddings of input vectors.

We discuss VAEs in detail in the sequel to this book, [Mur23]. However, in brief, the VAE combines two key ideas. First we create a non-linear extension of the factor analysis generative model, i.e., we

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

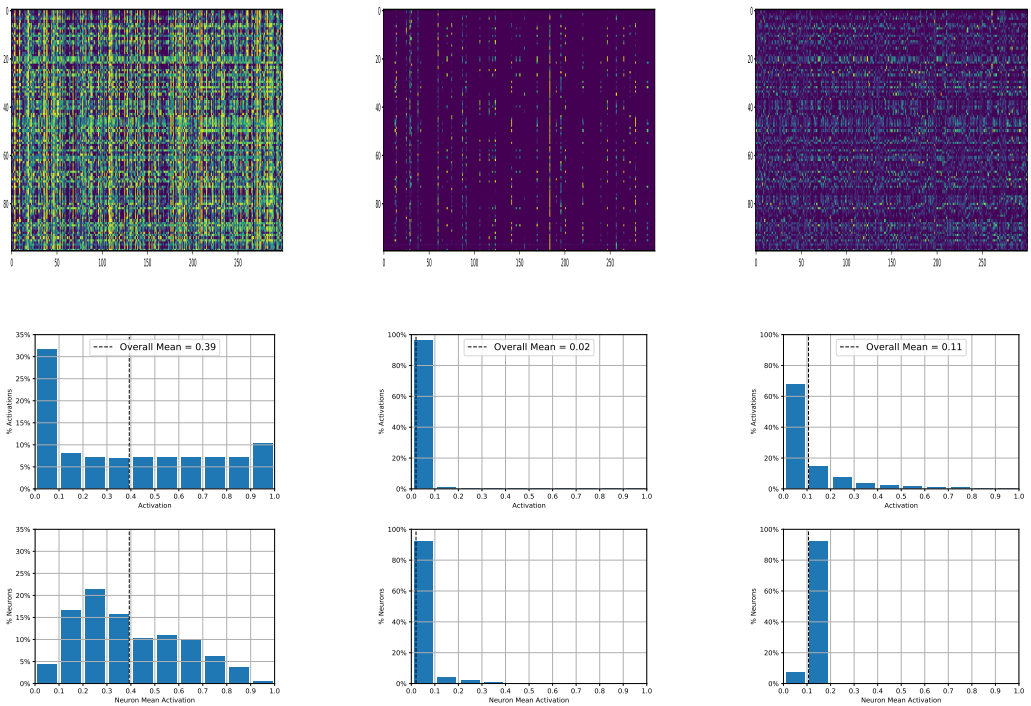


Figure 20.21: Neuron activity (in the bottleneck layer) for an autoencoder applied to Fashion MNIST. We show results for three models, with different kinds of sparsity penalty: no penalty (left column), ℓ_1 penalty (middle column), KL penalty (right column). Top row: Heatmap of 300 neuron activations (columns) across 100 examples (rows). Middle row: Histogram of activation levels derived from this heatmap. Bottom row: Histogram of the mean activation per neuron, averaged over all examples in the validation set. Adapted from Figure 17.11 of [Gér19]. Generated by ae_mnist_tf.ipynb.

replace $p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\mathbf{W}\boldsymbol{z}, \sigma^2\mathbf{I})$ with

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|f_d(\boldsymbol{z};\boldsymbol{\theta}), \sigma^2\mathbf{I}) \tag{20.94}$$

where f_d is the decoder. For binary observations we should use a Bernoulli likelihood:

$$p(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{\theta}) = \prod_{i=1}^{D} \text{Ber}(x_i|f_d(\boldsymbol{z};\boldsymbol{\theta}), \sigma^2\mathbf{I}) \tag{20.95}$$

Second, we create another model, $q(\boldsymbol{z}|\boldsymbol{x})$, called the **recognition network** or **inference network**, that is trained simultaneously with the generative model to do approximate posterior inference. If we assume the posterior is Gaussian, with diagonal covariance, we get

$$q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}|f_{e,\mu}(\boldsymbol{x};\boldsymbol{\phi}), \text{diag}(f_{e,\sigma}(\boldsymbol{x};\boldsymbol{\phi}))) \tag{20.96}$$

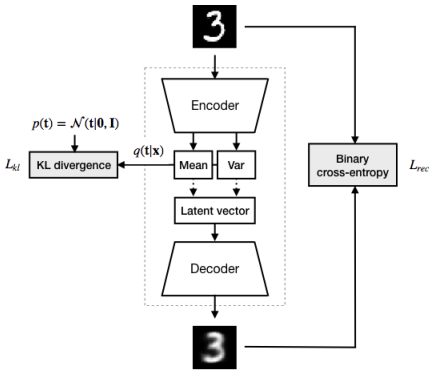


Figure 20.22: Schematic illustration of a VAE. From a figure from http://krasserm.github.io/2018/07/27/dfc-vae/. Used with kind permission of Martin Krasser.

where f_e is the encoder. See Figure 20.22 for a sketch.

The idea of training an inference network to "invert" a generative network, rather than running an optimization algorithm to infer the latent code, is called **amortized inference**. This idea was first proposed in the **Helmholtz machine** [Day+95]. However, that paper did not present a single unified objective function for inference and generation, but instead used the wake sleep method for training, which alternates between optimizing the generative model and inference model. By contrast, the VAE optimizes a variational lower bound on the log-likelihood, which is more principled, since it is a single unified objective.

20.3.5.1 Training VAEs

We cannot compute the exact marginal likelihood $p(\boldsymbol{x}|\boldsymbol{\theta})$ needed for MLE training, because posterior inference in a nonlinear FA model is intractable. However, we can use the inference network to compute an approximate posterior, $q(\boldsymbol{z}|\boldsymbol{x})$. We can then use this to compute the **evidence lower bound** or **ELBO**. For a single example $\boldsymbol{x}$, this is given by

$$\text{Ł}(\boldsymbol{\theta}, \boldsymbol{\phi}|\boldsymbol{x}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\right] \tag{20.97}$$

$$= \mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x},\boldsymbol{\phi})}\left[\log p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\theta})\right] - D_{\mathbb{KL}}\left(q(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\phi}) \parallel p(\boldsymbol{z})\right) \tag{20.98}$$

This can be interpreted as the expected log likelihood, plus a regularizer, that penalizes the posterior from deviating too much from the prior. (This is different than the approach in Section 20.3.4, where we applied the KL penalty to the aggregate posterior in each minibatch.)

The ELBO is a lower bound of the log marginal likelihood (aka evidence), as can be seen from Jensen's inequality:

$$\text{Ł}(\boldsymbol{\theta}, \boldsymbol{\phi}|\boldsymbol{x}) = \int q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})}{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} d\boldsymbol{z} \tag{20.99}$$

$$\leq \log \int q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})}{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} d\boldsymbol{z} = \log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \tag{20.100}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

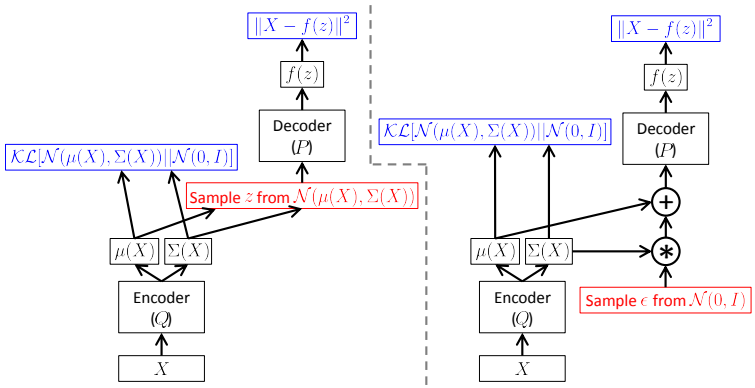


Figure 20.23: Computation graph for VAEs. where $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\mathbf{0}, \mathbf{I})$, $p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}|f(\boldsymbol{z}), \sigma^2\mathbf{I})$, and $q(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\phi}) = \mathcal{N}(\boldsymbol{z}|\mu(\boldsymbol{x}), \Sigma(\boldsymbol{x}))$. Red boxes show sampling operations which are not differentiable. Blue boxes show loss layers (we assume Gaussian likelihoods and priors). (Left) Without the reparameterization trick. (Right) With the reparameterization trick. Gradients can flow from the output loss, back through the decoder and into the encoder. From Figure 4 of [Doe16]. Used with kind permission of Carl Doersch.

Thus for fixed inference network parameters $\boldsymbol{\phi}$, increasing the ELBO should increase the log likelihood of the data, similar to EM Section 8.7.2.

20.3.5.2 The reparameterization trick

In this section, we discuss how to compute the ELBO and its gradient. For simplicity, let us suppose that the inference network estimates the parameters of a Gaussian posterior. Since $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ is Gaussian, we can write

$$\boldsymbol{z} = f_{e,\mu}(\boldsymbol{x}; \boldsymbol{\phi}) + f_{e,\sigma}(\boldsymbol{x}; \boldsymbol{\phi}) \odot \boldsymbol{\epsilon} \tag{20.101}$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Hence

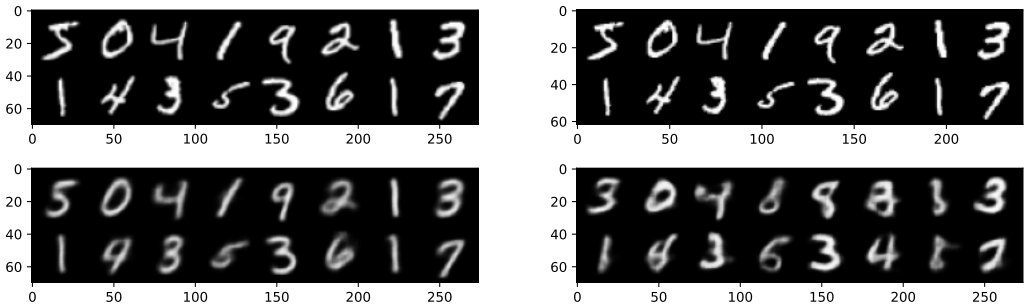
$$Ł(\boldsymbol{\theta}, \boldsymbol{\phi}|\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z} = \mu_{\boldsymbol{\phi}}(\boldsymbol{x}) + \sigma_{\boldsymbol{\phi}}(\boldsymbol{x}) \odot \boldsymbol{\epsilon})\right] - D_{\mathbb{KL}}\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p(\boldsymbol{z})\right) \tag{20.102}$$

Now the expectation is independent of the parameters of the model, so we can safely push gradients inside and use backpropagation for training in the usual way, by minimizing $-\mathbb{E}_{\boldsymbol{x} \sim \mathcal{D}}\left[Ł(\boldsymbol{\theta}, \boldsymbol{\phi}|\boldsymbol{x})\right]$ wrt $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$. This is known as the **reparameterization trick**. See Figure 20.23 for an illustration.

The first term in the ELBO can be approximated by sampling $\boldsymbol{\epsilon}$, scaling it by the output of the inference network to get $\boldsymbol{z}$, and then evaluating $\log p(\boldsymbol{x}|\boldsymbol{z})$ using the decoder network.

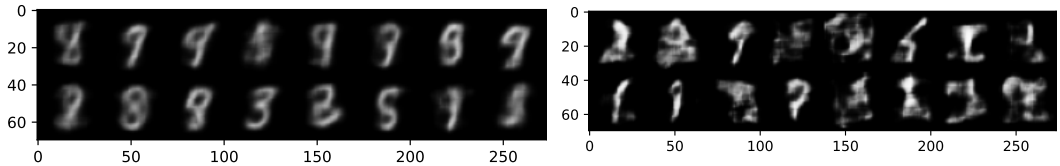
The second term in the ELBO is the KL of two Gaussians, which has a closed form solution. In particular, inserting $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\mathbf{0}, \mathbf{I})$ and $q(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}))$ into Equation (6.33), we get

$$D_{\mathbb{KL}}\left(q \parallel p\right) = \sum_{k=1}^{K} \left[\log(\frac{1}{\sigma_k}) + \frac{\sigma_k^2 + (\mu_k - 0)^2}{2 \cdot 1} - \frac{1}{2}\right] = -\frac{1}{2} \sum_{k=1}^{K} \left[\log \sigma_k^2 - \sigma_k^2 - \mu_k^2 + 1\right] \tag{20.103}$$



(a) (b)

Figure 20.24: Reconstructing MNIST digits using a 20 dimensional latent space. Top row: input images. Bottom row: reconstructions. (a) VAE. Generated by vae_mnist_conv_lightning.ipynb. (b) Deterministic AE. Generated by ae_mnist_conv.ipynb.



(a) (b)

Figure 20.25: Sampling MNIST digits using a 20 dimensional latent space. (a) VAE. Generated by vae_mnist_conv_lightning.ipynb. (b) Deterministic AE. Generated by ae_mnist_conv.ipynb.

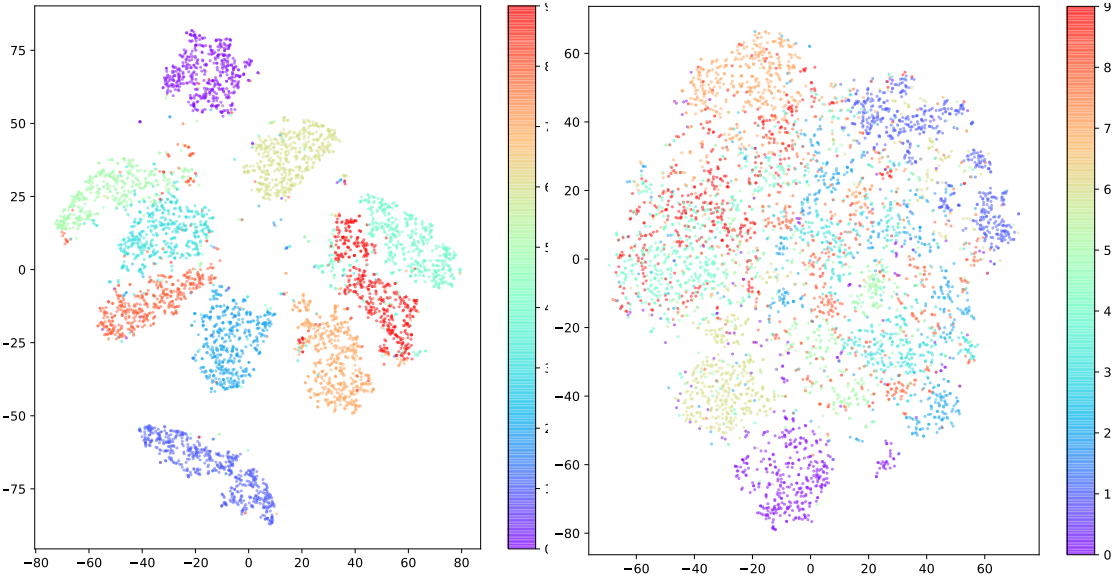
20.3.5.3 Comparison of VAEs and autoencoders

VAEs are very similar to autoencoders. In particular, the generative model, $p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$, acts like the decoder, and the inference network, $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, acts like the encoder. The reconstruction abilities of both models are similar, as can be seen by comparing Figure 20.24a with Figure 20.24b.

The primary advantage of the VAE is that it can be used to generate new data from random noise. In particular, we sample $\boldsymbol{z}$ from the Gaussian prior $\mathcal{N}(\boldsymbol{z}|\mathbf{0}, \mathbf{I})$, and then pass this through the decoder to get $\mathbb{E}[\boldsymbol{x}|\boldsymbol{z}] = f_d(\boldsymbol{z}; \boldsymbol{\theta})$. The VAE's decoder is trained to convert random points in the embedding space (generated by perturbing the input encodings) to sensible outputs. By contrast, the decoder for the deterministic autoencoder only ever gets as inputs the exact encodings of the training set, so it does not know what to do with random inputs that are outside what it was trained on. So a standard autoencoder cannot create new samples. This difference can be seen by comparing Figure 20.25a with Figure 20.25b.

The reason the VAE is better at sample is that it embeds images into Gaussians in latent space, whereas the AE embeds images into points, which are like delta functions. The advantage of using a

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 20.26: tSNE projection of a 20 dimensional latent space. (a) VAE. Generated by vae_mnist_conv_lightning.ipynb. (b) Deterministic AE. Generated by ae_mnist_conv.ipynb.



(a) (b)

Figure 20.27: Linear interpolation between the left and right images in a 20 dimensional latent space. (a) VAE. (b) Deterministic AE. Generated by vae_mnist_conv_lightning.ipynb.

latent *distribution* is that it encourages local smoothness, since a given image may map to multiple nearby places, depending on the stochastic sampling. By contrast, in an AE, the latent space is typically not smooth, so images from different classes often end up next to each other. This difference can be seen by comparing Figure 20.26a with Figure 20.26b.

We can leverage the smoothness of the latent space to perform **image interpolation**. Rather than working in pixel space, we can work in the latent space of the model. Specifically, let $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ be two images, and let $\boldsymbol{z}_1 = \mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x}_1)}[\boldsymbol{z}]$ and $\boldsymbol{z}_2 = \mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x}_2)}[\boldsymbol{z}]$ be their encodings. We can now generate new images that interpolate between these two anchors by computing $\boldsymbol{z} = \lambda \boldsymbol{z}_1 + (1-\lambda)\boldsymbol{z}_2$, where $0 \leq \lambda \leq 1$, and then decoding by computing $\mathbb{E}[\boldsymbol{x}|\boldsymbol{z}]$. This is called **latent space interpolation**. (The justification for taking a linear interpolation is that the learned manifold has approximately zero curvature, as shown in [SKTF18].) A VAE is more useful for latent space interpolation than an AE because its latent space is smoother, and because the model can generate from almost any point in latent space. This difference can be seen by comparing Figure 20.27a with Figure 20.27b.

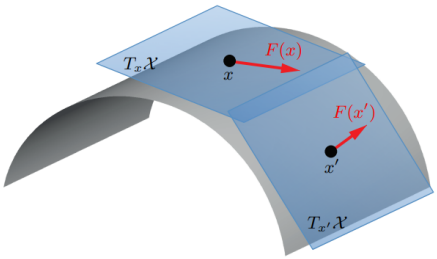


Figure 20.28: Illustration of the tangent space and tangent vectors at two different points on a 2d curved manifold. From Figure 1 of [Bro+17a]. Used with kind permission of Michael Bronstein.

20.4 Manifold learning *

In this section, we discuss the problem of recovering the underlying low-dimensional structure in a high-dimensional dataset. This structure is often assumed to be a curved manifold (explained in Section 20.4.1), so this problem is called **manifold learning** or **nonlinear dimensionality reduction**. The key difference from methods such as autoencoders (Section 20.3) is that we will focus on non-parametric methods, in which we compute an embedding for each point in the training set, as opposed to learning a generic model that can embed any input vector. That is, the methods we discuss do not (easily) support **out-of-sample generalization**. However, they can be easier to fit, and are quite flexible. Such methods can be a useful for unsupervised learning (knowledge discovery), data visualization, and as a preprocessing step for supervised learning. See [AAB21] for a recent review of this field.

20.4.1 What are manifolds?

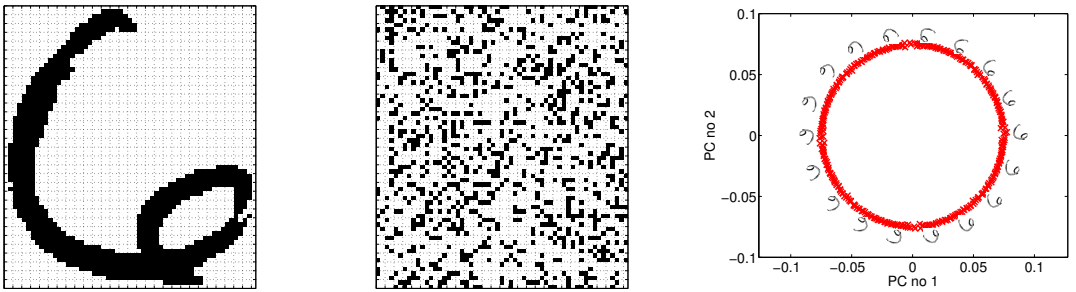
Roughly speaking, a **manifold** is a topological space which is locally Euclidean. One of the simplest examples is the surface of the earth, which is a curved 2d surface embedded in a 3d space. At each local point on the surface, the earth seems flat.

More formally, a d-dimensional manifold $\mathcal{X}$ is a space in which each point $x \in \mathcal{X}$ has a neighborhood which is topologically equivalent to a d-dimensional Euclidean space, called the **tangent space**, denoted $\mathcal{T}_x = T_x\mathcal{X}$. This is illustrated in Figure 20.28.

A **Riemannian manifold** is a differentiable manifold that associates an inner product operator at each point x in tangent space; this is assumed to depend smoothly on the position x. The inner product induces a notion of distance, angles, and volume. The collection of these inner products is called a **Riemannian metric**. It can be shown that any sufficiently smooth Riemannian manifold can be embedded into a Euclidean space of potentially higher dimension; the Riemannian inner product at a point then becomes Euclidean inner product in that tangent space.

20.4.2 The manifold hypothesis

Most "naturally occuring" high dimensional dataset lie a low dimensional manifold. This is called the **manifold hypothesis** [FMN16]. For example, consider the case of an image. Figure 20.29a shows a



(a) (b) (c)

Figure 20.29: Illustration of the image manifold. (a) An image of the digit 6 from the USPS dataset, of size $64 \times 57 = 3,648$. *(b) A random sample from the space* $\{0,1\}^{3648}$ *reshaped as an image. (c) A dataset created by rotating the original image by one degree 360 times. We project this data onto its first two principal components, to reveal the underlying 2d circular manifold. From Figure 1 of [Law12]. Used with kind permission of Neil Lawrence.*

single image of size 64×57. This is a vector in a 3,648-dimensional space, where each dimension corresponds to a pixel intensity. Suppose we try to generate an image by drawing a random point in this space; it is unlikely to look like the image of a digit, as shown in Figure 20.29b. However, the pixels are not independent of each other, since they are generated by some lower dimensional structure, namely the shape of the digit 6.

As we vary the shape, we will generate different images. We can often characterize the space of shape variations using a low-dimensional manifold. This is illustrated in Figure 20.29c, where we apply PCA (Section 20.1) to project a dataset of 360 images, each one a slightly rotated version of the digit 6, into a 2d space. We see that most of the variation in the data is captured by an underlying curved 2d manifold. We say that the **intrinsic dimensionality** d of the data is 2, even though the **ambient dimensionality** D is 3,648.

20.4.3 Approaches to manifold learning

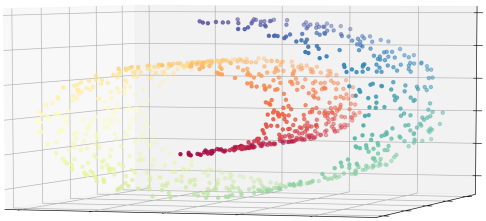
In the rest of this section, we discuss ways to learn manifolds from data. There are many different algorithms that have been proposed, which make different assumptions about the nature of the manifold, and which have different computational properties. We discuss a few of these methods in the following sections. For more details, see e.g., [Bur10].

The methods can be categorized as shown in Table 20.1. The term "nonparametric" refers to methods that learn a low dimensional embedding $\boldsymbol{z}_i$ for each datapoint $\boldsymbol{x}_i$, but do not learn a mapping function which can be applied to an out-of-sample datapoint. (However, [Ben+04b] discusses how to extend many of these methods beyond the training set by learning a kernel.)

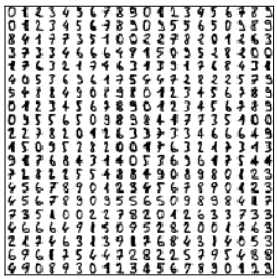
In the sections below, we compare some of these methods using 2 different datasets: a set of 1000 3d-points sampled from the 2d "**Swiss roll**" manifold, and a set of 1797 64-dimensional points sampled from the UCI digits dataset. See Figure 20.30 for an illustration of the data. We will learn a 2d manifold, so we can visualize the data.

Method	Parametric	Convex	Section
PCA / classical MDS	N	Y (Dense)	Section 20.1
Kernel PCA	N	Y (Dense)	Section 20.4.6
Isomap	N	Y (Dense)	Section 20.4.5
LLE	N	Y (Sparse)	Section 20.4.8
Laplacian Eigenmaps	N	Y (Sparse)	Section 20.4.9
tSNE	N	N	Section 20.4.10
Autoencoder	Y	N	Section 20.3

Table 20.1: A list of some approaches to dimensionality reduction. If a method is convex, we specify in parentheses whether it requires solving a sparse or dense eigenvalue problem.



(a)



(b)

Figure 20.30: Illustration of some data generated from low-dimensional manifolds. (a) The 2d Swiss-roll manifold embedded into 3d. Generated by manifold_swiss_sklearn.ipynb. (b) Sample of some UCI digits, which have size $8 \times 8 = 64$. Generated by manifold_digits_sklearn.ipynb.

20.4.4 Multi-dimensional scaling (MDS)

The simplest approach to manifold learning is **multidimensional scaling** (**MDS**). This tries to find a set of low dimensional vectors $\{\boldsymbol{z}_i \in \mathbb{R}^L : i = 1 : N\}$ such that the pairwise distances between these vectors is as similar as possible to a set of pairwise dissimilarities $\mathbf{D} = \{d_{ij}\}$ provided by the user. There are several variants of MDS, one of which turns out to be equivalent to PCA, as we discuss below.

20.4.4.1 Classical MDS

Suppose we start an $N \times D$ data matrix $\mathbf{X}$ with rows $\boldsymbol{x}_i$. Let us define the centered Gram (similarity) matrix as follows:

$$\tilde{K}_{ij} = \langle \boldsymbol{x}_i - \overline{\boldsymbol{x}}, \boldsymbol{x}_j - \overline{\boldsymbol{x}} \rangle \tag{20.104}$$

In matrix notation, we have $\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\mathsf{T}$, where $\tilde{\mathbf{X}} = \mathbf{C}_N\mathbf{X}$ and $\mathbf{C}_N = \mathbf{I}_N - \frac{1}{N}\mathbf{1}_N\mathbf{1}_N^\mathsf{T}$ is the centering matrix.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Now define the **strain** of a set of embeddings as follows:

$$\mathcal{L}_{\text{strain}}(\mathbf{Z}) = \sum_{i,j}(\tilde{K}_{ij} - \langle \tilde{\boldsymbol{z}}_i, \tilde{\boldsymbol{z}}_j \rangle)^2 = ||\tilde{\mathbf{K}} - \tilde{\mathbf{Z}}\tilde{\mathbf{Z}}^\mathsf{T}||_F^2 \tag{20.105}$$

where $\tilde{\boldsymbol{z}}_i = \boldsymbol{z}_i - \overline{\boldsymbol{z}}$ is the centered embedding vector. Intuitively this measures how well similarities in the high-dimensional data space, $\tilde{K}_{ij}$, are matched by similarities in the low-dimensional embedding space, $\langle \tilde{\boldsymbol{z}}_i, \tilde{\boldsymbol{z}}_j \rangle$. Minimizing this loss is called **classical MDS**.

We know from Section 7.5 that the best rank L approximation to a matrix is its truncated SVD representation, $\tilde{\mathbf{K}} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$. Since $\tilde{\mathbf{K}}$ is positive semi definite, we have that $\mathbf{V} = \mathbf{U}$. Hence the optimal embedding satisfies

$$\tilde{\mathbf{Z}}\tilde{\mathbf{Z}}^\mathsf{T} = \mathbf{U}\mathbf{S}\mathbf{U}^\mathsf{T} = (\mathbf{U}\mathbf{S}^{\frac{1}{2}})(\mathbf{S}^{\frac{1}{2}}\mathbf{U}^\mathsf{T}) \tag{20.106}$$

Thus we can set the embedding vectors to be the rows of $\tilde{\mathbf{Z}} = \mathbf{U}\mathbf{S}^{\frac{1}{2}}$.

Now we describe how to apply classical MDS to a dataset where we just have Euclidean distances, rather than raw features. First we compute a matrix of squared Euclidean distances, $\mathbf{D}^{(2)} = \mathbf{D} \odot \mathbf{D}$, which has the following entries:

$$D_{ij}^{(2)} = ||\boldsymbol{x}_i - \boldsymbol{x}_j||^2 = ||\boldsymbol{x}_i - \overline{\boldsymbol{x}}||^2 + ||\boldsymbol{x}_j - \overline{\boldsymbol{x}}||^2 - 2\langle \boldsymbol{x}_i - \overline{\boldsymbol{x}}, \boldsymbol{x}_j - \overline{\boldsymbol{x}} \rangle \tag{20.107}$$

$$= ||\boldsymbol{x}_i - \overline{\boldsymbol{x}}||^2 + ||\boldsymbol{x}_j - \overline{\boldsymbol{x}}||^2 - 2\tilde{K}_{ij} \tag{20.108}$$

We see that $\mathbf{D}^{(2)}$ only differs from $\tilde{\mathbf{K}}$ by some row and column constants (and a factor of -2). Hence we can compute $\tilde{\mathbf{K}}$ by double centering $\mathbf{D}^{(2)}$ using Equation (7.89) to get $\tilde{\mathbf{K}} = -\frac{1}{2}\mathbf{C}_N\mathbf{D}^{(2)}\mathbf{C}_N$. In other words,

$$\tilde{K}_{ij} = -\frac{1}{2}\left(d_{ij}^2 - \frac{1}{N}\sum_{l=1}^N d_{il}^2 - \frac{1}{N}\sum_{l=1}^N d_{jl}^2 + \frac{1}{N^2}\sum_{l=1}^N\sum_{m=1}^N d_{lm}^2\right) \tag{20.109}$$

We can then compute the embeddings as before.

It turns out that classical MDS is equivalent to PCA (Section 20.1). To see this, let $\tilde{\mathbf{K}} = \mathbf{U}_L\mathbf{S}_L\mathbf{U}_L^\mathsf{T}$ be the rank L truncated SVD of the centered kernel matrix. The MDS embedding is given by $\mathbf{Z}_{\text{MDS}} = \mathbf{U}_L\mathbf{S}_L^{\frac{1}{2}}$. Now consider the rank L SVD of the centered data matrix, $\tilde{\mathbf{X}} = \mathbf{U}_X\mathbf{S}_X\mathbf{V}_X^\mathsf{T}$. The PCA embedding is $\mathbf{Z}_{\text{PCA}} = \mathbf{U}_X\mathbf{S}_X$. Now

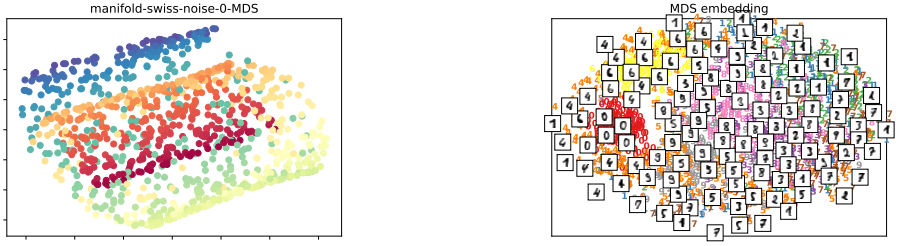
$$\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\mathsf{T} = \mathbf{U}_X\mathbf{S}_X\mathbf{V}_X^\mathsf{T}\mathbf{V}_X\mathbf{S}_X\mathbf{U}_X^\mathsf{T} = \mathbf{U}_X\mathbf{S}_X^2\mathbf{U}_X^\mathsf{T} = \mathbf{U}_L\mathbf{S}_L\mathbf{U}_L^\mathsf{T} \tag{20.110}$$

Hence $\mathbf{U}_X = \mathbf{U}_L$ and $\mathbf{S}_X = \mathbf{S}_L^2$, and so $\mathbf{Z}_{\text{PCA}} = \mathbf{Z}_{\text{MDS}}$.

20.4.4.2 Metric MDS

Classical MDS assumes Euclidean distances. We can generalize it to allow for any dissimilarity measure by defining the **stress function**

$$\mathcal{L}_{\text{stress}}(\mathbf{Z}) = \sqrt{\frac{\sum_{i<j}(d_{ij} - \hat{d}_{ij})^2}{\sum_{ij} d_{ij}^2}} \tag{20.111}$$



(a) (b)

Figure 20.31: Metric MDS applied to (a) Swiss roll. Generated by manifold_swiss_sklearn.ipynb. (b) UCI digits. Generated by manifold_digits_sklearn.ipynb.

where $\hat{d}_{ij} = ||\boldsymbol{z}_i - \boldsymbol{z}_j||$. This is called **metric MDS**. Note that this is a different objective than the one used by classical MDS, so even if d_{ij} are Euclidean distances, the results will be different.

We can use gradient descent to solve the optimization problem. However, it is better to use a bound optimization algorithm (Section 8.7) called **SMACOF** [Lee77], which stands for "Scaling by MAjorizing a COmplication Function". (This is the method implemented in scikit-learn.) See Figure 20.31 for the results of applying this to our running example.

20.4.4.3 Non-metric MDS

Instead of trying to match the distance between points, we can instead just try to match the ranking of how similar points are. To do this, let $f(d)$ be a monotonic transformation from distances to ranks. Now define the loss

$$\mathcal{L}_{\text{NM}}(\mathbf{Z}) = \sqrt{\frac{\sum_{i<j}(f(d_{ij}) - \hat{d}_{ij})^2}{\sum_{ij} \hat{d}_{ij}^2}} \tag{20.112}$$

where $\hat{d}_{ij} = ||\boldsymbol{z}_i - \boldsymbol{z}_j||$. (Note the $\hat{d}_{ij}^2$ in the denominator, not d_{ij}^2.) Minimizing this is known as **non-metric MDS**.

This objective can be optimized iteratively. First the function f is optimized, for a given $\mathbf{Z}$, using isotonic regression; this finds the optimal monotonic transformation of the input distances to match the current embedding distances. Then the embeddings $\mathbf{Z}$ are optimized, for a given f, using gradient descent, and the process repeats.

20.4.4.4 Sammon mapping

Metric MDS tries to minimize the sum of squared distances, so it puts the most emphasis on large distances. However, for many embedding methods, small distances matter more, since they capture local structure. One way to capture this is to divide each term of the loss by d_{ij}, so small distances

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 20.32: (a) If we measure distances along the manifold, we find $d(1,6) > d(1,4)$, whereas if we measure in ambient space, we find $d(1,6) < d(1,4)$. The plot at the bottom shows the underlying 1d manifold. (b) The K-nearest neighbors graph for some datapoints; the red path is the shortest distance between A and B on this graph. From [Hin13]. Used with kind permission of Geoff Hinton.

get upweighted:

$$\mathcal{L}_{\text{sammon}}(\mathbf{Z}) = \left(\frac{1}{\sum_{i<j} d_{ij}}\right) \sum_{i \neq j} \frac{(\hat{d}_{ij} - d_{ij})^2}{d_{ij}} \tag{20.113}$$

(Note the d_{ij} in the denominator, not d_{ij}^2.) Minimizing this results in a **Sammon mapping**. (The coefficient in front of the sum is just to simplify the gradient of the loss.) Unfortunately this is a non-convex objective, and it arguably puts too much emphasis on getting very small distances exactly right. We will discuss better methods for capturing local structure later on.

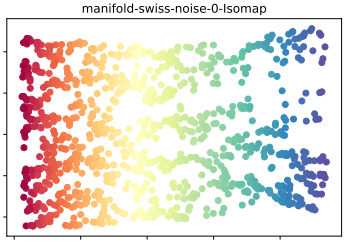
20.4.5 Isomap

If the high-dimensional data lies on or near a curved manifold, such as the Swiss roll example, then MDS might consider two points to be close even if their distance along the manifold is large. This is illustrated in Figure 20.32a.

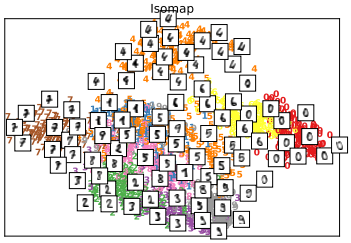
One way to capture this is to create the K-nearest neighbor graph between datapoints[5], and then approximate the manifold distance between a pair of points by the shortest distance along this graph; this can be computed efficiently using Dijkstra's shortest path algorithm. See Figure 20.32b for an illustration. Once we have computed this new distance metric, we can apply classical MDS (i.e., PCA). This is a way to capture local structure while avoiding local optima. The overall method is called **isomap** [TSL00].

See Figure 20.33 for the results of this method on our running example. We see that they are quite reasonable. However, if the data is noisy, there can be "false" edges in the nearest neighbor graph, which can result in "short circuits" which significantly distort the embedding, as shown in Figure 20.34. This problem is known as "**topological instability**" [BS02]. Choosing a very small neighborhood does not solve this problem, since this can fragment the manifold into a large number of disconnected regions. Various other solutions have been proposed, e.g., [CC07].

5. In scikit-learn, you can use the function `sklearn.neighbors.kneighbors_graph`.

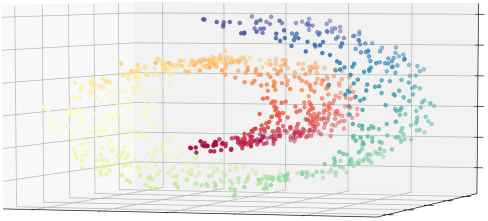


(a)

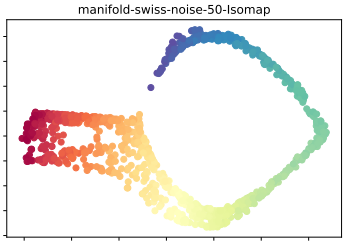


(b)

Figure 20.33: Isomap applied to (a) Swiss roll. Generated by manifold_swiss_sklearn.ipynb. (b) UCI digits. Generated by manifold_digits_sklearn.ipynb.



(a)



(b)

Figure 20.34: (a) Noisy version of Swiss roll data. We perturb each point by adding $\mathcal{N}(0, 0.5^2)$ noise. (b) Results of Isomap applied to this data. Generated by manifold_swiss_sklearn.ipynb.

20.4.6 Kernel PCA

PCA (and classical MDS) finds the best linear projection of the data, so as to preserve pairwise similarities between all the points. In this section, we consider nonlinear projections. The key idea is to solve PCA by finding the eigenvectors of the inner product (Gram) matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\mathsf{T}$, as in Section 20.1.3.2, and then to use the kernel trick (Section 17.3.4), which lets us replace inner products such as $\boldsymbol{x}_i^\mathsf{T}\boldsymbol{x}_j$ with a kernel function, $K_{ij} = \mathcal{K}(\boldsymbol{x}_i, \boldsymbol{x}_k)$. This is known as **kernel PCA** [SSM98].

Recall from Mercer's theorem that the use of a kernel implies some underlying feature space, so we are implicitly replacing $\boldsymbol{x}_i$ with $\boldsymbol{\phi}(\boldsymbol{x}_i) = \boldsymbol{\phi}_i$. Let $\boldsymbol{\Phi}$ be the corresponding (notional) design matrix, and $\mathbf{K} = \mathbf{X}\mathbf{X}^\mathsf{T}$ be the Gram matrix. Finally, let $\mathbf{S}_\phi = \frac{1}{N}\sum_i \boldsymbol{\phi}_i\boldsymbol{\phi}_i^\mathsf{T}$ be the covariance matrix in feature space. (We are assuming for now the features are centered.) From Equation (20.22), the normalized eigenvectors of $\mathbf{S}$ are given by $\mathbf{V}_{\text{kPCA}} = \boldsymbol{\Phi}^\mathsf{T}\mathbf{U}\boldsymbol{\Lambda}^{-\frac{1}{2}}$, where $\mathbf{U}$ and $\boldsymbol{\Lambda}$ contain the eigenvectors and eigenvalues of $\mathbf{K}$. Of course, we can't actually compute $\mathbf{V}_{\text{kPCA}}$, since $\boldsymbol{\phi}_i$ is potentially infinite dimensional. However, we can compute the projection of a test vector $\boldsymbol{x}_*$ onto the feature space as

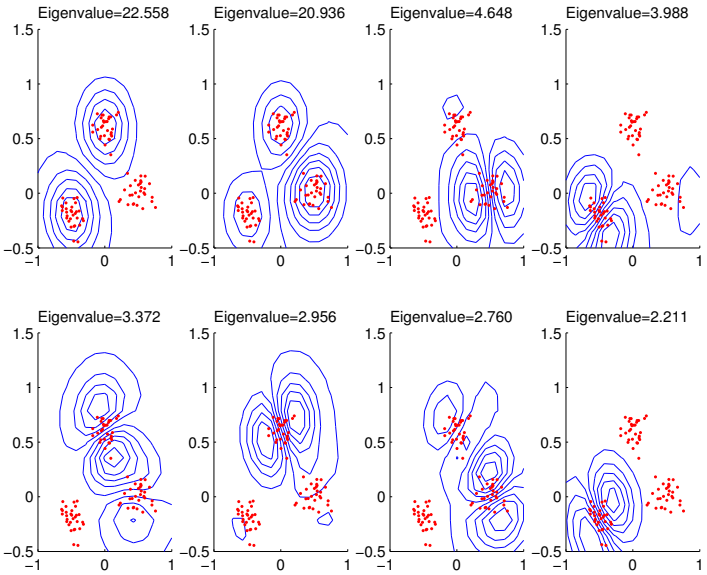


Figure 20.35: Visualization of the first 8 kernel principal component basis functions derived from some 2d data. We use an RBF kernel with $\sigma^2 = 0.1$. Generated by kpcaScholkopf.ipynb.

follows:

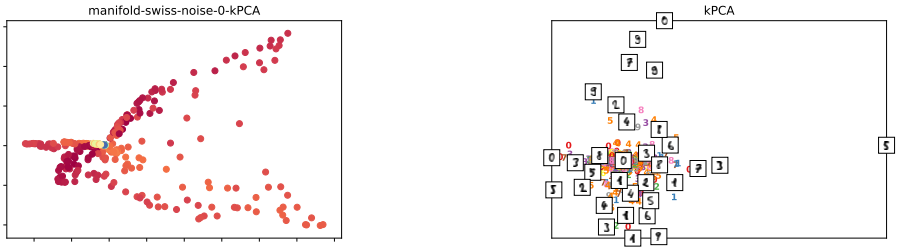
$$\boldsymbol{\phi}_*^\mathsf{T} \mathbf{V}_{\text{kPCA}} = \boldsymbol{\phi}_*^\mathsf{T} \boldsymbol{\Phi}^\mathsf{T} \mathbf{U} \boldsymbol{\Lambda}^{-\frac{1}{2}} = \boldsymbol{k}_*^\mathsf{T} \mathbf{U} \boldsymbol{\Lambda}^{-\frac{1}{2}} \tag{20.114}$$

where $\boldsymbol{k}_* = [\mathcal{K}(\boldsymbol{x}_*, \boldsymbol{x}_1), \ldots, \mathcal{K}(\boldsymbol{x}_*, \boldsymbol{x}_N)]$.

There is one final detail to worry about. The covariance matrix is only given by $\mathbf{S} = \boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi}$ if the features is zero-mean. Thus we can only use the Gram matrix $\mathbf{K} = \boldsymbol{\Phi}\boldsymbol{\Phi}^\mathsf{T}$ if $\mathbb{E}[\boldsymbol{\phi}_i] = \mathbf{0}$. Unfortunately, we cannot simply subtract off the mean in feature space, since it may be infinite dimensional. However, there is a trick we can use. Define the centered feature vector as $\tilde{\boldsymbol{\phi}}_i = \boldsymbol{\phi}(\boldsymbol{x}_i) - \frac{1}{N}\sum_{j=1}^N \boldsymbol{\phi}(\boldsymbol{x}_j)$. The Gram matrix of the centered feature vectors is given by $\tilde{K}_{ij} = \tilde{\boldsymbol{\phi}}_i^\mathsf{T} \tilde{\boldsymbol{\phi}}_j$. Using the double centering trick from Equation (7.89), we can write this in matrix form as $\tilde{\mathbf{K}} = \mathbf{C}_N \mathbf{K} \mathbf{C}_N$, where $\mathbf{C}_N \triangleq \mathbf{I}_N - \frac{1}{N}\mathbf{1}_N\mathbf{1}_N^\mathsf{T}$ is the centering matrix.

If we apply kPCA with a linear kernel, we recover regular PCA (classical MDS). This is limited to using $L \leq D$ embedding dimensions. If we use a non-degenerate kernel, we can use up to N components, since the size of $\boldsymbol{\Phi}$ is $N \times D^*$, where D^* is the (potentially infinite) dimensionality of embedded feature vectors. Figure 20.35 gives an example of the method applied to some $D = 2$ dimensional data using an RBF kernel. We project points in the unit grid onto the first 8 components and visualize the corresponding surfaces using a contour plot. We see that the first two components separate the three clusters, and the following components split the clusters.

See Figure 20.36 for some the results on kPCA (with an RBF kernel) on our running example. In this case, the results are arguably not very useful. In fact, it can be shown that kPCA with an RBF kernel expands the feature space instead of reducing it [WSS04], as we saw in Figure 20.35, which



(a) (b)

Figure 20.36: Kernel PCA applied to (a) Swiss roll. Generated by manifold_swiss_sklearn.ipynb. (b) UCI digits. Generated by manifold_digits_sklearn.ipynb.

makes it not very useful as a method for dimensionality reduction. We discuss a solution to this in Section 20.4.7.

20.4.7 Maximum variance unfolding (MVU)

kPCA with certain kernels, such as RBF, might not result in a low dimensional embedding, as discussed in Section 20.4.6. This observation led to the development of the **semidefinite embedding** algorithm [WSS04], also called **maximum variance unfolding**, which tries to learn an embedding $\{\boldsymbol{z}_i\}$ such that

$$\max \sum_{ij} ||\boldsymbol{z}_i - \boldsymbol{z}_j||_2^2 \;\; \text{s.t.} \;\; ||\boldsymbol{z}_i - \boldsymbol{z}_j||_2^2 = ||\boldsymbol{x}_i - \boldsymbol{x}_j||_2^2 \text{ for all } (i,j) \in G \tag{20.115}$$

where G is the nearest neighbor graph (as in Isomap). This approach explicitly tries to 'unfold' the data manifold while respecting the nearest neighbor constraints.

This can be reformulated as a **semidefinite programming** (SDP) problem by defining the kernel matrix $\mathbf{K} = \mathbf{Z}\mathbf{Z}^\mathsf{T}$ and then optimizing

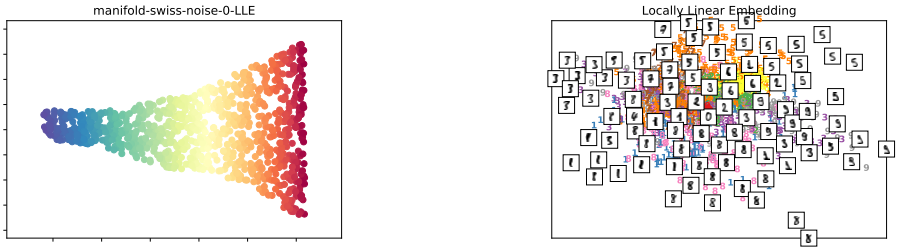
$$\max \text{tr}(\mathbf{K}) \;\; \text{s.t.} \;\; ||\boldsymbol{z}_i - \boldsymbol{z}_j||_2^2 = ||\boldsymbol{x}_i - \boldsymbol{x}_j||_2^2, \; \sum_{ij} K_{ij} = 0, \; \mathbf{K} \succ 0 \tag{20.116}$$

The resulting kernel is then passed to kPCA, and the resulting eigenvectors give the low dimensional embedding.

20.4.8 Local linear embedding (LLE)

The techniques we have discussed so far all rely on an eigendecomposition of a full matrix of pairwise similarities, either in the ambient space (PCA), in feature space (kPCA), or along the KNN graph (Isomap). In this section, we discuss **local linear embedding** (LLE) [RS00], a technique that solves a sparse eigenproblem, thus focusing more on local structure in the data.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 20.37: LLE applied to (a) Swiss roll. Generated by manifold_swiss_sklearn.ipynb. (b) UCI digits. Generated by manifold_digits_sklearn.ipynb.

LLE assumes the data manifold around each point $\boldsymbol{x}_i$ is locally linear. The best linear approximation can be found by predicting $\boldsymbol{x}_i$ as a linear combination of its K nearest neighbors using reconstruction weights $\boldsymbol{w}_i$. This can be found by solving

$$\hat{\mathbf{W}} = \min_{\mathbf{W}} \sum_{i=1}^{N} (\boldsymbol{x}_i - \sum_{j=1}^{N} w_{ij} \boldsymbol{x}_j)^2 \tag{20.117}$$

$$\text{subject to} \begin{cases} w_{ij} = 0 & \text{if } \boldsymbol{x}_j \notin \text{nbr}(\boldsymbol{x}_i, K) \\ \sum_{j=1}^{N} w_{ij} = 1 & \text{for } i = 1 : N \end{cases} \tag{20.118}$$

Note that we need the sum-to-one constraint on the weights to prevent the trivial solution $\mathbf{W} = \mathbf{0}$. The resulting vector of weights $\boldsymbol{w}_{i,:}$ constitute the **barycentric coordinates** of $\boldsymbol{x}_i$.

Any linear mapping of this hyperplane to a lower dimensional space preserves the reconstruction weights, and thus the local geometry. Thus we can solve for the low-dimensional embeddings for each point by solving

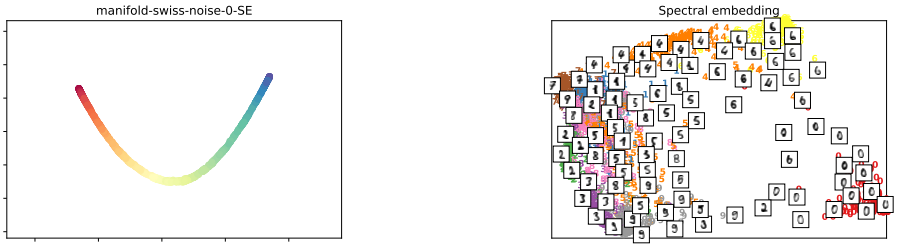
$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\text{argmin}} \sum_{i} ||\boldsymbol{z}_i - \sum_{j=1}^{N} \hat{w}_{ij} \boldsymbol{z}_j||_2^2 \tag{20.119}$$

where $\hat{w}_{ij} = 0$ if j is not one of the K nearest neighbors of i. We can rewrite this loss as

$$\mathcal{L}(\mathbf{Z}) = ||\mathbf{Z} - \mathbf{W}\mathbf{Z}||^2 = \mathbf{Z}^\mathsf{T}(\mathbf{I} - \mathbf{W})^\mathsf{T}(\mathbf{I} - \mathbf{W})\mathbf{Z} \tag{20.120}$$

Thus the solution is given by the eigenvectors of $(\mathbf{I} - \mathbf{W})^\mathsf{T}(\mathbf{I} - \mathbf{W})$ corresponding to the smallest nonzero eigenvalues, as shown in Section 7.4.8.

See Figure 20.37 for some the results on LLE on our running example. In this case, the results do not seem as good as those produced by Isomap. However, the method tends to be somewhat less sensitive to short-circuiting (noise).



(a) (b)

Figure 20.38: Laplacian eigenmaps applied to (a) Swiss roll. Generated by manifold_swiss_sklearn.ipynb. (b) UCI digits. Generated by manifold_digits_sklearn.ipynb.

20.4.9 Laplacian eigenmaps

In this section, we describe **Laplacian eigenmaps** or **spectral embedding** [BN01]. The idea is to compute a low-dimensional representation of the data in which the weighted distances between a datapoint and its K nearest neighbors are minimized. We put more weight on the first nearest neighbor than the second, etc. We give the details below.

20.4.9.1 Using eigenvectors of the graph Laplacian to compute embeddings

We want to find embeddings which minimize

$$\mathcal{L}(\mathbf{Z}) = \sum_{(i,j)\in E} W_{i,j}||\boldsymbol{z}_i - \boldsymbol{z}_j||_2^2 \tag{20.121}$$

where $W_{ij} = \exp(-\frac{1}{2\sigma^2}||\boldsymbol{x}_i - \boldsymbol{x}_j||_2^2)$ if $i-j$ are neighbors in the KNN graph and 0 otherwise. We add the constraint $\mathbf{Z}^\mathsf{T}\mathbf{D}\mathbf{Z} = \mathbf{I}$ to avoid the degenerate solution where $\mathbf{Z} = \mathbf{0}$, where $\mathbf{D}$ is the diagonal weight matrix storing the degree of each node, $D_{ii} = \sum_j W_{i,j}$.

We can rewrite the above objective as follows:

$$\mathcal{L}(\mathbf{Z}) = \sum_{ij} W_{ij}(||\boldsymbol{z}_i||^2 + ||\boldsymbol{z}_j||^2 - 2\boldsymbol{z}_i^\mathsf{T}\boldsymbol{z}_j) \tag{20.122}$$

$$= \sum_i D_{ii}||\boldsymbol{z}_i||^2 + \sum_j D_{jj}||\boldsymbol{z}_j||^2 - 2\sum_{ij} W_{ij}\boldsymbol{z}_i\boldsymbol{z}_j^\mathsf{T} \tag{20.123}$$

$$= 2\text{tr}(\mathbf{Z}^\mathsf{T}\mathbf{D}\mathbf{Z}) - 2\text{tr}(\mathbf{Z}^\mathsf{T}\mathbf{W}\mathbf{Z}) = 2\text{tr}(\mathbf{Z}^\mathsf{T}\mathbf{L}\mathbf{Z}) \tag{20.124}$$

where $\mathbf{L} = \mathbf{D} - \mathbf{W}$ is the graph Laplacian (see Section 20.4.9.2). One can show that minimizing this is equivalent to solving the (generalized) eigenvalue problem $\mathbf{L}\boldsymbol{z}_i = \lambda_i\mathbf{D}\boldsymbol{z}_i$ for the L smallest nonzero eigenvalues.

See Figure 20.38 for the results of applying this method (with an RBF kernel) to our running example.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

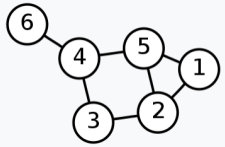
Labelled graph	Degree matrix	Adjacency matrix	Laplacian matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$

Figure 20.39: Illustration of the Laplacian matrix derived from an undirected graph. From https://en.wikipedia.org/wiki/Laplacian_matrix. Used with kind permission of Wikipedia author AzaToth.

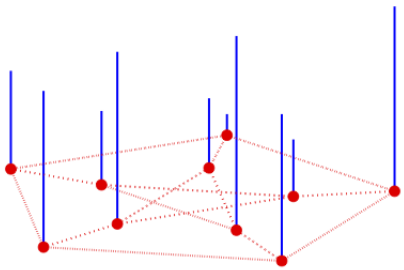


Figure 20.40: Illustration of a (positive) function defined on a graph. From Figure 1 of [Shu+13]. Used with kind permission of Pascal Frossard.

20.4.9.2 What is the graph Laplacian?

We saw above that we can compute the eigenvectors of the graph Laplacian in order to learn a good embedding of the high dimensional points. In this section, we give some intuition as to why this works.

Let $\mathbf{W}$ be a symmetric weight matrix for a graph, where $W_{ij} = W_{ji} \geq 0$. Let $\mathbf{D} = \text{diag}(d_i)$ be a diagonal matrix containing the weighted degree of each node, $d_i = \sum_j w_{ij}$. We define the **graph Laplacian** as follows:

$$\mathbf{L} \triangleq \mathbf{D} - \mathbf{W} \tag{20.125}$$

Thus the elements of $\mathbf{L}$ are given by

$$L_{ij} = \begin{cases} d_i & \text{if } i = j \\ -w_{ij} & \text{if } i \neq j \text{ and } w_{ij} \neq 0 \\ 0 & \text{otherwise} \end{cases} \tag{20.126}$$

See Figure 20.39 for an example of how to compute this.

Suppose we associate a value $f_i \in \mathbb{R}$ with each node i in the graph (see Figure 20.40 for example). Then we can use the graph Laplacian as a difference operator, to compute a discrete derivative of

the function at a point:

$$(\mathbf{L}\boldsymbol{f})(i) = \sum_{j \in \text{nbr}_i} W_{ij}[f(i) - f(j)] \tag{20.127}$$

where nbr_i is the set of neighbors of node i. We can also compute an overall measure of "smoothness" of the function f by computing its **Dirichlet energy** as follows:

$$\boldsymbol{f}^T\mathbf{L}\boldsymbol{f} = \boldsymbol{f}^T\mathbf{D}\boldsymbol{f} - \boldsymbol{f}^T\mathbf{W}\boldsymbol{f} = \sum_i d_i f_i^2 - \sum_{i,j} f_i f_j w_{ij} \tag{20.128}$$

$$= \frac{1}{2}\left(\sum_i d_i f_i^2 - 2\sum_{i,j} f_i f_j w_{ij} + \sum_j d_j f_j^2\right) = \frac{1}{2}\sum_{i,j} w_{ij}(f_i - f_j)^2 \tag{20.129}$$

By studying the eigenvalues and eigenvectors of the Laplacian matrix, we can determine various useful properties of the function. (Applying linear algebra to study the adjacency matrix of a graph, or related matrices, is called **spectral graph theory** [Chu97].) For example, we see that $\mathbf{L}$ is symmetric and positive semi-definite, since we have $\boldsymbol{f}^T\mathbf{L}\boldsymbol{f} \geq 0$ for all $\boldsymbol{f} \in \mathbb{R}^N$, which follows from Equation (20.129) due to the assumption that $w_{ij} \geq 0$. Consequently $\mathbf{L}$ has N non-negative, real-valued eigenvalues, $0 \leq \lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_N$. The corresponding eigenvectors form an orthogonal basis for the function f defined on the graph, in order of decreasing smoothness.

In Section 20.4.9.1, we discuss Laplacian eigenmaps, which is a way to learn low dimensional embeddings for high dimensional data vectors. The approach is to let $z_{id} = f_i^d$ be the d'th embedding dimension for input i, and then to find a basis for these functions (i.e., embedding of the points) that varies smoothly over the graph, thus respecting distance of the points in ambient space.

There are many other applications of the graph Laplacian in ML. For example, in Section 21.5.1, we discuss normalized cuts, which is a way to learn a clustering of high dimensional data vectors based on pairwise similarity; and [WTN19] discusses how to use the eigenvectors of the state transition matrix to learn representations for RL.

20.4.10 t-SNE

In this section, we describe a very popular nonconvex technique for learning low dimensional embeddings called **t-SNE** [MH08]. This extends the earlier **stochastic neighbor embedding** method of [HR03], so we first describe SNE, before describing the t-SNE extension.

20.4.10.1 Stochastic neighborhood embedding (SNE)

The basic idea in SNE is to convert high-dimensional Euclidean distances into conditional probabilities that represent similarities. More precisely, we define $p_{j|i}$ to be the probability that point i would pick point j as its neighbor if neighbors were picked in proportion to their probability under a Gaussian centered at $\boldsymbol{x}_i$:

$$p_{j|i} = \frac{\exp(-\frac{1}{2\sigma_i^2}||\boldsymbol{x}_i - \boldsymbol{x}_j||^2)}{\sum_{k \neq i}\exp(-\frac{1}{2\sigma_i^2}||\boldsymbol{x}_i - \boldsymbol{x}_k||^2)} \tag{20.130}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Here σ_i^2 is the variance for data point i, which can be used to "magnify" the scale of points in dense regions of input space, and diminish the scale in sparser regions. (We discuss how to estimate the length scales σ_i^2 shortly).

Let $\boldsymbol{z}_i$ be the low dimensional embedding representing $\boldsymbol{x}_i$. We define similarities in the low dimensional space in an analogous way:

$$q_{j|i} = \frac{\exp(-||\boldsymbol{z}_i - \boldsymbol{z}_j||^2)}{\sum_{k \neq i} \exp(-||\boldsymbol{z}_i - \boldsymbol{z}_k||^2)} \tag{20.131}$$

In this case, the variance is fixed to a constant; changing it would just rescale the learned map, and not change its topology.

If the embedding is a good one, then $q_{j|i}$ should match $p_{j|i}$. Therefore, SNE defines the objective to be

$$\mathcal{L} = \sum_i D_{\mathbb{KL}}\left(P_i \parallel Q_i\right) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}} \tag{20.132}$$

where P_i is the conditional distribution over all other data points given $\boldsymbol{x}_i$, Q_i is the conditional distribution over all other latent points given $\boldsymbol{z}_i$, and $D_{\mathbb{KL}}\left(P_i \parallel Q_i\right)$ is the KL divergence (Section 6.2) between the distributions.

Note that this is an asymmetric objective. In particular, there is a large cost if a small $q_{j|i}$ is used to model a large $p_{j|i}$. This objective will prefer to pull distant points together rather than push nearby points apart. We can get a better idea of the geometry by looking at the gradient for each embedding vector, which is given by

$$\nabla_{\boldsymbol{z}_i} \mathcal{L}(\mathbf{Z}) = 2 \sum_j (\boldsymbol{z}_i - \boldsymbol{z}_j)(p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j}) \tag{20.133}$$

Thus points are pulled towards each other if the p's are bigger than the q's, and repelled if the q's are bigger than the p's.

Although this is an intuitively sensible objective, it is not convex. Nevertheless it can be minimized using SGD. In practice, it helps to add Gaussian noise to the embedding points, and to gradually anneal the amount of noise. [Hin13] recommends to "spend a long time at the noise level at which the global structure starts to form from the hot plasma of map points" before reducing it.[6]

20.4.10.2 Symmetric SNE

There is a slightly simpler version of SNE that minimizes a single KL between the joint distribution P in high dimensional space and Q in low dimensional space:

$$\mathcal{L} = D_{\mathbb{KL}}\left(P \parallel Q\right) = \sum_{i<j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{20.134}$$

This is called **symmetric SNE**.

6. See [Ros98; WF20] for a discussion of annealing and phase transitions in unsupervised learning. See also [CP10] for a discussion of the **elastic embedding** algorithm, which uses a homotopy method to more efficiently optimize a model that is related to both SNE and Laplacian eigenmaps.

The obvious way to define p_{ij} is to use

$$p_{ij} = \frac{\exp(-\frac{1}{2\sigma^2}||\boldsymbol{x}_i - \boldsymbol{x}_j||^2)}{\sum_{k \neq l} \exp(-\frac{1}{2\sigma^2}||\boldsymbol{x}_k - \boldsymbol{x}_l||^2)} \tag{20.135}$$

We can define q_{ij} similarily.

The corresponding gradient becomes

$$\nabla_{\boldsymbol{z}_i} \mathcal{L}(\mathbf{Z}) = 4 \sum_j (\boldsymbol{z}_i - \boldsymbol{z}_j)(p_{ij} - q_{ij}) \tag{20.136}$$

As before, points are pulled towards each other if the p's are bigger than the q's, and repelled if the q's are bigger than the p's.

Although symmetric SNE is slightly easier to implement, it loses the nice property of regular SNE that the data is its own optimal embedding if the embedding dimension L is set equal to the ambient dimension D. Nevertheless, the methods seems to give similar results in practice on real datasets where $L \ll D$.

20.4.10.3 t-distributed SNE

A fundamental problem with SNE and many other embedding techniques is that they tend to squeeze points that are relatively far away in the high dimensional space close together in the low dimensional (usually 2d) embedding space; this is called the **crowding problem**, and arises due to the use of squared errors (or Gaussian probabilities).

One solution to this is to use a probability distribution in latent space that has heavier tails, which eliminates the unwanted attractive forces between points that are relatively far in the high dimensional space. An obvious choice is the Student-t distribution (Section 2.7.1). In t-SNE, they set the degree of freedom parameter to $\nu = 1$, so the distribution becomes equivalent to a Cauchy:

$$q_{ij} = \frac{(1 + ||\boldsymbol{z}_i - \boldsymbol{z}_j||^2)^{-1}}{\sum_{k<l}(1 + ||\boldsymbol{z}_k - \boldsymbol{z}_l||^2)^{-1}} \tag{20.137}$$

We can use the same global KL objective as in Equation (20.134). For t-SNE, the gradient turns out to be

$$\nabla \boldsymbol{z}_i \mathcal{L} = 4 \sum_j (p_{ij} - q_{ij})(\boldsymbol{z}_i - \boldsymbol{z}_j)(1 + ||\boldsymbol{z}_i - \boldsymbol{z}_j||^2)^{-1} \tag{20.138}$$

The gradient for symmetric (Gaussian) SNE is the same, but lacks the $(1 + ||\boldsymbol{z}_i - \boldsymbol{z}_j||^2)^{-1}$ term. This term is useful because $(1 + ||\boldsymbol{z}_i - \boldsymbol{z}_j||^2)^{-1}$ acts like an inverse square law. This means that points in embedding space act like stars and galaxies, forming many well-separated clusters (galaxies) each of which has many stars tightly packed inside. This can be useful for separating different classes of data in an unsupervised way (see Figure 20.41 for an example).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 20.41: tSNE applied to (a) Swiss roll. Generated by manifold_swiss_sklearn.ipynb. (b) UCI digits. Generated by manifold_digits_sklearn.ipynb.



Figure 20.42: Illustration of the effect of changing the perplexity parameter when t-SNE is applied to some 2d data. From [WVJ16]. See http://distill.pub/2016/misread-tsne for an animated version of these figures. Used with kind permission of Martin Wattenberg.

20.4.10.4 Choosing the length scale

An important parameter in t-SNE is the local bandwidth σ_i^2. This is usually chosen so that P_i has a perplexity chosen by the user.[7] This can be interpreted as a smooth measure of the effective number of neighbors.

Unfortunately, the results of t-SNE can be quite sensitive to the perplexity parameter, so it is wise to run the algorithm with many different values. This is illustrated in Figure 20.42. The input data is 2d, so there is no distortion generating by mapping to a 2d latent space. If the perplexity is too small, the method tends to find structure within each cluster which is not truly present. At perplexity 30 (the default for scikit-learn), the clusters seem equi-distant in embedding space, even though some are closer than others in the data space. Many other caveats in interpreting t-SNE plots can be found in [WVJ16].

7. The perplexity is defined to be $2^{\mathbb{H}(P_i)}$, where $\mathbb{H}(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}$ is the entropy; see Section 6.1.5 for details. A big radius around each point (large value of σ_i) will result in a high entropy, and thus high perplexity.

20.4.10.5 Computational issues

The naive implementation of t-SNE takes $O(N^2)$ time, as can be seen from the gradient term in Equation (20.138). A faster version can be created by leveraging an analogy to N-body simulation in physics. In particular, the gradient requires computing the force of N points on each of N points. However, points that are far away can be grouped into clusters (computationally speaking), and their effective force can be approximated by a few representative points per cluster. We can then approximate the forces using the **Barnes-Hut algorithm** [BH86], which takes $O(N \log N)$ time, as proposed in [Maa14]. Unfortunately, this only works well for low dimensional embeddings, such as $L = 2$.

20.4.10.6 UMAP

Various extensions of tSNE have been proposed, that try to improve its speed, the quality of the embedding space, or the ability to embed into more than 2 dimensions.

One popular recent extension is called **UMAP** (which stands for "Uniform Manifold Approximation and Projection"), was proposed in [MHM18]. At a high level, this is similar to tSNE, but it tends to preserve global structure better, and it is much faster. This makes it easier to try multiple values of the hyperparameters. For an interactive tutorial on UMAP, and a comparison to tSNE, see [CP19].

20.5 Word embeddings

Words are categorical random variables, so their corresponding one-hot vector representations are sparse. The problem with this binary representation is that semantically similar words may have very different vector representations. For example, the pair of related words "man" and "woman" will be Hamming distance 1 apart, as will the pair of unrelated words "man" and "banana".

The standard way to solve this problem is to use **word embeddings**, in which we map each sparse one-hot vector, $\boldsymbol{s}_{n,t} \in \{0,1\}^M$, representing the t'th word in document n, to a lower-dimensional dense vector, $\boldsymbol{z}_{n,t} \in \mathbb{R}^D$, such that semantically similar words are placed close by. This can significantly help with data sparsity. There are many ways to learn such embeddings, as we discuss below.

Before discussing methods, we have to define what we mean by "semantically similar" words. We will assume that two words are semantically similar if they occur in similar contexts. This is known as the **distributional hypothesis** [Har54], which is often summarized by the phase (originally from [Fir57]) "a word is characterized by the company it keeps". Thus the methods we discuss will all learn a mapping from a word's context to an embedding vector for that word.

20.5.1 Latent semantic analysis / indexing

In this section, we discuss a simple way to learn word embeddings based on singular value decomposition (Section 7.5) of a term-frequency count matrix.

20.5.1.1 Latent semantic indexing (LSI)

Let C_{ij} be the number of times "term" i occurs in "context" j. The definition of what we mean by "term" is application-specific. In English, we often take it to be the set of unique tokens that are separated by punctuation or whitespace; for simplicity, we will call these "words". However, we may

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

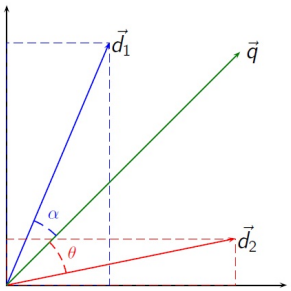


Figure 20.43: Illustration of the cosine similarity between a query vector $\boldsymbol{q}$ and two document vectors $\boldsymbol{d}_1$ and $\boldsymbol{d}_2$. Since angle α is less than angle θ, we see that the query is more similar to document 1. From https://en.wikipedia.org/wiki/Vector_space_model. Used with kind permission of Wikipedia author Riclas.

preprocess the text data to remove very frequent or infrequent words, or perform other kinds of preprocessing. as we discuss in Section 1.5.4.1.

The definition of what we mean by "context" is also application-specific. In this section, we count how many times word i occurs in each document $j \in \{1, \ldots, N\}$ from a set or **corpus** of documents; the resulting matrx $\mathbf{C}$ is called a **term-document frequency matrix**, as in Figure 1.15. (Sometimes we apply the TF-IDF transformation to the counts, as discussed in Section 1.5.4.2.)

Let $\mathbf{C} \in \mathbb{R}^{M \times N}$ be the count matrix, and let $\hat{\mathbf{C}}$ be the rank K approximation that minimizes the following loss:

$$\mathcal{L}(\hat{\mathbf{C}}) = ||\mathbf{C} - \hat{\mathbf{C}}||_F = \sum_{ij} (C_{ij} - \hat{C}_{ij})^2 \tag{20.139}$$

One can show that the minimizer of this is given by the rank K truncated SVD approximation, $\hat{\mathbf{C}} = \mathbf{USV}$. This means we can represent each c_{ij} as a bilinear product:

$$c_{ij} \approx \sum_{k=1}^{K} u_{ik} s_k v_{kj} \tag{20.140}$$

We define $\boldsymbol{u}_i$ to be the embedding for word i, and $\boldsymbol{s} \odot \boldsymbol{v}_j$ to be the embedding for context j.

We can use these embeddings for **document retrieval**. The idea is to compute an embedding for the query words using $\boldsymbol{u}_i$, and to compare this to the embedding of all the documents or contexts $\boldsymbol{v}_j$. This is known as **latent semantic indexing** or **LSI** [Dee+90].

In more detail, suppose the query is a bag of words $w_1, \ldots, w_B$; we represent this by the vector $\boldsymbol{q} = \frac{1}{B} \sum_{b=1}^{B} \boldsymbol{u}_{w_b}$, where $\boldsymbol{u}_{w_b}$ is the embedding for word w_b. Let document j be represented by $\boldsymbol{v}_j$. We then rank documents by the **cosine similarity** between the query vector and document, defined by

$$\text{sim}(\boldsymbol{q}, \boldsymbol{d}) = \frac{\boldsymbol{q}^\mathsf{T} \boldsymbol{d}}{||\boldsymbol{q}||\ ||\boldsymbol{d}||} \tag{20.141}$$

where $||\boldsymbol{q}|| = \sqrt{\sum_i q_i^2}$ is the ℓ_2-norm of $\boldsymbol{q}$. This measures the angles between the two vectors, as shown in Figure 20.43. Note that if the vectors are unit norm, cosine similarity is the same as inner

product; it is also equal to the squared Euclidean distance, up to a change of sign and an irrelevant additive constant:

$$||\boldsymbol{q}-\boldsymbol{d}||^2 = (\boldsymbol{q}-\boldsymbol{d})^\mathsf{T}(\boldsymbol{q}-\boldsymbol{d}) = \boldsymbol{q}^\mathsf{T}\boldsymbol{q} + \boldsymbol{d}^\mathsf{T}\boldsymbol{d} - 2\boldsymbol{q}^\mathsf{T}\boldsymbol{d} = 2(1-\text{sim}(\boldsymbol{q},\boldsymbol{d})) \tag{20.142}$$

20.5.1.2 Latent semantic analysis (LSA)

Now suppose we define context more generally to be some local neighborhood of words $j \in \{1,\ldots,M^h\}$, where h is the window size. Thus C_{ij} is how many times word i occurs in a neighborhood of type j. We can compute the SVD of this matrix as before, to get $c_{ij} \approx \sum_{k=1}^K u_{ik}s_k v_{jk}$. We define $\boldsymbol{u}_i$ to be the embedding for word i, and $\boldsymbol{s} \odot \boldsymbol{v}_j$ to be the embedding for context j. This is known as **latent semantic analysis** or **LSA** [Dee+90].

For example, suppose we compute $\mathbf{C}$ on the British National Corpus.[8] For each word, let us retrieve the K nearest neighbors in embedding space ranked by cosine similarity (i.e., normalized inner product). If the query word is "dog", and we use $h=2$ or $h=30$, the nearest neighbors are as follows:

```
h=2: cat, horse, fox, pet, rabbit, pig, animal, mongrel, sheep, pigeon
h=30: kennel, puppy, pet, bitch, terrier, rottweiler, canine, cat, to bark
```

The 2-word context window is more sensitive to syntax, while the 30-word window is more sensitive to semantics. The "optimal" value of context size h depends on the application.

20.5.1.3 PMI

In practice LSA (and other similar methods) give much better results if we replace the raw counts C_{ij} with **pointwise mutual information** (**PMI**) [CH90], defined as

$$\mathbb{PMI}(i,j) = \log \frac{p(i,j)}{p(i)p(j)} \tag{20.143}$$

If word i is strongly associated with context j, we will have $\mathbb{PMI}(i,j) > 0$. If the PMI is negative, it means i and j co-occur less often that if they were independent; however, such negative correlations can be unreliable, so it is common to use the **positive PMI**: $\mathbb{PPMI}(i,j) = \max(\mathbb{PMI}(i,j), 0)$. In [BL07b], they show that SVD applied to the PPMI matrix results in word embeddings that perform well on a many tasks related to word meaning. See Section 20.5.5 for a theoretical model that explains this empirical performance.

20.5.2 Word2vec

In this section, we discuss the popular **word2vec** model from [Mik+13a; Mik+13b], which are "shallow" neural nets for predicting a word given its context. In Section 20.5.5, we will discuss the connections with SVD of the PMI matrix.

There are two versions of the word2vec model. The first is called CBOW, which stands for "continuous bag of words". The second is called skipgram. We discuss both of these below.

8. This example is taken from [Eis19, p312].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

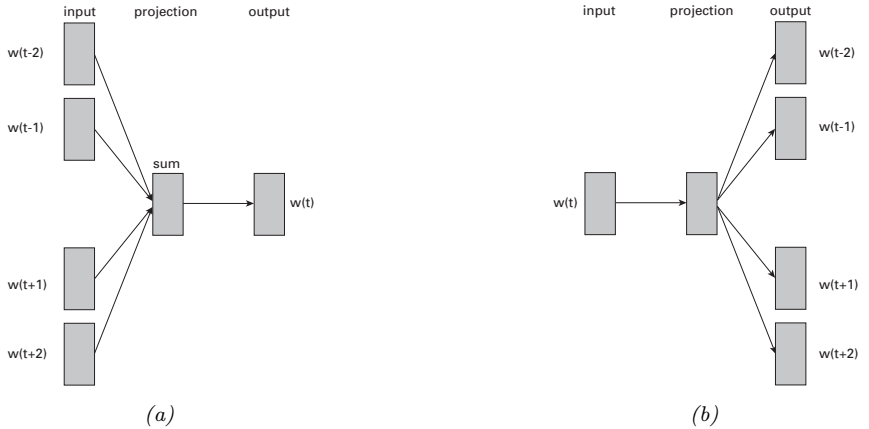


Figure 20.44: Illustration of word2vec model with window size of 2. (a) CBOW version. (b) Skip-gram version.

20.5.2.1 Word2vec CBOW model

In the continuous bag of words (**CBOW**) model (see Figure 20.44(a)), the log likelihood of a sequence of words is computed using the following model:

$$\log p(\boldsymbol{w}) = \sum_{t=1}^{T} \log p(w_t|\boldsymbol{w}_{t-m:t+m}) = \sum_{t=1}^{T} \log \frac{\exp(\boldsymbol{v}_{w_t}^\mathsf{T} \overline{\boldsymbol{v}}_t)}{\sum_{i \in \mathcal{V}} \exp(\boldsymbol{v}_i^\mathsf{T} \overline{\boldsymbol{v}}_t)} \tag{20.144}$$

$$= \sum_{t=1}^{T} \boldsymbol{v}_{w_t}^\mathsf{T} \overline{\boldsymbol{v}}_t - \log \sum_{i \in \mathcal{V}} \exp(\boldsymbol{v}_i^\mathsf{T} \overline{\boldsymbol{v}}_t) \tag{20.145}$$

where $\boldsymbol{v}_{w_t}$ is the vector for the word at location w_t, $\mathcal{V}$ is the set of all words, m is the context size, and

$$\overline{\boldsymbol{v}}_t = \frac{1}{2m} \sum_{h=1}^{m} (\boldsymbol{v}_{w_{t+h}} + \boldsymbol{v}_{w_{t-h}}) \tag{20.146}$$

is the average of the word vectors in the window around word w_t. Thus we try to predict each word given its context. The model is called CBOW because it uses a bag of words assumption for the context, and represents each word by a continuous embedding.

20.5.2.2 Word2vec Skip-gram model

In CBOW, each word is predicted from its context. A variant of this is to predict the context (surrounding words) given each word. This yields the following objective:

$$-\log p(\boldsymbol{w}) = -\sum_{t=1}^{T}\left[\sum_{j=1}^{m}\log p(w_{t-j}|w_t) + \log p(w_{t+j}|w_t)\right] \tag{20.147}$$

$$= -\sum_{t=1}^{T}\sum_{-m\le j\le m, j\neq 0}\log p(w_{t+j}|w_t) \tag{20.148}$$

where m is the context window length. We define the log probability of some other context word w_o given the central word w_c to be

$$\log p(w_o|w_c) = \boldsymbol{u}_o^{\mathsf{T}}\boldsymbol{v}_c - \log\left(\sum_{i\in\mathcal{V}}\exp(\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{v}_c)\right) \tag{20.149}$$

where $\mathcal{V}$ is the vocabulary. Here $\boldsymbol{u}_i$ is the embedding of a word if used as context, and $\boldsymbol{v}_i$ is the embedding of a word if used as the central word. This model is known as the **skipgram model**. See Figure 20.44(b) for an illustration.

20.5.2.3 Negative sampling

Computing the conditional probability of each word using Equation (20.149) is expensive, due to the need to normalize over all possible words in the vocabulary. This makes it slow to compute the log likelihood and its gradient, for both the CBOW and skip-gram models.

In [Mik+13b], they propose a fast approximation, called **skip-gram with negative sampling** (**SGNS**). The basic idea is to create a set of $K+1$ context words for each central word w_t, and to label the one that actually occurs as positive, and the rest as negative. The negative words are called noise words, and can be sampled from a reweighted unigram distribution, $p(w) \propto \text{freq}(w)^{3/4}$, which has the effect of redistributing probability mass from common to rare words. The conditional probability is now approximated by

$$p(w_{t+j}|w_t) = p(D=1|w_t, w_{t+j})\prod_{k=1}^{K} p(D=0|w_t, w_k) \tag{20.150}$$

where $w_k \sim p(w)$ are noise words, and $D=1$ is the event that the word pair actually occurs in the data, and $D=0$ is the event that the word pair does not occur. The binary probabilities are given by

$$p(D=1|w_t, w_{t+j}) = \sigma(\boldsymbol{u}_{w_{t+j}}^{\mathsf{T}}\boldsymbol{v}_{w_t}) \tag{20.151}$$

$$p(D=0|w_t, w_k) = 1 - \sigma(\boldsymbol{u}_{w_k}^{\mathsf{T}}\boldsymbol{v}_{w_t}) \tag{20.152}$$

To train this model, we just need to compute the contexts for each central word, and a set of negative noise words. We associate a label of 1 with the context words, and a label of 0 with the noise words. We can then compute the log probability of the data, and optimize the embedding vectors $\boldsymbol{u}_i$ and $\boldsymbol{v}_i$ for each word using SGD. See skipgram_jax.ipynb for some sample code.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

20.5.3 GloVE

A popular alternative to Skipgram is the **GloVe** model of [PSM14a]. (GloVe stands for "global vectors for word representation".) This method uses a simpler objective, which is much faster to optimize.

To explain the method, recall that in the skipgram model, the predicted conditional probability of word j occuring in the context window of central word i as

$$q_{ij} = \frac{\exp(\boldsymbol{u}_j^\mathsf{T} \boldsymbol{v}_i)}{\sum_{k \in \mathcal{V}} \exp(\boldsymbol{u}_k^\mathsf{T} \boldsymbol{v}_i)} \tag{20.153}$$

Let x_{ij} be the number of times word j occurs in any context window of i. (Note that if word i occurs in the window of j, then j will occur in the window of i, so we have $x_{ij} = x_{ji}$.) Then we can rewrite Equation (20.148) as follows:

$$\mathcal{L} = -\sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}} x_{ij} \log q_{ij} \tag{20.154}$$

If we define $p_{ij} = x_{ij}/x_i$ to be the empirical probability of word j occuring in the context window of central word i, we can rewrite the skipgram loss as a cross entropy loss:

$$\mathcal{L} = -\sum_{i \in \mathcal{V}} x_i \sum_{j \in \mathcal{V}} p_{ij} \log q_{ij} \tag{20.155}$$

The problem with this objective is that computing q_{ij} is expensive, due to the need to normalize over all words. In GloVe, we work with unnormalized probabilities, $p'_{ij} = x_{ij}$ and $q'_{ij} = \exp(\boldsymbol{u}_j^\mathsf{T} \boldsymbol{v}_i + b_i + c_j)$, where b_i and c_j are bias terms to capture marginal probabilities. In addition, we minimize the squared loss, $(\log p'_{ij} - \log q'_{ij})^2$, which is more robust to errors in estimating small probablities than log loss. Finally, we upweight rare words for which $x_{ij} < c$, where $c = 100$, by weighting the squared errors by $h(x_{ij})$, where $h(x) = (x/c)^{0.75}$ if $x < c$, and $h(x) = 1$ otherwise. This gives the final GloVe objective:

$$\mathcal{L} = -\sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}} h(x_{ij})(\boldsymbol{u}_j^\mathsf{T} \boldsymbol{v}_i + b_i + c_j - \log x_{ij})^2 \tag{20.156}$$

We can precompute x_{ij} offline, and then optimize the above objective using SGD. After training, we define the embedding of word i to be the average of $\boldsymbol{v}_i$ and $\boldsymbol{u}_i$.

Empirically GloVe gives similar results to skigram, but it is faster to train. See Section 20.5.5 for a theoretical model that explains why these methods work.

20.5.4 Word analogies

One of the most remarkable properties of word embeddings produced by word2vec, GloVe, and other similar methods is that the learned vector space seems to capture relational semantics in terms of simple vector addition. For example, consider the **word analogy problem** "man is to woman as king is to queen", often written as man:woman::king:queen. Suppose we are given the words a=man, b=woman, c=king; how do we find d=queen? Let $\boldsymbol{\delta} = \boldsymbol{v}_b - \boldsymbol{v}_a$ be the vector representing the

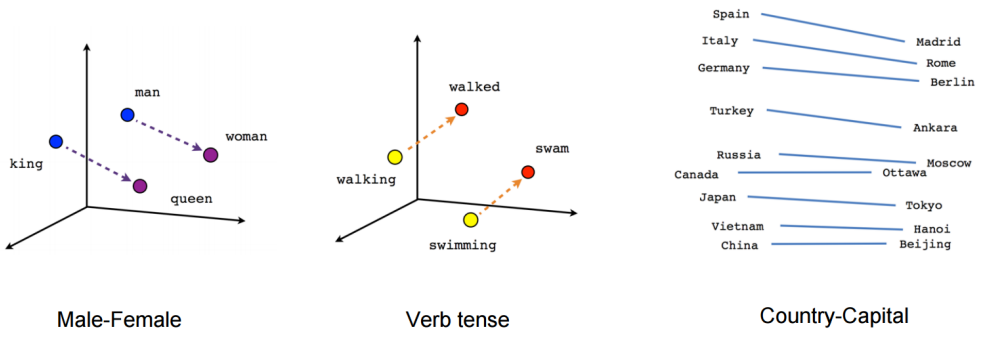


Figure 20.45: Visualization of arithmetic operations in word2vec embedding space. From https://www.tensorflow.org/tutorials/representation/word2vec .

concept of "converting the gender from male to female". Intuitively we can find word d by computing $\boldsymbol{v}_d = \boldsymbol{c} + \boldsymbol{\delta}$, and then finding the closest word in the vocabulary to $\boldsymbol{v}_d$. See Figure 20.45 for an illustration of this process, and word_analogies_jax.ipynb for some code.

In [PSM14a], they conjecture that $a : b :: c : d$ holds iff for every word w in the vocabulary, we have

$$\frac{p(w|a)}{p(w|b)} \approx \frac{p(w|c)}{p(w|d)} \tag{20.157}$$

In [Aro+16], they show that this follows from the RAND-WALK modeling assumptions in Section 20.5.5. See also [AH19; EDH19] for other explanations of why word analogies work, based on different modeling assumptions.

20.5.5 RAND-WALK model of word embeddings

Word embeddings significantly improve the performance of various kinds of NLP models compared to using one-hot encodings for words. It is natural to wonder why the above word embeddings work so well. In this section, we give a simple generative model for text documents that explains this phenomenon, based on [Aro+16].

Consider a sequence of words $w_1, \ldots, w_T$. We assume each word is generated by a latent context or discourse vector $\boldsymbol{z}_t \in \mathbb{R}^D$ using the following **log bilinear language model**, similar to [MH07]:

$$p(w_t = w|\boldsymbol{z}_t) = \frac{\exp(\boldsymbol{z}_t^\mathsf{T} \boldsymbol{v}_w)}{\sum_{w'} \exp(\boldsymbol{z}_t^\mathsf{T} \boldsymbol{v}_{w'})} = \frac{\exp(\boldsymbol{z}_t^\mathsf{T} \boldsymbol{v}_w)}{Z(\boldsymbol{z}_t)} \tag{20.158}$$

where $\boldsymbol{v}_w \in \mathbb{R}^D$ is the embedding for word w, and $Z(\boldsymbol{z}_t)$ is the partition function. We assume $D < M$, the number of words in the vocabulary.

Let us further assume the prior for the word embeddings $\boldsymbol{v}_w$ is an isotropic Gaussian, and that the latent topic $\boldsymbol{z}_t$ undergoes a slow Gaussian random walk. (This is therefore called the **RAND-WALK** model.) Under this model, one can show that $Z(\boldsymbol{z}_t)$ is approximately equal to a fixed constant, Z, independent of the context. This is known as the **self-normalization property** of log-linear models

[AK15]. Furthermore, one can show that the pointwise mutual information of predictions from the model is given by

$$\mathbb{PMI}(w, w') = \frac{p(w, w')}{p(w)p(w')} \approx \frac{\boldsymbol{v}_w^\mathsf{T} \boldsymbol{v}_{w'}}{D} \tag{20.159}$$

We can therefore fit the RAND-WALK model by matching the model's predicted values for PMI with the empirical values, i.e., we minimize

$$\mathcal{L} = \sum_{w,w'} X_{w,w'} (\mathbb{PMI}(w, w') - \boldsymbol{v}_w^\mathsf{T} \boldsymbol{v}_{w'})^2 \tag{20.160}$$

where $X_{w,w'}$ is the number of times w and w' occur next to each other. This objective can be seen as a frequency-weighted version of the SVD loss in Equation (20.139). (See [LG14] for more connections between word embeddings and SVD.)

Furthermore, some additional approximations can be used to show that the NLL for the RAND-WALK model is equivalent to the CBOW and SGNS word2vec objectives. We can also derive the objective for GloVE from this approach.

20.5.6 Contextual word embeddings

Consider the sentences "I was eating an apple" and "I bought a new phone from Apple". The meaning of the word "apple" is different in both cases, but a fixed word embedding, of the type discussed in Section 20.5, would not be able to capture this. In Section 15.7, we discuss **contextual word embeddings**, where the embedding of a word is a function of all the words in its context (usually a sentence). This can give much improved results, and is currently the standard approach to representing natural language data, as a pre-processing step before doing transfer learning (see Section 19.2).

20.6 Exercises

Exercise 20.1 [EM for FA]

Derive the EM updates for the factor analysis model. For simplicity, you can optionally assume $\boldsymbol{\mu} = \mathbf{0}$ is fixed.

Exercise 20.2 [EM for mixFA †]

Derive the EM updates for a mixture of factor analysers.

Exercise 20.3 [Deriving the second principal component]

a. Let

$$J(\boldsymbol{v}_2, \boldsymbol{z}_2) = \frac{1}{n} \sum_{i=1}^{n} (\boldsymbol{x}_i - z_{i1}\boldsymbol{v}_1 - z_{i2}\boldsymbol{v}_2)^T (\boldsymbol{x}_i - z_{i1}\boldsymbol{v}_1 - z_{i2}\boldsymbol{v}_2) \tag{20.161}$$

Show that $\frac{\partial J}{\partial \boldsymbol{z}_2} = 0$ yields $z_{i2} = \boldsymbol{v}_2^T \boldsymbol{x}_i$.

b. Show that the value of $\boldsymbol{v}_2$ that minimizes

$$\tilde{J}(\boldsymbol{v}_2) = -\boldsymbol{v}_2^T \mathbf{C} \boldsymbol{v}_2 + \lambda_2(\boldsymbol{v}_2^T \boldsymbol{v}_2 - 1) + \lambda_{12}(\boldsymbol{v}_2^T \boldsymbol{v}_1 - 0) \tag{20.162}$$

is given by the eigenvector of $\mathbf{C}$ with the second largest eigenvalue. Hint: recall that $\mathbf{C}\boldsymbol{v}_1 = \lambda_1 \boldsymbol{v}_1$ and $\frac{\partial \boldsymbol{x}^T \mathbf{A} \boldsymbol{x}}{\partial \boldsymbol{x}} = (\mathbf{A} + \mathbf{A}^T)\boldsymbol{x}$.

Exercise 20.4 [Deriving the residual error for PCA †]

a. Prove that

$$||\boldsymbol{x}_i - \sum_{j=1}^{K} z_{ij}\boldsymbol{v}_j||^2 = \boldsymbol{x}_i^T\boldsymbol{x}_i - \sum_{j=1}^{K} \boldsymbol{v}_j^T\boldsymbol{x}_i\boldsymbol{x}_i^T\boldsymbol{v}_j \tag{20.163}$$

Hint: first consider the case $K = 2$. Use the fact that $\boldsymbol{v}_j^T\boldsymbol{v}_j = 1$ and $\boldsymbol{v}_j^T\boldsymbol{v}_k = 0$ for $k \neq j$. Also, recall $z_{ij} = \boldsymbol{x}_i^T\boldsymbol{v}_j$.

b. Now show that

$$J_K \triangleq \frac{1}{n}\sum_{i=1}^{n}\left(\boldsymbol{x}_i^T\boldsymbol{x}_i - \sum_{j=1}^{K}\boldsymbol{v}_j^T\boldsymbol{x}_i\boldsymbol{x}_i^T\boldsymbol{v}_j\right) = \frac{1}{n}\sum_{i=1}^{n}\boldsymbol{x}_i^T\boldsymbol{x}_i - \sum_{j=1}^{K}\lambda_j \tag{20.164}$$

Hint: recall $\boldsymbol{v}_j^T\mathbf{C}\boldsymbol{v}_j = \lambda_j\boldsymbol{v}_j^T\boldsymbol{v}_j = \lambda_j$.

c. If $K = d$ there is no truncation, so $J_d = 0$. Use this to show that the error from only using $K < d$ terms is given by

$$J_K = \sum_{j=K+1}^{d}\lambda_j \tag{20.165}$$

Hint: partition the sum $\sum_{j=1}^{d}\lambda_j$ into $\sum_{j=1}^{K}\lambda_j$ and $\sum_{j=K+1}^{d}\lambda_j$.

Exercise 20.5 [PCA via successive deflation]

Let $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k$ be the first k eigenvectors with largest eigenvalues of $\mathbf{C} = \frac{1}{n}\mathbf{X}^T\mathbf{X}$, where $\mathbf{X}$ is the centered $N \times D$ design matrix; these are known as the principal basis vectors. These satisfy

$$\boldsymbol{v}_j^T\boldsymbol{v}_k = \begin{cases} 0 & \text{if } j \neq k \\ 1 & \text{if } j = k \end{cases} \tag{20.166}$$

We will construct a method for finding the $\boldsymbol{v}_j$ sequentially.

As we showed in class, $\boldsymbol{v}_1$ is the first principal eigenvector of $\mathbf{C}$, and satisfies $\mathbf{C}\boldsymbol{v}_1 = \lambda_1\boldsymbol{v}_1$. Now define $\tilde{\boldsymbol{x}}_i$ as the orthogonal projection of $\boldsymbol{x}_i$ onto the space orthogonal to $\boldsymbol{v}_1$:

$$\tilde{\boldsymbol{x}}_i = \mathbf{P}_{\perp\boldsymbol{v}_1}\,\boldsymbol{x}_i = (\mathbf{I} - \boldsymbol{v}_1\boldsymbol{v}_1^T)\boldsymbol{x}_i \tag{20.167}$$

Define $\tilde{\mathbf{X}} = [\tilde{\boldsymbol{x}}_1; ...; \tilde{\boldsymbol{x}}_n]$ as the **deflated matrix** of rank $d-1$, which is obtained by removing from the d dimensional data the component that lies in the direction of the first principal direction:

$$\tilde{\mathbf{X}} = (\mathbf{I} - \boldsymbol{v}_1\boldsymbol{v}_1^T)^T\mathbf{X} = (\mathbf{I} - \boldsymbol{v}_1\boldsymbol{v}_1^T)\mathbf{X} \tag{20.168}$$

a. Using the facts that $\mathbf{X}^T\mathbf{X}\boldsymbol{v}_1 = n\lambda_1\boldsymbol{v}_1$ (and hence $\boldsymbol{v}_1^T\mathbf{X}^T\mathbf{X} = n\lambda_1\boldsymbol{v}_1^T$) and $\boldsymbol{v}_1^T\boldsymbol{v}_1 = 1$, show that the covariance of the deflated matrix is given by

$$\tilde{\mathbf{C}} \triangleq \frac{1}{n}\tilde{\mathbf{X}}^T\tilde{\mathbf{X}} = \frac{1}{n}\mathbf{X}^T\mathbf{X} - \lambda_1\boldsymbol{v}_1\boldsymbol{v}_1^T \tag{20.169}$$

b. Let $\boldsymbol{u}$ be the principal eigenvector of $\tilde{\mathbf{C}}$. Explain why $\boldsymbol{u} = \boldsymbol{v}_2$. (You may assume $\boldsymbol{u}$ is unit norm.)

c. Suppose we have a simple method for finding the leading eigenvector and eigenvalue of a pd matrix, denoted by $[\lambda, \boldsymbol{u}] = f(\mathbf{C})$. Write some pseudo code for finding the first K principal basis vectors of $\mathbf{X}$ that only uses the special f function and simple vector arithmetic, i.e., your code should not use SVD or the `eig` function. Hint: this should be a simple iterative routine that takes 2–3 lines to write. The input is $\mathbf{C}$, K and the function f, the output should be $\boldsymbol{v}_j$ and λ_j for $j = 1 : K$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Exercise 20.6 [PPCA variance terms]

Recall that in the PPCA model, $\mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}$. We will show that this model correctly captures the variance of the data along the principal axes, and approximates the variance in all the remaining directions with a single average value σ^2.

Consider the variance of the predictive distribution $p(\boldsymbol{x})$ along some direction specified by the unit vector $\boldsymbol{v}$, where $\boldsymbol{v}^T\boldsymbol{v} = 1$, which is given by $\boldsymbol{v}^T C\boldsymbol{v}$.

a. First suppose $\boldsymbol{v}$ is orthogonal to the principal subspace. and hence $\boldsymbol{v}^T\mathbf{U} = \mathbf{0}$. Show that $\boldsymbol{v}^T\mathbf{C}\boldsymbol{v} = \sigma^2$.
b. Now suppose $\boldsymbol{v}$ is parallel to the principal subspace. and hence $\boldsymbol{v} = \boldsymbol{u}_i$ for some eigenvector $\boldsymbol{u}_i$. Show that $\boldsymbol{v}^T\mathbf{C}\boldsymbol{v} = (\lambda_i - \sigma^2) + \sigma^2 = \lambda_i$.

Exercise 20.7 [Posterior inference in PPCA †]

Derive $p(\boldsymbol{z}_n|\boldsymbol{x}_n)$ for the PPCA model.

Exercise 20.8 [Imputation in a FA model †]

Derive an expression for $p(\boldsymbol{x}_h|\boldsymbol{x}_v, \boldsymbol{\theta})$ for a FA model, where $\boldsymbol{x} = (\boldsymbol{x}_h, \boldsymbol{x}_v)$ is a partition of the data vector.

Exercise 20.9 [Efficiently evaluating the PPCA density]

Derive an expression for $p(\boldsymbol{x}|\hat{\mathbf{W}}, \hat{\sigma}^2)$ for the PPCA model based on plugging in the MLEs and using the matrix inversion lemma.

21 Clustering

21.1 Introduction

Clustering is a very common form of unsupervised learning. There are two main kinds of methods. In the first approach, the input is a set of data samples $\mathcal{D} = \{\boldsymbol{x}_n : n = 1 : N\}$, where $\boldsymbol{x}_n \in \mathcal{X}$, where typically $\mathcal{X} = \mathbb{R}^D$. In the second approach, the input is an $N \times N$ pairwise dissimilarity metric $D_{ij} \geq 0$. In both cases, the goal is to assign similar data points to the same cluster.

As is often the case with unsupervised learning, it is hard to evaluate the quality of a clustering algorithm. If we have labels for some of the data, we can use the similarity (or equality) between the labels of two data points as a metric for determining if the two inputs "should" be assigned to the same cluster or not. If we don't have labels, but the method is based on a generative model of the data, we can use log likelihood as a metric. We will see examples of both approaches below.

21.1.1 Evaluating the output of clustering methods

> The validation of clustering structures is the most difficult and frustrating part of cluster analysis. Without a strong effort in this direction, cluster analysis will remain a black art accessible only to those true believers who have experience and great courage. — Jain and Dubes [JD88]

Clustering is an unsupervised learning technique, so it is hard to evaluate the quality of the output of any given method [Kle02; LWG12]. If we use probabilistic models, we can always evaluate the likelihood of the data, but this has two drawbacks: first, it does not directly assess any clustering that is discovered by the model; and second, it does not apply to non-probabilistic methods. So now we discuss some performance measures not based on likelihood.

Intuitively, the goal of clustering is to assign points that are similar to the same cluster, and to ensure that points that are dissimilar are in different clusters. There are several ways of measuring these quantities e.g., see [JD88; KR90]. However, these internal criteria may be of limited use. An alternative is to rely on some external form of data with which to validate the method. For example, if we have labels for each object, then we can assume that objects with the same label are similar. We can then use the metrics we discuss below to quantify the quality of the clusters. (If we do not have labels, but we have a reference clustering, we can derive labels from that clustering.)



Figure 21.1: Three clusters with labeled objects inside.

21.1.1.1 Purity

Let N_{ij} be the number of objects in cluster i that belong to class j, and let $N_i = \sum_{j=1}^{C} N_{ij}$ be the total number of objects in cluster i. Define $p_{ij} = N_{ij}/N_i$; this is the empirical distribution over class labels for cluster i. We define the **purity** of a cluster as $p_i \triangleq \max_j p_{ij}$, and the overall purity of a clustering as

$$\text{purity} \triangleq \sum_i \frac{N_i}{N} p_i \tag{21.1}$$

For example, in Figure 21.1, we have that the purity is

$$\frac{6}{17}\frac{5}{6} + \frac{6}{17}\frac{4}{6} + \frac{5}{17}\frac{3}{5} = \frac{5+4+3}{17} = 0.71 \tag{21.2}$$

The purity ranges between 0 (bad) and 1 (good). However, we can trivially achieve a purity of 1 by putting each object into its own cluster, so this measure does not penalize for the number of clusters.

21.1.1.2 Rand index

Let $U = \{u_1, \ldots, u_R\}$ and $V = \{v_1, \ldots, v_C\}$ be two different partitions of the N data points. For example, U might be the estimated clustering and V is reference clustering derived from the class labels. Now define a 2×2 contingency table, containing the following numbers: TP is the number of pairs that are in the same cluster in both U and V (true positives); TN is the number of pairs that are in the different clusters in both U and V (true negatives); FN is the number of pairs that are in the different clusters in U but the same cluster in V (false negatives); and FP is the number of pairs that are in the same cluster in U but different clusters in V (false positives). A common summary statistic is the **Rand index**:

$$R \triangleq \frac{TP + TN}{TP + FP + FN + TN} \tag{21.3}$$

This can be interpreted as the fraction of clustering decisions that are correct. Clearly $0 \leq R \leq 1$.

For example, consider Figure 21.1, The three clusters contain 6, 6 and 5 points, so the number of "positives" (i.e., pairs of objects put in the same cluster, regardless of label) is

$$TP + FP = \binom{6}{2} + \binom{6}{2} + \binom{5}{2} = 40 \tag{21.4}$$

Of these, the number of true positives is given by

$$TP = \binom{5}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2} = 20 \tag{21.5}$$

where the last two terms come from cluster 3: there are $\binom{3}{2}$ pairs labeled C and $\binom{2}{2}$ pairs labeled A. So $FP = 40 - 20 = 20$. Similarly, one can show $FN = 24$ and $TN = 72$. So the Rand index is $(20 + 72)/(20 + 20 + 24 + 72) = 0.68$.

The Rand index only achieves its lower bound of 0 if $TP = TN = 0$, which is a rare event. One can define an **adjusted Rand index** [HA85] as follows:

$$AR \triangleq \frac{\text{index} - \text{expected index}}{\text{max index} - \text{expected index}} \tag{21.6}$$

Here the model of randomness is based on using the generalized hyper-geometric distribution, i.e., the two partitions are picked at random subject to having the original number of classes and objects in each, and then the expected value of $TP + TN$ is computed. This model can be used to compute the statistical significance of the Rand index.

The Rand index weights false positives and false negatives equally. Various other summary statistics for binary decision problems, such as the F-score (Section 5.1.4), can also be used.

21.1.1.3 Mutual information

Another way to measure cluster quality is to compute the mutual information between two candidate partitions U and V, as proposed in [VD99]. To do this, let $p_{UV}(i,j) = \frac{|u_i \cap v_j|}{N}$ be the probability that a randomly chosen object belongs to cluster u_i in U and v_j in V. Also, let $p_U(i) = |u_i|/N$ be the be the probability that a randomly chosen object belongs to cluster u_i in U; define $p_V(j) = |v_j|/N$ similarly. Then we have

$$\mathbb{I}(U,V) = \sum_{i=1}^{R}\sum_{j=1}^{C} p_{UV}(i,j) \log \frac{p_{UV}(i,j)}{p_U(i)p_V(j)} \tag{21.7}$$

This lies between 0 and $\min\{\mathbb{H}(U), \mathbb{H}(V)\}$. Unfortunately, the maximum value can be achieved by using lots of small clusters, which have low entropy. To compensate for this, we can use the **normalized mutual information**,

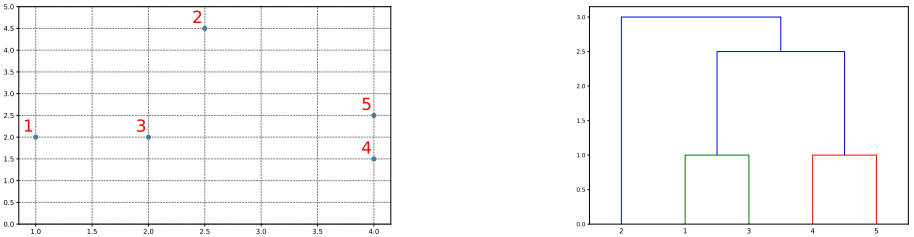
$$NMI(U,V) \triangleq \frac{\mathbb{I}(U,V)}{(\mathbb{H}(U) + \mathbb{H}(V))/2} \tag{21.8}$$

This lies between 0 and 1. A version of this that is adjusted for chance (under a particular random data model) is described in [VEB09]. Another variant, called **variation of information**, is described in [Mei05].

21.2 Hierarchical agglomerative clustering

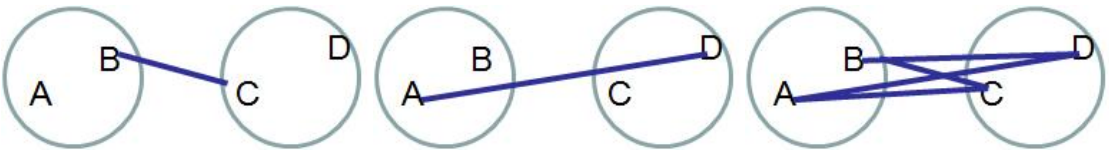
A common form of clustering is known as **hierarchical agglomerative clustering** or **HAC**. The input to the algorithm is an $N \times N$ dissimilarity matrix $D_{ij} \geq 0$, and the output is a tree structure in which groups i and j with small disimilarity are grouped together in a hierarchical fashion.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 21.2: (a) An example of single link clustering using city block distance. Pairs (1,3) and (4,5) are both distance 1 apart, so get merged first. (b) The resulting dendrogram. Adapted from Figure 7.5 of [Alp04]. Generated by agglomDemo.ipynb.



(a) (b) (c)

Figure 21.3: Illustration of (a) Single linkage. (b) Complete linkage. (c) Average linkage.

For example, consider the set of 5 inputs points in Figure 21.2(a), $\boldsymbol{x}_n \in \mathbb{R}^2$. We will use **city block distance** between the points to define the dissimilarity, i.e.,

$$d_{ij} = \sum_{k=1}^{2} |x_{ik} - x_{jk}| \tag{21.9}$$

We start with a tree with N leaves, each corresponding to a cluster with a single data point. Next we compute the pair of points that are closest, and merge them. We see that (1,3) and (4,5) are both distance 1 apart, so they get merged first. We then measure the dissimilarity between the sets $\{1, 3\}$, $\{4, 5\}$ and $\{2\}$ using some measure (details below), and group them, and repeat. The result is a binary tree known as a **dendrogram**, as shown in Figure 21.2(b). By cutting this tree at different heights, we can induce a different number of (nested) clusters. We give more details below.

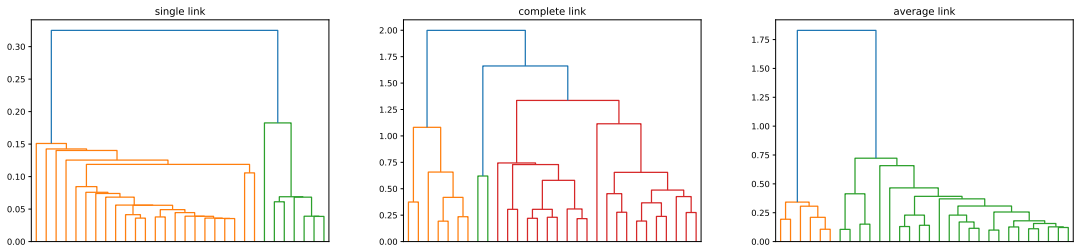
21.2.1 The algorithm

Agglomerative clustering starts with N groups, each initially containing one object, and then at each step it merges the two most similar groups until there is a single group, containing all the data. See Algorithm 21.1 for the pseudocode. Since picking the two most similar clusters to merge takes $O(N^2)$ time, and there are $O(N)$ steps in the algorithm, the total running time is $O(N^3)$. However, by using a priority queue, this can be reduced to $O(N^2 \log N)$ (see e.g., [MRS08, ch. 17] for details).

Algorithm 21.1: Agglomerative clustering

Initialize clusters as singletons: **for** $i \leftarrow 1$ **to** n **do** $C_i \leftarrow \{i\}$

Initialize set of clusters available for merging: $S \leftarrow \{1, \ldots, n\}$; **repeat**
  Pick 2 most similar clusters to merge: $(j, k) \leftarrow \arg\min_{j,k \in S} d_{j,k}$
  Create new cluster $C_\ell \leftarrow C_j \cup C_k$
  Mark j and k as unavailable: $S \leftarrow S \setminus \{j, k\}$
  if $C_\ell \neq \{1, \ldots, n\}$ **then**
    Mark ℓ as available, $S \leftarrow S \cup \{\ell\}$
  foreach $i \in S$ **do**
    Update dissimilarity matrix $d(i, \ell)$
until *no more clusters are available for merging*



(a) *(b)* *(c)*

Figure 21.4: Hierarchical clustering of yeast gene expression data. (a) Single linkage. (b) Complete linkage. (c) Average linkage. Generated by hclust_yeast_demo.ipynb.

There are actually three variants of agglomerative clustering, depending on how we define the dissimilarity between groups of objects. We give the details below.

21.2.1.1 Single link

In **single link clustering**, also called **nearest neighbor clustering**, the distance between two groups G and H is defined as the distance between the two closest members of each group:

$$d_{SL}(G, H) = \min_{i \in G, i' \in H} d_{i,i'} \tag{21.10}$$

See Figure 21.3(a).

The tree built using single link clustering is a minimum spanning tree of the data, which is a tree that connects all the objects in a way that minimizes the sum of the edge weights (distances). To see this, note that when we merge two clusters, we connect together the two closest members of the clusters; this adds an edge between the corresponding nodes, and this is guaranteed to be the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

"lightest weight" edge joining these two clusters. And once two clusters have been merged, they will never be considered again, so we cannot create cycles. As a consequence of this, we can actually implement single link clustering in $O(N^2)$ time, whereas the other variants take $O(N^3)$ time.

21.2.1.2 Complete link

In **complete link clustering**, also called **furthest neighbor clustering**, the distance between two groups is defined as the distance between the two most distant pairs:

$$d_{CL}(G, H) = \max_{i \in G, i' \in H} d_{i,i'} \tag{21.11}$$

See Figure 21.3(b).

Single linkage only requires that a single pair of objects be close for the two groups to be considered close together, regardless of the similarity of the other members of the group. Thus clusters can be formed that violate the **compactness** property, which says that all the observations within a group should be similar to each other. In particular if we define the **diameter** of a group as the largest dissimilarity of its members, $d_G = \max_{i \in G, i' \in G} d_{i,i'}$, then we can see that single linkage can produce clusters with large diameters. Complete linkage represents the opposite extreme: two groups are considered close only if all of the observations in their union are relatively similar. This will tend to produce clusterings with small diameter, i.e., compact clusters. (Compare Figure 21.4(a) with Figure 21.4(b).)

21.2.1.3 Average link

In practice, the preferred method is **average link clustering**, which measures the average distance between all pairs:

$$d_{avg}(G, H) = \frac{1}{n_G n_H} \sum_{i \in G} \sum_{i' \in H} d_{i,i'} \tag{21.12}$$

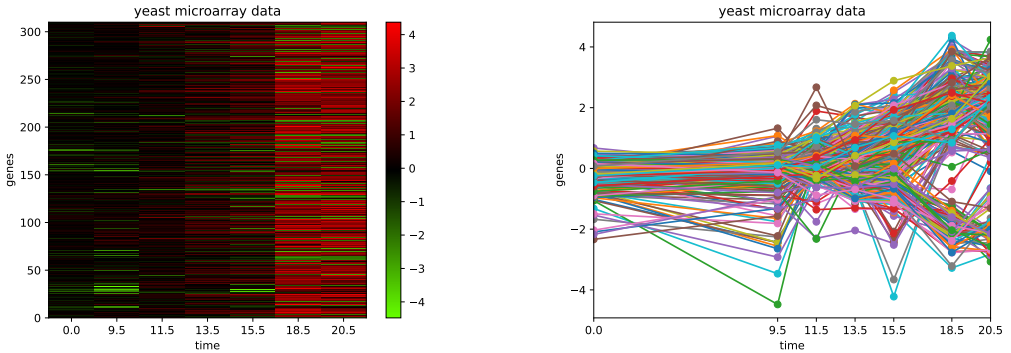
where n_G and n_H are the number of elements in groups G and H. See Figure 21.3(c).

Average link clustering represents a compromise between single and complete link clustering. It tends to produce relatively compact clusters that are relatively far apart. (See Figure 21.4(c).) However, since it involves averaging of the $d_{i,i'}$'s, any change to the measurement scale can change the result. In contrast, single linkage and complete linkage are invariant to monotonic transformations of $d_{i,i'}$, since they leave the relative ordering the same.

21.2.2 Example

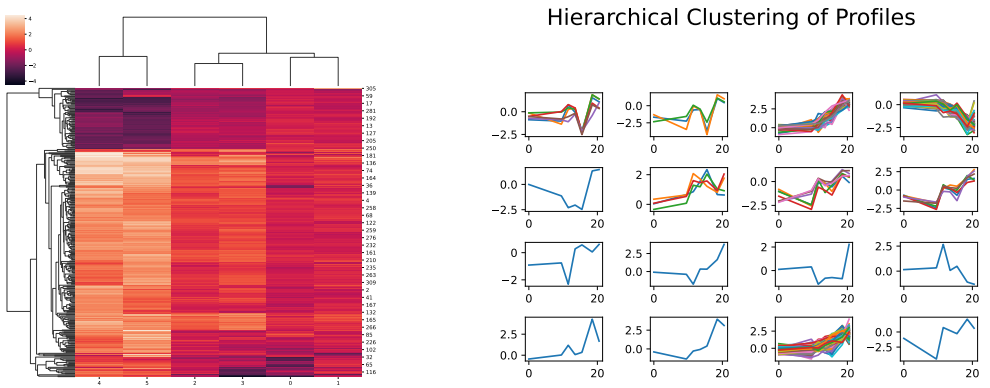
Suppose we have a set of time series measurements of the expression levels for $N = 300$ genes at $T = 7$ points. Thus each data sample is a vector $\boldsymbol{x}_n \in \mathbb{R}^7$. See Figure 21.5 for a visualization of the data. We see that there are several kinds of genes, such as those whose expression level goes up monotonically over time (in response to a given stimulus), those whose expression level goes down monotonically, and those with more complex response patterns.

Suppose we use Euclidean distance to compute a pairwise dissimilarity matrix, $\mathbf{D} \in \mathbb{R}^{300 \times 300}$, and apply HAC using average linkage. We get the dendrogram in Figure 21.6(a). If we cut the tree at



(a) (b)

Figure 21.5: (a) Some yeast gene expression data plotted as a heat map. (b) Same data plotted as a time series. Generated by yeast_ data_ viz.ipynb.



(a) (b)

Figure 21.6: Hierarchical clustering applied to the yeast gene expression data. (a) The rows are permuted according to a hierarchical clustering scheme (average link agglomerative clustering), in order to bring similar rows close together. (b) 16 clusters induced by cutting the average linkage tree at a certain height. Generated by hclust_ yeast_ demo.ipynb.

a certain height, we get the 16 clusters shown in Figure 21.6(b). The time series assigned to each cluster do indeed "look like" each other.

21.2.3 Extensions

There are many extensions to the basic HAC algorithm. For example, [Mon+21] present a more scalable version of the bottom up algorithm that builds sub-clusters in parallel. And [Mon+19] discusses an online version of the algorithm, that can cluster data as it arrives, while reconsidering

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

previous clustering decisions (as opposed to only making greedy decisions). Under certain assumptions, this can provably recover the true underlying structure. This can be useful for clustering "mentions" of "entities" (such as people or things) in streaming text data. (This problem is called **entity discovery**.)

21.3 K means clustering

There are several problems with hierarchical agglomerative clustering (Section 21.2). First, it takes $O(N^3)$ time (for the average link method), making it hard to apply to big datasets. Second, it assumes that a dissimilarity matrix has already been computed, whereas the notion of "similarity" is often unclear and needs to be learned. Third, it is just an algorithm, not a model, and so it is hard to evaluate how good it is. That is, there is no clear objective that it is optimizing.

In this section, we discuss the **K-means algorithm** [Mac67; Llo82], which addresses these issues. First, it runs in $O(NKT)$ time, where T is the number of iterations. Second, it computes similarity in terms of Euclidean distance to learned cluster centers $\boldsymbol{\mu}_k \in \mathbb{R}^D$, rather than requiring a dissimilarity matrix. Third, it optimizes a well-defined cost function, as we will see.

21.3.1 The algorithm

We assume there are K cluster centers $\boldsymbol{\mu}_k \in \mathbb{R}^D$, so we can cluster the data by assigning each data point $\boldsymbol{x}_n \in \mathbb{R}^D$ to it closest center:

$$z_n^* = \arg\min_k ||\boldsymbol{x}_n - \boldsymbol{\mu}_k||_2^2 \tag{21.13}$$

Of course, we don't know the cluster centers, but we can estimate them by computing the average value of all points assigned to them:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n:z_n=k} \boldsymbol{x}_n \tag{21.14}$$

We can then iterate these steps to convergence.

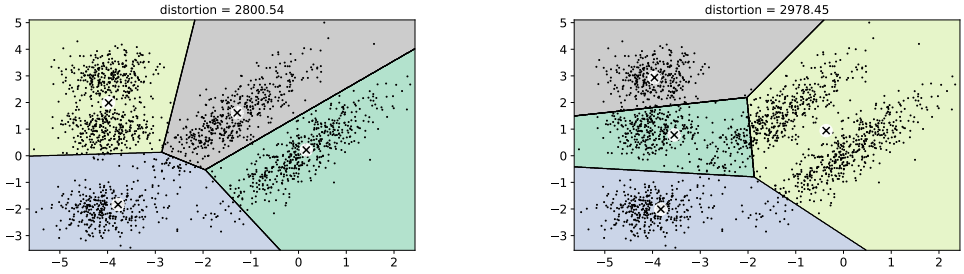
More formally, we can view this as finding a local minimum of the following cost function, known as the **distortion**:

$$J(\mathbf{M}, \mathbf{Z}) = \sum_{n=1}^{N} ||\boldsymbol{x}_n - \boldsymbol{\mu}_{z_n}||^2 = ||\mathbf{X} - \mathbf{Z}\mathbf{M}^\mathsf{T}||_F^2 \tag{21.15}$$

where $\mathbf{X} \in \mathbb{R}^{N \times D}$, $\mathbf{Z} \in [0,1]^{N \times K}$, and $\mathbf{M} \in \mathbb{R}^{D \times K}$ contains the cluster centers $\boldsymbol{\mu}_k$ in its columns. K-means optimizes this using alternating minimization. (This is closely related to the EM algorithm for GMMs, as we discuss in Section 21.4.1.1.)

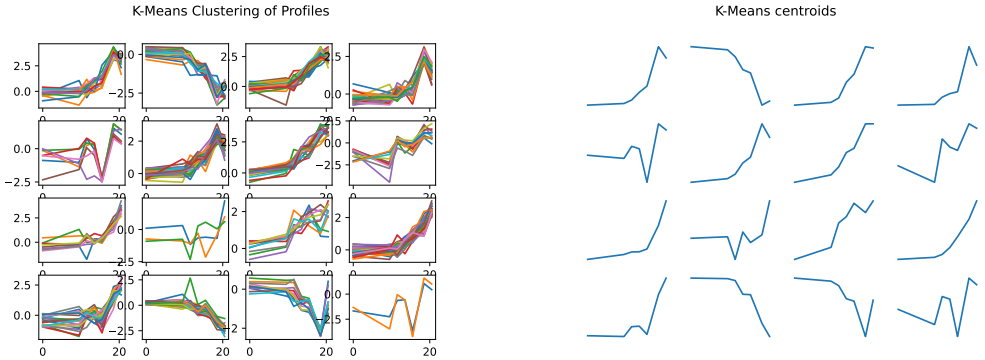
21.3.2 Examples

In this section, we give some examples of K-means clustering.



(a) (b)

Figure 21.7: Illustration of K-means clustering in 2d. We show the result of using two different random seeds. Adapted from Figure 9.5 of [Gér19]. Generated by kmeans_voronoi.ipynb.



(a) (b)

Figure 21.8: Clustering the yeast data from Figure 21.5 using K-means clustering with $K = 16$. (a) Visualizing all the time series assigned to each cluster. (b) Visualizing the 16 cluster centers as prototypical time series. Generated by kmeans_yeast_demo.ipynb.

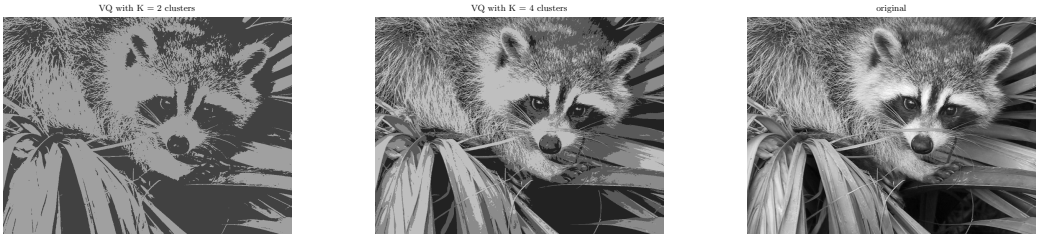
21.3.2.1 Clustering points in the 2d plane

Figure 21.7 gives an illustration of K-means clustering applied to some points in the 2d plane. We see that the method induces a **Voronoi tessellation** of the points. The resulting clustering is sensitive to the initialization. Indeed, we see that the lower quality clustering on the right has higher distortion. By default, sklearn uses 10 random restarts (combined with the K-means++ initialization described in Section 21.3.4) and returns the clustering with lowest distortion. (In sklearn, the distortion is called the "inertia".)

21.3.2.2 Clustering gene expression time series data from yeast cells

In Figure 21.8, we show the result of applying K-means clustering with $K = 16$ to the 300×7 yeast time series matrix shown in Figure 21.5. We see that time series that "look similar" to each other are

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b) (c)

Figure 21.9: An image compressed using vector quantization with a codebook of size K. (a) $K = 2$. (b) $K = 4$. (c) Original uncompressed image. Generated by vqDemo.ipynb.

assigned to the same cluster. We also see that the centroid of each cluster is a reasonabe summary all the data points assigned to that cluster. Finally we notice that group 6 was not used, since no points were assigned to it. However, this is just an accident of the initialization process, and we are not guaranteed to get the same clustering, or number of clusters, if we repeat the algorithm. (We discuss good ways to initialize the method in Section 21.3.4, and ways to choose K in Section 21.3.7.)

21.3.3 Vector quantization

Suppose we want to perform lossy compression of some real-valued vectors, $\boldsymbol{x}_n \in \mathbb{R}^D$. A very simple approach to this is to use **vector quantization** or **VQ**. The basic idea is to replace each real-valued vector $\boldsymbol{x}_n \in \mathbb{R}^D$ with a discrete symbol $z_n \in \{1, \ldots, K\}$, which is an index into a **codebook** of K prototypes, $\boldsymbol{\mu}_k \in \mathbb{R}^D$. Each data vector is encoded by using the index of the most similar prototype, where similarity is measured in terms of Euclidean distance:

$$\text{encode}(\boldsymbol{x}_n) = \arg\min_k ||\boldsymbol{x}_n - \boldsymbol{\mu}_k||^2 \tag{21.16}$$

We can define a cost function that measures the quality of a codebook by computing the **reconstruction error** or **distortion** it induces:

$$J \triangleq \frac{1}{N} \sum_{n=1}^{N} ||\boldsymbol{x}_n - \text{decode}(\text{encode}(\boldsymbol{x}_n))||^2 = \frac{1}{N} \sum_{n=1}^{N} ||\boldsymbol{x}_n - \boldsymbol{\mu}_{z_n}||^2 \tag{21.17}$$

where $\text{decode}(k) = \boldsymbol{\mu}_k$. This is exactly the cost function that is minimized by the K-means algorithm.

Of course, we can achieve zero distortion if we assign one prototype to every data vector, by using $K = N$ and assigning $\boldsymbol{\mu}_n = \boldsymbol{x}_n$. However, this does not compress the data at all. In particular, it takes $O(NDB)$ bits, where N is the number of real-valued data vectors, each of length D, and B is the number of bits needed to represent a real-valued scalar (the quantization accuracy to represent each $\boldsymbol{x}_n$).

We can do better by detecting similar vectors in the data, creating prototypes or centroids for them, and then representing the data as deviations from these prototypes. This reduces the space

requirement to $O(N \log_2 K + KDB)$ bits. The $O(N \log_2 K)$ term arises because each of the N data vectors needs to specify which of the K codewords it is using; and the $O(KDB)$ term arises because we have to store each codebook entry, each of which is a D-dimensional vector. When N is large, the first term dominates the second, so we can approximate the **rate** of the encoding scheme (number of bits needed per object) as $O(\log_2 K)$, which is typically much less than $O(DB)$.

One application of VQ is to image compression. Consider the 200×320 pixel image in Figure 21.9; we will treat this as a set of $N = 64,000$ scalars. If we use one byte to represent each pixel (a gray-scale intensity of 0 to 255), then $B = 8$, so we need $NB = 512,000$ bits to represent the image in uncompressed form. For the compressed image, we need $O(N \log_2 K)$ bits. For $K = 4$, this is about 128kb, a factor of 4 compression, yet it results in negligible perceptual loss (see Figure 21.9(b)).

Greater compression could be achieved if we modeled spatial correlation between the pixels, e.g., if we encoded 5x5 blocks (as used by JPEG). This is because the residual errors (differences from the model's predictions) would be smaller, and would take fewer bits to encode. This shows the deep connection between data compression and density estimation. See the sequel to this book, [Mur23], for more information.

21.3.4 The K-means++ algorithm

K-means is optimizing a non-convex objective, and hence needs to be initialized carefully. A simple approach is to pick K data points at random, and to use these as the initial values for $\boldsymbol{\mu}_k$. We can improve on this by using **multiple restarts**, i.e., we run the algorithm multiple times from different random starting points, and then pick the best solution. However, this can be slow.

A better approach is to pick the centers sequentially so as to try to "cover" the data. That is, we pick the initial point uniformly at random, and then each subsequent point is picked from the remaining points, with probability proportional to its squared distance to the point's closest cluster center. That is, at iteration t, we pick the next cluster center $\boldsymbol{\mu}_t$ to be $\boldsymbol{x}_n$ with probability

$$p(\boldsymbol{\mu}_t = \boldsymbol{x}_n) = \frac{D_{t-1}(\boldsymbol{x}_n)}{\sum_{n'=1}^{N} D_{t-1}(\boldsymbol{x}_{n'})} \tag{21.18}$$

where

$$D_{t-1}(\boldsymbol{x}) = \min_{k=1}^{t-1} ||\boldsymbol{x} - \boldsymbol{\mu}_k||_2^2 \tag{21.19}$$

is the squared distance of $\boldsymbol{x}$ to the closest existing centroid up to time step $t-1$. Thus points that are far away from a centroid are more likely to be picked, thus reducing the distortion. This is known as **farthest point clustering** [Gon85], or **K-means++** [AV07; Bah+12; Bac+16; BLK17; LS19a]. Surprisingly, this simple trick can be shown to guarantee that the recontruction error is never more than $O(\log K)$ worse than optimal [AV07].

21.3.5 The K-medoids algorithm

There is a variant of K-means called **K-medoids** algorithm, in which we estimate each cluster center $\boldsymbol{\mu}_k$ by choosing the data example $\boldsymbol{x}_n \in \mathcal{X}$ whose average dissimilarity to all other points in that cluster is minimal; such a point is known as a **medoid**. By contrast, in K-means, we take averages over points $\boldsymbol{x}_n \in \mathbb{R}^D$ assigned to the cluster to compute the center. K-medoids can be more robust to

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

outliers (although that issue can also be tackled by using mixtures of Student distributions, instead of mixtures of Gaussians). More importantly, K-medoids can be applied to data that does not live in $\mathbb{R}^D$, where averaging may not be well defined. In K-medoids, the input to the algorithm is $N \times N$ pairwise distance matrix, $D(n, n')$, not an $N \times D$ feature matrix.

The classic algorithm for solving the K-medoids is the **partitioning around medoids** or **PAM** method [KR87]. In this approach, at each iteration, we loop over all K medoids. For each medoid m, we consider each non-medoid point o, swap m and o, and recompute the cost (sum of all the distances of points to their medoid). If the cost has decreased, we keep this swap. The running time of this algorithm is $O(N^2KT)$, where T is the number of iterations.

There is also a simpler and faster method, known as the **Voronoi iteration** method due to [PJ09]. In this approach, at each iteration, we have two steps, similar to K-means. First, for each cluster k, look at all the points currently assigned to that cluster, $S_k = \{n : z_n = k\}$, and then set m_k to be the index of the medoid of that set. (To find the medoid requires examining all $|S_k|$ candidate points, and choosing the one that has the smallest sum of distances to all the other points in S_k.) Second, for each point n, assign it to its closest medoid, $z_n = \text{argmin}_k D(n, k)$. The pseudo-code is given in Algorithm 21.2.

Algorithm 21.2: K-medoids algorithm

Initialize $m_{1:K}$ as a random subset of size K from $\{1, \ldots, N\}$
repeat
  $z_n = \text{argmin}_k d(n, m_k)$ for $n = 1 : N$
  $m_k = \text{argmin}_{n:z_n=k} \sum_{n':z_{n'}=k} d(n, n')$ for $k = 1 : K$
until *converged*

21.3.6 Speedup tricks

K-means clustering takes $O(NKI)$ time, where I is the number of iterations, but we can reduce the constant factors using various tricks. For example, [Elk03] shows how to use the triangle inequality to keep track of lower and upper bounds for the distances between inputs and the centroids; this can be used to eliminate some redundant computations. Another approach is to use a minibatch approximation, as proposed in [Scu10]. This can be significantly faster, although can result in slightly worse loss (see Figure 21.10).

21.3.7 Choosing the number of clusters K

In this section, we discuss how to choose the number of clusters K in the K-means algorithm and other related methods.

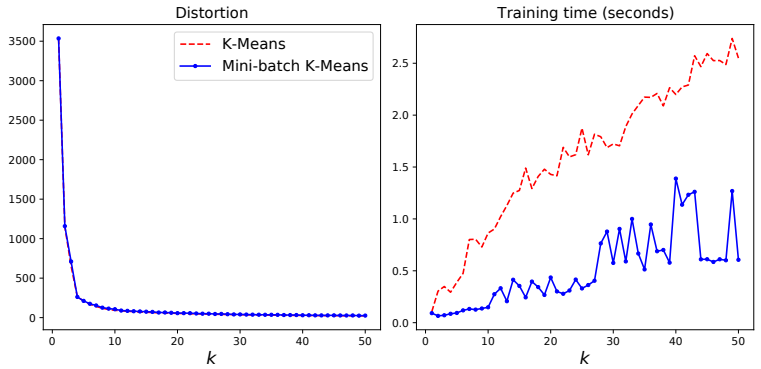
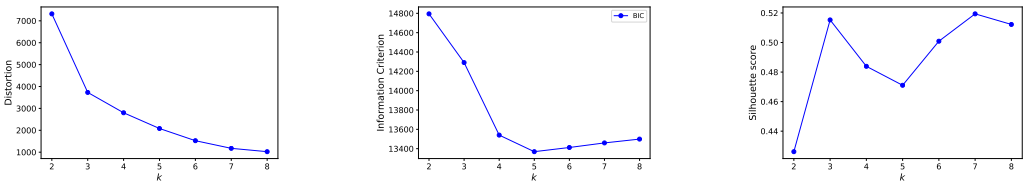


Figure 21.10: Illustration of batch vs mini-batch K-means clustering on the 2d data from Figure 21.7. Left: distortion vs K. Right: Training time vs K. Adapted from Figure 9.6 of [Gér19]. Generated by kmeans_minibatch.ipynb.



(a) (b) (c)

Figure 21.11: Performance of K-means and GMM vs K on the 2d dataset from Figure 21.7. (a) Distortion on validation set vs K. Generated by kmeans_silhouette.ipynb. (b) BIC vs K. Generated by gmm_2d.ipynb. (c) Silhouette score vs K. Generated by kmeans_silhouette.ipynb.

21.3.7.1 Minimizing the distortion

Based on our experience with supervised learning, a natural choice for picking K is to pick the value that minimizes the reconstruction error on a validation set, defined as follows:

$$\text{err}(\mathcal{D}_{\text{valid}}, K) = \frac{1}{|\mathcal{D}_{\text{valid}}|} \sum_{n \in \mathcal{D}_{\text{valid}}} ||\boldsymbol{x}_n - \hat{\boldsymbol{x}}_n||_2^2 \tag{21.20}$$

where $\hat{\boldsymbol{x}}_n = \text{decode}(\text{encode}(\boldsymbol{x}_n))$ is the reconstruction of $\boldsymbol{x}_n$.

Unfortunately, this technique will not work. Indeed, as we see in Figure 21.11a, the distortion monotonically decreases with K. To see why, note that the K-means model is a degenerate density model which consists of K "spikes" at the $\boldsymbol{\mu}_k$ centers. As we increase K, we "cover" more of the input space. Hence any given input point is more likely to find a close prototype to accurately represent it as K increases, thus decreasing reconstruction error. Thus unlike with supervised learning, *we cannot use reconstruction error on a validation set as a way to select the best unsupervised model.* (This comment also applies to picking the dimensionality for PCA, see Section 20.1.4.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

21.3.7.2 Maximizing the marginal likelihood

A method that does work is to use a proper probabilistic model, such as a GMM, as we describe in Section 21.4.1. We can then use the log marginal likelihood (LML) of the data to perform model selection.

We can approximate the LML using the BIC score as we discussed in Section 5.2.5.1. From Equation (5.59), we have

$$\text{BIC}(K) = \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}_k) - \frac{D_K}{2}\log(N) \tag{21.21}$$

where D_K is the number of parameters in a model with K clusters, and $\hat{\boldsymbol{\theta}}_K$ is the MLE. We see from Figure 21.11b that this exhibits the typical U-shaped curve, where the penalty decreases and then increases.

The reason this works is that each cluster is associated with a Gaussian distribution that fills a volume of the input space, rather than being a degenerate spike. Once we have enough clusters to cover the true modes of the distribution, the Bayesian Occam's razor (Section 5.2.3) kicks in, and starts penalizing the model for being unncessarily complex.

See Section 21.4.1.3 for more discussion of Bayesian model selection for mixture models.

21.3.7.3 Silhouette coefficient

In this section, we describe a common heuristic method for picking the number of clusters in a K-means clustering model. This is designed to work for spherical (not elongated) clusters. First we define the **silhouette coefficient** of an instance i to be $sc(i) = (b_i - a_i)/\max(a_i, b_i)$, where a_i is the mean distance to the other instances in cluster $k_i = \text{argmin}_k ||\boldsymbol{\mu}_k - \boldsymbol{x}_i||$, and b_i is the mean distance to the other instances in the next closest cluster, $k'_i = \text{argmin}_{k \neq k_i} ||\boldsymbol{\mu}_k - \boldsymbol{x}_i||$. Thus a_i is a measure of compactness of i's cluster, and b_i is a measure of distance between the clusters. The silhouette coefficient varies from -1 to +1. A value of +1 means the instance is close to all the members of its cluster, and far from other clusters; a value of 0 means it is close to a cluster boundary; and a value of -1 means it may be in the wrong cluster. We define the **silhouette score** of a clustering K to be the mean silhouette coefficient over all instances.

In Figure 21.11a, we plot the distortion vs K for the data in Figure 21.7. As we explained above, it goes down monotonically with K. There is a slight "**kink**" or "**elbow**" in the curve at $K = 3$, but this is hard to detect. In Figure 21.11c, we plot the silhouette score vs K. Now we see a more prominent peak at $K = 3$, although it seems $K = 7$ is almost as good. See Figure 21.12 for a comparison of some of these clusterings.

It can be informative to look at the individual silhouette coefficients, and not just the mean score. We can plot these in a **silhouette diagram**, as shown in Figure 21.13, where each colored region corresponds to a different cluster. The dotted vertical line is the average coefficient. Clusters with many points to the left of this line are likely to be of low quality. We can also use the silhouette diagram to look at the size of each cluster, even if the data is not 2d.

21.3.7.4 Incrementally growing the number of mixture components

An alternative to searching for the best value of K is to incrementally "grow" GMMs. We can start with a small value of K, and after each round of training, we consider splitting the cluster with the

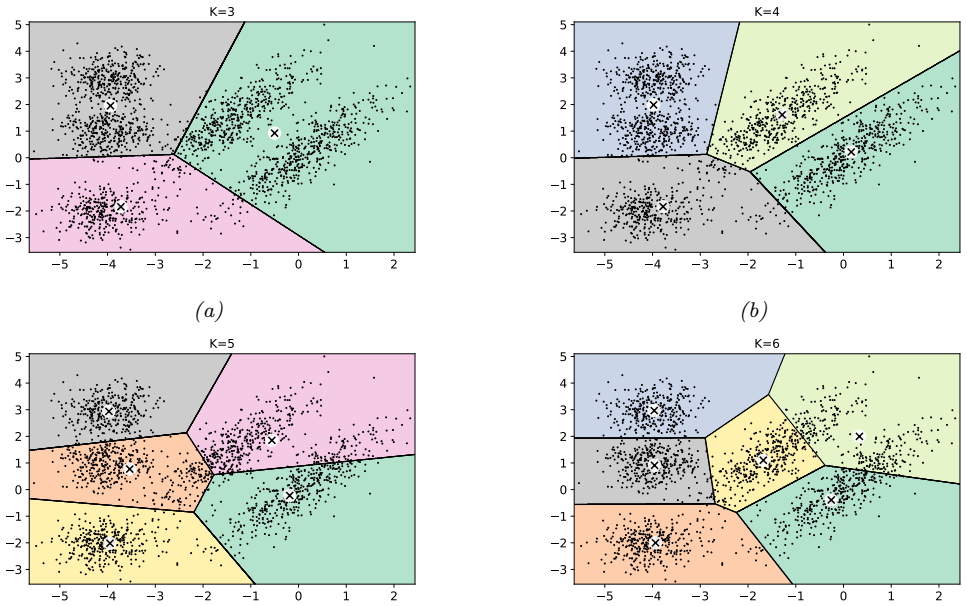


Figure 21.12: Voronoi diagrams for K-means for different K on the 2d dataset from Figure 21.7. Generated by kmeans_silhouette.ipynb.

highest mixing weight into two, with the new centroids being random perturbations of the original centroid, and the new scores being half of the old scores. If a new cluster has too small a score, or too narrow a variance, it is removed. We continue in this way until the desired number of clusters is reached. See [FJ02] for details.

21.3.7.5 Sparse estimation methods

Another approach is to pick a large value of K, and then to use some kind of sparsity-promoting prior or inference method to "kill off" unneeded mixture components, such as variational Bayes. See the sequel to this book, [Mur23], for details.

21.4 Clustering using mixture models

We have seen how the K-means algorithm can be used to cluster data vectors in $\mathbb{R}^D$. However, this method assumes that all clusters have the same spherical shape, which is a very restrictive assumption. In addition, K-means assumes that all clusters can be described by Gaussians in the input space, so it cannot be applied to discrete data. By using mixture models (Section 3.5), we can overcome both of these problems, as we illustrate below.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

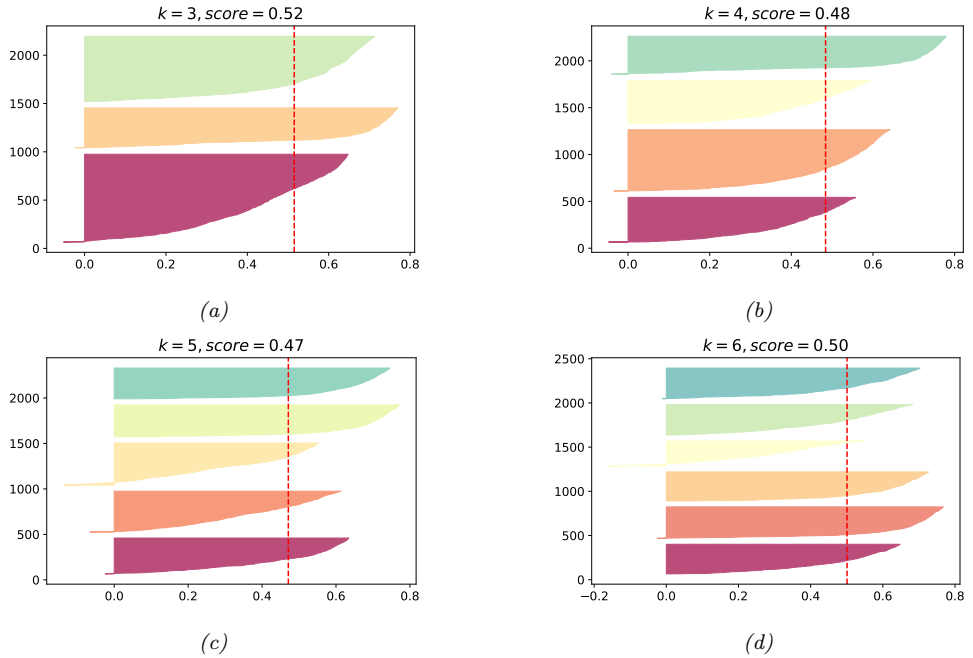


Figure 21.13: Silhouette diagrams for K-means for different K on the 2d dataset from Figure 21.7. Generated by kmeans_silhouette.ipynb.

21.4.1 Mixtures of Gaussians

Recall from Section 3.5.1 that a Gaussian mixture model (GMM) is a model of the form

$$p(\boldsymbol{x}|\boldsymbol{\theta}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{21.22}$$

If we know the model parameters $\boldsymbol{\theta} = (\boldsymbol{\pi}, \{\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\})$, we can use Bayes rule to compute the responsibility (posterior membership probability) of cluster k for data point $\boldsymbol{x}_n$:

$$r_{nk} \triangleq p(z_n = k|\boldsymbol{x}_n, \boldsymbol{\theta}) = \frac{p(z_n = k|\boldsymbol{\theta})p(\boldsymbol{x}_n|z_n = k, \boldsymbol{\theta})}{\sum_{k'=1}^{K} p(z_n = k'|\boldsymbol{\theta})p(\boldsymbol{x}_n|z_n = k', \boldsymbol{\theta})} \tag{21.23}$$

Given the responsibilities, we can compute the most probable cluster assignment as follows:

$$\hat{z}_n = \arg\max_k r_{nk} = \arg\max_k \left[\log p(\boldsymbol{x}_n|z_n = k, \boldsymbol{\theta}) + \log p(z_n = k|\boldsymbol{\theta})\right] \tag{21.24}$$

This is known as **hard clustering**.

21.4.1.1 K-means is a special case of EM

We can estimate the parameters of a GMM using the EM algorithm (Section 8.7.3). It turns out that the K-means algorithm is a special case of this algorithm, in which we make two approximations:

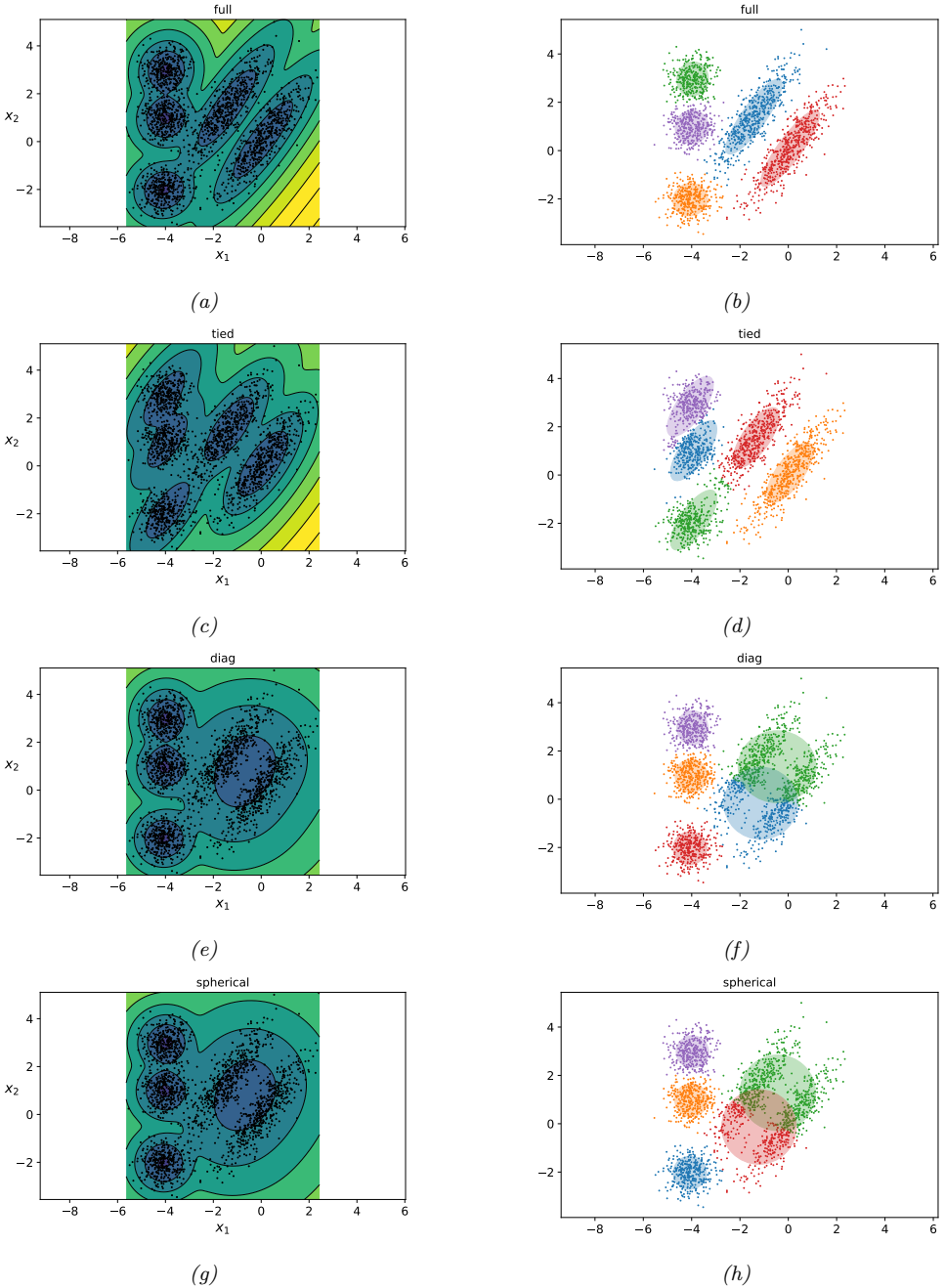


Figure 21.14: Some data in 2d fit using a GMM with $K = 5$ components. Left column: marginal distribution $p(\boldsymbol{x})$. Right column: visualization of each mixture distribution, and the hard assignment of points to their most likely cluster. (a-b) Full covariance. (c-d) Tied full covariance. (e-f) Diagonal covairance, (g-h) Spherical covariance. Color coding is arbitrary. Generated by gmm_2d.ipynb.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

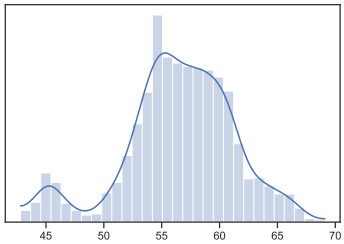


Figure 21.15: Some 1d data, with a kernel density estimate superimposed. Adapted from Figure 6.2 of [Mar18]. Generated by gmm_identifiability_pymc3.ipynb.

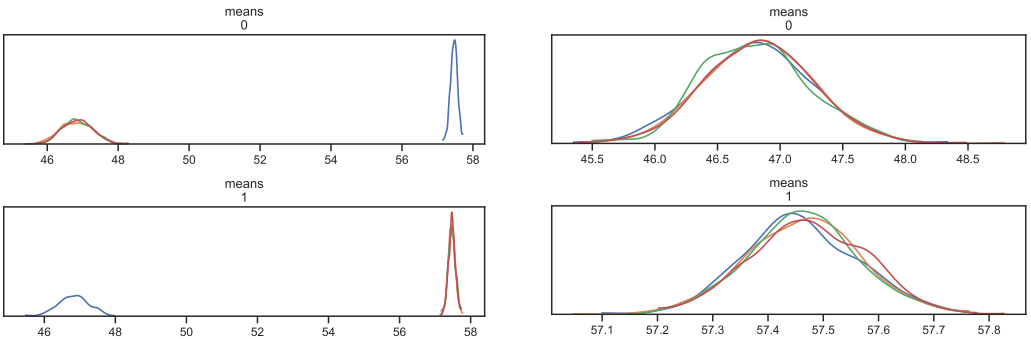


Figure 21.16: Illustration of the label switching problem when performing posterior inference for the parameters of a GMM. We show a KDE estimate of the posterior marginals derived from 1000 samples from 4 HMC chains. (a) Unconstrained model. Posterior is symmetric. (b) Constrained model, where we add a penalty to ensure $\mu_0 < \mu_1$. Adapted from Figure 6.6-6.7 of [Mar18]. Generated by gmm_identifiability_pymc3.ipynb.

we fix $\boldsymbol{\Sigma}_k = \mathbf{I}$ and $\pi_k = 1/K$ for all the clusters (so we just have to estimate the means $\boldsymbol{\mu}_k$), and we approximate the E step, by replacing the soft responsibilities with hard cluster assignments, i.e., we compute $z_n^* = \text{argmax}_k \, r_{nk}$, and set $r_{nk} \approx \mathbb{I}(k = z_n^*)$ instead of using the soft responsibilities, $r_{nk} = p(z_n = k|\boldsymbol{x}_n, \boldsymbol{\theta})$. With this approximation, the weighted MLE problem in Equation (8.165) of the M step reduces to Equation (21.14), so we recover K-means.

However, the assumption that all the clusters have the same spherical shape is very restrictive. For example, Figure 21.14 shows the marginal density and clustering induced using different shaped covariance matrices for some 2d data. We see that modeling this particular dataset needs the ability to capture off-diagonal covariance for some clusters (top row).

21.4.1.2 Unidentifiability and label switching

Note that we are free to permute the labels in a mixture model without changing the likelihood. This is called the **label switching problem**, and is an example of **non-identifiability** of the parameters.

This can cause problems if we wish to perform posterior inference over the parameters (as opposed to just computing the MLE or a MAP estimate). For example, suppose we fit a GMM with $K = 2$ components to the data in Figure 21.15 using HMC. The posterior over the means, $p(\mu_1, \mu_2|\mathcal{D})$, is shown in Figure 21.16a. We see that the marginal posterior for each component, $p(\mu_k|\mathcal{D})$, is bimodal. This reflects the fact that there are two equally good explanations of the data: either $\mu_1 \approx 47$ and $\mu_2 \approx 57$, or vice versa.

To break symmetry, we can add an **ordering constraint** on the centers, so that $\mu_1 < \mu_2$. We can do this by adding a penalty or potential function to the objective if the constraint is violated. More precisely, the penalized log joint becomes

$$\ell'(\boldsymbol{\theta}) = \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) + \phi(\boldsymbol{\mu}) \tag{21.25}$$

where

$$\phi(\boldsymbol{\mu}) = \begin{cases} -\infty & \text{if } \mu_1 < \mu_0 \\ 0 & \text{otherwise} \end{cases} \tag{21.26}$$

This has the desired effect, as shown in Figure 21.16b.

A more general approach is to apply a transformation to the parameters, to ensure identifiability. That is, we sample the parameters $\boldsymbol{\theta}$ from a proposal, and then apply an invertible transformation $\boldsymbol{\theta}' = f(\boldsymbol{\theta})$ to them before computing the log joint, $\log p(\mathcal{D}, \boldsymbol{\theta}')$. To account for the change of variables (Section 2.8.3), we add the log of the determinant of the Jacobian. In the case of a 1d ordering transformation, which just sorts its inputs, the determinant of the Jacobian is 1, so the log-det-Jacobian term vanishes.

Unfortunately, this approach does not scale to more than 1 dimensional problems, because there is no obvious way to enforce an ordering constraint on the centers $\boldsymbol{\mu}_k$.

21.4.1.3 Bayesian model selection

Once we have a reliable way to ensure identifiability, we can use Bayesian model selection techniques from Section 5.2.2 to select the number of clusters K. In Figure 21.17, we illustrate the results of fitting a GMM with $K = 3 - 6$ components to the data in Figure 21.15. We use the ordering transform on the means, and perform inference using HMC. We compare the resulting GMM model fits to the fit of a kernel density estimate (Section 16.3), which often over-smooths the data. We see fairly strong evidence for two bumps, corresponding to different subpopulations.

We can compare these models more quantitatively by computing their WAIC scores (widely applicable information criterion) which is an approximation to the log marginal likelihood (see [Wat10; Wat13; VGG17] for details). The results are shown in Figure 21.18. (This kind of visualization was proposed in [McE20, p228].) We see that the model with $K = 6$ scores significantly higher than the other models, although $K = 5$ is a close second. This is consistent with the plot in Figure 21.17.

21.4.2 Mixtures of Bernoullis

As we discussed in Section 3.5.2, we can use a mixtures of Bernoullis to cluster binary data. The model has the form

$$p(\boldsymbol{y}|z = k, \boldsymbol{\theta}) = \prod_{d=1}^{D} \text{Ber}(y_d|\mu_{dk}) = \prod_{d=1}^{D} \mu_{dk}^{y_d}(1 - \mu_{dk})^{1-y_d} \tag{21.27}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

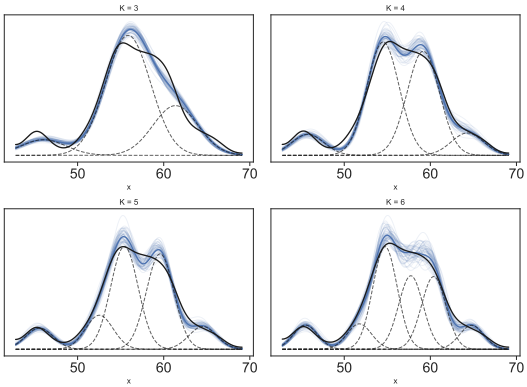


Figure 21.17: Fitting GMMs with different numbers of clusters K to the data in Figure 21.15. Black solid line is KDE fit. Solid blue line is posterior mean; feint blue lines are posterior samples. Dotted lines show the individual Gaussian mixture components, evaluated by plugging in their posterior mean parameters. Adapted from Figure 6.8 of [Mar18]. Generated by gmm_chooseK_pymc3.ipynb.

Here μ_{dk} is the probability that bit d turns on in cluster k. We can fit this model with EM, SGD, MCMC, etc. See Figure 3.13 for an example, where we cluster some binarized MNIST digits.

21.5 Spectral clustering *

In this section, we discuss an approach to clustering based on eigenvalue analysis of a pairwise similarity matrix. It uses the eigenvectors to derive feature vectors for each datapoint, which are then clustered using a feature-based clustering method, such as K-means (Section 21.3). This is known as **spectral clustering** [SM00; Lux07].

21.5.1 Normalized cuts

We start by creating a weighted undirected graph $\mathbf{W}$, where each data vector is a node, and the strength of the $i-j$ edge is a measure of similarity. Typically we only connected a node to its most similar neighbors, to ensure the graph is sparse, which speeds computation.

Our goal is to find K clusters of similar points. That is, we want to find a **graph partition** into $S_1, \ldots, S_K$ disjoint sets of nodes so as to minimize some kind of cost.

Our first attempt at a cost function is to compute the weight of connections between nodes in each cluster to nodes outside each cluster:

$$\text{cut}(S_1, \ldots, S_K) \triangleq \frac{1}{2} \sum_{k=1}^{K} W(S_k, \overline{S}_k) \tag{21.28}$$

where $W(A, B) \triangleq \sum_{i \in A, j \in B} w_{ij}$ and $\overline{S}_k = V \setminus S_k$ is the complement of S_k, where $V = \{1, \ldots, N\}$.

Unfortunately the optimal solution to this often just partitions off a single node from the rest, since that minimizes the weight of the cut. To prevent this, we can divide by the size of each set, to

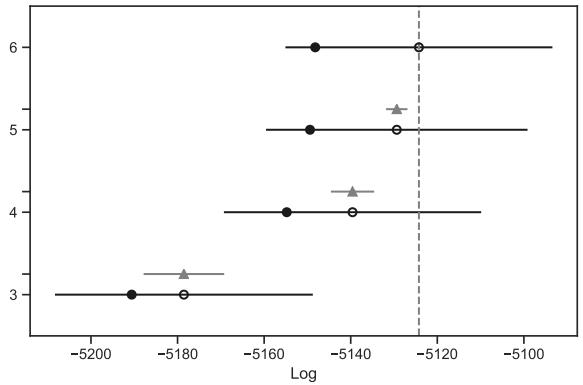


Figure 21.18: WAIC scores for the different GMMs. The empty circle is the posterior mean WAIC score for each model, and the black lines represent the standard error of the mean. The solid circle is the in-sample deviance of each model, i.e., the unpenalized log-likelihood. The dashed vertical line corresponds to the maximum WAIC value. The gray triangle is the difference in WAIC score for that model compared to the best model. Adapted from Figure 6.10 of [Mar18]. Generated by gmm_chooseK_pymc3.ipynb.

get the following objective, known as the **normalized cut**:

$$\text{Ncut}(S_1, \ldots, S_K) \triangleq \frac{1}{2} \sum_{k=1}^{K} \frac{\text{cut}(S_k, \overline{S}_k)}{\text{vol}(S_k)} \tag{21.29}$$

where $\text{vol}(A) \triangleq \sum_{i \in A} d_i$ is the total weight of set A and $d_i = \sum_{j=1}^{N} w_{ij}$ is the weighted degree of node i. This splits the graph into K clusters such that nodes within each cluster are similar to each other, but are different to nodes in other clusters.

We can formulate the Ncut problem in terms of searching for binary vectors $\boldsymbol{c}_i \in \{0,1\}^N$ that minimizes the above objective, where $c_{ik} = 1$ iff point i belongs to cluster k. Unfortunately this is NP-hard [WW93]. Below we discuss a continuous relaxation of the problem based on eigenvector methods that is easier to solve.

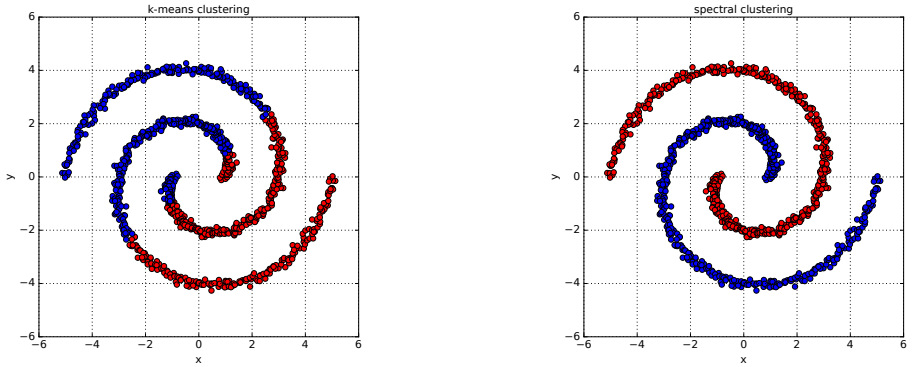
21.5.2 Eigenvectors of the graph Laplacian encode the clustering

In Section 20.4.9.2, we discussed the graph Laplacian, which is defined as $\mathbf{L} \triangleq \mathbf{D} - \mathbf{W}$, where $\mathbf{W}$ is a symmetric weight matrix for the graph, and $\mathbf{D} = \text{diag}(d_i)$ is a diagonal matrix containing the weighted degree of each node, $d_i = \sum_j w_{ij}$. To get some intuition as to why $\mathbf{L}$ might be useful for graph-based clustering, we note the following result.

Theorem 21.5.1. *The set of eigenvectors of* $\mathbf{L}$ *with eigenvalue 0 is spanned by the indicator vectors* $\mathbf{1}_{S_1}, \ldots, \mathbf{1}_{S_K}$*, where* S_k *are the* K *connected components of the graph.*

Proof. Let us start with the case $K = 1$. If $\boldsymbol{f}$ is an eigenvector with eigenvalue 0, then $0 = \sum_{ij} w_{ij}(f_i - f_j)^2$. If two nodes are connected, so $w_{ij} > 0$, we must have that $f_i = f_j$. Hence $\boldsymbol{f}$ is constant for all vertices which are connected by a path in the graph. Now suppose $K > 1$. In this

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 21.19: Results of clustering some data. (a) K-means. (b) Spectral clustering. Generated by spectral_clustering_demo.ipynb.

case, $\mathbf{L}$ will be block diagonal. A similar argument to the above shows that we will have K indicator functions, which "select out" the connected components. □

This suggests the following clustering algorithm. Compute the eigenvectors and values of $\mathbf{L}$, and let $\mathbf{U}$ be an $N \times K$ matrix with the K eigenvectors with smallest eigenvalue in its columns. (Fast methods for computing such "bottom" eigenvectors are discussed in [YHJ09]). Let $\boldsymbol{u}_i \in \mathbb{R}^K$ be the i'th *row* of $\mathbf{U}$. Since these $\boldsymbol{u}_i$ will be piecewise constant, we can apply K-means clustering (Section 21.3) to them to recover the connected components. (Note that the vectors $\boldsymbol{u}_i$ are the same as those computed by Laplacian eigenmaps discussed in Section 20.4.9.)

Real data may not exhibit such clean block structure, but one can show, using results from perturbation theory, that the eigenvectors of a "perturbed" Laplacian will be close to these ideal indicator functions [NJW01].

In practice, it is important to normalize the graph Laplacian, to account for the fact that some nodes are more highly connected than others. One way to do this (proposed in [NJW01]) is to create a symmetric matrix

$$\mathbf{L}_{sym} \triangleq \mathbf{D}^{-\frac{1}{2}}\mathbf{L}\mathbf{D}^{-\frac{1}{2}} = \mathbf{I} - \mathbf{D}^{-\frac{1}{2}}\mathbf{W}\mathbf{D}^{-\frac{1}{2}} \tag{21.30}$$

This time the eigenspace of 0 is spanned by $\mathbf{D}^{\frac{1}{2}}\mathbf{1}_{S_k}$. This suggests the following algorithm: find the smallest K eigenvectors of $\mathbf{L}_{sym}$, stack them into the matrix $\mathbf{U}$, normalize each *row* to unit norm by creating $t_{ij} = u_{ij}/\sqrt{(\sum_k u_{ik}^2)}$ to make the matrix $\mathbf{T}$, cluster the rows of $\mathbf{T}$ using K-means, then infer the partitioning of the original points.

21.5.3 Example

Figure 21.19 illustrates the method in action. In Figure 21.19(a), we see that K-means does a poor job of clustering, since it implicitly assumes each cluster corresponds to a spherical Gaussian. Next we try spectral clustering. We compute a dense similarity matrix $\mathbf{W}$ using a Gaussian kernel,

$W_{ij} = \exp(-\frac{1}{2\sigma^2}||\boldsymbol{x}_i - \boldsymbol{x}_j||_2^2)$. We then compute the first two eigenvectors of the normalized Laplacian L_{sym}. From this we infer the clustering using K-means, with $K = 2$; the results are shown in Figure 21.19(b).

21.5.4 Connection with other methods

Spectral clustering is closely related to several other methods for unsupervised learning, some of which we discuss below.

21.5.4.1 Connection with kPCA

Spectral clustering is closely related to kernel PCA (Section 20.4.6). In particular, kPCA uses the largest eigenvectors of $\mathbf{W}$; these are equivalent to the smallest eigenvectors of $\mathbf{I} - \mathbf{W}$. This is similar to the above method, which computes the smallest eigenvectors of $\mathbf{L} = \mathbf{D} - \mathbf{W}$. See [Ben+04a] for details. In practice, spectral clustering tends to give better results than kPCA.

21.5.4.2 Connection with random walk analysis

In practice we get better results by computing the eigenvectors of the normalized graph Laplacian. One way to normalize the graph Laplacian, which is used in [SM00; Mei01], is to define

$$\mathbf{L}_{rw} \triangleq \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{W} \tag{21.31}$$

One can show that for $\mathbf{L}_{rw}$, the eigenspace of 0 is again spanned by the indicator vectors $\mathbf{1}_{S_k}$ [Lux07], so we can perform clustering directly on the K smallest eigenvectors $\mathbf{U}$.

There is an interesting connection between this approach and random walks on a graph. First note that $\mathbf{P} = \mathbf{D}^{-1}\mathbf{W} = \mathbf{I} - \mathbf{L}_{rw}$ is a stochastic matrix, where $p_{ij} = w_{ij}/d_i$ can be interpreted as the probability of going from i to j. If the graph is connected and non-bipartite, it possesses a unique stationary distribution $\boldsymbol{\pi} = (\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_N)$, where $\pi_i = d_i/\text{vol}(V)$, and $\text{vol}(V) = \sum_i d_i$ is the sum of all the node degrees. Furthermore, one can show that for a partition of size 2,

$$\text{Ncut}(S, \overline{S}) = p(\overline{S}|S) + p(S|\overline{S}) \tag{21.32}$$

This means that we are looking for a cut such that a random walk spends more time transitioning to similar points, and rarely makes transitions from S to $\overline{S}$ or vice versa. This analysis can be extended to $K > 2$; for details, see [Mei01].

21.6 Biclustering *

In some cases, we have a data matrix $\mathbf{X} \in \mathbb{R}^{N_r \times N_c}$ and we want to cluster the rows *and* the columns; this is known as **biclustering** or **coclustering**. This is widely used in bioinformatics, where the rows often represent genes and the columns represent conditions. It can also be used for collaborative filtering, where the rows represent users and the columns represent movies.

A variety of ad hoc methods for biclustering have been proposed; see [MO04] for a review. In Section 21.6.1, we present a simple probabilistic generative model in which we assign a latent cluster id to each row, and a differnet latent cluster id to each column. In Section 21.6.2, we extend this to the case where each row can belong to multiple clusters, depending on which groups of features (columns) we choose to use to define the different groups of objects (rows).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

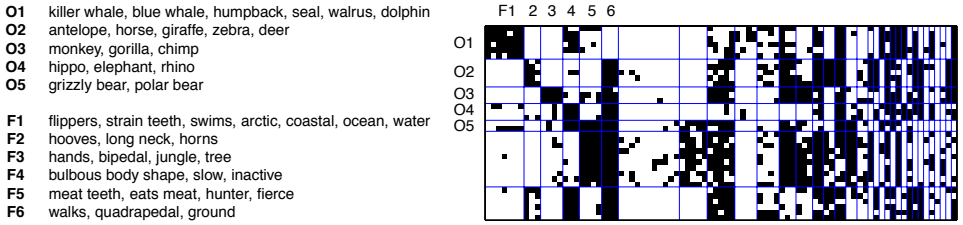


Figure 21.20: Illustration of biclustering. We show 5 of the 12 organism clusters, and 6 of the 33 feature clusters. The original data matrix is shown, partitioned according to the discovered clusters. From Figure 3 of [Kem+06]. Used with kind permission of Charles Kemp.

21.6.1 Basic biclustering

Here we present a simple probabilistic generative model for biclustering based on [Kem+06] (see also [SMM03] for a related approach). The idea is to associate each row and each column with a latent indicator, $u_i \in \{1, \ldots, N_u\}$, $v_j \in \{1, \ldots, N_v\}$, where N_u is the number of row clusters, and N_v is the number of column clusters. We then use the following generative model:

$$p(\mathbf{U}) = \prod_{i=1}^{N_r} \text{Unif}(u_i | \{1, \ldots, N_u\}) \tag{21.33}$$

$$p(\mathbf{V}) = \prod_{j=1}^{N_c} \text{Cat}(v_j | \{1, \ldots, N_v\}) \tag{21.34}$$

$$p(\mathbf{X}|\mathbf{U}, \mathbf{V}, \boldsymbol{\theta}) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} p(X_{ij} | \boldsymbol{\theta}_{u_i, v_j}) \tag{21.35}$$

where $\boldsymbol{\theta}_{a,b}$ are the parameters for row cluster a and column cluster b.

Figure 21.20 shows a simple example. The data has the form $X_{ij} = 1$ iff animal i has feature j, where $i = 1 : 50$ and $j = 1 : 85$. The animals represent whales, bears, horses, etc. The features represent properties of the habitat (jungle, tree, coastal), or anatomical properties (has teeth, quadripedal), or behavioral properties (swims, eats meat), etc. The method discovered 12 animal clusters and 33 feature clusters. ([Kem+06] use a Bayesian nonparametric method to infer the number of clusters.) For example, the O2 cluster is { antelope, horse, giraffe, zebra, deer }, which is characterized by feature clusters F2 = { hooves, long neck, horns} and F6 = { walks, quadripedal, ground }, whereas the O4 cluster is { hippo, elephant, rhino }, which is characterized by feature clusters F4 = { bulbous body shape, slow, inactive } and F6.

21.6.2 Nested partition models (Crosscat)

The problem with basic biclustering (Section 21.6.1) is that each object (row) can only belong to one cluster. Intuitively, an object can have multiple roles, and can be assigned to different clusters depending on which subset of features you use. For example, in the animal dataset, we may want to group the animals on the basis of anatomical features (e.g., mammals are warm blooded, reptiles are

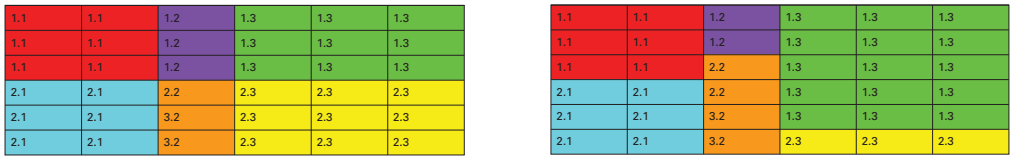


Figure 21.21: (a) Example of biclustering. Each row is assigned to a unique cluster, and each column is assigned to a unique cluster. (b) Example of multi-clustering using a nested partition model. The rows can belong to different clusters depending on which subset of column features we are looking at.

not), or on the basis of behavioral features (e.g., predators vs prey).

We now present a model that can capture this phenomenon. We illustrate the method with an example. Suppose we have a 6×6 matrix, with $N_u = 2$ row clusters and $N_v = 3$ column clusters. Furthermore, suppose the latent column assignments are as follows: $\boldsymbol{v} = [1, 1, 2, 3, 3, 3]$. This means we put columns 1 and 2 into group 1, column 3 into group 2, and columns 4 to 6 into group 3. For the columns that get clustered into group 1, we cluster the rows as follows: $\boldsymbol{u}_{:,1} = [1, 1, 1, 2, 2, 2]$; For the columns that get clustered into group 2, we cluster the rows as follows: $\boldsymbol{u}_{:,2} = [1, 1, 2, 2, 2, 2]$; and for the columns that get clustered into group 3, we cluster the rows as follows: $\boldsymbol{u}_{:,3} = [1, 1, 1, 1, 1, 2]$. The resulting partition is shown in Figure 21.21(b). We see that the clustering of the rows depends on which group of columns we choose to focus on.

Formally, we can define the model as follows:

$$p(\mathbf{U}) = \prod_{i=1}^{N_r} \prod_{l=1}^{N_v} \text{Unif}(u_{il} | \{1, \ldots, N_u\}) \tag{21.36}$$

$$p(\mathbf{V}) = \prod_{j=1}^{N_c} \text{Unif}(v_j | \{1, \ldots, N_v\}) \tag{21.37}$$

$$p(\mathbf{Z}|\mathbf{U}, \mathbf{V}) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} \mathbb{I}\left(Z_{ij} = (u_{i,v_j}, v_j)\right) \tag{21.38}$$

$$p(\mathbf{X}|\mathbf{Z}, \boldsymbol{\theta}) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} p(X_{ij} | \boldsymbol{\theta}_{z_{ij}}) \tag{21.39}$$

where $\boldsymbol{\theta}_{k,l}$ are the parameters for cocluster $k \in \{1, \ldots, N_u\}$ and $l \in \{1, \ldots, N_v\}$.

This model was independently proposed in [Sha+06; Man+16] who call it **crosscat** (for cross-categorization), in [Gua+10; CFD10], who call it **multi-clust**, and in [RG11], who call it **nested partitioning**. In all of these papers, the authors propose to use Dirichlet processes, to avoid the problem of estimating the number of clusters. Here we assume the number of clusters is known, and show the parameters explicitly, for notational simplicity.

Figure 21.22 illustrates the model applied to some binary data containing 22 animals and 106 features. The figure shows the (approximate) MAP partition. The first partition of the columns contains taxonomic features, such as "has bones", "is warm-blooded", "lays eggs", etc. This divides the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

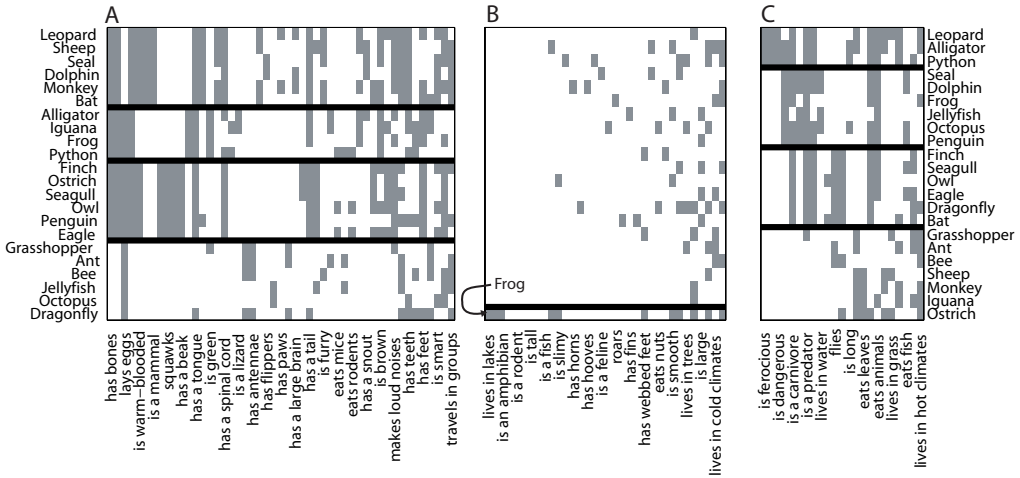


Figure 21.22: MAP estimate produced by the crosscat system when applied to a binary data matrix of animals (rows) by features (columns). See text for details. From Figure 7 of [Sha+06]. Used with kind permission of Vikash Mansingkha.

animals into birds, reptiles/ amphibians, mammals, and invertebrates. The second partition of the columns contains features that are treated as noise, with no apparent structure (except for the single row labeled "frog"). The third partition of the columns contains ecological features like "dangerous", "carnivorous", "lives in water", etc. This divides the animals into prey, land predators, sea predators and air predators. Thus each animal (row) can belong to a different cluster depending on what set of features are considered.

22 Recommender Systems

Recommender systems are systems which recommend **items** (such as movies, books, ads) to **users** based on various information, such as their past viewing/ purchasing behavior (e.g., which movies they rated high or low, which ads they clicked on), as well as optional "side information" such as demographics about the user, or information about the content of the item (e.g., its title, genre or price). Such systems are widely used by various internet companies, such as Facebook, Amazon, Netflix, Google, etc. In this chapter, we give a brief introduction to the topic. More details can be found in e.g., [DKK12; Pat12; Yan+14; AC16; Agg16; Zha+19b]..

22.1 Explicit feedback

In this section, we consider the simplest setting in which the user gives **explicit feedback** to the system in terms of a **rating**, such as +1 or -1 (for like/dislike) or a score from 1 to 5. Let $Y_{ui} \in \mathbb{R}$ be the rating that user u gives to item i. We can represent this as an $M \times N$ matrix, where M is the number of users, and N is the number of items. Typically this matrix will be very large but very sparse, since most users will not provide any feedback on most items. See Figure 22.1(a) for an example. We can also view this sparse matrix as a bipartite graph, where the weight of the $u - i$ edge is Y_{ui}. This reflects the fact that we are dealing with **relational data**, i.e., the values of u and i have no intrinsic meaning (they are just arbitrary indices), it is the fact that u and i are connected that matters.

If Y_{ui} is missing, it could be because user u has not interacted with item i, or it could be that they knew they wouldn't like it and so they chose not to engage with it. In the former case, some of the data is **missing at random**; in the latter case, the missingness is informative about the true value of Y_{ui}. (See e.g., [Mar+11] for further discussion of this point.) We will assume the data is missing at random, for simplicity.

22.1.1 Datasets

A famous example of an explicit ratings matrix was made available by the movie streaming company Netflix. In 2006, they released a large dataset of 100,480,507 movie ratings (on a scale of 1 to 5) from 480,189 users of 17,770 movies. Despite the large size of the training set, the ratings matrix is still 99% sparse (unknown). Along with the data, they offered a prize of $1M, known as the **Netflix Prize**, to any team that could predict the true ratings of a set of test (user, item) pairs more accurately than their incumbent system. The prize was claimed on September 21, 2009 by a team known as "BellKor's Pragmatic Chaos". They used an ensemble of different methods, as described in

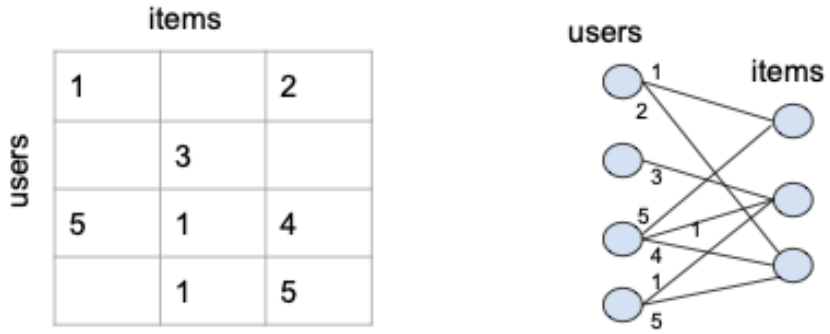


Figure 22.1: Example of a relational dataset represented as a sparse matrix (left) or a sparse bipartite graph (right). Values corresponding to empty cells (missing edges) are unknown. Rows 3 and 4 are similar to each other, indicating that users 3 and 4 might have similar preferences, so we can use the data from user 3 to predict user 4's preferences. However, user 1 seems quite different in their preferences, and seems to give low ratings to all items. For user 2, we have very little observed data, so it is hard to make reliable predictions.

[Kor09; BK07; FHK12]. However, a key component in their ensemble was the method described in Section 22.1.3.

Unfortunately the Netflix data is no longer available due to privacy concerns. Fortunately the **MovieLens** group at the University of Minnesota have released an anonymized public dataset of movie ratings, on a scale of 1-5, that can be used for research [HK15]. There are also various other public explicit ratings datasets, such as the **Jester** jokes dataset from [Gol+01] and the **BookCrossing** dataset from [Zie+05].

22.1.2 Collaborative filtering

The original approach to the recommendation problem is called **collaborative filtering** [Gol+92]. The idea is that users collaborate on recommending items by sharing their ratings with other users; then if u wants to know if they interact with i, they can see what ratings other users u' have given to i, and take a weighted average:

$$\hat{Y}_{ui} = \sum_{u': Y_{u',i} \neq ?} \text{sim}(u, u')\, Y_{u',i} \tag{22.1}$$

where we assume $Y_{u',i} =?$ if the entry is unknown. The traditional approach measured the similarity of two users by comparing the sets $S_u = \{Y_{u,i} \neq ? : i \in \mathcal{I}\}$ and $S_{u'} = \{Y_{u',i} \neq ? : i \in \mathcal{I}\}$, where $\mathcal{I}$ is the set of items. However, this can suffer from data sparsity. In Section 22.1.3 we discuss an approach based on learning dense embedding vectors for each item and each user, so we can compute similarity in a low dimensional feature space.

22.1.3 Matrix factorization

We can view the recommender problem as one of **matrix completion**, in which we wish to predict all the missing entries of $\mathbf{Y}$. We can formulate this as the following optimization problem:

$$\mathcal{L}(\mathbf{Z}) = \sum_{ij:Y_{ij} \neq ?} (Z_{ij} - Y_{ij})^2 = ||\mathbf{Z} - \mathbf{Y}||_F^2 \tag{22.2}$$

However, this is an under-specified problem, since there are an infinite number of ways of filling in the missing entries of $\mathbf{Z}$.

We need to add some constraints. Suppose we assume that $\mathbf{Y}$ is low rank. Then we can write it in the form $\mathbf{Z} = \mathbf{U}\mathbf{V}^\mathsf{T} \approx \mathbf{Y}$, where $\mathbf{U}$ is an $M \times K$ matrix, $\mathbf{V}$ is a $N \times K$ matrix, K is the rank of the matrix, M is the number of users, and N is the number of items. This corresponds to a prediction of the form by writing

$$\hat{y}_{ui} = \boldsymbol{u}_u^\mathsf{T} \boldsymbol{v}_i \tag{22.3}$$

This is called **matrix factorization**.

If we observe all the Y_{ij} entries, we can find the optimal $\mathbf{Z}$ using SVD (Section 7.5). However, when $\mathbf{Y}$ has missing entries, the corresponding objective is no longer convex, and does not have a unique optimum [SJ03]. We can fit this using **alternating least squares** (**ALS**), where we estimate $\mathbf{U}$ given $\mathbf{V}$ and then estimate $\mathbf{V}$ given $\mathbf{U}$ (for details, see e.g., [KBV09]). Alternatively we can just use SGD.

In practice, it is important to also allow for user-specific and item-specific baselines, by writing

$$\hat{y}_{ui} = \mu + b_u + c_i + \boldsymbol{u}_u^\mathsf{T} \boldsymbol{v}_i \tag{22.4}$$

This can capture the fact that some users might always tend to give low ratings and others may give high ratings; in addition, some items (e.g., very popular movies) might have unusually high ratings.

In addition, we can add some ℓ_2 regularization to the parameters to get the objective

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{ij:Y_{ij} \neq ?} (y_{ij} - \hat{y}_{ij})^2 + \lambda(b_u^2 + c_i^2 + ||\boldsymbol{u}_u||^2 + ||\boldsymbol{v}_i||^2) \tag{22.5}$$

We can optimize this using SGD by sampling a random (u, i) entry from the set of observed values, and performing the following updates:

$$b_u = b_u + \eta(e_{ui} - \lambda b_u) \tag{22.6}$$
$$c_i = c_i + \eta(e_{ui} - \lambda c_i) \tag{22.7}$$
$$\boldsymbol{u}_u = \boldsymbol{u}_u + \eta(e_{ui}\boldsymbol{v}_i - \lambda \boldsymbol{u}_u) \tag{22.8}$$
$$\boldsymbol{v}_i = \boldsymbol{v}_i + \eta(e_{ui}\boldsymbol{u}_u - \lambda \boldsymbol{v}_i) \tag{22.9}$$

where $e_{ui} = y_{ui} - \hat{y}_{ui}$ is the error term, and $\eta \geq 0$ is the learning rate. This approach was first proposed by Simon Funk, who was one of the first to do well in the early days of the Netflix competition.[1]

1. https://sifter.org/~simon/journal/20061211.html.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

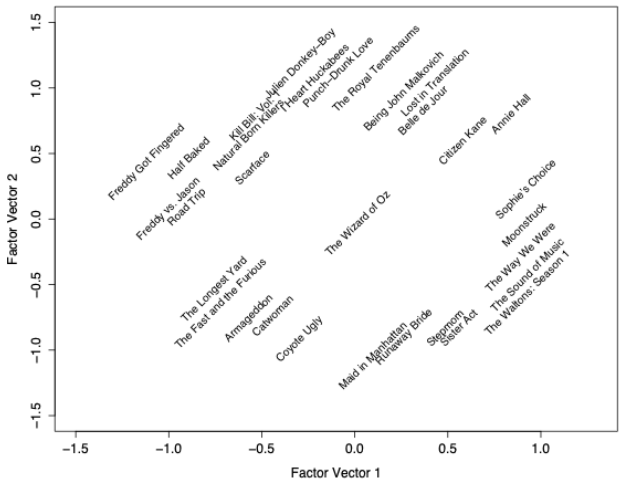


Figure 22.2: Visualization of the first two latent movie factors estimated from the Netflix challenge data. Each movie j is plotted at the location specified by $\boldsymbol{v}_j$. See text for details. From Figure 3 of [KBV09]. Used with kind permission of Yehuda Koren.

22.1.3.1 Probabilistic matrix factorization (PMF)

We can convert matrix factorization into a probabilistic model by defining

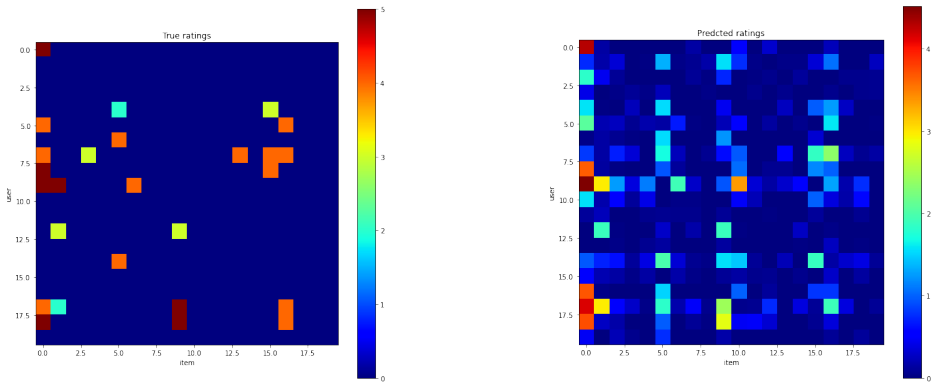
$$p(y_{ui} = y) = \mathcal{N}(y|\mu + b_u + c_i + \boldsymbol{u}_u^\mathsf{T}\boldsymbol{v}_i, \sigma^2) \tag{22.10}$$

This is known as **probabilistic matrix factorization** (**PMF**) [SM08]. The NLL of this model is equivalent to the matrix factorization objective in Equation (22.2). However, the probabilistic perspective allows us to generalize the model more easily. For example, we can capture the fact that the ratings are integers (often mostly 0s), and not reals, using a Poisson or negative Binomial likelihood (see e.g., [GOF18]). This is similar to exponential family PCA (Section 20.2.7), except that we view rows and columns symmetrically.

22.1.3.2 Example: Netflix

Suppose we apply PMF to the Netflix dataset using $K = 2$ latent factors. Figure 22.2 visualizes the learned embedding vectors $\boldsymbol{u}_i$ for a few movies. On the left of the plot we have low-brow humor and horror movies (*Half Baked*, *Freddy vs Jason*), and on the right we have more serious dramas (*Sophie's Choice*, *Moonstruck*). On the top we have critically acclaimed independent movies (*Punch-Drunk Love*, *I Heart Huckabees*), and on the bottom we have mainstream Hollywood blockbusters (*Armageddon*, *Runway Bride*). The *Wizard of Oz* is right in the middle of these axes, since it is in some senses an "average movie".

Users are embedded into the same spaces as movies. We can then predict the rating for any user-video pair using proximity in the latent embedding space.



(a) (b)

Figure 22.3: (a) A fragment of the observed ratings matrix from the MovieLens-1M dataset. (b) Predictions using SVD with 50 latent components. Generated by matrix_factorization_recommender.ipynb.

22.1.3.3 Example: MovieLens

Now suppose we apply PMF to the MovieLens-1M dataset with 6040 users, 3706 movies, and 1,000,209 ratings. We will use $K = 50$ factors. For simplicity, we fit this using SVD applied to the dense ratings matrix, where we replace missing values with 0. (This is just a simple approximation to keep the demo code simple.) In Figure 22.3 we show a snippet of the true and predicted ratings matrix. (We truncate the predictions to lie in the range [1,5].) We see that the model is not particularly accurate, but does capture some structure in the data.

Furthermore, it seems to behave in a qualitatively sensible way. For example, in Figure 22.4 we show the top 10 movies rated by a given user as well as the top 10 predictions for movies they had not seen. The model seems to have "picked up" on the underlying preferences of the user. For example, we see that many of the predicted movies are action or film-noir, and both of these genres feature in the user's own top-10 list, even though explicit genre information is not used during model training.

22.1.4 Autoencoders

Matrix factorization is a (bi)linear model. We can make a nonlinear version using autoencoders. Let $\boldsymbol{y}_{:,i} \in \mathbb{R}^M$ be the i'th column of the ratings matrix, where unknown ratings are set to 0. We can predict this ratings vector using an autoencoder of the form

$$f(\boldsymbol{y}_{:,i}; \boldsymbol{\theta}) = \mathbf{W}^{\mathsf{T}} \varphi(\mathbf{V}\boldsymbol{y}_{:,i} + \boldsymbol{\mu}) + \boldsymbol{b} \tag{22.11}$$

where $\mathbf{V} \in \mathbb{R}^{KM}$ maps the ratings to an embedding space, $\mathbf{W} \in \mathbb{R}^{KM}$ maps the embedding space to a distribution over ratings, $\boldsymbol{\mu} \in \mathbb{R}^K$ are the biases of the hidden units, and $\boldsymbol{b} \in \mathbb{R}^M$ are the biases of the output units. This is called the (item-based) version of the **AutoRec** model [Sed+15]. This has $2MK + M + K$ parameters. There is also a user-based version, that can be derived in a similar manner, which has $2NK + N + K$ parameters. (On MovieLens and Netflix, the authors find that the item-based method works better.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

	MovieID	Title	Genres
36	858	Godfather, The (1972)	Action|Crime|Drama
35	1387	Jaws (1975)	Action|Horror
65	2028	Saving Private Ryan (1998)	Action|Drama|War
63	1221	Godfather: Part II, The (1974)	Action|Crime|Drama
11	913	Maltese Falcon, The (1941)	Film-Noir|Mystery
20	3417	Crimson Pirate, The (1952)	Adventure|Comedy|Sci-Fi
34	2186	Strangers on a Train (1951)	Film-Noir|Thriller
55	2791	Airplane! (1980)	Comedy
31	1188	Strictly Ballroom (1992)	Comedy|Romance
28	1304	Butch Cassidy and the Sundance Kid (1969)	Action|Comedy|Western

(a)

	MovieID	Title	Genres
516	527	Schindler's List (1993)	Drama|War
1848	1953	French Connection, The (1971)	Action|Crime|Drama|Thriller
596	608	Fargo (1996)	Crime|Drama|Thriller
1235	1284	Big Sleep, The (1946)	Film-Noir|Mystery
2085	2194	Untouchables, The (1987)	Action|Crime|Drama
1188	1230	Annie Hall (1977)	Comedy|Romance
1198	1242	Glory (1989)	Action|Drama|War
897	922	Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)	Film-Noir
1849	1954	Rocky (1976)	Action|Drama
581	593	Silence of the Lambs, The (1991)	Drama|Thriller

(b)

Figure 22.4: (a) Top 10 movies (from a list of 69) that user "837" has already highly rated. (b) Top 10 predictions (from a list of 3637) from the algorithm. Generated by matrix_factorization_recommender.ipynb.

We can fit this by only updating parameters that are associated with the observed entries of $\boldsymbol{y}_{:,i}$. Furthermore, we can add an ℓ_2 regularizer to the weight matrices to get the objective

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \sum_{u:y_{ui}\neq ?} (y_{u,i} - f(\boldsymbol{y}_{:,i};\boldsymbol{\theta})_u)^2 + \frac{\lambda}{2}(||\mathbf{W}||_F^2 + ||\mathbf{V}||_F^2) \tag{22.12}$$

Despite the simplicity of this method, the authors find that this does better than more complex methods such as restricted Boltzmann machines (RBMs, [SMH07]) and local low-rank matrix

approximation (LLORMA, [Lee+13]).

22.2 Implicit feedback

So far, we have assumed that the user gives explicit ratings for each item that they interact with. This is a very restrictive assumption. More generally, we would like to learn from the **implicit feedback** that users give just by interacting with a system. For example, we can treat the list of movies that user u watches as positives, and regard all the other movies as negatives. Thus we get a sparse, positive-only ratings matrix.

Alternatively, we can view the fact that they watched movie i but did not watch movie j as an implicit signal that they prefer i to j. The resulting data can be represented as a set of tuples of the form $y_n = (u, i, j)$, where (u, i) is a positive pair, and (u, j) is a negative (or unlabeled) pair.

22.2.1 Bayesian personalized ranking

To fit a model to data of the form (u, i, j), we need to use a **ranking loss**, so that the model ranks i ahead of j for user u. A simple way to do this is to use a Bernoulli model of the form

$$p(y_n = (u, i, j)|\boldsymbol{\theta}) = \sigma(f(u, i; \boldsymbol{\theta}) - f(u, j; \boldsymbol{\theta})) \tag{22.13}$$

If we combine this with a Gaussian prior for $\boldsymbol{\theta}$, we get the following MAP estimation problem:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{(u,i,j)\in\mathcal{D}} \log \sigma(f(u, i; \boldsymbol{\theta}) - f(u, j; \boldsymbol{\theta})) - \lambda||\boldsymbol{\theta}||^2 \tag{22.14}$$

where $\mathcal{D} = \{(u, i, j) : i \in \mathcal{I}_u^+, j \in \mathcal{I} \setminus \mathcal{I}_u^+\}$, where $\mathcal{I}_u^+$ are the set of all items that user u selected, and $\mathcal{I} \setminus \mathcal{I}_u^+$ are all the other items (which they may dislike, or simply may not have seen). This is known as **Bayesian personalized ranking** or BPR [Ren+09].

Let us consider this example from [Zha+20, Sec 16.5]. There are 4 items in total, $\mathcal{I} = \{i_1, i_2, i_3, i_4\}$, and user u chose to interact with $\mathcal{I}_u^+ = \{i_2, i_3\}$. In this case, the implicit item-item preference matrix for user u has the form

$$\mathbf{Y}_u = \begin{pmatrix} . & + & + & ? \\ - & . & ? & - \\ - & ? & . & - \\ ? & + & + & . \end{pmatrix} \tag{22.15}$$

where $Y_{u,i,i'} = +$ means user u prefers i' to i, $Y_{u,i,i'} = -$ means user u prefers i to i', and $Y_{u,i,i'} =?$ means we cannot tell what the user's preference is. For example, focusing on the second column, we see that this user rates i_2 higher than i_1 and i_4, since they selected i_2 but not i_1 or i_4; however, we cannot tell if they prefer i_2 over i_3 or vice versa.

When the set of posssible items is large, the number of negatives in $\mathcal{I} \setminus \mathcal{I}_u^+$ can be very large. Fortunately we can approximate the loss by subsampling negatives.

Note that an alternative to the log-loss above is to use a hinge loss, similar to the approach used in SVMs (Section 17.3). This has the form

$$\mathcal{L}(y_n = (u, i, j), f) = \max(m - (f(u, i) - f(u, j)), 0) = \max(m - f(u, i) + f(u, j), 0) \tag{22.16}$$

where $m \geq 0$ is the safety margin. This tries to ensure the negative items j never score more than m higher than the positive items i.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

22.2.2 Factorization machines

The AutoRec approach of Section 22.1.4 is nonlinear, but treats users and items asymmetrically. In this section, we discuss a more symmetric discriminative modeling approach. We start with a linear version. The basic idea is to predict the output (such as a rating) for any given user-item pair, $\boldsymbol{x} = [\text{one-hot}(u), \text{one-hot}(i)]$, using

$$f(\boldsymbol{x}) = \mu + \sum_{i=1}^{D} w_i x_i + \sum_{i=1}^{D} \sum_{j=i+1}^{D} (\boldsymbol{v}_i^\mathsf{T} \boldsymbol{v}_j) x_i x_j \tag{22.17}$$

where $\boldsymbol{x} \in \mathbb{R}^D$ where $D = (M + N)$ is the number of inputs, $\mathbf{V} \in \mathbb{R}^{D \times K}$ is a weight matrix, $\boldsymbol{w} \in \mathbb{R}^D$ is a weight vector, and $\mu \in \mathbb{R}$ is a global offset. This is known as a **factorization machine** (FM) [Ren12].

The term $(\boldsymbol{v}_i^\mathsf{T} \boldsymbol{v}_j) x_i x_j$ measures the interaction between feature i and j in the input. This generalizes the matrix factorization model of Equation (22.4), since it can handle other kinds of information in the input $\boldsymbol{x}$, beyond just user and item, as we discuss in Section 22.3.

Computing Equation (22.17) takes $O(KD^2)$ time, since it considers all possible pairwise interactions between every user and every item. Fortunately we can rewrite this so that we can compute it in $O(KD)$ time as follows:

$$\sum_{i=1}^{D} \sum_{j=i+1}^{D} (\boldsymbol{v}_i^\mathsf{T} \boldsymbol{v}_j) x_i x_j = \frac{1}{2} \sum_{i=1}^{D} \sum_{j=1}^{D} (\boldsymbol{v}_i^\mathsf{T} \boldsymbol{v}_j) x_i x_j - \frac{1}{2} \sum_{i=1}^{D} (\boldsymbol{v}_i^\mathsf{T} \boldsymbol{v}_i) x_i x_i \tag{22.18}$$

$$= -\frac{1}{2} \left(\sum_{i=1}^{D} \sum_{j=1}^{D} \sum_{k=1}^{K} v_{ik} v_{jk} x_i x_j - \sum_{i=1}^{D} \sum_{k=1}^{K} v_{ik} v_{ik} x_i x_i \right) \tag{22.19}$$

$$= -\frac{1}{2} \sum_{k=1}^{K} \left((\sum_{i=1}^{D} v_{ik} x_i)^2 - \sum_{i=1}^{D} v_{ik}^2 x_i^2 \right) \tag{22.20}$$

For sparse vectors, the overall complexity is linear in the number of non-zero components. So if we use one-hot encodings of the user and item id, the complexity is just $O(K)$, analogous to the original matrix factorization objective of Equation (22.4).

We can fit this model to minimize any loss we want. For example, if we have explicit feedback, we may choose MSE loss, and if we have implicit feedback, we may choose ranking loss.

In [Guo+17], they propose a model called **deep factorization machines**, which combines the above method with an MLP applied to a concatenation of the embedding vectors, instead of the inner product. More precisely, it is a model of the form

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \sigma(\text{FM}(\boldsymbol{x}) + \text{MLP}(\boldsymbol{x})) \tag{22.21}$$

This is closely related to the **wide and deep** model proposed in [Che+16]. The idea is that the bilinear FM model captures explicit interactions between specific users and items (a form of memorization), whereas the MLP captures implicit interactions between user features and item features, which allows the model to generalize.

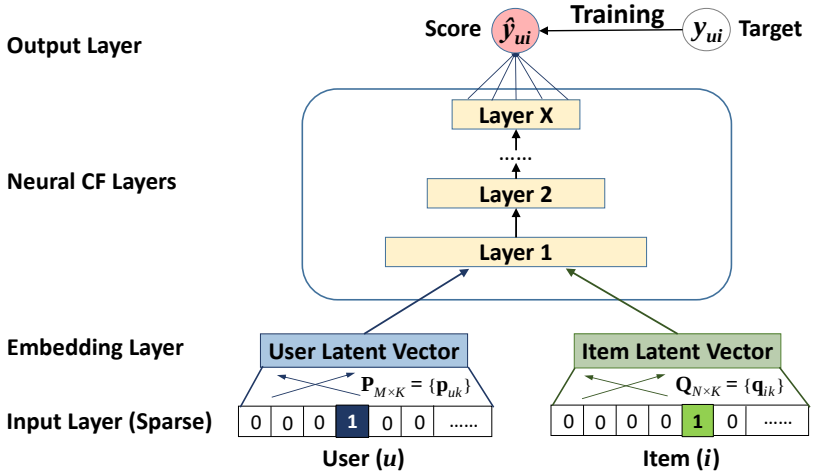


Figure 22.5: Illustration of the neural matrix factorization model. From Figure 2 of [He+17]. Used with kind permission of Xiangnan He.

22.2.3 Neural matrix factorization

In this section, we describe the **neural matrix factorization** model of [He+17]. This is another way to combine bilinear models with deep neural networks. The bilinear part is used to define the following generalized matrix factorization (GMF) pathway, which computes the following feature vector for user u and item i:

$$\boldsymbol{z}_{ui}^1 = \mathbf{P}_{u,:} \odot \mathbf{Q}_{i,:} \tag{22.22}$$

where $\mathbf{P} \in \mathbb{R}^{MK}$ is a user embedding matrix, and $\mathbf{Q} \in \mathbb{R}^{NK}$ is an item embedding matrix. The DNN part is just an MLP applied to a concatenation of the embedding vectors (using different embedding matrices):

$$\boldsymbol{z}_{ui}^2 = \text{MLP}([\tilde{\mathbf{U}}_{u,:}, \tilde{\mathbf{V}}_{i,:}]) \tag{22.23}$$

Finally, the model combines these to get

$$f(u, i; \boldsymbol{\theta}) = \sigma(\boldsymbol{w}^\mathsf{T}[\boldsymbol{z}_{ui}^1, \boldsymbol{z}_{ui}^2]) \tag{22.24}$$

See Figure 22.5 for an illustration.

In [He+17], the model is trained on implicit feedback, where $y_{ui} = 1$ if the interaction of user u with item i is observed, and $y_{ui} = 0$ otherwise. However, it could be trained to minimize BPR loss.

22.3 Leveraging side information

So far, we have assumed that the only information available to the predictor are the integer id of the user and the integer id of the item. This is an extremely impoverished representation, and will fail to

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

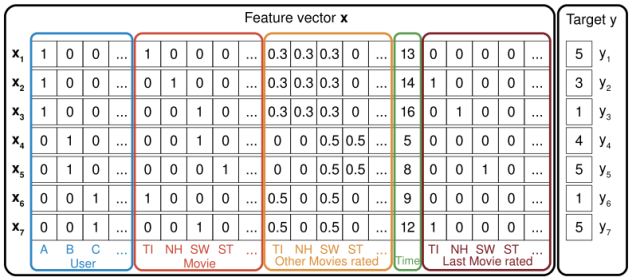


Figure 22.6: Illustration of a design matrix for a movie recommender system, where we show the id of the user and movie, as well as other side information. From Figure 1 of [Ren12]. Used with kind permission of Stefen Rendle.

work if we encounter a new user or new item (the so-called **cold start** problem). To overcome this, we need to leverage "**side information**", beyond just the id of the user/item.

There are many forms of side information we can use. For items, we often have rich meta-data, such as text (e.g., title), images (e.g., cover), high-dimensional categorical variables (e.g., location), or just scalars (eg., price). For users, the side information available depends on the specific form of the interactive system. For search engines, it is the list of queries the user has issued, and (if they are logged in), information derived from websites they have visited (which is tracked via cookies). For online shopping sites, it is the list of searches plus past viewing and purchasing behavior. For social networking sites, there is information about the friendship graph of each user.

It is very easy to capture this side information in the factorization machines framework, by expanding our definition of $\boldsymbol{x}$ beyond the two one-hot vectors, as illustrated in Figure 22.6. The same input encoding can of course be fed into other kinds of models, such as deepFM or neuralMF.

In addition to features about the user and item, there may be other contextual features, such as the time of the interaction (e.g., the day or evening). The order (sequence) of the most recently viewed items is often also a useful signal. The "Convolutional Sequence Embedding Recommendation" or **Caser** model proposed in [TW18] captures this by embedding the last M items, and then treating the $M \times K$ input as an image, by using a convolutional layer as part of the model.

Many other kinds of neural models can be designed for the recommender task. See e.g., [Zha+19b] for a review.

22.4 Exploration-exploitation tradeoff

An interesting "twist" to recommender systems that does not arise in other kinds of prediction problems is the fact that the data that the system is trained on is a consequence of recommendations made by earlier versions of the system. Thus there is a feedback loop [Bot+13]. For example, consider the YouTube video recommendation system [CAS16]. There are millions of videos on the site, so the system must come up with a shortlist, or "**slate**", of videos to show the user, to help them find what they want (see e.g., [Ie+19]). If the user watches one of these videos, the system can consider this positive feedback that it made a good recommendation, and it can update the model parameters accordingly. However, maybe there was some other video that the user would have liked even more?

It is impossible to answer this counterfactual unless the system takes a chance and shows some items for which the user response is uncertain. This is an example of the **exploration-exploitation tradeoff**.

In addition to needing to explore, the system may have to wait for a long time until it can detect if a change it made to its recommendation policies was beneficial. It is common to use **reinforcement learning** to learn policies which optimize long-term reward. See the sequel to this book, [Mur23], for details.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

23 Graph Embeddings *

This chapter is coauthored with Bryan Perozzi, Sami Abu-El-Haija and Ines Chami, and is based on [Cha+21].

23.1 Introduction

We now turn our focus to data which has semantic relationships between training samples $\{\mathbf{x}_n\}_{n=1}^N$. The relationships (known as edges) connect training samples (nodes) with an application specific meaning (commonly similarity). Graphs provide the mathematical foundations for reasoning about these kind of relationships

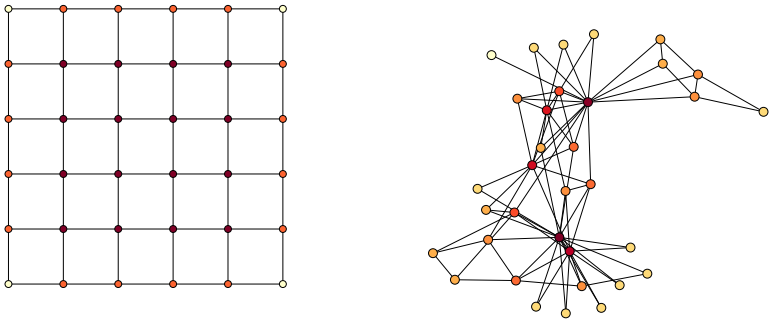
Graphs are universal data structures that can represent complex relational data (composed of nodes and edges), and appear in multiple domains such as social networks, computational chemistry [Gil+17], biology [Sta+06], recommendation systems [KSJ09], semi-supervised learning [GB18], and others.

Let $\mathbf{A} \in \{0,1\}^{N \times N}$ be the adjacency matrix, where N is the number of nodes, and let $\mathbf{W} \in \mathbb{R}^{N \times N}$ be a weighted version. In the methods we discuss below, some set $\mathbf{W} = \mathbf{A}$ while others set $\mathbf{W}$ to a transformation of $\mathbf{A}$, such as row-wise normalization. Finally, let $\mathbf{X} \in \mathbb{R}^{N \times D}$ be a matrix of node features.

When designing and training a neural network model over graph data, we desire the designed method to be applicable to nodes which participate in different graph settings (e.g. have differing connections and community structure). Contrast this with a neural network model designed for images, where each pixel (node) has the same neighborhood structure. By contrast, an arbitrary graph has no specified alignment of nodes, and further, each node might have a different neighborhood structure. See Figure 23.1 for a comparison. Consequently, operations like Euclidean spatial convolution cannot be directly applied on irregular graphs: Euclidean convolutions strongly rely on geometric priors (such as shift invariance), which don't generalize to non-Euclidean domains.

These challenges led to the development of **Geometric Deep Learning** (GDL) research [Bro+17b], which aims at applying deep learning techniques to non-Euclidean data. In particular, given the widespread prevalence of graphs in real-world applications, there has been a surge of interest in applying machine learning methods to graph-structured data. Among these, **Graph Representation Learning** (GRL) [Cha+21] methods aim at learning low-dimensional continuous vector representations for graph-structured data, also called embeddings.

We divide GRL here into two classes of problems: **unsupervised** and **supervised** (or semi-supervised) GRL. The first class aims at learning low-dimensional Euclidean representations optimizing



(a) Grid (Euclidean). *(b) Arbitrary graph (Non-Euclidean).*

Figure 23.1: An illustration of Euclidean vs. non-Euclidean graphs. Used with permission from [Cha+21].

an objective, e.g. one that preserve the structure of an input graph. The second class also learns low-dimensional Euclidean representations but for a specific downstream prediction task such as node or graph classification. Further, the graph structure can be fixed throughout training and testing, which is known as the **transductive** learning setting (e.g. predicting user properties in a large social network), or alternatively the model is expected to answer questions about graphs not seen during training, known as the **inductive** learning setting (e.g. classifying molecular structures). Finally, while most supervised and unsupervised methods learn representations in Euclidean vector spaces, there recently has been interest for **non-Euclidean representation learning**, which aims at learning non-Euclidean embedding spaces such as hyperbolic or spherical spaces. The main motivations for this body of work is to use a continuous embedding space that resembles the underlying discrete structure of the input data it tries to embed (e.g. the hyperbolic space is a continuous version of trees [Sar11]).

23.2 Graph Embedding as an Encoder/Decoder Problem

While there are many approaches to GRL, many methods follow a similar pattern. First, the network input (node features $\mathbf{X} \in \mathbb{R}^{N \times D}$ and graph edges in $\mathbf{A}$ or $\mathbf{W} \in \mathbb{R}^{N \times N}$) is encoded from the discrete domain of the graph into a continuous representation (embedding), $\mathbf{Z} \in \mathbb{R}^{N \times L}$. Next, the learned representation $\mathbf{Z}$ is used to optimize a particular objective (such as reconstructing the links of the graph). In this section we will use the graph encoder-decoder model (GraphEDM) proposed by Chami et al. [Cha+21] to analyze popular families of GRL methods.

The GraphEDM framework (Figure 23.2, [Cha+21]) provides a general framework that encapsulates a wide variety of supervised and unsupervised graph embedding methods: including ones utilizing the graph as a regularizer (e.g. [ZG02]), positional embeddings(e.g. [PARS14]), and graph neural networks such as ones based on message passing [Gil+17; Sca+09] or graph convolutions

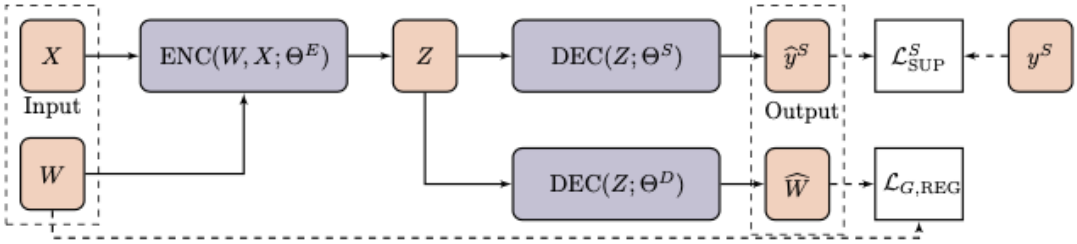


Figure 23.2: Illustration of the GraphEDM framework from Chami et al. [Cha+21]. Based on the supervision available, methods will use some or all of the branches. In particular, unsupervised methods do not leverage label decoding for training and only optimize the similarity decoder (lower branch). On the other hand, semi-supervised and supervised methods leverage the additional supervision to learn models' parameters (upper branch). Reprinted with permission from [Cha+21].

[Bru+14; KW16a]).

The GraphEDM framework takes as input a weighted graph $\mathbf{W} \in \mathbb{R}^{N \times N}$, and optional node features $\mathbf{X} \in \mathbb{R}^{N \times D}$. In (semi-)supervised settings, we assume that we are given training target labels for nodes (denoted N), edges (denoted E), and/or for the entire graph (denoted G). We denote the supervision signal as $S \in \{N, E, G\}$, as presented below.

The GraphEDM model itself can be decomposed into the following components:

- **Graph encoder network** $\text{ENC}_{\Theta^E} : \mathbb{R}^{N \times N} \times \mathbb{R}^{N \times D} \to \mathbb{R}^{N \times L}$, parameterized by Θ^E, which combines the graph structure with optional node features to produce a node embedding matrix $\mathbf{Z} \in \mathbb{R}^{N \times L}$ as follows:

$$\mathbf{Z} = \text{ENC}(\mathbf{W}, \mathbf{X}; \Theta^E). \tag{23.1}$$

  As we shall see next, this node embedding matrix might capture different graph properties depending on the supervision used for training.

- **Graph decoder network** $\text{DEC}_{\Theta^D} : \mathbb{R}^{N \times L} \to \mathbb{R}^{N \times N}$, parameterized by Θ^D, which uses the node embeddings Z to compute similarity scores for all node pairs in matrix $\widehat{\mathbf{W}} \in \mathbb{R}^{N \times N}$ as follows:

$$\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D). \tag{23.2}$$

- **Classification network** $\text{DEC}_{\Theta^S} : \mathbb{R}^{N \times L} \to \mathbb{R}^{N \times |\mathcal{Y}|}$, where $\mathcal{Y}$ is the label space. This network is used in (semi-)supervised settings and parameterized by Θ^S. The output is a distribution over the labels $\hat{y}^S$, using node embeddings, as follows:

$$\widehat{y}^S = \text{DEC}(\mathbf{Z}; \Theta^S). \tag{23.3}$$

Specific choices of the aforementioned (encoder and decoder) networks allows GraphEDM to realize specific graph embedding methods, as we explain in the next subsections.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

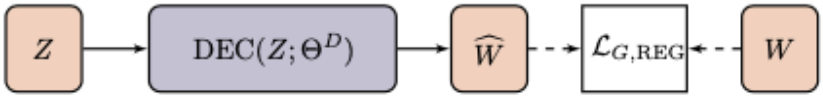


Figure 23.3: Shallow embedding methods. The encoder is a simple embedding look-up and the graph structure is only used in the loss function. Reprinted with permission from [Cha+21].

The output of a model, as described by GraphEDM framework, is a reconstructed graph similarity matrix $\widehat{W}$ (often used to train *unsupervised* embedding algorithms), and/or labels $\widehat{y}^S$ for *supervised* applications. The label output space $\mathcal{Y}$ is application dependent. For instance, in node-level classification, $\widehat{y}^N \in \mathcal{Y}^N$, with $\mathcal{Y}$ representing the node label space. Alternately, for edge-level labeling, $\widehat{y}^E \in \mathcal{Y}^{N \times N}$, with $\mathcal{Y}$ representing the edge label space. Finally, we note that other kinds of labeling are possible, such as graph-level labeling (where we would say $\widehat{y}^G \in \mathcal{Y}$, with $\mathcal{Y}$ representing the graph label space).

Finally, a loss must be specified. This can be used to optimize the parameters $\Theta = \{\Theta^E, \Theta^D, \Theta^S\}$. GraphEDM models can be optimized using a combination of three different terms. First, a supervised loss term, $\mathcal{L}^S_{\text{SUP}}$, compares the predicted labels $\hat{y}^S$ to the ground truth labels y^S. Next, a graph reconstruction loss term, $\mathcal{L}_{G,\text{RECON}}$, may leverage the graph structure to impose regularization constraints on the model parameters. Finally, a weight regularization loss term, $\mathcal{L}_{\text{REG}}$, allows representing priors on trainable model parameters for reducing overfitting. Models realizable by GraphEDM framework are trained by minimizing the total loss $\mathcal{L}$ defined as:

$$\mathcal{L} = \alpha \mathcal{L}^S_{\text{SUP}}(y^S, \hat{y}^S; \Theta) + \beta \mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) + \gamma \mathcal{L}_{\text{REG}}(\Theta), \tag{23.4}$$

where α, β and γ are hyper-parameters, that can be tuned or set to zero. Note that graph embedding methods can be trained in a *supervised* ($\alpha \neq 0$) or *unsupervised* ($\alpha = 0$) fashion. Supervised graph embedding approaches leverage an additional source of information to learn embeddings such as node or graph labels. On the other hand, unsupervised network embedding approaches rely on the graph structure only to learn node embeddings.

23.3 Shallow graph embeddings

Shallow embedding methods are transductive graph embedding methods, where the encoder function maps categorical node IDs onto a Euclidean space through an embedding matrix. Each node $v_i \in V$ has a corresponding low-dimensional learnable embedding vector $\mathbf{Z}_i \in \mathbb{R}^L$ and the shallow encoder function is

$$\mathbf{Z} = \text{ENC}(\Theta^E) \triangleq \Theta^E \quad \text{where} \quad \Theta^E \in \mathbb{R}^{N \times L}. \tag{23.5}$$

Crucially, the embedding dictionary $\mathbf{Z}$ is directly learned as model parameters. In the unsupervised case, embeddings $\mathbf{Z}$ are optimized to recover some information about the input graph (e.g., the adjacency matrix $\mathbf{W}$, or some transformation of it). This is somewhat similar to dimensionality reduction methods, such as PCA (Section 20.1), but for graph data structures. In the supervised case, the embeddings are optimized to predict some labels, for nodes, edges and/or the whole graph.

23.3.1 Unsupervised embeddings

In the unsupervised case, we will consider two main types of shallow graph embedding methods: distance-based and outer product-based. Distance-based methods optimize the embedding dictionary $\mathbf{Z} = \Theta^E \in \mathbb{R}^{N \times L}$ such that nodes i and j which are close in the graph (as measured by some graph distance function) are embedded in $\mathbf{Z}$ such that $d_2(\mathbf{Z}_i, \mathbf{Z}_j)$ is small, where $d_2(.,.)$ is a pairwise distance function between embedding vectors. The distance function $d_2(\cdot,\cdot)$ can be customized, which can lead to Euclidean (Section 23.3.2) or non-Euclidean (Section 23.3.3) embeddings. The decoder outputs a node-to-node matrix $\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D)$, with $\widehat{W}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j)$.

Alternatively, some methods rely on pairwise dot-products to compute node similarities. The decoder network can be written as: $\widehat{W} = \text{DEC}(\mathbf{Z}; \Theta^D) = \mathbf{Z}\mathbf{Z}^\top$.

In both cases, unsupervised embeddings for distance- and product-based methods are learned by minimizing the graph reconstruction loss:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = d_1(s(\mathbf{W}), \widehat{\mathbf{W}}), \tag{23.6}$$

where $s(\mathbf{W})$ is an optional transformation of the adjacency matrix $\mathbf{W}$, and d_1 is pairwise distance function between matrices, which does not need to be of the same form as d_2. As we shall see, there are many plausible choices for s, d_1, d_2. For instance, we can let s be the adjacency matrix itself, $s(\mathbf{W}) = \mathbf{W}$ or a power of it e.g. $s(\mathbf{W}) = \mathbf{W}^2$. If the input is a weighted binary matrix $\mathbf{W} = \mathbf{A}$, we can set $s(\mathbf{W}) = 1 - \mathbf{W}$, so that connected nodes with $A_{ij} = 1$ get a weight (distance) of 0.

23.3.2 Distance-based: Euclidean methods

Distance-based methods minimize Euclidean distances between similar (connected) nodes. We give some examples below.

Multi-dimensional scaling (MDS, Section 20.4.4) is equivalent to setting $s(\mathbf{W})$ to some distance matrix measuring the dissimilarity between nodes (e.g. proportional to pairwise shortest distance) and then defining

$$d_1(s(W), \widehat{W}) = \sum_{i,j} (s(W)_{ij} - \widehat{W}_{ij})^2 = ||s(\mathbf{W}) - \widehat{\mathbf{W}}||_F^2 \tag{23.7}$$

where $\widehat{W}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j) = ||\mathbf{Z}_i - \mathbf{Z}_j||$ (although other distance metrics are plausible).

Laplacian eigenmaps (Section 20.4.9) learn embeddings by solving the generalized eigenvector problem

$$\min_{\mathbf{Z} \in \mathbb{R}^{|V| \times d}} \text{tr}(\mathbf{Z}^\mathsf{T} \mathbf{L}\, \mathbf{Z}) \;\; \text{s.t.} \;\; \mathbf{Z}^\mathsf{T}\mathbf{D}\mathbf{Z} = \mathbf{I} \;\; \text{and} \;\; \mathbf{Z}^\mathsf{T}\mathbf{D}\mathbf{1} = \mathbf{0} \tag{23.8}$$

where $\mathbf{L} = \mathbf{D} - \mathbf{W}$ is the graph Laplacian (Section 20.4.9.2), and $\mathbf{D}$ is a diagonal matrix containing the sum across columns for each row. The first constraint removes an arbitrary scaling factor in the embedding and the second one removes trivial solutions corresponding to the constant eigenvector (with eigenvalue zero for connected graphs). Further, note that $\text{tr}(\mathbf{Z}^\mathsf{T}\mathbf{L}\mathbf{Z}) = \frac{1}{2}\sum_{i,j} W_{ij}||\mathbf{Z}_i - \mathbf{Z}_j||_2^2$, where $\mathbf{Z}_i$ is the i'th row of $\mathbf{Z}$; therefore the minimization objective can be equivalently written as a

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

graph reconstruction term, as follows:

$$d_1(s(\mathbf{W}), \widehat{\mathbf{W}}) = \sum_{i,j} \mathbf{W}_{ij} \times \widehat{\mathbf{W}}_{ij} \tag{23.9}$$

$$\widehat{\mathbf{W}}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j) = ||\mathbf{Z}_i - \mathbf{Z}_j||_2^2 \tag{23.10}$$

where $s(\mathbf{W}) = \mathbf{W}$.

23.3.3 Distance-based: non-Euclidean methods

So far, we have discussed methods which assume that embeddings lie in an Euclidean Space. However, recent work has considered hyperbolic geometry for graph embedding. In particular, hyperbolic embeddings are ideal for embedding trees and offer an exciting alternative to Euclidean geometry for graphs that exhibit hierarchical structures. We give some examples below.

Nickel and Kiela [NK17] learn embeddings of hierarchical graphs using the **Poincaré model** of hyperbolic space. This is simple to represent in our notation as we only need to change $d_2(\mathbf{Z}_i, \mathbf{Z}_j)$ to the Poincaré distance function:

$$d_2(\mathbf{Z}_i, \mathbf{Z}_j) = d_{\text{Poincaré}}(\mathbf{Z}_i, \mathbf{Z}_j) = \text{arcosh}\left(1 + 2\frac{||\mathbf{Z}_i - \mathbf{Z}_j||_2^2}{(1 - ||\mathbf{Z}_i||_2^2)(1 - ||\mathbf{Z}_j||_2^2)}\right). \tag{23.11}$$

The optimization then learns embeddings which minimize distances between connected nodes while maximizing distances between disconnected nodes:

$$d_1(\mathbf{W}, \widehat{\mathbf{W}}) = \sum_{i,j} \mathbf{W}_{ij} \text{log} \frac{e^{-\widehat{\mathbf{W}}_{ij}}}{\sum_{k|\mathbf{W}_{ik}=0} e^{-\widehat{\mathbf{W}}_{ik}}} \tag{23.12}$$

where the denominator is approximated using negative sampling. Note that since the hyperbolic space has a manifold structure, care needs to be taken to ensure that the embeddings remain on the manifold (using Riemannian optimization techniques [Bon13]).

Other variants of these methods have been proposed. Nickel and Kiela [NK18] explore the **Lorentz model** of hyperbolic space , and show that it provides better numerical stability than the Poincaré model. Another line of work extends non-Euclidean embeddings to mixed-curvature product spaces [Gu+18], which provide more flexibility for other types of graphs (e.g. ring of trees). Finally, work by Chamberlain, Clough, and Deisenroth [CCD17] extends Poincaré embeddings using skip-gram losses with hyperbolic inner products.

23.3.4 Outer product-based: Matrix factorization methods

Matrix factorization approaches learn embeddings that lead to a low rank representation of some similarity matrix $s(\mathbf{W})$, with $s : \mathbb{R}^{N \times N} \to \mathbb{R}^{N \times N}$. The following are frequent choices: $s(\mathbf{W}) = \mathbf{W}$, $s(\mathbf{W}) = L$ (Graph Laplacian), or other proximity measure such as the Katz centrality index, Common Neighbors or Adamic/Adar index.

The decoder function in matrix factorization methods is just a dot product:

$$\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D) = \mathbf{Z}\mathbf{Z}^\mathsf{T} \tag{23.13}$$

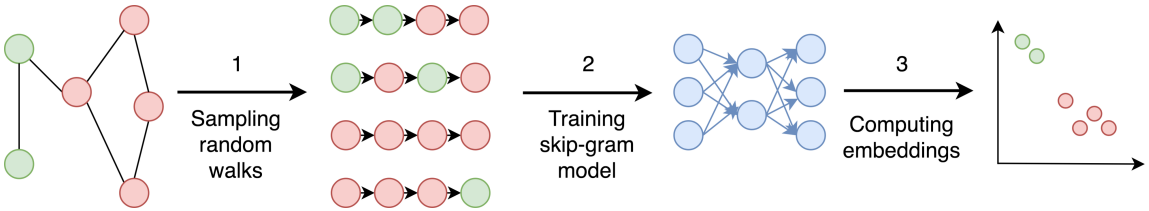


Figure 23.4: An overview of the pipeline for random-walk graph embedding methods. Reprinted with permission from [God18].

Matrix factorization methods learn $\mathbf{Z}$ by minimizing a regularization loss $\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = ||s(\mathbf{W}) - \widehat{\mathbf{W}}||_F^2$.

The **graph factorization** method of [Ahm+13] learns a low-rank factorization of a graph by minimizing the graph regularization loss $\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = \sum_{(v_i,v_j)\in E}(\mathbf{W}_{ij} - \widehat{\mathbf{W}}_{ij})^2$.

Note that if $\mathbf{A}$ is the binary adjacency matrix, ($\mathbf{A}_{ij} = 1$ iff $(v_i, v_j) \in E$ and $\mathbf{A}_{ij} = 0$ otherwise), the graph reconstruction loss can be expressed in terms of the Frobenius norm:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = ||\mathbf{A} \odot (\mathbf{W} - \widehat{\mathbf{W}})||_F^2, \tag{23.14}$$

where $\odot$ is the element-wise matrix multiplication operator. Therefore, GF also learns a low-rank factorization of the adjacency matrix W measured in Frobenuis norm. We note that this is a sparse operation (summing only over edges which exist in the graph), and so the method has computational complexity $O(M)$.

The methods described so far are all symmetric, that is, they assume that $\mathbf{W}_{ij} = \mathbf{W}_{ji}$. This is a limiting assumption when working with directed graphs as some relationships are not reciprocal. The **GraRep** method of [CLX15] overcomes this limitation by learning two embeddings per node, a source embedding $\mathbf{Z}^s$ and a target embedding $\mathbf{Z}^t$, which capture asymmetric proximity in directed networks. In addition to asymmetry, GraRep learns embeddings that preserve k-hop neighborhoods via powers of the adjacency matrix and minimizes a graph reconstruction loss with:

$$\widehat{\mathbf{W}}^{(k)} = \mathbf{Z}^{(k),s}\mathbf{Z}^{(k),t^\mathsf{T}} \tag{23.15}$$

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}^{(k)}; \Theta) = ||\mathbf{D}^{-k}\mathbf{W}^k - \widehat{\mathbf{W}}^{(k)}||_F^2, \tag{23.16}$$

for each $1 \le k \le K$. GraRep concatenates all representations to get source embeddings $\mathbf{Z}^s = [\mathbf{Z}^{(1),s}|\ldots|\mathbf{Z}^{(K),s}]$ and target embeddings $\mathbf{Z}^t = [\mathbf{Z}^{(1),t}|\ldots|\mathbf{Z}^{(K),t}]$. Unfortunately, GraRep is not very scalable, since it uses a matrix power, $\mathbf{D}^{-1}\mathbf{W}$, making it increasingly more dense. This limitation can be circumvented by using implicit matrix factorization [Per+17] as discussed below.

23.3.5 Outer product-based: Skip-gram methods

Skip-gram graph embedding models were inspired by research in natural language processing to model the distributional behavior of words [Mik+13c; PSM14b]. Skip-gram word embeddings are optimized to predict words in their context (the surrounding words) for each target word in a sentence. Given

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

a sequence of words $(w_1, \ldots, w_T)$, skip-gram will minimize the objective:

$$\mathcal{L} = -\sum_{-K \le i \le K, i \neq 0} \log \mathbb{P}(w_{k-i}|w_k),$$

for each target words w_k. These conditional probabilities can be efficiently estimated using neural networks. See Section 20.5.2.2 for details.

This idea has been leveraged for graph embeddings in the **DeepWalk** framework of [PARS14]. They justified this by showing empirically how the frequency statistics induced by random walks in real graphs follow a distribution similar to that of words used in natural language. In terms of GraphEDM, skip-gram graph embedding methods use an outer product (Equation 23.13) as their decoder function and a graph reconstruction term computed over random walks on the graph.

In more detail, DeepWalk trains node embeddings to maximize the probability of predicting *context nodes* for each *center node*. The context nodes are nodes appearing adjacent to the center node, in simulated random walks on $\mathbf{A}$. To train embeddings, DeepWalk generates sequences of nodes using truncated unbiased random walks on the graph — which can be compared to sentences in natural language models — and then maximizes their log-likelihood. Each random walk starts with a node $v_{i_1} \in V$ and repeatedly samples the next node uniformly at random: $v_{i_{j+1}} \in \{v \in V \mid (v_{i_j}, v) \in E\}$. The walk length is a hyperparameter. All generated random-walks can then be encoded by a sequence model. This two-step paradigm introduced by [PARS14] has been followed by many subsequent works, such as **node2vec** [GL16].

We note that it is common for underlying implementations to use two distinct representations for each node, one for when a node is center of a truncated random walk, and one when it is in the context. The implications of this modeling choice are studied further in [AEHPAR17].

To present DeepWalk in the GraphEDM framework, we can set:

$$s(\mathbf{W}) = \mathbb{E}_q\left[\left(\mathbf{D}^{-1}\mathbf{W}\right)^q\right] \text{ with } q \sim P(Q) = \text{Categorical}([1, 2, \ldots, T_{\max}]) \tag{23.17}$$

where $P(Q = q) = \frac{T_{\max} - 1 + q}{T_{\max}}$ (see [AEH+18] for the derivation).

Training DeepWalk is equivalent to minimizing:

$$\mathcal{L}_{G,\text{RECON}}(W, \widehat{W}; \Theta) = \log Z(\mathbf{Z}) - \sum_{v_i \in V, v_j \in V} s(\mathbf{W})_{ij} \widehat{\mathbf{W}}_{ij}, \tag{23.18}$$

where $\widehat{\mathbf{W}} = \mathbf{Z}\mathbf{Z}^\intercal$, and the partition function $Z(\mathbf{Z}) = \prod_i \sum_j \exp(\widehat{\mathbf{W}}_{ij})$ can be approximated in $O(N)$ time via hierarchical softmax (see Section 20.5.2). (It is also common to model $\widehat{\mathbf{W}} = \mathbf{Z}_{\text{out}}\mathbf{Z}_{\text{in}}^\intercal$ for directed graphs using embedding dictionaries $\mathbf{Z}_{\text{out}}, \mathbf{Z}_{\text{in}} \in \mathbb{R}^{N \times L}$.)

As noted by [LG14], Skip-gram methods can be viewed as implicit matrix factorization, and the methods discussed here are related to those of Matrix Factorization (see Section 23.3.4). This relationship is discussed in depth by [Qiu+18], who propose a general matrix factorization framework, **NetMF**, which uses the same underlying graph proximity information as DeepWalk, LINE [Tan+15], and node2vec [GL16]. Casting the node embedding problem as matrix factorization can inherit benefits of efficient sparse matrix operations [Qiu+19a].

23.3.6 Supervised embeddings

In many applications, we have labeled data in addition to node features and graph structure. While it is possible to tackle a supervised task by first learning unsupervised representations and then using them as features in a secondary model, this is not the ideal workflow. Unsupervised node embeddings might not preserve important properties of graphs (e.g., node neighborhoods or attributes), that are most useful for a downstream supervised task.

In light of this limitation, a number of methods combining these two steps, namely learning embeddings and predicting node or graph labels, have been proposed. Here, we focus on simple shallow methods. We discuss deep, nonlinear embeddings later on.

23.3.6.1 Label propagation

Label propagation (LP) [ZG02] is a very popular algorithm for graph-based semi-supervised node classification. The encoder is a shallow model represented by a lookup table $\mathbf{Z}$. LP uses the label space to represent the node embeddings directly (i.e. the decoder in LP is simply the identity function):

$$\hat{y}^N = \text{DEC}(\mathbf{Z}; \Theta^C) = \mathbf{Z}.$$

In particular, LP uses the graph structure to smooth the label distribution over the graph by adding a regularization term to the loss function, using the underlying assumption that neighbor nodes should have similar labels (i.e. there exist some label consistency between connected nodes). Laplacian eigenmaps are utilized in the regularization to enforce this smoothness:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = \sum_{i,j} \mathbf{W}_{ij} ||y_i^N - \hat{y}_j^N||_2^2 \tag{23.19}$$

LP minimizes this energy function over the space of functions that take fixed values on labeled nodes (i.e. $\hat{y}_i^N = y_i^N \ \forall i | v_i \in V_L$) using an iterative algorithm that updates an unlabeled node's label distribution via the weighted average of its neighbors' labels.

Label spreading (LS) [Zho+04] is a variant of label propagation which minimizes the following energy function:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = \sum_{i,j} \mathbf{W}_{ij} \left|\left| \frac{\hat{y}_i^N}{\sqrt{D_i}} - \frac{\hat{y}_j^N}{\sqrt{D_j}} \right|\right|_2^2, \tag{23.20}$$

where $D_i = \sum_j W_{ij}$ is the degree of node v_i.

In both methods, the supervised loss is simply the sum of distances between predicted labels and ground truth labels (one-hot vectors):

$$\mathcal{L}_{\text{SUP}}^N(y^N, \hat{y}^N; \Theta) = \sum_{i | v_i \in V_L} ||y_i^N - \hat{y}_i^N||_2^2. \tag{23.21}$$

Note that while the regularization term is computed over all nodes in the graph, the supervised loss is computed over labeled nodes only. These methods are expected to work well with *consistent* graphs, that is graphs where node proximity in the graph is positively correlated with label similarity.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

23.4 Graph Neural Networks

An extensive area of research focuses on defining convolutions over graph data. In the notation of Chami et al. [Cha+21], these (semi-)supervised neighborhood aggregation methods can be represented by an encoder of the form $\mathbf{Z} = \text{ENC}(\mathbf{X}, \mathbf{W}; \Theta^E)$, and decoders of the form $\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D)$ and/or $\widehat{y}^S = \text{DEC}(\mathbf{Z}; \Theta^S)$. There are many models in this family; we review some of them below.

23.4.1 Message passing GNNs

The original **graph neural network** (**GNN**) model of [GMS05; Sca+09] was the first formulation of deep learning methods for graph-structured data. It views the supervised graph embedding problem as an information diffusion mechanism, where nodes send information to their neighbors until some stable equilibrium state is reached. More concretely, given randomly initialized node embeddings $\mathbf{Z}^0$, it applies the following recursion:

$$\mathbf{Z}^{t+1} = \text{ENC}(\mathbf{X}, \mathbf{W}, \mathbf{Z}^t; \Theta^E), \tag{23.22}$$

where parameters Θ^E are reused at every iteration. After convergence ($t = T$), the node embeddings $\mathbf{Z}^T$ are used to predict the final output such as node or graph labels:

$$\hat{y}^S = \text{DEC}(\mathbf{X}, \mathbf{Z}^T; \Theta^S). \tag{23.23}$$

This process is repeated several times and the GNN parameters Θ^E and Θ^D are learned with backpropagation via the Almeda-Pineda algorithm [Alm87; Pin88]. By Banach's fixed point theorem, this process is guaranteed to converge to a unique solution when the recursion provides a contraction mapping. In light of this, Scarselli et al. [Sca+09] explore maps that can be expressed using message passing networks:

$$\mathbf{Z}_i^{t+1} = \sum_{j|(v_i,v_j)\in E} f(\mathbf{X}_i, \mathbf{X}_j, \mathbf{Z}_j^t; \Theta^E), \tag{23.24}$$

where $f(\cdot)$ is a multi-layer perception (MLP) constrained to be a contraction mapping. The decoder function, however, has no constraints and can be any MLP.

Li et al. [Li+15] propose **Gated Graph Sequence Neural Networks** (GGSNNs), which remove the contraction mapping requirement from GNNs. In GGSNNs, the recursive algorithm in Equation 23.22 is relaxed by applying mapping functions for a fixed number of steps, where each mapping function is a gated recurrent unit [Cho+14b] with parameters shared for every iteration. The GGSNN model outputs predictions at every step, and so is particularly useful for tasks which have sequential structure (such as temporal graphs).

Gilmer et al. [Gil+17] provide a framework for graph neural networks called **message passing neural networks** (MPNNs), which encapsulates many recent models. In contrast with the GNN model which runs for an indefinite number of iterations, MPNNs provide an abstraction for modern approaches, which consist of multi-layer neural networks with a *fixed* number of layers. At every layer ℓ, message functions $f^\ell(.)$ receive messages from neighbors (based on neighbor's hidden state),

which are then passed to aggregation functions $h^\ell(.)$:

$$\mathbf{m}_i^{\ell+1} = \sum_{j|(v_i,v_j)\in E} f^\ell(\mathbf{H}_i^\ell, \mathbf{H}_j^\ell) \tag{23.25}$$

$$\mathbf{H}_i^{\ell+1} = h^\ell(\mathbf{H}_i^\ell, \mathbf{m}_i^{\ell+1}), \tag{23.26}$$

where $\mathbf{H}^0 = \mathbf{X}$. After ℓ layers of message passing, nodes' hidden representations encode information within ℓ-hop neighborhoods.

Battaglia et al. [Bat+18] propose **GraphNet**, which further extends the MPNN framework to learn representations for edges, nodes and the entire graph using message passing functions. The explicit addition of edge and graph representations adds additional expressivity to the MPNN model, and allows the application of graph models to additional domains.

23.4.2 Spectral Graph Convolutions

Spectral methods define graph convolutions using the spectral domain of the graph Laplacian matrix. These methods broadly fall into two categories: *spectrum-based methods*, which explicitly compute an eigendecomposition of the Laplacian (e.g., **spectral CNNs** [Bru+14]) and *spectrum-free* methods, which are motivated by spectral graph theory but do not actually perform a spectral decomposition (e.g., **Graph convolutional networks** or **GCN** [KW16a]).

A major disadvantage of spectrum-based methods is that they rely on the spectrum of the graph Laplacian and are therefore domain-dependent (i.e. cannot generalize to new graphs). Moreover, computing the Laplacian's spectral decomposition is computationally expensive. Spectrum-free methods overcome these limitations by utilizing approximations of these spectral filters. However, spectrum-free methods require using the whole graph $\mathbf{W}$, and so do not scale well.

For more details on spectral approaches, see e.g., [Bro+17b; Cha+21].

23.4.3 Spatial Graph Convolutions

Spectrum-based methods have an inherent domain dependency which limits the application of a model trained on one graph to a new dataset. Additionally, *spectrum-free* methods (e.g. GCNs) require using the entire graph $\mathbf{A}$, which can quickly become unfeasible as the size of the graph grows.

To overcome these limitations, another branch of graph convolutions (*spatial* methods) borrow ideas from standard CNNs – applying convolutions in the spatial domain as defined by the graph topology. For instance, in computer vision, convolutional filters are spatially localized by using fixed rectangular patches around each pixel. Combined with the natural ordering of pixels in images (top, left, bottom, right), it is possible to reuse filters' weights at every location. This process significantly reduces the total number of parameters needed for a model. While such spatial convolutions cannot directly be applied in graph domains, spatial graph convolutions take inspiration from them. The core idea is to use *neighborhood sampling* and *attention mechanisms* to create fixed-size graph patches, overcoming the irregularity of graphs.

23.4.3.1 Sampling-based spatial methods

To overcome the domain dependency and storage limitations of GCNs, Hamilton, Ying, and Leskovec [HYL17] propose **GraphSAGE**, a framework to learn inductive node embeddings. Instead of

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

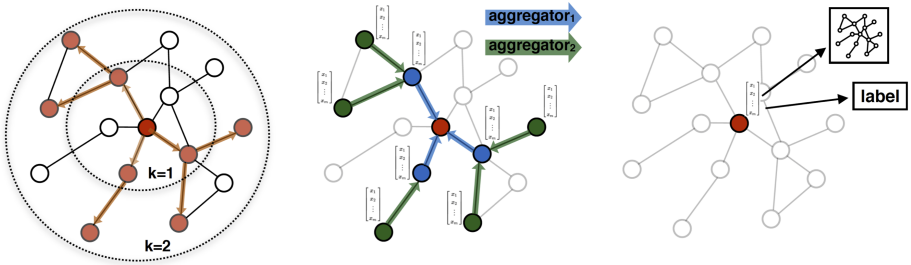


Figure 23.5: Illustration of the GraphSAGE model. Reprinted with permission from [HYL17].

averaging signals from all one-hop neighbors (via multiplications with the Laplacian matrix), SAGE samples fixed neighborhoods (of size q) for each node. This removes the strong dependency on fixed graph structure and allows generalization to new graphs. At every SAGE layer, nodes aggregate information from nodes sampled from their neighborhood (see Figure 23.5). In the GraphEDM notation, the propagation rule can be written as:

$$\mathbf{H}_{:,i}^{\ell+1} = \sigma(\Theta_1^\ell \mathbf{H}_{:,i}^\ell + \Theta_2^\ell \text{AGG}(\{\mathbf{H}_{:,j}^\ell \mid v_j \in \text{Sample}(\text{nbr}(v_i), q)\})), \tag{23.27}$$

where $\text{AGG}(\cdot)$ is an aggregation function. This aggregation function can be any permutation invariant operator such as averaging (SAGE-mean) or max-pooling (SAGE-pool). As SAGE works with fixed size neighborhoods (and not the entire adjacency matrix), it also reduces the computational complexity of training GCNs.

23.4.3.2 Attention-based spatial methods

Attention mechanisms (Section 15.4) have been successfully used in language models where they, for example, allow models to identify relevant parts of long sequence inputs. Inspired by their success in language, similar ideas have been proposed for graph convolution networks. Such graph-based attention models learn to focus their attention on important neighbors during the message passing step via parametric patches which are learned on top of node features. This provides more flexibility in inductive settings, compared to methods that rely on fixed weights such as GCNs.

The **Graph attention network** (GAT) model of [Vel+18] is an attention-based version of GCNs. At every GAT layer, it attends over the neighborhood of each node and learns to selectively pick nodes which lead to the best performance for some downstream task. The intuition behind this is similar to SAGE [HYL17] and makes GAT suitable for inductive and transductive problems. However unlike SAGE, which limits the convolution step to fixed size-neighborhoods, GAT allows each node to attend over the entirety of its neighbors – assigning each of them different weights. The attention parameters are trained through backpropagation, and the attention scores are then row-normalized with a softmax activation.

23.4.3.3 Geometric spatial methods

Monti et al. [Mon+17] propose **MoNet**, a general framework that works particularly well when the node features lie in a geometric space, such as 3D point clouds or meshes. MoNet learns attention patches using parametric functions in a pre-defined spatial domain (e.g. spatial coordinates), and then applies convolution filters in the resulting graph domain.

MoNet generalizes spatial approaches which introduce constructions for convolutions on manifolds, such as the Geodesic CNN (GCNN) [Mas+15] and the Anisotropic CNN (ACNN) [Bos+16]. Both GCNN and ACNN use fixed patches that are defined on a specific coordinate system and therefore cannot generalize to graph-structured data. However, the MoNet framework is more general; any pseudo-coordinates (i.e. node features) can be used to induce the patches. More formally, if $\mathbf{U}^s$ are pseudo-coordinates and $\mathbf{H}^\ell$ are features from another domain, the MoNet layer can be expressed in our notation as:

$$\mathbf{H}^{\ell+1} = \sigma\left(\sum_{k=1}^{K}(\mathbf{W} \odot g_k(\mathbf{U}^s))\mathbf{H}^\ell \Theta_k^\ell\right), \tag{23.28}$$

where $g_k(U^s)$ are the learned parametric patches, which are $N \times N$ matrices. In practice, MoNet uses Gaussian kernels to learn patches, such that:

$$g_k(\mathbf{U}^s) = \exp\left(-\frac{1}{2}(\mathbf{U}^s - \boldsymbol{\mu}_k)^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}(\mathbf{U}^s - \boldsymbol{\mu}_k)\right), \tag{23.29}$$

where $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$ are learned parameters, and $\boldsymbol{\Sigma}_k$ is restricted to be diagonal.

23.4.4 Non-Euclidean Graph Convolutions

As we discussed in Section 23.3.3, hyperbolic geometry enables learning of shallow embeddings of hierarchical graphs which have smaller distortion than Euclidean embeddings. However, one major downside of shallow embeddings is that they do not generalize well (if at all) across graphs. On the other hand, Graph Neural Networks, which leverage node features, have achieved good results on many inductive graph embedding tasks.

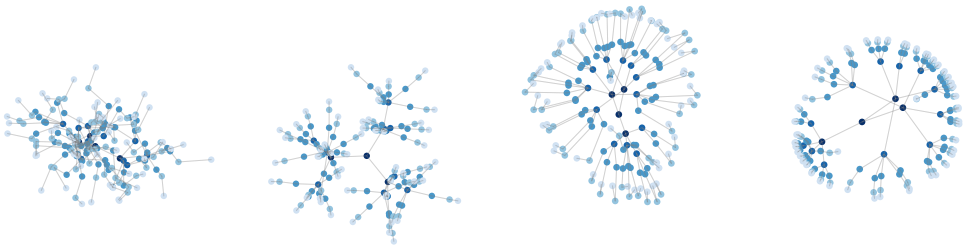
It is natural then, that there has been recent interest in extending Graph Neural Networks to learn non-Euclidean embeddings. One major challenge in doing so again revolves around the nature of convolution itself. How should we perform convolutions in a non-Euclidean space, where standard operations such as inner products and matrix multiplications are not defined?

Hyperbolic Graph Convolution Networks (HGCN) [Cha+19a] and Hyperbolic Graph Neural Networks (HGNN) [LNK19] apply graph convolutions in hyperbolic space by leveraging the Euclidean tangent space, which provides a first-order approximation of the hyperbolic manifold at a point. For every graph convolution step, node embeddings are mapped to the Euclidean tangent space at the origin, where convolutions are applied, and then mapped back to the hyperbolic space. These approaches yield significant improvements on graphs that exhibit hierarchical structure (Figure 23.6).

23.5 Deep graph embeddings

In this section, we use graph neural networks to devise graph embeddings in the unsupervised and semi-supervised cases.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) GCN layers. (b) HGCN layers.

Figure 23.6: Euclidean (left) and hyperbolic (right) embeddings of a tree graph. Hyperbolic embeddings learn natural hierarchies in the embedding space (depth indicated by color). Reprinted with permission from [Cha+19a].

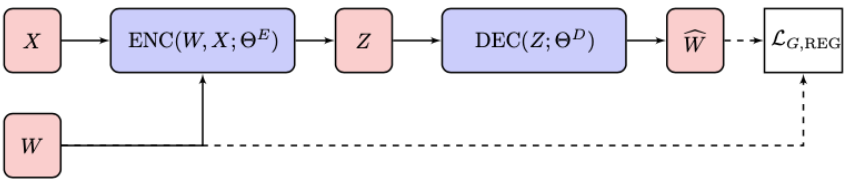


Figure 23.7: Unsupervised graph neural networks. Graph structure and input features are mapped to low-dimensional embeddings using a graph neural network encoder. Embeddings are then decoded to compute a graph regularization loss (unsupervised). Reprinted with permission from [Cha+21].

23.5.1 Unsupervised embeddings

In this section, we discuss unsupervised losses for GNNs, as illustrated in Figure 23.7.

23.5.1.1 Structural deep network embedding

The **structural deep network embedding** (SDNE) method of [WCZ16] uses auto-encoders which preserve first and second-order node proximity. The SDNE encoder takes a row of the adjacency matrix as input (setting $s(\mathbf{W}) = \mathbf{W}$) and produces node embeddings $\mathbf{Z} = \text{ENC}(\mathbf{W}; \theta^E)$. (Note that this ignores any node features.) The SDNE decoder returns $\widehat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \Theta^D)$, a reconstruction trained to recover the original graph adjacency matrix. SDNE preserves second order node proximity by minimizing the following loss:

$$||(s(\mathbf{W}) - \widehat{\mathbf{W}}) \cdot \mathbb{I}\left(s(\mathbf{W}) > 0\right)||_F^2 + \alpha_{\text{SDNE}} \sum_{ij} s(\mathbf{W})_{ij} ||\mathbf{Z}_i - \mathbf{Z}_j||_2^2 \tag{23.30}$$

The first term is similar to the matrix factorization regularization objective, except that $\widehat{\mathbf{W}}$ is not computed using outer products. The second term is used by distance-based shallow embedding methods.

23.5.1.2 (Variational) graph auto-encoders

Kipf and Welling [KW16b] use graph convolutions (Section 23.4.2) to learn node embeddings $\mathbf{Z} = \text{GCN}(\mathbf{W}, \mathbf{X}; \Theta^E)$. The decoder is an outer product: $\text{DEC}(\mathbf{Z}; \Theta^D) = \mathbf{Z}\mathbf{Z}^\mathsf{T}$. The graph reconstruction term is the sigmoid cross entropy between the true adjacency and the predicted edge similarity scores:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = -\left(\sum_{i,j} (1 - \mathbf{W}_{ij}) \log(1 - \sigma(\widehat{\mathbf{W}}_{ij})) + \mathbf{W}_{ij} \log \sigma(\widehat{\mathbf{W}}_{ij}) \right). \tag{23.31}$$

Computing the regularization term over all possible nodes pairs is computationally challenging in practice, so the Graph Auto Encoders (GAE) model uses negative sampling to overcome this challenge.

Whereas GAE is a deterministic model, the authors also introduce variational graph auto-encoders (VGAE), which rely on variational auto-encoders (as in Section 20.3.5) to encode and decode the graph structure. In VGAE, the embedding $\mathbf{Z}$ is modeled as a latent variable with a standard multivariate normal prior $p(\mathbf{Z}) = \mathcal{N}(\mathbf{Z}|\mathbf{0}, \mathbf{I})$ and a graph convolution is used as the amortized inference network, $q_\Phi(\mathbf{Z}|\mathbf{W}, \mathbf{X})$. The model is trained by minimizing the corresponding negative evidence lower bound:

$$\text{NELBO}(\mathbf{W}, \mathbf{X}; \Theta) = -\mathbb{E}_{q_\Phi(\mathbf{Z}|\mathbf{W},\mathbf{X})}[\log p(\mathbf{W}|\mathbf{Z})] + \text{KL}(q_\Phi(\mathbf{Z}|\mathbf{W}, \mathbf{X})||p(\mathbf{Z})) \tag{23.32}$$

$$= \mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) + \text{KL}(q_\Phi(\mathbf{Z}|\mathbf{W}, \mathbf{X})||p(\mathbf{Z})). \tag{23.33}$$

23.5.1.3 Iterative generative modelling of graphs (Graphite)

The **graphite** model of [GZE19] extends GAE and VGAE by introducing a more complex decoder. This decoder iterates between pairwise decoding functions and graph convolutions, as follows:

$$\widehat{\mathbf{W}}^{(k)} = \frac{\mathbf{Z}^{(k)}\mathbf{Z}^{(k)\mathsf{T}}}{||\mathbf{Z}^{(k)}||_2^2} + \frac{\mathbf{1}\mathbf{1}^\mathsf{T}}{N}$$

$$\mathbf{Z}^{(k+1)} = \text{GCN}(\widehat{\mathbf{W}}^{(k)}, \mathbf{Z}^{(k)})$$

where $\mathbf{Z}^{(0)}$ is initialized using the output of the encoder network. This process allows Graphite to learn more expressive decoders. Finally, similar to GAE, Graphite can be deterministic or variational.

23.5.1.4 Methods based on contrastive losses

The **deep graph infomax** method of [Vel+19] is a GAN-like method for creating graph-level embeddings. Given one or more *real* (positive) graphs, each with its adjacency matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$ and node features $\mathbf{X} \in \mathbb{R}^{N \times D}$, this method creates *fake* (negative) adjacency matrices $\mathbf{W}^- \in \mathbb{R}^{N^- \times N^-}$ and their features $X^- \in \mathbb{R}^{N^- \times D}$. It trains (i) an encoder that processes both real and fake samples, respectively giving $Z = \text{ENC}(\mathbf{X}, \mathbf{W}; \Theta^E) \in \mathbb{R}^{N \times L}$ and $\mathbf{Z}^- = \text{ENC}(\mathbf{X}^-, \mathbf{W}^-; \Theta^E) \in \mathbb{R}^{N^- \times L}$, (ii) a (readout) graph pooling function $\mathcal{R} : \mathbb{R}^{N \times L} \to \mathbb{R}^L$, and (iii) a discriminator function $\mathcal{D} : \mathbb{R}^L \times \mathbb{R}^L \to [0, 1]$ which is trained to output $\mathcal{D}(\mathbf{Z}_i, \mathcal{R}(\mathbf{Z})) \approx 1$ and $\mathcal{D}(\mathbf{Z}_j^-, \mathcal{R}(\mathbf{Z}^-)) \approx 0$, respectively, for nodes corresponding to given graph $i \in V$ and fake graph $j \in V^-$. Specifically, DGI optimizes:

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

$$\min_{\Theta} - \mathop{\mathbb{E}}_{\mathbf{X},\mathbf{W}} \sum_{i=1}^{N} \log \mathcal{D}(\mathbf{Z}_i, \mathcal{R}(\mathbf{Z})) - \mathop{\mathbb{E}}_{\mathbf{X}^-,\mathbf{W}^-} \sum_{j=1}^{N^-} \log \left(1 - \mathcal{D}(\mathbf{Z}_j^-, \mathcal{R}(\mathbf{Z}^-))\right), \tag{23.34}$$

where Θ contains Θ^E and the parameters of $\mathcal{R}, \mathcal{D}$. In the first expectation, DGI samples from the real (positive) graphs. If only one graph is given, it could sample some subgraphs from it (e.g. connected components). The second expectation samples fake (negative) graphs. In DGI, fake samples use the real adjacency $W^- := W$ but fake features X^- are a row-wise random permutation of real X. The ENC used in DGI is a graph convolutional network, though any GNN can be used. The readout $\mathcal{R}$ summarizes an entire (variable-size) graph to a single (fixed-dimension) vector. Veličković et al. [Vel+19] use $\mathcal{R}$ as a row-wise mean, though other graph pooling might be used e.g. ones aware of the adjacency.

The optimization of Equation (23.34) is shown by [Vel+19] to maximize a lower-bound on the mutual information between the outputs of the encoder and the graph pooling function, i.e., between individual node representations and the graph representation.

In [Pen+20] they present a variant called **Graphical Mutual Information**. Rather than maximizing MI of node information and an entire graph, GMI maximizes the MI between the representation of a node and its neighbors.

23.5.2 Semi-supervised embeddings

In this section, we discuss semi-supervised losses for GNNs. We consider the simple special case in which we use a nonlinear encoder of the node features, but ignore the graph structure, i.e., we use $\mathbf{Z} = \text{ENC}(\mathbf{X}; \Theta^E)$.

23.5.2.1 SemiEmb

[WRC08] propose an approach called **semi-supervised embeddings** (SemiEmb) They use an MLP for the encoder of $\mathbf{X}$. For the decoder, we can use a distance-based graph decoder: $\widehat{\mathbf{W}}_{ij} = \text{DEC}(\mathbf{Z}; \Theta^D)_{ij} = ||\mathbf{Z}_i - \mathbf{Z}_j||^2$, where $||\cdot||$ can be the L2 or L1 norm.

SemiEmb regularizes intermediate or auxiliary layers in the network using the same regularizer as the label propagation loss in Equation (23.19). SemiEmb uses a feed forward network to predict labels from intermediate embeddings, which are then compared to ground truth labels using the Hinge loss.

23.5.2.2 Planetoid

Unsupervised skip-gram methods like DeepWalk and node2vec learn embeddings in a multi-step pipeline, where random walks are first generated from the graph and then used to learn embeddings. These embeddings are likely not optimal for downstream classification tasks. The **Planetoid** method of [YCS16] extends such random walk methods to leverage node label information during the embedding algorithm.

Planetoid first maps nodes to embeddings $\mathbf{Z} = [\mathbf{Z}^c || \mathbf{Z}^F] = \text{ENC}(\mathbf{X}; \Theta^E)$ using a neural network (again ignoring graph structure). The node embeddings $\mathbf{Z}^c$ capture structural information while the

node embeddings $\mathbf{Z}^F$ capture feature information. There are two variants, a transductive version that directly learns $\mathbf{Z}^c$ (as an embedding lookup), and an inductive model where $\mathbf{Z}^c$ is computed with parametric mappings that act on input features $\mathbf{X}$. The Planetoid objective contains both a supervised loss and a graph regularization loss. The graph regularization loss measures the ability to predict context using nodes embeddings:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = -\mathbb{E}_{(i,j,\gamma)} \log \sigma\left(\gamma \widehat{\mathbf{W}}_{ij}\right), \tag{23.35}$$

with $\widehat{\mathbf{W}}_{ij} = \mathbf{Z}_i \mathbf{Z}_j^\intercal$ and $\gamma \in \{-1, 1\}$ with $\gamma = 1$ if $(v_i, v_j) \in E$ is a positive pair and $\gamma = -1$ if (v_i, v_j) is a negative pair. The distribution under the expectation is directly defined through a sampling process.

The supervised loss in Planetoid is the negative log-likelihood of predicting the correct labels:

$$\mathcal{L}^N_{\text{SUP}}(y^N, \widehat{y}^N; \Theta) = -\frac{1}{|V_L|} \sum_{i|v_i \in V_L} \sum_{1 \le k \le C} y^N_{ik} \log \widehat{y}^N_{ik}, \tag{23.36}$$

where i is a node's index while k indicates label classes, and $\widehat{y}^N_i$ are computed using a neural network followed by a softmax activation, mapping $\mathbf{Z}_i$ to predicted labels.

23.6 Applications

There are many applications of graph embeddings, both unsupervised and supervised. We give some examples in the sections below.

23.6.1 Unsupervised applications

In this section, we discuss common unsupervised applications.

23.6.1.1 Graph reconstruction

A popular unsupervised graph application is graph reconstruction. In this setting, the goal is to learn mapping functions (which can be parametric or not) that map nodes onto a manifold which can reconstruct the graph. This is regarded as *unsupervised* in the sense that there is no supervision beyond the graph structure. Models can be trained by minimizing a reconstruction error, which is the error in recovering the original graph from learned embeddings. Several algorithms were designed specifically for this task, and we refer to Section 23.3.1 and Section 23.5.1 for some examples of reconstruction objectives. At a high level, graph reconstruction is similar to dimensionality reduction in the sense that the main goal is to summarize some input data into a low-dimensional embedding. Instead of compressing high dimensional vectors into low-dimensional ones as standard dimensionality reduction methods (e.g. PCA) do, the goal of graph reconstruction models is to compress data defined on graphs into low-dimensional vectors.

23.6.1.2 Link prediction

The goal in **link prediction** is to predict missing or unobserved links (e.g., links that may appear in the future for dynamic and temporal networks). Link prediction can also help identify spurious

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

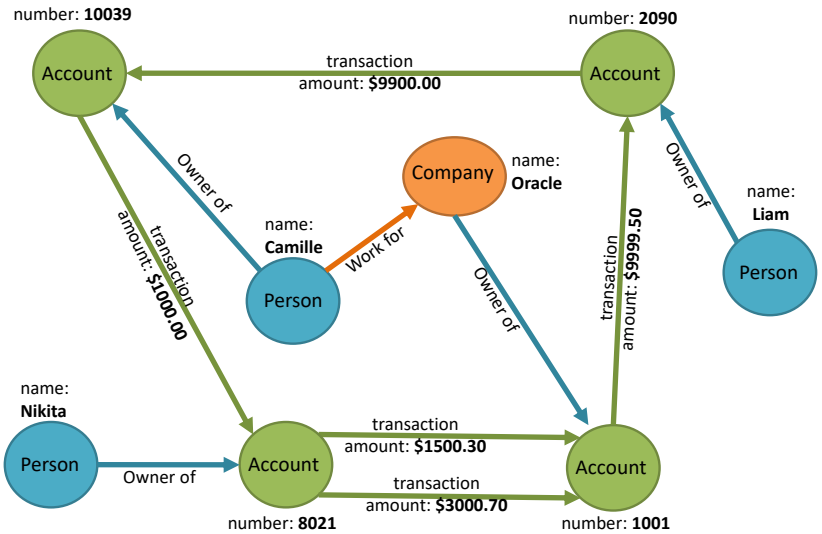


Figure 23.8: A graph representation of some financial transactions. Adapted from http://pgql-lang.org/spec/1.2/.

links and remove them. It is a major application of graph learning models in industry, and common example of applications include predicting friendships in **social networks**, predicting user-product interactions in **recommendation systems**, predicting suspicious links in a **fraud detection system** (see Figure 23.8), or predicting missing relationships between entities in a **knowledge graph** (see e.g., [Nic+15]).

A common approach for training link prediction models is to mask some edges in the graph (positive and negative edges), train a model with the remaining edges and then test it on the masked set of edges. Note that link prediction is different from graph reconstruction. In link prediction, we aim at predicting links that are not observed in the original graph while in graph reconstruction, we only want to compute embeddings that preserve the graph structure through reconstruction error minimization.

Finally, while link prediction has similarities with supervised tasks in the sense that we have labels for edges (positive, negative, unobserved), we group it under the unsupervised class of applications since edge labels are usually not used during training, but only used to measure the predictive quality of embeddings.

23.6.1.3 Clustering

Clustering is particularly useful for discovering communities and has many real-world applications. For instance, clusters exist in biological networks (e.g. as groups of proteins with similar properties), or in social networks (e.g. as groups of people with similar interests).

The unsupervised methods introduced in this chapter can be used to solve clustering problems

by applying the clustering algorithm (e.g. k-means) to embeddings that are output by an encoder. Further, clustering can be joined with the learning algorithm while learning a shallow [Roz+19] or Graph Convolution [Chi+19a; CEL19] embedding model.

23.6.1.4 Visualization

There are many off-the-shelf tools for mapping graph nodes onto two-dimensional manifolds for the purpose of visualization. Visualizations allow network scientists to qualitatively understand graph properties, understand relationships between nodes or visualize node clusters. Among the popular tools are methods based on *Force-Directed Layouts*, with various web-app Javascript implementations.

Unsupervised graph embedding methods are also used for visualization purposes: by first training an encoder-decoder model (corresponding to a shallow embedding or graph convolution network), and then mapping every node representation onto a two-dimensional space using t-SNE (Section 20.4.10) or PCA (Section 20.1). Such a process (embedding $\rightarrow$ dimensionality reduction) is commonly used to qualitatively evaluate the performance of graph learning algorithms. If nodes have attributes, one can use these attributes to color the nodes on 2D visualization plots. Good embedding algorithms embed nodes that have similar attributes nearby in the embedding space, as demonstrated in visualizations of various methods [PARS14; KW16a; AEH+18]. Finally, beyond mapping every node to a 2D coordinate, methods which map every graph to a representation [ARZP19] can similarly be projected into two dimensions to visualize and qualitatively analyze graph-level properties.

23.6.2 Supervised applications

In this section, we discuss common supervised applications.

23.6.2.1 Node classification

Node classification is an important supervised graph application, where the goal is to learn node representations that can accurately predict node labels. (This is sometimes called **statistical relational learning** [GT07].) For instance, node labels could be scientific topics in citation networks, or gender and other attributes in social networks.

Since labeling large graphs can be time-consuming and expensive, semi-supervised node classification is a particularly common application. In semi-supervised settings, only a fraction of nodes are labeled and the goal is to leverage links between nodes to predict attributes of unlabeled nodes. This setting is transductive since there is only one partially labeled fixed graph. It is also possible to do inductive node classification, which corresponds to the task of classifying nodes in multiple graphs.

Note that node features can significantly boost the performance on node classification tasks if these are descriptive for the target label. Indeed, recent methods such as GCN (Section 23.4.2) GraphSAGE (Section 23.4.3.1) have achieved state-of-the-art performance on multiple node classification benchmarks due to their ability to combine structural information and semantics coming from features. On the other hand, other methods such as random walks on graphs fail to leverage feature information and therefore achieve lower performance on these tasks.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

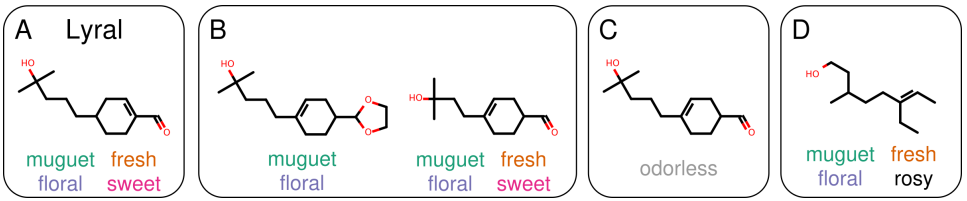


Figure 23.9: Structurally similar molecules do not necessarily have similar odor descriptors. (A) Lyral, the reference molecule. (B) Molecules with similar structure can share similar odor descriptors. (C) However, a small structural change can render the molecule odorless. (D) Further, large structural changes can leave the odor of the molecule largely unchanged. From Figure 1 of [SL+19], originally from [OPK12]. Used with kind permission of Benjamin Sanchez-Lengeling.

23.6.2.2 Graph classification

Graph classification is a supervised application where the goal is to predict graph labels. Graph classification problems are inductive and a common example is classifying chemical compounds (e.g. predicting toxicity or odor from a molecule, as shown in Figure 23.9).

Graph classification requires some notion of pooling, in order to aggregate node-level information into graph-level information. As discussed earlier, generalizing this notion of pooling to arbitrary graphs is non trivial because of the lack of regularity in the graph structure making graph pooling an active research area. In addition to the supervised methods discussed above, a number of unsupervised methods for learning graph-level representations have been proposed [Tsi+18; ARZP19; TMP20].

A Notation

A.1 Introduction

It is very difficult to come up with a single, consistent notation to cover the wide variety of data, models and algorithms that we discuss in this book. Furthermore, conventions differ between different fields (such as machine learning, statistics and optimization), and between different books and papers within the same field. Nevertheless, we have tried to be as consistent as possible. Below we summarize most of the notation used in this book, although individual sections may introduce new notation. Note also that the same symbol may have different meanings depending on the context, although we try to avoid this where possible.

A.2 Common mathematical symbols

We list some common symbols below.

Symbol	Meaning
∞	Infinity
$\rightarrow$	Tends towards, e.g., $n \rightarrow \infty$
$\propto$	Proportional to, so $y = ax$ can be written as $y \propto x$
$\triangleq$	Defined as
$O(\cdot)$	Big-O: roughly means order of magnitude
$\mathbb{Z}_+$	The positive integers
$\mathbb{R}$	The real numbers
$\mathbb{R}_+$	The positive reals
$\mathcal{S}_K$	The K-dimensional probability simplex
$\mathcal{S}_{++}^D$	Cone of positive definite $D \times D$ matrices
$\approx$	Approximately equal to
$\{1, \ldots, N\}$	The finite set $\{1, 2, \ldots, N\}$
$1 : N$	The finite set $\{1, 2, \ldots, N\}$
$[\ell, u]$	The continuous interval $\{\ell \leq x \leq u\}$.

A.3 Functions

Generic functions will be denoted by f (and sometimes g or h). We will encounter many named functions, such as $\tanh(x)$ or $\sigma(x)$. A scalar function applied to a vector is assumed to be applied elementwise, e.g., $\boldsymbol{x}^2 = [x_1^2, \ldots, x_D^2]$. Functionals (functions of a function) are written using "blackboard" font, e.g., $\mathbb{H}(p)$ for the entropy of a distribution p. A function parameterized by fixed parameters $\boldsymbol{\theta}$ will be denoted by $f(\boldsymbol{x}; \boldsymbol{\theta})$ or sometimes $f_{\boldsymbol{\theta}}(\boldsymbol{x})$. We list some common functions (with no free parameters) below.

A.3.1 Common functions of one argument

Symbol	Meaning
$\lfloor x \rfloor$	Floor of x, i.e., round down to nearest integer
$\lceil x \rceil$	Ceiling of x, i.e., round up to nearest integer
$\neg a$	logical NOT
$\mathbb{I}(x)$	Indicator function, $\mathbb{I}(x) = 1$ if x is true, else $\mathbb{I}(x) = 0$
$\delta(x)$	Dirac delta function, $\delta(x) = \infty$ if $x = 0$, else $\delta(x) = 0$
$\lvert x \rvert$	Absolute value
$\lvert \mathcal{S} \rvert$	Size (cardinality) of a set
$n!$	Factorial function
$\log(x)$	Natural logarithm of x
$\exp(x)$	Exponential function e^x
$\Gamma(x)$	Gamma function, $\Gamma(x) = \int_0^\infty u^{x-1} e^{-u} du$
$\Psi(x)$	Digamma function, $\Psi(x) = \frac{d}{dx} \log \Gamma(x)$
$\sigma(x)$	Sigmoid (logistic) function, $\frac{1}{1+e^{-x}}$

A.3.2 Common functions of two arguments

Symbol	Meaning
$a \wedge b$	logical AND
$a \vee b$	logical OR
$B(a, b)$	Beta function, $B(a, b) = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}$
$\binom{n}{k}$	n choose k, equal to $n!/(k!(n-k)!)$
δ_{ij}	Kronecker delta, equals $\mathbb{I}(i = j)$
$\boldsymbol{u} \odot \boldsymbol{v}$	Elementwise product of two vectors
$\boldsymbol{u} \circledast \boldsymbol{v}$	Convolution of two vectors

A.3.3 Common functions of > 2 arguments

Symbol	Meaning
$B(\boldsymbol{x})$	Multivariate beta function, $\frac{\prod_k \Gamma(x_k)}{\Gamma(\sum_k x_k)}$
$\Gamma(\boldsymbol{x})$	Multi. gamma function, $\pi^{D(D-1)/4} \prod_{d=1}^{D} \Gamma\left(x + (1-d)/2\right)$

$\text{softmax}(\boldsymbol{x})$	Softmax function, $[\frac{e^{x_c}}{\sum_{c'=1}^{C} e^{x_{c'}}}]_{c=1}^{C}$

A.4 Linear algebra

In this section, we summarize the notation we use for linear algebra (see Chapter 7 for details).

A.4.1 General notation

Vectors are bold lower case letters such as $\boldsymbol{x}$, $\boldsymbol{w}$. Matrices are bold upper case letters, such as $\mathbf{X}$, $\mathbf{W}$. Scalars are non-bold lower case. When creating a vector from a list of N scalars, we write $\boldsymbol{x} = [x_1, \ldots, x_N]$; this may be a column vector or a row vector, depending on the context. (Vectors are assumed to be column vectors, unless noted otherwise.) When creating an $M \times N$ matrix from a list of vectors, we write $\mathbf{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ if we stack along the columns, or $\mathbf{X} = [\boldsymbol{x}_1; \ldots; \boldsymbol{x}_M]$ if we stack along the rows.

A.4.2 Vectors

Here is some standard notation for vectors. (We assume $\boldsymbol{u}$ and $\boldsymbol{v}$ are both N-dimensional vectors.)

Symbol	Meaning
$\boldsymbol{u}^\mathsf{T}\boldsymbol{v}$	Inner (scalar) product, $\sum_{i=1}^{N} u_i v_i$
$\boldsymbol{u}\boldsymbol{v}^\mathsf{T}$	Outer product ($N \times N$ matrix)
$\boldsymbol{u} \odot \boldsymbol{v}$	Elementwise product, $[u_1 v_1, \ldots, u_N v_N]$
$\boldsymbol{v}^\mathsf{T}$	Transpose of $\boldsymbol{v}$
$\dim(\boldsymbol{v})$	Dimensionality of $\boldsymbol{v}$ (namely N)
$\text{diag}(\boldsymbol{v})$	Diagonal $N \times N$ matrix made from vector $\boldsymbol{v}$
$\mathbf{1}$ or $\mathbf{1}_N$	Vector of ones (of length N)
$\mathbf{0}$ or $\mathbf{0}_N$	Vector of zeros (of length N)
$\|\|\boldsymbol{v}\|\| = \|\|\boldsymbol{v}\|\|_2$	Euclidean or ℓ_2 norm $\sqrt{\sum_{i=1}^{N} v_i^2}$
$\|\|\boldsymbol{v}\|\|_1$	ℓ_1 norm $\sum_{i=1}^{N} \|v_i\|$

A.4.3 Matrices

Here is some standard notation for matrices. (We assume $\mathbf{S}$ is a square $N \times N$ matrix, $\mathbf{X}$ and $\mathbf{Y}$ are of size $M \times N$, and $\mathbf{Z}$ is of size $M' \times N'$.)

Symbol	Meaning
$\mathbf{X}_{:,j}$	j'th column of matrix
$\mathbf{X}_{i,:}$	i'th row of matrix (treated as a column vector)
X_{ij}	Element (i, j) of matrix
$\mathbf{S} \succ 0$	True iff $\mathbf{S}$ is a positive definite matrix
$\text{tr}(\mathbf{S})$	Trace of a square matrix
$\det(\mathbf{S})$	Determinant of a square matrix
$\|\mathbf{S}\|$	Determinant of a square matrix

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

$\mathbf{S}^{-1}$	Inverse of a square matrix
$\mathbf{X}^\dagger$	Pseudo-inverse of a matrix
$\mathbf{X}^\mathsf{T}$	Transpose of a matrix
$\text{diag}(\mathbf{S})$	Diagonal vector extracted from square matrix
$\mathbf{I}$ or $\mathbf{I}_N$	Identity matrix of size $N \times N$
$\mathbf{X} \odot \mathbf{Y}$	Elementwise product
$\mathbf{X} \otimes \mathbf{Z}$	Kronecker product (see Section 7.2.5)

A.4.4 Matrix calculus

In this section, we summarize the notation we use for matrix calculus (see Section 7.8 for details).

Let $\boldsymbol{\theta} \in \mathbb{R}^N$ be a vector and $f : \mathbb{R}^N \to \mathbb{R}$ be a scalar valued function. The derivative of f wrt its argument is denoted by the following:

$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \triangleq \nabla f(\boldsymbol{\theta}) \triangleq \nabla f \triangleq \begin{pmatrix} \frac{\partial f}{\partial \theta_1} & \cdots & \frac{\partial f}{\partial \theta_N} \end{pmatrix} \tag{A.1}$$

The gradient is a vector that must be evaluated at a point in space. To emphasize this, we will sometimes write

$$\boldsymbol{g}_t \triangleq \boldsymbol{g}(\boldsymbol{\theta}_t) \triangleq \nabla f(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}_t} \tag{A.2}$$

We can also compute the (symmetric) $N \times N$ matrix of second partial derivatives, known as the **Hessian**:

$$\nabla^2 f \triangleq \begin{pmatrix} \frac{\partial^2 f}{\partial \theta_1^2} & \cdots & \frac{\partial^2 f}{\partial \theta_1 \partial \theta_N} \\ & \vdots & \\ \frac{\partial^2 f}{\partial \theta_N \theta_1} & \cdots & \frac{\partial^2 f}{\partial \theta_N^2} \end{pmatrix} \tag{A.3}$$

The Hessian is a matrix that must be evaluated at a point in space. To emphasize this, we will sometimes write

$$\mathbf{H}_t \triangleq \mathbf{H}(\boldsymbol{\theta}_t) \triangleq \nabla^2 f(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}_t} \tag{A.4}$$

A.5 Optimization

In this section, we summarize the notation we use for optimization (see Chapter 8 for details).

We will often write an objective or cost function that we wish to minimize as $\mathcal{L}(\boldsymbol{\theta})$, where $\boldsymbol{\theta}$ are the variables to be optimized (often thought of as parameters of a statistical model). We denote the parameter value that achieves the minimum as $\boldsymbol{\theta}_* = \text{argmin}_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\boldsymbol{\theta})$, where Θ is the set we are optimizing over. (Note that there may be more than one such optimal value, so we should really write $\boldsymbol{\theta}_* \in \text{argmin}_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\boldsymbol{\theta})$.)

When performing iterative optimization, we use t to index the iteration number. We use η as a step size (learning rate) parameter. Thus we can write the gradient descent algorithm (explained in Section 8.4) as follows: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \boldsymbol{g}_t$.

We often use a hat symbol to denote an estimate or prediction (e.g., $\hat{\boldsymbol{\theta}}$, $\hat{y}$), a star subscript or superscript to denote a true (but usually unknown) value (e.g., $\boldsymbol{\theta}_*$ or $\boldsymbol{\theta}^*$), an overline to denote a mean value (e.g., $\overline{\boldsymbol{\theta}}$).

A.6 Probability

In this section, we summarize the notation we use for probability theory (see Chapter 2 for details).

We denote a probability density function (pdf) or probability mass function (pmf) by p, a cumulative distribution function (cdf) by P, and the probability of a binary event by Pr. We write $p(X)$ for the distribution for random variable X, and $p(Y)$ for the distribution for random variable Y — these refer to different distributions, even though we use the same p symbol in both cases. (In cases where confusion may arise, we write $p_X(\cdot)$ and $p_Y(\cdot)$.) Approximations to a distribution p will often be represented by q, or sometimes $\hat{p}$.

In some cases, we distinguish between a random variable (rv) and the values it can take on. In this case, we denote the variable in upper case (e.g., X), and its value in lower case (e.g., x). However, we often ignore this distinction between variables and values. For example, we sometimes write $p(x)$ to denote either the scalar value (the distribution evaluated at a point) or the distribution itself, depending on whether X is observed or not.

We write $X \sim p$ to denote that X is distributed according to distribution p. We write $X \perp Y \mid Z$ to denote that X is conditionally independent of Y given Z. If $X \sim p$, we denote the expected value of $f(X)$ using

$$\mathbb{E}\left[f(X)\right] = \mathbb{E}_{p(X)}\left[f(X)\right] = \mathbb{E}_X\left[f(X)\right] = \int_x f(x)p(x)dx \tag{A.5}$$

If f is the identity function, we write $\overline{X} \triangleq \mathbb{E}\left[X\right]$. Similarly, the variance is denoted by

$$\mathbb{V}\left[f(X)\right] = \mathbb{V}_{p(X)}\left[f(X)\right] = \mathbb{V}_X\left[f(X)\right] = \int_x (f(x) - \mathbb{E}\left[f(X)\right])^2 p(x)dx \tag{A.6}$$

If $\boldsymbol{x}$ is a random vector, the covariance matrix is denoted

$$\mathrm{Cov}\left[\boldsymbol{x}\right] = \mathbb{E}\left[(\boldsymbol{x} - \overline{\boldsymbol{x}})(\boldsymbol{x} - \overline{\boldsymbol{x}})^{\mathsf{T}}\right] \tag{A.7}$$

If $X \sim p$, the mode of a distribution is denoted by

$$\hat{x} = \mathrm{mode}\left[p\right] = \operatorname*{argmax}_x p(x) \tag{A.8}$$

We denote parametric distributions using $p(\boldsymbol{x}|\boldsymbol{\theta})$, where $\boldsymbol{x}$ are the random variables, $\boldsymbol{\theta}$ are the parameters and p is a pdf or pmf. For example, $\mathcal{N}(x|\mu, \sigma^2)$ is a Gaussian (normal) distribution with mean μ and standard deviation σ.

A.7 Information theory

In this section, we summarize the notation we use for information theory (see Chapter 6 for details).

If $X \sim p$, we denote the (differential) entropy of the distribution by $\mathbb{H}(X)$ or $\mathbb{H}(p)$. If $Y \sim q$, we denote the KL divergence from distribution p to q by $D_{\mathbb{KL}}(p \parallel q)$. If $(X, Y) \sim p$, we denote the mutual information between X and Y by $\mathbb{I}(X; Y)$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

A.8 Statistics and machine learning

We briefly summarize the notation we use for statistical learning.

A.8.1 Supervised learning

For supervised learning, we denote the observed features (also called inputs or **covariates**) by $\boldsymbol{x} \in \mathcal{X}$. Often $\mathcal{X} = \mathbb{R}^D$, meaning the features are real-valued. (Note that this includes the case of discrete-valued inputs, which can be represented as one-hot vectors.) Sometimes we compute manually-specified features of the input; we denote these by $\boldsymbol{\phi}(\boldsymbol{x})$. We also have outputs (also called **targets** or **response variables**) $\boldsymbol{y} \in \mathcal{Y}$ that we wish to predict. Our task is to learn a conditional probability distribution $p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ are the parameters of the model. If $\mathcal{Y} = \{1, \ldots, C\}$, we call this **classification**. If $\mathcal{Y} = \mathbb{R}^C$, we call this **regression** (often $C = 1$, so we are just predicting a scalar response).

The parameters $\boldsymbol{\theta}$ are estimated from **training data**, denoted by $\mathcal{D} = \{(\boldsymbol{x}_n, \boldsymbol{y}_n) : n \in \{1, \ldots, N\}\}$ (so N is the number of training cases). If $\mathcal{X} = \mathbb{R}^D$, we can store the training inputs in an $N \times D$ **design matrix** denoted by $\mathbf{X}$. If $\mathcal{Y} = \mathbb{R}^C$, we can store the training outputs in an $N \times C$ matrix $\mathbf{Y}$. If $\mathcal{Y} = \{1, \ldots, C\}$, we can represent each class label as a C-dimensional bit vector, with one element turned on (this is known as a **one-hot encoding**), so we can store the training outputs in an $N \times C$ binary matrix $\mathbf{Y}$.

A.8.2 Unsupervised learning and generative models

Unsupervised learning is usually formalized as the task of unconditional density estimation, namely modeling $p(\boldsymbol{x}|\boldsymbol{\theta})$. In some cases, we want to perform conditional density estimation; we denote the values we are conditioning on by $\boldsymbol{u}$, so the model becomes $p(\boldsymbol{x}|\boldsymbol{u}, \boldsymbol{\theta})$. This is similar to supervised learning, except that $\boldsymbol{x}$ is usually high dimensional (e.g., an image) and $\boldsymbol{u}$ is usually low dimensional (e.g., a class label or a text description).

In some models, we have **latent variables**, also called **hidden variables**, which are never observed in the training data. We call such models **latent variable models** (LVM). We denote the latent variables for data case n by $\boldsymbol{z}_n \in \mathcal{Z}$. Sometimes latent variables are known as **hidden variables**, and are denoted by $\boldsymbol{h}_n$. By contrast, the **visible variables** will be denoted by $\boldsymbol{v}_n$. Typically the latent variables are continuous or discrete, i.e., $\mathcal{Z} = \mathbb{R}^L$ or $\mathcal{Z} = \{1, \ldots, K\}$.

Most LVMs have the form $p(\boldsymbol{x}_n, \boldsymbol{z}_n|\boldsymbol{\theta})$; such models can be used for unsupervised learning. However, LVMs can also be used for supervised learning. In particular, we can either create a generative (unconditional) model of the form $p(\boldsymbol{x}_n, \boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{\theta})$, or a discriminative (conditional) model of the form $p(\boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{x}_n, \boldsymbol{\theta})$.

A.8.3 Bayesian inference

When working with Bayesian inference, we write the prior over the parameters as $p(\boldsymbol{\theta}|\boldsymbol{\xi})$, where $\boldsymbol{\xi}$ are the hyperparameters. For conjugate models, the posterior has the same form as the prior (by definition). We can therefore just update the hyperparameters from their prior value, $\breve{\boldsymbol{\xi}}$, to their posterior value, $\widehat{\boldsymbol{\xi}}$.

In variational inference (Section 4.6.8.3), we use $\boldsymbol{\psi}$ to represent the parameters of the variational posterior, i.e., $p(\boldsymbol{\theta}|\mathcal{D}) \approx q(\boldsymbol{\theta}|\boldsymbol{\psi})$. We optimize the ELBO wrt $\boldsymbol{\psi}$ to make this a good approximation.

When performing Monte Carlo sampling, we use a s subscript or superscript to denote a sample (e.g., $\boldsymbol{\theta}_s$ or $\boldsymbol{\theta}^s$).

A.9 Abbreviations

Here are some of the abbreviations used in the book.

Abbreviation	Meaning
cdf	Cumulative distribution function
CNN	Convolutional neural network
DAG	Directed acyclic graph
DML	Deep metric learning
DNN	Deep neural network
dof	Degrees of freedom
EB	Empirical Bayes
EM	Expectation maximization algorithm
GLM	Generalized linear model
GMM	Gaussian mixture model
HMC	Hamiltonian Monte Carlo
HMM	Hidden Markov model
iid	Independent and identically distributed
iff	If and only if
KDE	Kernel density estimation
KL	Kullback Leibler divergence
KNN	K nearest neighbor
LHS	Left hand side (of an equation)
LSTM	Long short term memory (a kind of RNN)
LVM	Latent variable model
MAP	Maximum A Posterior estimate
MCMC	Markov chain Monte Carlo
MLE	Maximum likelihood estimate
MLP	Multilayer perceptron
MSE	Mean squared error
NLL	Negative log likelihood
OLS	Ordinary least squares
psd	Positive definite (matrix)
pdf	Probability density function
pmf	Probability mass function
PNLL	Penalized NLL
PGM	Probabilistic graphical model
RNN	Recurrent neural network
RHS	Right hand side (of an equation)
RSS	Residual sum of squares
rv	Random variable
RVM	Relevance vector machine

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

SGD	Stochastic gradient descent
SSE	Sum of squared errors
SVI	Stochastic variational inference
SVM	Support vector machine
VB	Variational Bayes
wrt	With respect to

Index

à trous algorithm, **486**
AMSGrad, 301
AdaDelta, **300**
AdaGrad, **299**
Adam, **300**
Padam, 301
RMSProp, **299**
RPROP, **299**
Yogi, 301
1x1 convolution, **474**

abstractive summarization, **543**
action potential, **436**
actions, **167**
activation function, **428**
activation maximization, **495**
active, **305**
active learning, 407, **650**
active set, **399**
activity regularization, **683**
Adaboost.M1, **615**
AdaBoostClassifier, 615
Adam, 447
Adamic/Adar, 758
adapters, **631**
adaptive basis functions, **611**
adaptive instance normalization, **501**
adaptive learning rate, **299**, **301**
add-one smoothing, **122**, **132**, **335**
additive attention, **521**
additive model, **611**
adjoint, **445**
adjusted Rand index, **717**
admissible, **192**
ADOPT, **301**
affine function, **8**, 8
agent, **17**, **167**
aggregated gradient, **298**
AGI, **28**
AI, **28**
AI ethics, **28**
AI safety, **28**
Akaike information criterion, **186**
aleatoric uncertainty, **7**, **34**
AlexNet, **481**
alignment, **522**
alignment problem, **28**
all pairs, **593**
all-reduce, **454**
ALS, **743**
alternating least squares, **743**
alternative hypothesis, **179**, **198**
ambient dimensionality, **690**
amortized inference, **685**
anchor, **555**
anchor boxes, **490**
ANN, **436**
Anscombe's quartet, **43**, 43
approximate posterior inference, **151**
approximation error, **194**
ARD, **411**, **570**, **598**
ARD kernel, **570**
area under the curve, **173**
Armijo backtracking method, **286**
Armijo-Goldstein, **286**
artificial general intelligence, **28**
artificial intelligence, **28**
artificial neural networks, **436**
associative, **242**
asymptotic normality, **155**
asymptotically optimal, **160**
asynchronous training, **454**
atomic bomb, 73
attention, **518**, **522**
attention kernel, **654**
attention score, **519**
attention weight, **519**
AUC, **173**
augmented intelligence, **28**
auto-covariance matrix, **79**
AutoAugment, **627**
autocorrelation matrix, **79**
autodiff, **438**
autoencoder, **679**
automatic differentiation, **438**
automatic relevancy determination, **411**, **570**, **598**
AutoML, **485**
AutoRec, **745**
autoregressive model, **101**
average link clustering, **720**
average pooling, **475**
average precision, **175**
axis aligned, **82**
axis parallel splits, **603**
axon, 436

B-splines, **399**
backbone, **433**
backfitting, **402**
backpropagation, **428**
backpropagation algorithm, **438**
backpropagation through time, **510**
backslash operator, **267**
backsubstitution, **267**, 375
bag, **655**
bag of word embeddings, **26**
bag of words, **24**, 432
bagging, **609**
BALD, **651**
balloon kernel density estimator, **563**
band-diagonal matrix, **239**
bandwidth, **459**, **560**, **568**
Barnes-Hut algorithm, **705**
barycentric coordinates, **698**
base measure, **93**
basis, **233**
basis function expansion, **425**
basis vectors, **243**
batch learning, **119**
batch normalization, **477**, **478**
batch renormalization, **478**
BatchBALD, **651**
Bayes decision rule, **168**
Bayes error, **127**, 547
Bayes estimator, **168**, **191**
Bayes factor, **180**
Bayes model averaging, **129**
Bayes risk, **191**
Bayes rule, 45
Bayes rule for Gaussians, **87**
Bayes' rule, **45**, 45
Bayes's rule, 45
Bayesian, **33**
Bayesian χ^2-test, **188**
Bayesian active learning by disagreement, **651**
Bayesian decision theory, **167**
Bayesian deep learning, **457**
Bayesian factor regression, **677**
Bayesian inference, **44**, **46**
Bayesian information criterion, **185**
Bayesian machine learning, **147**
Bayesian model selection, 185
Bayesian network, **100**
Bayesian neural network, **457**
Bayesian Occam's razor, **183**
Bayesian optimization, **650**
Bayesian personalized ranking, **747**
Bayesian statistics, **129**, 154
Bayesian t-test, **188**
BBO, **319**
Beam search, **515**
belief state, **46**
Berkson's paradox, **102**
Bernoulli distribution, **49**
Bernoulli mixture model, **98**
BERT, **539**
Bessel function, 570
beta distribution, **63**, 121, 131
beta function, **63**
beta-binomial, **135**
BFGS, **290**
bi-tempered logistic regression, **362**
bias, **8**, **159**, **371**
bias-variance tradeoff, **161**, 459
BIC, **185**
BIC loss, **186**
BIC score, **186**, 728
biclustering, **737**
bidirectional RNN, **506**
big data, **3**
bigram model, **102**, 212
bijector, **67**
bilevel optimization, **195**
binary classification, **2**, **46**
binary connect, **311**
binary cross entropy, **342**
binary entropy function, **208**
binary logistic regression, **339**
binomial coefficient, **50**
binomial distribution, **50**, 50
binomial regression, **416**
BinomialBoost, **618**
BIO, **541**
BiT, **532**
bits, **207**
bivariate Gaussian, **81**
black swan paradox, **122**
blackbox, **319**
blackbox optimization, **319**
block diagonal, **239**
block structured matrices, 250
Blue Brain Project, **438**
BMA, **129**
BMM, **98**
BN, **477**
BNN, **457**
Boltzmann distribution, **55**
BookCrossing, **742**
Boolean logic, **34**
Boosting, **611**
bootstrap, **156**
bottleneck, **679**
bound optimization, **312**, 354
bounding boxes, **489**
bowl shape, **344**
box constraints, **308**
box plots, **44**
boxcar kernel, **560**, 562
branching factor, **212**
Brier score, **179**, 645
Brier Skill Score, **179**
Brownian motion, 571
byte-pair encoding, **26**

C-way N-shot classification, **653**
C4, **544**
C4.5, **605**
calculus, **269**
calculus of variations, 95
calibration plot, **420**
canonical correlation analysis, **679**
canonical form, **94**
canonical link function, **417**
canonical parameters, **93**
CART, **603**, 605
Cartesian, 68
Caser, **750**
CatBoost, **619**
categorical, **53**
categorical PCA, **677**

CatPCA, **677**
Cauchy, **63**
causal, 80
causal CNN, **517**
causal convolution, **518**
CBOW, 707, **708**, 708
CCA, **679**
cdf, **37**, **57**
center, 408
centering matrix, **113**, **247**, 696
central interval, **146**
central limit theorem, 60, **72**
centroids, **459**
chain rule for entropy, **211**
chain rule for mutual information, **219**
chain rule of calculus, **273**
chain rule of probability, **39**
change of variables, **67**
channels, **467**, **473**
characteristic equation, **254**
characteristic length scale, **570**
characteristic matrix, **222**
chatbots, **543**
ChatGPT, **542**, 543
Chi-squared distribution, **65**
Cholesky decomposition, 382
Cholesky factorization, **266**
Chow's rule, **170**
CIFAR, **20**
city block distance, **718**
class conditional density, **323**
class confusion matrix, **172**
class imbalance, **173**, **359**, 593
class-balanced sampling, **359**
classes, **2**
classical MDS, **692**
classical statistics, **154**
classification, **2**, **778**
Classification and regression trees, **603**
CLIP, **635**
closed world assumption, **550**
cloze, **539**
cloze task, **632**
cluster assumption, **640**
Clustering, **715**
clustering, **97**
clusters, **14**
CNN, **3**, **426**, **467**
co-adaptation, 455
Co-training, **642**
coclustering, **737**
code generation, **543**
codebook, **724**
coefficient of determination, **381**
CoLA, **543**
cold start, **750**
collaborative filtering, 737, **742**
column rank, **237**
column space, **233**
column vector, **229**
column-major order, **231**
committee method, **608**
commutative, **242**
compactness, **720**
comparison of classifiers, **187**
complementary log-log, **418**
complementary slackness, **305**
complete link clustering, **720**
completing the square, **88**
complexity penalty, **121**, **185**
composite objective, **282**
compositional, **434**
compound hypothesis, **199**
computation graph, **444**
computer graphics, **493**
concave, **278**
condition number, 123, **238**, 286
conditional computation, **463**
conditional distribution, **38**
conditional entropy, **210**
conditional instance normalization, **500**
conditional mixture model, **462**
conditional mutual information, **219**
conditional probability, **35**
conditional probability distribution, **7**, **50**, **100**
conditional probability table, **100**
conditional variance formula, **42**
conditionally independent, **35**, **39**, **99**
confidence interval, 146, **157**
confirmation bias, **639**
conformer, **533**
conjugate function, **279**
conjugate gradient, **286**, **375**
conjugate prior, **87**, **130**, 130, 131
consensus sequence, **209**
conservation of probability mass, **183**
Consistency regularization, **644**
consistent estimator, **192**, 406
constrained optimization, **277**
constrained optimization problem, **302**
constrained optimization problems, 403
constraints, **277**
contextual word embeddings, 26, **537**, **712**
contingency table, **189**
continual learning, **551**
continuation method, **398**
continuous optimization, **275**
continuous random variable, **36**
contraction, **683**
contractive autoencoder, **682**
contradicts, **523**
contrastive loss, **555**
contrastive tasks, **633**
control variate, 297
convex, **344**
convex combination, 133
convex function, **278**
convex optimization, **277**
convex relaxation, **386**
convex set, **277**
ConvNeXt, **485**, 533
convolution, **71**, **468**
convolution theorem, **71**
convolution with holes, **486**
convolutional Markov model, **517**
convolutional neural network, **3**, 21
convolutional neural networks, **12**, **426**, **467**
coordinate descent, **397**
coordinate vectors, **233**
coordinated based representations, **429**
coreference resolution, **527**
coreset, 559
corpus, **706**
correlation coefficient, **78**, **82**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

correlation does not imply causation, **79**
correlation matrix, **78**, 114
cosine kernel, **571**
cosine similarity, **706**
cost function, **275**
covariance, **77**
covariance matrix, **77**, **81**
covariates, **2**, **371**, **778**
COVID-19, **46**
CPD, **100**
CPT, **100**
Cramer-Rao lower bound, **160**
credible interval, **143**, **146**, 146
critical point, **303**
cross correlation, **469**
cross entropy, **209**, 214, **216**
cross validation, **126**, **196**
cross-covariance, **77**
cross-entropy, **178**
cross-over rate, **173**
cross-validated risk, **126**, **196**
crosscat, **739**
crowding problem, **703**
cubic splines, **399**
cumulants, **95**
cumulative distribution function, **37**, **57**
curse of dimensionality, **548**
curve fitting, 14
curved exponential family, **94**
CV, **126**, **196**
cyclic permutation property, **236**
cyclical learning rate, **296**

DAG, **100**, **444**
data augmentation, **216**, **627**
data compression, **16**, 725
data fragmentation, **605**
Data mining, **27**
data parallelism, **454**
data processing inequality, **223**
Data science, **27**
data uncertainty, **7**, **34**
Datasaurus Dozen, **43**, 44
dead ReLU, **450**
debiasing, **389**
decision boundary, **5**, **53**, **149**, **340**
decision making under uncertainty, **1**
decision rule, **5**
decision surface, **6**
decision tree, **6**
decision trees, **603**
decode, **657**
decoder, **679**, 687
deconvolution, **487**
deduction, **201**
deep CCA, **679**
deep factorization machines, **748**
deep graph infomax, **767**
deep metric learning, **552**, **554**
deep mixture of experts, **463**
deep neural networks, **12**, **425**
DeepDream, **497**
DeepWalk, **760**
default prior, **145**
defender's fallacy, **75**
deflated matrix, **713**
deflation, **259**
degree of normality, **62**
degrees of freedom, **13**, **62**, **383**
delta rule, **294**
demonstrations, 18
dendrites, **436**
dendrogram, **718**
denoising autoencoder, **681**
dense prediction, **492**
dense sequence labeling, **507**
DenseNets, **484**
density estimation, **16**
density kernel, 520, **560**
dependent variable, **371**
depth prediction, **492**
depthwise separable convolution, **488**
derivative, **269**
derivative free optimization, **319**
descent direction, **283**, **284**
design matrix, **3**, 245, 426, **778**
determinant, **237**
development set, **124**
deviance, **185**, **420**, **606**
DFO, **319**
diagonal covariance matrix, **83**
diagonal matrix, **239**
diagonalizable, **255**
diagonally dominant, **240**
diameter, **720**
differentiable decision tree, **606**
differentiable programming, **444**
differential entropy, **212**
differentiating under the integral sign, **71**
differentiation, 270
diffuse prior, **145**
dilated convolution, **486**, 491
dilation factor, **486**
dimensionality reduction, **4**, **657**
Dirac delta function, **60**, 148
directed acyclic graph, **100**, **444**
directional derivative, **270**
Dirichlet distribution, **138**, 334
Dirichlet energy, **701**
discrete AdaBoost, **614**
discrete optimization, **275**
discrete random variable, **35**
discretize, **213**, **220**
discriminant function, **324**
discriminative classifier, **323**, **336**
dispersion parameter, **415**
distance metric, **213**
distant supervision, **655**
distortion, **657**, **722**, **724**
distributional hypothesis, **705**
distributive, **242**
divergence measure, **213**
diverse beam search, **516**
DNA sequence motifs, **208**
DNN, **12**, **425**
document retrieval, **706**
document summarization, **22**
domain adaptation, 630, **637**
domain adversarial learning, **637**
dominates, **192**, **200**
dot product, **242**
double centering trick, **247**
double descent, **459**
double sided exponential, **63**

dropout, **455**
dual feasibility, **305**
dual form, **588**
dual problem, **587**
dual variables, **594**
dummy encoding, **23**, **53**
Dutch book theorem, 203
dynamic graph, **446**
dynamic programming, **515**

E step, **312**, **314**
early stopping, **126**, **455**
EB, **145**
echo state network, **511**
ECM, 670
economy sized QR, **265**
economy sized SVD, **260**
edge devices, **311**, **488**
EER, **173**
effect size, **187**
EfficientNetv2, **485**
eigenfaces, **659**
eigenvalue, **253**
eigenvalue decomposition, **253**
eigenvalue spectrum, 123
eigenvector, **253**
Einstein summation, **248**
einsum, **248**
elastic embedding, **702**
elastic net, 390, **396**
ELBO, **153**, **314**, **685**
elbow, **728**
electronic health records, **523**
ell-2 loss, 9
ELM, **584**
ELMo, **538**
ELPD, **187**
ELU, **450**
EM, 85, **312**
EMA, **119**
email spam classification, **22**
embarassingly parallel, **454**
embedding, **657**
EMNIST, **20**
empirical Bayes, **145**, 182, 411
empirical distribution, **65**, 73, **109**, 193, 215
empirical risk, **6**, **194**
empirical risk minimization, **7**, **115**, **194**, 293
encode, **657**
encoder, **679**, 687
encoder-decoder, **491**
encoder-decoder architecture, **509**
endogenous variables, **2**
energy based model, **635**
energy function, 152
ensemble, 296, **457**
ensemble learning, **608**
entails, **523**
entity discovery, **722**
entity linking, **551**
entity resolution, 542, **551**
entropy, **178**, **207**, 314, 606
entropy minimization, **639**
entropy SGD, **458**
Epanechnikov kernel, 561
epigraph, **278**
epistemic uncertainty, **7**, **34**
epistemology, 34
epoch, **293**
epsilon insensitive loss function, **595**
equal error rate, **173**
equality constraints, **277**, **302**
equitability, **222**
equivalent sample size, **131**
equivariance, **475**
ERM, **115**, **194**
error function, **57**
estimation error, **194**
estimator, **154**
EVD, **253**
event, **34**, 34, **35**, 35
events, **33**
evidence, **135**, **182**
evidence lower bound, **153**, **314**, **685**
EWMA, **119**, 299
exact line search, **286**
exchangeable, **102**
exclusive KL, **216**
exclusive or, 460
exemplar-based models, **547**
exemplars, **459**
exogenous variables, **2**
expectation maximization, **312**
expected complete data log likelihood, **315**
expected log pointwise predictive density, **187**
expected sufficient statistics, **315**
expected value, **40**, **58**
experiment design, **650**
explaining away, **102**
explanatory variables, **371**
explicit feedback, **741**
exploding gradient problem, **447**
exploration-exploitation tradeoff, **751**
exploratory data analysis, **4**
exponential dispersion family, **415**
Exponential distribution, **65**
exponential family, **93**, 96, 144
exponential family factor analysis, **675**
exponential family PCA, **675**
Exponential linear unit, 448
exponential loss, **614**
exponential moving average, **119**
exponentially weighted moving average, **119**
exponentiated cross entropy, **212**
exponentiated quadratic kernel, **568**
extractive summarization, **543**
extreme learning machine, **584**

F score, 175
face detection, **489**
face recognition, **489**
face verification, **551**
FaceNet, **557**
factor analysis, **16**, **666**
factor loading matrix, **667**
factorization machine, **748**
FAISS, **550**
false alarm rate, **172**
false negative rate, **46**
false positive rate, **46**, **172**
fan-in, **453**
fan-out, **453**
Fano's inequality, **225**
farthest point clustering, **725**

Fashion-MNIST, **20**
fast adapation, **652**
fast Hadamard transform, **584**
fastfood, **584**
feasibility, **304**
feasibility problem, **277**
feasible set, **277**
feature crosses, **24**
feature detection, **471**
feature engineering, **12**
feature extraction, **12**
feature extractor, **372**
feature importance, **621**, 621
feature map, **471**
feature preprocessing, **12**
feature selection, **225**, **310**, **385**
features, **1**
featurization, **4**
feedforward neural network, **425**
few-shot classification, **551**
few-shot learning, **653**
FFNN, **425**
fill in, **27**
fill-in-the-blank, **539**, **632**
filter, **469**
filter response normalization, **479**
filters, **467**
FIM, **155**
fine-grained classification, **21**, **653**
fine-grained visual classification, **629**
fine-tune, **537**
fine-tuning phase, **629**
finite difference, **270**
finite sum problem, **293**
first order, **346**
first order Markov condition, **101**
first-order, **282**, **289**
Fisher information matrix, **155**, 348
Fisher scoring, **348**
Fisher's linear discriminant analysis, **328**
FISTA, **398**
FLAN-T5, **544**
flat local minimum, **276**
flat minima, **457**
flat prior, **144**
flatten, **431**
FLDA, **328**
folds, **126**, **196**
forget gate, **513**
forward mode differentiation, **439**
forward stagewise additive modeling, **612**
forwards KL, **216**
forwards model, **49**
founder variables, **673**
fraction of variance explained, **665**
fraud detection system, **770**
frequentist, **33**
frequentist decision theory, **189**
frequentist statistics, **154**
Frobenius norm, **236**
frozen parameters, **630**
full covariance matrix, **82**
full rank, **237**
full-matrix Adagrad, **301**
function space, **577**
furthest neighbor clustering, **720**
fused batchnorm, **477**
gallery, **490**, **550**
GAM, **401**
gamma distribution, **64**
GANs, 637
Gated Graph Sequence Neural Networks, **762**
gated recurrent units, **512**
gating function, **462**
Gaussian, **9**
Gaussian discriminant analysis, **323**
Gaussian distribution, **57**
Gaussian kernel, **459**, 535, **560**, **568**
Gaussian mixture model, **97**
Gaussian process, **460**
Gaussian process regression, 401
Gaussian processes, **574**
Gaussian scale mixture, **105**
GCN, **763**
GDA, **323**
GELU, 448, **451**
generalization error, 197
generalization gap, **13**, **194**
generalize, **7**, **121**
generalized additive model, **401**
generalized CCA, **679**
generalized eigenvalue, **330**
generalized Lagrangian, **304**, 587
generalized linear models, **415**
generalized low rank models, **675**
generalized probit approximation, **367**
Generative adversarial networks, **647**
generative classifier, **323**, **336**
generative image model, **493**
Geometric Deep Learning, **753**
geometric series, **120**, 287
Gini index, **605**
glmnet, **397**
GLMs, **415**
global average pooling, **432**, **476**
global optimization, **275**
global optimum, **275**, 344
globally convergent, **276**
Glorot initialization, **453**
GloVe, **710**
GMM, **97**
GMRES, **376**
GNN, **426**, **762**
goodness of fit, **380**
GoogLeNet, **482**
GPT, **542**
GPT-2, **542**
GPT-3, **542**
GPT-4, **542**
GPUs, **435**, **454**
GPyTorch, **583**
gradient, **270**, **283**
gradient boosted regression trees, **618**
gradient boosting, **616**
gradient clipping, **447**
gradient sign reversal, **637**
gradient tree boosting, **618**
Gram matrix, **241**, **247**, **500**, **568**
Gram Schmidt, 242
Graph attention network, **764**
Graph convolutional networks, **763**
graph factorization, **759**
graph Laplacian, **700**, 735
graph neural network, **762**

graph neural networks, **426**
graph partition, **734**
Graph Representation Learning, **753**
graphical models, **40**
Graphical Mutual Information, **768**
graphics processing units, **454**
graphite, **767**
GraphNet, **763**
GraphSAGE, **763**
GraRep, **759**
greedy decoding, **515**
greedy forward selection, **399**
grid approximation, **152**
grid search, 125, **320**
group lasso, **394**
group normalization, **479**
group sparsity, **393**
grouping effect, **396**
GRU, **512**
Gshard, **533**
Gumbel noise, 516

HAC, **717**
half Cauchy, **63**
half spaces, **340**
Hamiltonian Monte Carlo, **154**
hard attention, **525**
hard clustering, **98**, **730**
hard negatives, **556**
hard thresholding, **388**, 391
hardware accelerators, **437**
harmonic mean, 175
hat matrix, **375**
HDI, **147**
He initialization, **453**
heads, **433**
heat map, **471**
Heaviside, 346
Heaviside step function, 443
heaviside step function, **52**, **426**
heavy ball, **287**
heavy tails, **62**, **402**
Helmholtz machine, **685**
Hessian, 344, **776**
Hessian matrix, **272**
heteroskedastic regression, **59**, **376**
heuristics, **438**
hidden, **45**
hidden common cause, **79**
hidden units, **427**
hidden variables, **102**, **312**, **778**
hierarchical, **434**
hierarchical agglomerative clustering, **717**
hierarchical Bayesian model, **145**
hierarchical mixture of experts, **463**
hierarchical softmax, **358**
hierarchy, **357**
highest density interval, **147**
highest posterior density, **147**
hinge loss, **116**, 320, **591**, 747
Hinton diagram, **86**
hit rate, **172**
HMC, **154**
Hoeffding's inequality, **197**
hogwild training, **454**
holdout set, **196**
homogeneous, **101**
homoscedastic regression, **59**
homotopy, **398**
HPD, **147**
Huber loss, **177**, **404**, 617
Huffman encoding, 358
human pose estimation, **493**
Hutchinson trace estimator, 236, **237**
hyper-parameters, **131**, 319
hypercolumn, **474**
hypernyms, **358**
hyperparameter, **195**
hyperparameters, **145**
hyperplane, **340**
hypothesis, **523**
hypothesis space, **194**
hypothesis testing, **179**

I-projection, **216**
IA, **28**
ID3, **605**
identifiability, **355**
identifiable, **192**, 355
identity matrix, **239**
iid, **72**, **108**, **130**
ill-conditioned, 114, **238**
ill-posed, **49**
ILP, **307**
ILSVRC, **21**
image captioning, **505**
image classification, **3**
image compression, 725
image interpolation, **688**
image patches, **467**
image tagging, **350**, **488**
image-to-image, **492**
ImageNet, **21**, 435, 481
ImageNet-21k, 533
IMDB, 128
IMDB movie review dataset, **22**
implicit feedback, **747**
implicit regularization, **457**
impostors, **552**
imputation tasks, **632**
inception block, **482**
Inceptionism, **497**
inclusive KL, **216**
incremental learning, **551**
indefinite, **240**
independent, **35**, **39**
independent and identically distributed, **72**, **130**
independent variables, **371**
indicator function, **6**, 36
induced norm, **235**
inducing points, **583**
induction, **122**, **201**
inductive bias, **13**, 427, 532
inductive learning, **643**
inequality constraints, **277**, **302**
infeasible, 306
inference, **107**, **129**
inference network, **684**
infinitely wide, **582**
InfoNCE, **556**
information, 33
information content, **207**
information criteria, **185**
information diagram, **218**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

information diagrams, 219
information extraction, **542**
information gain, **213**, **650**
information gathering action, **7**
information projection, **216**
information retrieval, 173
information theory, **178**, **207**
inner product, **242**
input gate, **513**
Instagram, **489**
instance normalization, **478**
instance segmentation, **490**
instance-balanced sampling, **359**
instance-based learning, **547**
InstructGPT, **543**
instruction fine-tuning, **544**
Integer linear programming, **307**
integrated risk, **191**
integrating out, **129**
intelligence augmentation, **28**
inter-quartile range, **44**
interaction effects, **23**
intercept, **8**
interior point method, 306
internal covariate shift, **477**
interpolate, **12**
interpolated precision, **175**
interpolator, **574**
interpretable, **17**
intrinsic dimensionality, **690**
inverse, **249**
inverse cdf, **38**
inverse document frequency, **25**
inverse Gamma distribution, **65**
inverse probability, **49**
inverse problems, **49**
inverse reinforcement learning, **28**
inverse Wishart, 122
Iris, 2
Iris dataset, **3**
IRLS, **348**
isomap, **694**
isotropic covariance matrix, **83**
ISTA, **398**
items, **741**
iterate averaging, **297**
iterative soft thresholding algorithm, **398**
iteratively reweighted least squares, **348**

Jacobian, 68, 352
Jacobian formulation, 271
Jacobian matrix, **271**
Jacobian vector product, **271**
Jensen's inequality, **214**, 314
Jeopardy, 170
Jester, **742**
JFT, 533
jittered, **461**
joint distribution, **38**
joint probability, **34**
JPEG, 725
just in time, **446**
JVP, **271**

K nearest neighbor, **547**
k-d tree, **550**
K-means algorithm, **722**
K-means clustering, **98**
K-means++, **725**
K-medoids, **725**
Kalman filter, **91**
Karl Popper, 122
Karush-Kuhn-Tucker, **305**
Katz centrality index, 758
KDE, **560**, **562**
kernel, **469**, 560
kernel density estimation, **560**
kernel density estimator, **562**
kernel function, **459**, **567**, 567
kernel PCA, **695**, 737
kernel regression, 520, **564**, 576
kernel ridge regression, 576, **595**
kernel smoothing, **564**
kernel trick, **590**
keys, **518**
keywords, 261
kink, **728**
KKT, **305**
KL divergence, **109**, **177**, **213**, 314
KNN, **547**
knots, **399**
Knowledge distillation, **649**
knowledge graph, **770**
Kronecker product, **248**
Krylov subspace methods, **583**
KSG estimator, **220**
Kullback Leibler divergence, **109**, **177**
Kullback-Leibler divergence, **213**, 314

L-BFGS, **291**
L0-norm, **385**, 386
L1 loss, 176
L1 regularization, 385
L1VM, **598**
L2 loss, 176
L2 regularization, 123, 349, 381
L2VM, **597**
label, **2**
label noise, **360**, 655
Label propagation, **643**, **761**
label smearing, **358**
label smoothing, **650**, **655**
Label spreading, **761**
label switching problem, **319**, **732**
Lagrange multiplier, **303**
Lagrange multipliers, 95, 111
Lagrange notation, **270**
Lagrangian, 95, 259, **277**, **303**, 386
Lanczos algorithm, 671
language model, **101**
language modeling, **22**, **505**
language models, 211, **537**
Laplace, 385
Laplace approximation, **152**, 363
Laplace distribution, **63**
Laplace smoothing, **335**
Laplace vector machine, **598**
Laplace's rule of succession, **134**
Laplacian eigenmaps, **699**, 736, **757**
LAR, **399**
large language models, **542**
large margin classifier, **585**
large margin nearest neighbor, **552**

LARS, **398**
lasso, 307, **385**
latent coincidence analysis, **553**
latent factors, **15**, **659**
latent semantic analysis, **707**
latent semantic indexing, **706**
latent space interpolation, **688**
latent variable, **96**
latent variable models, **778**
latent variables, **778**
latent vector, **659**
law of iterated expectations, **41**
law of total expectation, **41**
law of total variance, **42**
layer normalization, **478**
layer-sequential unit-variance, **453**
LCA, **553**
LDA, **323**, **325**
Leaky ReLU, 448
leaky ReLU, **450**
learning curve, **127**
learning rate, **283**
learning rate schedule, **284**, **294**, 295
learning rate warmup, **296**
learning to learn, **652**
learning with a critic, **18**
learning with a teacher, **18**
least angle regression, **399**
least favorable prior, **192**
least mean squares, **294**, **378**
least squares boosting, **399**, **612**
least squares objective, **268**
least squares solution, **10**
leave-one-out cross-validation, **126**, **196**
LeCun initialization, **453**
left pseudo inverse, **269**
Leibniz notation, **270**
LeNet, **476**, **479**
level sets, 82, **83**
life-long learning, **551**
LightGBM, **619**
likelihood, **45**
likelihood function, **129**
likelihood principle, **202**
likelihood ratio, **180**, **201**
likelihood ratio test, **198**
limited memory BFGS, **291**, 354
line search, **285**
Linear algebra, **229**
linear autoencoder, **680**
linear combination, **243**
linear discriminant analysis, **323**, **325**
linear function, **8**
linear Gaussian system, **86**
linear kernel, **581**
linear map, **233**
linear operator, **376**
linear programming, 403
linear rate, **286**
linear regression, **59**, **371**, 415, 425
linear subspace, **243**
linear threshold function, **426**
linear transformation, **233**
linearity of expectation, **40**
linearly dependent, 232
linearly independent, 232
linearly separable, **340**
Linformer, **535**
link function, **415**, 417
link prediction, **769**
Lipschitz constant, **281**
liquid state machine, **511**
LMNN, **552**
LMS, **294**
local linear embedding, **697**
local maximum, **276**
local minimum, **275**
local optimum, **275**, 344
locality sensitive hashing, **550**
locally linear regression, **565**
locally-weighted scatterplot smoothing, **565**
LOESS, **565**
log bilinear language model, **711**
log likelihood, **108**
log loss, **178**, **592**
log odds, **52**
log partition function, **93**
log-sum-exp trick, **56**
logistic, **51**
logistic function, **52**, **148**
Logistic regression, **339**
logistic regression, **8**, **53**, **148**, 415
logit, **51**, **339**, **342**
logit adjustment, **359**
logit function, **52**
logitBoost, **616**
logits, **7**, **55**, **350**
long short term memory, **513**
long tail, 151, **358**
Lorentz, **63**
Lorentz model, **758**
loss function, **6**, 9, **167**, **275**
lossy compression, 724
lower triangular matrix, **240**
LOWESS, **565**
LSA, **707**
lse, **56**
LSH, **550**
LSI, **706**
LSTM, **513**

M step, **312**
M'th order Markov model, 101
M-projection, **216**
M1, **646**
M2, **646**
machine learning, **1**
machine translation, **22**, **509**
Mahalanobis distance, **83**, **547**
Mahalanobis whitening, **258**
main effects, **23**
majorize-minimize, **312**
MALA, 493
MAML, **652**
manifold, **689**, 689
manifold assumption, **643**
manifold hypothesis, **689**
manifold learning, **689**
mAP, **175**
MAP estimate, **169**
MAP estimation, **121**, 195
MAR, **27**
margin, **116**, **585**, **614**
margin errors, **590**

marginal distribution, **38**
marginal likelihood, **45**, **129**, **135**, 137, 145, **182**
marginalizing out, **129**, 148
marginalizing over, **129**
marginally independent, **39**
Markov chain, **101**
Markov chain Monte Carlo, **153**
Markov kernel, **101**
Markov model, **101**
MART, **618**
masked attention, **520**
masked language model, **539**
matched filter, **468**
matching network, **654**
Matern kernel, **570**
matrix, **229**
matrix completion, **743**
matrix determinant lemma, **252**
matrix factorization, **743**
matrix inversion lemma, **251**, 594
matrix square root, 235, **245**, **266**
matrix vector multiplication, **583**
max pooling, **475**
maxent classifier, **357**
maximal information coefficient, 221
maximum a posterior estimation, **121**
maximum a posteriori, **169**
maximum entropy, 60, **207**
maximum entropy classifer, **357**
maximum entropy model, **95**
maximum entropy sampling, **651**
maximum expected utility principle, **168**
maximum likelihood estimate, **8**
maximum likelihood estimation, **107**
maximum risk, **191**
maximum variance unfolding, **697**
MCAR, **27**
McCulloch-Pitts model, **436**
McKernel, **584**
MCMC, **153**
MDL, **186**
MDN, **463**
MDS, **691**
mean, **40**, **58**
mean average precision, **175**
mean function, **415**
mean squared error, **9**, **115**
mean value imputation, **27**
median, **38**, **57**
median absolute deviation, **563**
medoid, **725**
memory cell, **513**
memory-based learning, **547**
Mercer kernel, **567**
Mercer's theorem, **568**
message passing neural networks, **762**
meta-learning, **652**, 654
method of moments, **117**
metric MDS, **693**
Metropolis-adjusted Langevin algorithm, **493**
MICe, **222**
min-max scaling, **350**
minibatch, **293**
minimal, **93**
minimal representation, **94**
minimal sufficient statistic, **224**
minimally informative prior, **145**
minimax estimator, **192**
minimum description length, **186**
minimum mean squared error, **176**
minimum message length, **186**
minimum spanning tree, 719
minorize-maximize, **312**
MIP, **307**
misclassification rate, **6**, **116**
missing at random, **27**, **741**
missing completely at random, **27**
missing data, **26**, **312**
missing data mechanism, **27**, 647
missing value imputation, **85**
mixed ILP, **307**
mixing weights, 137
mixmatch, **640**
mixture density network, **463**
mixture model, **96**
mixture of Bernoullis, **98**
mixture of beta distributions, **136**
mixture of experts, **462**, 533
mixture of factor analyzers, **674**
mixture of Gaussians, **97**
ML, **1**
MLE, **8**, **107**
MLP, **425**, **427**
MLP-mixer, **427**
MM, **312**
MMSE, **176**
MNIST, **19**, 479
MobileNet, **488**
MoCo, **635**
mode, **41**, **169**
mode-covering, **216**
mode-seeking, **217**
model compression, **455**
model fitting, **7**, **107**
model parallelism, **454**
model selection, **181**
model selection consistent, **392**
model uncertainty, **7**, **34**
model-agnostic meta-learning, **652**
modus tollens, **201**
MoE, **462**
MoG, **97**
moment projection, **216**
momentum, **287**
momentum contrastive learning, **635**
MoNet, **765**
Monte Carlo approximation, **73**, **153**, **366**
Monte Carlo dropout, **457**
Monty Hall problem, **47**
Moore-Penrose pseudo-inverse, **261**
most powerful test, **200**
motes, 11
MovieLens, **742**
moving average, **119**
MSE, **9**, **115**
multi-class classification, **350**
multi-clust, **739**
Multi-dimensional scaling, **757**
multi-headed attention, **527**
multi-instance learning, **655**
multi-label classification, **350**
multi-label classifier, 358
multi-level model, **145**
multi-object tracking, **551**

multiclass logistic regression, **339**
multidimensional scaling, **691**
multilayer perceptron, **425**, **427**
multimodal, **41**
multinomial coefficient, **54**
multinomial distribution, **53**, **54**
Multinomial logistic regression, **350**
multinomial logistic regression, **55**, **339**
multinomial logit, **54**
multiple imputation, **85**
multiple linear regression, **10**, **371**
multiple restarts, **725**
multiplicative interaction, **518**
multivariate Bernoulli naive Bayes, **332**
multivariate Gaussian, **80**
multivariate linear regression, **372**
multivariate normal, **80**
mutual information, 79, **217**
mutually independent, 74
MVM, **583**
MVN, **80**
myopic, **651**

N-pairs loss, **556**
Nadaraya-Watson, **564**
naive Bayes assumption, 327, **332**
naive Bayes classifier, **332**
named entity recognition, **542**
NAS, **485**
nats, **207**
natural exponential family, **94**
natural language inference, **523**
natural language processing, **21**, 357
natural language understanding, **49**
natural parameters, **93**
NBC, **332**
NCA, **552**
NCM, **328**
nearest centroid classifier, **328**
nearest class mean classifier, **328**, **359**
nearest class mean metric learning, **328**
nearest neighbor clustering, **719**
NEF, **94**
negative definite, **240**
negative log likelihood, **8**, **108**
negative semidefinite, **240**
neighborhood components analysis, **552**
neocognitron, **476**
nested optimization, **195**
nested partitioning, **739**
Nesterov accelerated gradient, **288**
Netflix Prize, **741**
NetMF, **760**
neural architecture search, **485**
neural implicit representations, **429**
neural language model, **102**
neural machine translation, **509**
neural matrix factorization, **749**
neural style transfer, **497**
neural tangent kernel, **582**
NeurIPS, 18
neutral, **523**
Newton's method, **289**, **347**
next sentence prediction, **539**
Neyman-Pearson lemma, **200**
NHST, **200**
NHWC, **474**
NIPS, 18
NLL, **108**
NLP, **21**
NMAR, **27**
no free lunch theorem, **13**
node2vec, **760**
noise floor, **127**
non-identifiability, **732**
non-identifiable, **410**
non-metric MDS, **693**
non-parametric bootstrap, **157**
non-parametric methods, 689
non-parametric model, **460**
non-saturating activation functions, 429, **449**
noninformative, **144**
nonlinear dimensionality reduction, **689**
nonlinear factor analysis, **674**
nonparametric methods, **567**
nonparametric models, **547**
nonsmooth optimization, **281**
norm, **234**, 238
normal, **9**
normal distribution, **57**
normal equations, **269**, **373**
Normal-Inverse-Wishart distribution, **318**
normalization layers, **476**
normalized, **241**
normalized cut, **735**
normalized mutual information, **221**, **717**
normalizer-free networks, **479**
Normalizing flows, **648**
not missing at random, **27**
noun phrase chunking, **541**
novelty detection, **551**
NT-Xent, **556**
nu-SVM classifier, 590
nuclear norm, **235**
nucleotide, 208
null hypothesis, **179**, 187, **198**
null hypothesis significance testing, **200**
nullspace, **233**
numerator layout, 271

object detection, **489**
objective, **144**
objective function, **108**, **275**
observation distribution, **45**
Occam factor, **185**
Occam's razor, **183**
offset, **10**, **371**
Old Faithful, 316
Olivetti face dataset, 658
OLS, **115**, **269**, **373**
one-cycle learning rate schedule, **296**
one-hot, **178**
one-hot encoding, **23**, 352, **778**
one-hot vector, **53**, **229**
one-shot learning, **328**, **653**
one-sided p-value, **200**
one-sided test, **187**
one-standard error rule, **126**
one-to-many functions, **461**
one-versus-one, **593**
one-versus-the-rest, **593**
one-vs-all, **593**
online learning, **119**, **311**, **551**
OOD, **551**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

OOV, **24**, **26**
open class, **26**
open set recognition, **550**
open world, **489**
open world assumption, **551**
OpenPose, **493**
opt-einsum, **249**
optimal policy, **168**
optimism of the training error, **195**
optimization problem, **275**
order, **231**
order statistics, **118**
ordered Markov property, **100**
ordering constraint, **733**
ordinary least squares, **115**, **269**, **373**
Ornstein-Uhlenbeck process, **571**
orthodox statistics, **154**
orthogonal, **241**, 255
orthogonal projection, **375**
orthogonal random features, **584**
orthonormal, **241**, **255**
out of vocabulary, **26**
out-of-bag instances, **609**
out-of-distribution, **551**
out-of-sample generalization, **689**
out-of-vocabulary, **24**
outer product, **243**
outliers, **61**, **176**, **360**, **402**
output gate, **513**
over-complete representation, **94**
over-parameterized, **55**, **459**
overcomplete representation, **679**
overdetermined, **266**
overdetermined system, **374**
overfitting, **13**, **120**, 133

p-value, 180, **200**
PAC learnable, **197**
PageRank, **258**
pair plot, **4**
paired test, **187**
pairwise independent, 74
PAM, **726**
panoptic segmentation, **491**
parameter space, **275**
parameter tying, **101**
parameters, **6**
parametric bootstrap, **157**
parametric models, **547**
parametric ReLU, **450**
part of speech tagging, 541
part-of-speech, **538**
partial dependency plot, **623**
partial derivative, **270**
partial least squares, **678**
partial pivoting, **264**
partial regression coefficient, **377**
partially observed, **167**
partition function, **56**, **93**, **635**
partitioned inverse formulae, **250**
partitioning around medoids, **726**
Parzen window density estimator, **562**
pathologies, 203
pattern recognition, **2**
PCA, **16**, **657**, 658
PCA whitening, **257**
pdf, **37**, **58**
peephole connections, **514**
penalty term, **309**
percent point function, **38**
perceptron, **346**, **426**
perceptron learning algorithm, **346**
Performer, **535**
periodic kernel, **571**
permutation test, **201**
perplexity, **211**, 505
person re-identification, **551**
PersonLab, **493**
perturbation theory, 736
PGM, **100**
Planetoid, **768**
plates, **103**
Platt scaling, **591**
PLS, **678**
plug-in approximation, **133**, **148**
plugin approximation, 366
PMF, **744**
pmf, **36**
PMI, **707**
Poincaré model, **758**
point estimate, **107**
point null hypothesis, **187**
pointwise convolution, **474**
pointwise mutual information, **707**
Poisson regression, **417**
polar, 69
policy, **17**
Polyak-Ruppert averaging, **297**
polynomial expansion, 372
polynomial regression, **10**, 123, **372**
polytope, **305**
pool-based active learning, **650**
population risk, **13**, **125**, **193**
POS, **538**
position weight matrix, 208, **209**
positional embedding, **528**
positive definite, **240**
positive definite kernel, **567**
positive PMI, **707**
positive semidefinite, **240**
post-norm, **530**
posterior, 129
posterior distribution, **46**, **129**
posterior expected loss, **167**
posterior inference, **46**
posterior mean, 176
posterior median, **177**
posterior predictive distribution, **129**, **134**, 148, **366**, 406
power, **200**
power method, **258**
PPCA, **668**
ppf, **38**
pre-activation, **339**
pre-activations, **428**
pre-norm, **530**
pre-train, **537**
pre-trained word embedding, **26**
pre-training phase, **629**
preactivation resnet, **484**
precision, **57**, 141, **174**, 174
precision at K, **174**
precision matrix, **84**, **112**
precision-recall curve, **174**
preconditioned SGD, **298**

preconditioner, **298**
preconditioning matrix, **298**
predictive analytics, **27**
predictors, **2**
preferences, **167**
premise, **523**
PreResnet, **484**
pretext tasks, **633**
prevalence, **46**, **175**
primal problem, **587**
primal variables, **595**
principal components analysis, **16**, **657**
principal components regression, **384**
prior, **121**, **129**
prior distribution, **45**
probabilistic forecasting, **177**
probabilistic graphical model, **100**
probabilistic inference, **46**
probabilistic matrix factorization, **744**
probabilistic PCA, 657
probabilistic perspective, **1**
probabilistic prediction, **177**
probabilistic principal components analysis, **668**
probability density function, **37**, **58**
probability distribution, **177**
probability distributions, **1**
probability mass function, **36**
probability simplex, **138**
probability theory, 45
probably approximately correct, **197**
probit approximation, **367**
probit function, **57**, 367
probit link function, **418**
product rule, **39**
product rule of probability, **45**
profile likelihood, **665**
profile log likelihood, 666
projected gradient descent, **309**, 398
projection, **234**
projection matrix, **375**
prompt, **543**
prompt engineering, **543**, **637**
proper scoring rule, **179**
prosecutor's fallacy, **75**
proxies, **557**
proximal gradient descent, 398
proximal gradient method, **308**
proximal operator, **308**
ProxQuant, **311**
proxy tasks, **633**
prune, **606**
psd, **240**
pseudo counts, **131**, **334**
pseudo inputs, **583**
pseudo inverse, 373
pseudo norm, **235**
pseudo-labeling, **639**
pseudo-likelihood, **539**
pure, **606**
purity, **716**
Pythagoras's theorem, **268**

QALY, **167**
QP, **306**
quadratic approximation, **152**
quadratic discriminant analysis, **324**
quadratic form, **240**, **256**
quadratic kernel, **568**
quadratic loss, **9**, **176**
quadratic program, **306**, 386, 596
quality-adjusted life years, **167**
quantile, **38**, **57**
quantile function, **38**
quantization, 212
quantize, **213**, **220**
quantized, **311**
quartiles, **38**, **57**
Quasi-Newton, **290**
quasi-Newton, 354
queries, **518**
query synthesis, **650**
question answering, **22**, **542**

radial basis function, **560**
radial basis function kernel, **460**, 535
Rand index, **716**
RAND-WALK, **711**
random finite sets, **551**
random forests, **610**
random Fourier features, **584**
random number generator, **73**
random shuffling, 293
random variable, **35**
random variables, **1**
random walk kernel, **573**
range, **233**
rank, **231**, **237**
rank deficient, **237**
rank one update, **251**
rank-nullity theorem, **262**
ranking loss, **555**, **747**
RANSAC, **404**
rate, **486**, **725**
rate of convergence, **286**
rating, **741**
Rayleigh quotient, **258**
RBF, **560**
RBF kernel, **460**, **568**
RBF network, **460**
real AdaBoost, **614**
recall, **172**, **174**
receiver operating characteristic, **173**
receptive field, **473**, 486
recognition network, **684**
recommendation systems, **770**
Recommender systems, **741**
reconstruction error, **657**, **659**, **724**
Rectified linear unit, 448
rectified linear unit, **429**, **449**
recurrent neural network, **503**
recurrent neural networks, **12**, **426**
recursive update, **119**
recursively, **377**
reduce-on-plateau, **296**
reduced QR, **265**
Reformer, **535**
region of practical equivalence, **187**
regression, **8**, **778**
regression coefficient, **377**
regression coefficients, **8**, **371**
regularization, **121**
regularization parameter, **121**
regularization path, **384**, **389**, 398
regularized discriminant analysis, **327**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

regularized empirical risk, **195**
reinforcement learning, **17**, **751**
reinforcement learning from human feedback, **542**
reject option, **170**
reject the null hypothesis, **200**
relational data, **741**
relative entropy, **213**
relevance vector machine, **598**
ReLU, **429**, **449**
reparameterization trick, **686**
representation learning, **633**
reservoir computing, **512**
reset gate, **513**
reshape, **231**
residual block, **451**, **483**
residual error, **115**
residual network, **451**
residual plot, **380**
residual sum of squares, **115**, **373**
residuals, **9**, 380
ResNet, **451**, **483**
ResNet-18, **484**
response, **2**
response variables, **778**
responsibility, **98**, **315**, **463**
reverse KL, **216**
reverse mode differentiation, **440**
reward, **18**
reward function, **275**
reward hacking, **28**
RFF, **584**
ridge regression, **123**, 162, **381**, 455
Riemannian manifold, **689**
Riemannian metric, **689**
right pseudo inverse, **268**
risk, **167**, **189**
risk averse, **169**, 170
risk neutral, **169**
risk sensitive, **169**
RL, **17**
RLHF, **542**
RMSE, **115**, **381**
RNN, **426**, **503**
Robbins-Monro conditions, **295**
robust, **9**, **61**, **176**
robust linear regression, 306
robust logistic regression, **360**
robustness, **402**
ROC, **173**
root mean squared error, **115**, **381**
ROPE, **187**
rotation matrix, **241**
row rank, **237**
row-major order, **231**
RSS, **115**
rule of iterated expectation, **104**
rule of total probability, **38**
running sum, **119**
rv, **35**
RVM, **598**

saddle point, **277**, 280
SAGA, **298**, 346
same convolution, **471**
SAMME, **615**
Sammon mapping, **694**
sample efficiency, **17**
sample mean, 160
sample size, **2**, **110**, 130
sample space, **35**
sample variance, **143**
sampling distribution, **154**
SARS-CoV-2, **46**
saturated model, 420
saturates, **429**
scalar field, **270**
scalar product, **242**
scalars, **232**
scale of evidence, 180
scaled dot-product attention, **521**
scaling hypothesis, **542**
scatter matrix, **113**, **246**
Schatten p-norm, **235**
scheduled sampling, **510**
Schur complement, 84, **250**
score function, **155**, **275**, **682**
scree plot, **664**
second order, **346**
Second-order, **289**
self attention, **526**
self-normalization property, **711**
self-supervised, **632**
self-supervised learning, **16**
self-training, **638**, **650**
SELU, **450**
semantic role labeling, **357**
semantic segmentation, **491**
semantic textual similarity, **525**
semi-hard negatives, **557**
semi-supervised embeddings, **768**
Semi-supervised learning, **638**
semi-supervised learning, 337
semidefinite embedding, **697**
semidefinite programming, **552**, **697**
sensible PCA, **668**
sensitivity, **46**, **172**
sensor fusion, **92**
sentiment analysis, **22**
seq2seq, **507**
seq2seq model, **22**
seq2vec, **506**
sequence logo, **209**
sequence motif, **209**
sequential minimal optimization, **588**
SGD, **292**
SGNS, **709**
shaded nodes, **102**
shallow parsing, **541**
Shampoo, **301**
Shannon's source coding theorem, **209**
shared, **325**
sharp minima, **457**
sharpness aware minimization, **458**
Sherman-Morrison formula, **251**
Sherman-Morrison-Woodbury formula, **251**
shooting, **397**
short and fat, **3**
shrinkage, **90**, **143**, **384**
shrinkage estimation, **122**
shrinkage factor, **390**, **617**
Siamese network, **555**, 633
side information, **750**
sifting property, **61**, 148
sigmoid, **51**, **52**, **148**, 326

signal-to-noise ratio, **90**
significance, **199**
significance level, **200**
silhouette coefficient, **728**, 728
silhouette diagram, **728**
silhouette score, **728**
SiLU, **451**
SimCLR, **633**
similarity, **547**
simple hypothesis, **199**
simple linear regression, **9**, **371**
simplex algorithm, **306**
Simpson's paradox, **80**
simulated annealing, 43
single link clustering, **719**
single shot detector, **490**
singular, **249**
singular statistical model, **186**
singular value decomposition, **260**
singular values, 238, **260**
singular vectors, **260**
skip connections, 484
skip-gram with negative sampling, **709**
skipgram, 707
skipgram model, **709**
slack variables, **589**, **596**
slate, **750**
slope, **10**
SMACOF, **693**
SmelU, **451**
SMO, **588**
smooth optimization, **281**
Smoothing splines, **401**
SNLI, **523**
Sobel edge detector, **494**
social networks, **770**
soft clustering, **98**
soft decision tree, **606**
soft margin constraints, **589**
soft thresholding, **388**, 391
soft thresholding operator, **310**
soft triple, **557**
softmax, **54**
softmax function, **7**
Softplus, 448
softplus, **59**
solver, **275**
source dataset, **629**
source domain, **637**
span, **232**
sparse, 138, **385**
sparse Bayesian learning, **411**
sparse factor analysis, 673
sparse GP, **583**
sparse kernel machine, **460**, 550
sparse linear regression, 307
sparse vector machines, **597**
sparsity inducing regularizer, **310**
specificity, **46**
spectral clustering, **734**
spectral CNNs, **763**
spectral embedding, **699**
spectral graph theory, **701**
spectral radius, **447**
spherical covariance matrix, **83**
spherical embedding constraint, **559**
split variable trick, **397**
spurious correlation, **79**
spurious correlations, 80
spurious features, **337**
square, **230**
square-root sampling, **359**
square-root schedule, **296**
squared error, **176**
squared exponential kernel, **568**
stacking, **609**
standard basis, **233**
standard deviation, **41**, **58**
standard error, **133**, **156**
standard error of the mean, **126**, **143**
standard form, **305**
standard normal, **57**, **58**
standardization operation, 246
standardize, **350**, **376**
standardized, 316
standardizing, 256
Stanford Natural Language Inference, **523**
state of nature, **167**
state space, **35**
state transition matrix, **101**
static graph, **446**
stationary, **101**
stationary kernels, **569**
stationary point, **276**
statistical learning theory, **196**
statistical machine translation, **509**
statistical relational learning, **771**
statistics, **27**
steepest descent, **284**
Stein's paradox, **193**
step decay, **296**
step function, **66**
step size, **283**
stochastic averaged gradient accelerated, **298**
stochastic beam search, **516**
stochastic gradient boosting, **618**
stochastic gradient descent, **292**, 345
stochastic gradient descent with warm restarts, **296**
stochastic matrix, **101**
stochastic neighbor embedding, **701**
stochastic optimization, **292**
stochastic variance reduced gradient, **297**
stochastic volatility model, **434**
Stochastic Weight Averaging, **297**
stochastic weight averaging, **458**, **645**
stop word removal, **24**
storks, 80
straight-through estimator, **311**
strain, **692**
stream-based active learning, **650**
stress function, **692**
strict, 192
strict local minimum, **276**
strictly concave, **278**
strictly convex, **278**
strided convolution, **473**
string kernel, **573**
strong learner, **612**
strongly convex, **280**
structural deep network embedding, **766**
structural risk minimization, **195**
structured data, **425**
STS Benchmark, **525**
STSB, **543**

Student distribution, **61**
Student t distribution, 61
subderivative, 443
subdifferentiable, **282**
subdifferential, **282**
subgradient, **282**
submodular, **651**
subword units, **26**
sufficient statistic, **224**
sufficient statistics, **93**, **110**, **112**, **130**
sum of squares matrix, **246**
sum rule, **38**
supervised learning, **1**
supervised PCA, **677**
support vector machine, **585**
support vector machine regression, **597**
support vectors, **585**, **588**, **597**
surface normal prediction, **492**
surrogate function, **312**
surrogate loss function, **116**
surrogate splits, **606**
suspicious coincidence, 181
SVD, **260**, 383
SVM, 307, **585**
SVM regression, **597**
SVRG, **297**
Swish, 448
swish, **450**
Swiss roll, **690**
symmetric, **230**
symmetric SNE, **702**
synaptic connection, 436
synchronous training, **454**
syntactic sugar, **103**
systems of linear equations, 266

t-SNE, **701**
T5, **543**
tabular data, **3**, **425**
tall and skinny, **3**
tangent space, **689**
target, **2**, **371**
target dataset, **629**
target domain, **637**
target neighbors, **552**
targets, **778**
taxonomy, **357**
Taylor series, 152
Taylor series expansion, 226
teacher forcing, **509**
temperature, **55**
tempered cross entropy, **361**
tempered softmax, **361**
template matching, **467**, **471**
tensor, **230**, **473**
tensor processing units, **437**, **454**
term frequency matrix, **25**
term-document frequency matrix, **706**
test risk, **13**
test set, **13**
test statistic, **200**
text to speech, **518**
textual entailment, **523**
TF-IDF, **25**
thin SVD, **260**
thresholded linear unit, **479**
TICe, **222**
tied, **325**
Tikhonov damping, **292**
Tikhonov regularization, **292**
time series forecasting, 505
time-invariant, **101**
TinyImages, 20
TL;DR, **543**
token, **24**
topological instability, **694**
topological order, **100**
total derivative, **271**
total differential, **271**
total variation, **494**
TPUs, **437**, **454**
trace, **236**
trace norm, **235**
trace trick, **236**
tracing, **446**
training, **7**, **107**
training data, **778**
training set, **2**
transductive learning, **643**
transfer learning, **537**, 559, **629**
transformer, **526**
transformers, **426**
transition function, **101**
transition kernel, **101**
translation invariance, **467**
transpose, **230**
transposed convolution, **487**, 492
treewidth, 249
Tri-cube kernel, 561
Tri-Training, **642**
triangle inequality, **213**
tridiagonal, **239**
trigram model, **101**
triplet loss, **555**
true negative rate, **46**
true positive rate, **46**, **172**
truncate, 511
truncated Gaussian, **61**
truncated SVD, **264**
trust-region optimization, **292**
tube, **596**
Turing machine, **504**
TV, **494**
two-part code, **186**
two-sided p-value, **200**
two-sided test, **187**
type I error, **199**
type I error rate, **172**
type II error, **199**
type II maximum likelihood, **145**
typical patterns, **16**

U-net, **491**, **492**
U-shaped curve, **13**
UMAP, **705**
unadjusted Langevin algorithm, **493**
unbiased, **160**
uncertainty, **33**
unconditionally independent, **39**
unconstrained optimization, **277**
undercomplete representation, **679**
underdetermined, **266**
underfitting, **13**, **124**, 127
unidentifiable, 356

uninformative, **132**, **144**
uninformative prior, 143
union bound, **197**
uniqueness, **667**
unit vector, **229**
unit vectors, **53**
unitary, **241**
universal function approximator, **434**
UNK, **26**
unrolled, **103**
unstable, **608**
unstructured data, **426**
unsupervised learning, **14**
unsupervised pre-training, **632**
update gate, **513**
upper triangular matrix, **240**
users, **741**
utility function, **168**

VAE, **683**
valid convolution, **471**
validation risk, **125**, **196**
validation set, **13**, **124**, **196**
value of information, **650**
values, **518**
vanishing gradient problem, **429**, **447**
variable metric, **290**
variable selection, **392**
variance, **40**, **58**
variation of information, **717**
variational autoencoder, **16**, 646, **683**
variational autoencoders, 674
variational EM, **315**
variational inference, **153**
variational RNN, **505**
varimax, **673**
VC dimension, **198**
vec2seq, **503**
vector, **229**
vector addition, 710
vector field, **270**, **682**
vector Jacobian product, **272**
vector quantization, **724**
vector space, **232**
vector space model, **24**
VI, **153**
vicinal risk minimization, **628**
violin plot, **44**
virtual adversarial training, **644**
visible variables, **778**
visual n-grams, 637
visual scene understanding, **49**
ViT, **532**
Viterbi decoding, **515**
VJP, **272**
Voronoi iteration, **726**
Voronoi tessellation, **548**, **723**
VQ, **724**

WAIC, 733
wake sleep, 685
Wald interval, **159**
Wald statistic, **200**
warm start, **384**
warm starting, **398**
Watanabe–Akaike information criterion, **187**
Watson, **170**
wavenet, **518**
weak learner, **612**
weakly supervised learning, **655**
WebText, **542**
weight decay, **123**, **349**, **381**, **455**
weight space, **577**
weighted least squares, **376**
weighted least squares problem, 347
weighted linear regression, **376**
weights, **8**, **371**
well-conditioned, **238**
whiten, **256**
wide and deep, **748**
wide data, **3**
wide format, **24**
wide resnet, **484**
widely applicable information criterion, **187**
Widrow-Hoff rule, **294**
winner takes all, **55**
WMT dataset, **22**
Wolfe conditions, **291**
word analogy problem, **710**
word embeddings, **26**, **705**, 705
word sense disambiguation, **538**
word stemming, **24**
word2vec, **707**
wordpieces, **26**
working response, **347**
World Health Organization, 222
WSD, **538**

Xavier initialization, **453**
XGBoost, **619**
XOR problem, **427**

YOLO, **490**

ZCA, **258**
zero count, **122**
zero-avoiding, **216**
zero-forcing, **217**
zero-one loss, **6**, **169**
zero-padding, **471**
zero-shot classification, **637**
zero-shot learning, **653**
zero-shot task transfer, **543**
zig-zag, **286**

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Bibliography

[AAB21] A. Agrawal, A. Ali, and S. Boyd. "Minimum-distortion embedding". en. In: *Foundations and Trends in Machine Learning* 14.3 (2021), pp. 211–378.

[AB08] C. Archambeau and F. Bach. "Sparse probabilistic projections". In: *NIPS*. 2008.

[AB14] G. Alain and Y. Bengio. "What Regularized Auto-Encoders Learn from the Data-Generating Distribution". In: *JMLR* (2014).

[AC16] D. K. Agarwal and B.-C. Chen. *Statistical Methods for Recommender Systems*. en. 1st edition. Cambridge University Press, 2016.

[Ace] "The Turing Test is Bad for Business". In: (2021).

[AEH+18] S. Abu-El-Haija, B. Perozzi, R. Al-Rfou, and A. A. Alemi. "Watch your step: Learning node embeddings via graph attention". In: *Advances in Neural Information Processing Systems*. 2018, pp. 9180–9190.

[AEHPAR17] S. Abu-El-Haija, B. Perozzi, and R. Al-Rfou. "Learning Edge Representations via Low-Rank Asymmetric Projections". In: *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. CIKM '17. 2017, 1787–1796.

[AEM18] Ö. D. Akyildiz, V. Elvira, and J. Miguez. "The Incremental Proximal Method: A Probabilistic Perspective". In: *ICASSP*. 2018.

[AFF19] C. Aicher, N. J. Foti, and E. B. Fox. "Adaptively Truncating Backpropagation Through Time to Control Gradient Bias". In: (2019). arXiv: 1905.07473 [cs.LG].

[Agg16] C. C. Aggarwal. *Recommender Systems: The Textbook*. en. 1st ed. 2016 edition. Springer, 2016.

[Agg20] C. C. Aggarwal. *Linear Algebra and Optimization for Machine Learning: A Textbook*. en. 1st ed. 2020 edition. Springer, 2020.

[AGM19] V. Amrhein, S. Greenland, and B. McShane. "Scientists rise up against statistical significance". In: *Nature* 567.7748 (2019), p. 305.

[Agr70] A. Agrawala. "Learning with a probabilistic teacher". In: *IEEE Transactions on Information Theory* 16.4 (1970), pp. 373–379.

[AH19] C. Allen and T. Hospedales. "Analogies Explained: Towards Understanding Word Embeddings". In: *ICML*. 2019.

[AHK12] A. Anandkumar, D. Hsu, and S. M. Kakade. "A Method of Moments for Mixture Models and Hidden Markov Models". In: *COLT*. Vol. 23. Proceedings of Machine Learning Research. PMLR, 2012, pp. 33.1–33.34.

[Ahm+13] A. Ahmed, N. Shervashidze, S. Narayanamurthy, V. Josifovski, and A. J. Smola. "Distributed large-scale natural graph factorization". In: *Proceedings of the 22nd international conference on World Wide Web*. ACM. 2013, pp. 37–48.

[AK15] J. Andreas and D. Klein. "When and why are log-linear models self-normalizing?" In: *Proc. ACL*. Association for Computational Linguistics, 2015, pp. 244–249.

[Aka74] H. Akaike. "A new look at the statistical model identification". In: *IEEE Trans. on Automatic Control* 19.6 (1974).

[AKA91] D. W. Aha, D. Kibler, and M. K. Albert. "Instance-based learning algorithms". In: *Mach. Learn.* 6.1 (1991), pp. 37–66.

[Aky+19] Ö. D. Akyildiz, É. Chouzenoux, V. Elvira, and J. Míguez. "A probabilistic incremental proximal gradient method". In: *IEEE Signal Process. Lett.* 26.8 (2019).

[AL13] N. Ailon and E. Liberty. "An Almost Optimal Unrestricted Fast Johnson-Lindenstrauss Transform". In: *ACM Trans. Algorithms* 9.3 (2013), 21:1–21:12.

[Ala18] J. Alammar. *Illustrated Transformer*. Tech. rep. 2018.

[Alb+17] M. Alber, P.-J. Kindermans, K. Schütt, K.-R. Müller, and F. Sha. "An Empirical Study on The Properties of Random Bases for Kernel Methods". In: *NIPS*. Curran Associates, Inc., 2017, pp. 2763–2774.

[Alb+18] D. Albanese, S. Riccadonna, C. Donati, and P. Franceschi. "A practical tool for maximal information coefficient analysis". en. In: *Gigascience* 7.4 (2018), pp. 1–8.

[ALL18] S. Arora, Z. Li, and K. Lyu. "Theoretical Analysis of Auto Rate-Tuning by Batch Normalization". In: (2018). arXiv: 1812.03981 [cs.LG].

[Alm87] L. B. Almeida. "A learning rule for asynchronous perceptrons with feedback in a combinatorial environment." In: *Proceedings, 1st First International Conference on Neural Networks*. Vol. 2. IEEE. 1987, pp. 609–618.

[Alo+09] D. Aloise, A. Deshpande, P. Hansen, and P. Popat. "NP-hardness of Euclidean sum-of-squares clustering". In: *Machine Learning* 75 (2009), pp. 245–249.

[Alp04] E. Alpaydin. *Introduction to machine learning*. MIT Press, 2004.

[Ami+19] E. Amid, M. K. Warmuth, R. Anil, and T. Koren. "Robust Bi-Tempered Logistic Loss Based on Bregman Divergences". In: *NIPS*. 2019.

[Amo+16] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. "Concrete

Problems in AI Safety". In: (2016). arXiv: 1606.06565 [cs.AI].

[Amo17] Amoeba. *What is the difference between ZCA whitening and PCA whitening*. Stackexchange. 2017.

[And01] C. A. Anderson. "Heat and Violence". In: *Current Directions in Psychological Science* 10.1 (2001), pp. 33–38.

[And+18] R. Anderson, J. Huchette, C. Tjandraatmadja, and J. P. Vielma. "Strong convex relaxations and mixed-integer programming formulations for trained neural networks". In: (2018). arXiv: 1811.01988 [math.OC].

[Ani+20] R. Anil, V. Gupta, T. Koren, K. Regan, and Y. Singer. "Scalable Second Order Optimization for Deep Learning". In: (2020). arXiv: 2002.09018 [cs.LG].

[Ans73] F. J. Anscombe. "Graphs in Statistical Analysis". In: *Am. Stat.* 27.1 (1973), pp. 17–21.

[AO03] J.-H. Ahn and J.-H. Oh. "A Constrained EM Algorithm for Principal Component Analysis". In: *Neural Computation* 15 (2003), pp. 57–65.

[Arc+19] F. Arcadu, F. Benmansour, A. Maunz, J. Willis, Z. Haskova, and M. Prunotto. "Deep learning algorithm predicts diabetic retinopathy progression in individual patients". en. In: *NPJ Digit Med* 2 (2019), p. 92.

[Ard+20] R. Ardila, M. Branson, K. Davis, M. Kohler, J. Meyer, M. Henretty, R. Morais, L. Saunders, F. Tyers, and G. Weber. "Common Voice: A Massively-Multilingual Speech Corpus". In: *Proceedings of The 12th Language Resources and Evaluation Conference*. 2020, pp. 4218–4222.

[Arj21] M. Arjovsky. "Out of Distribution Generalization in Machine Learning". In: (2021). arXiv: 2103.02667 [stat.ML].

[Arn+19] S. M. R. Arnold, P.-A. Manzagol, R. Babanezhad, I. Mitliagkas, and N. Le Roux. "Reducing the variance in online optimization by transporting past gradients". In: *NIPS*. 2019.

[Aro+16] S. Arora, Y. Li, Y. Liang, T. Ma, and A. Risteski. "A Latent Variable Model Approach to PMI-based Word Embeddings". In: *TACL* 4 (2016), pp. 385–399.

[Aro+19] L. Aroyo, A. Dumitrache, O. Inel, Z. Szlávik, B. Timmermans, and C. Welty. "Crowdsourcing Inclusivity: Dealing with Diversity of Opinions, Perspectives and Ambiguity in Annotated Data". In: *WWW*. WWW '19. Association for Computing Machinery, 2019, pp. 1294–1295.

[Aro+21] R. Arora et al. *Theory of deep learning*. 2021.

[ARZP19] R. Al-Rfou, D. Zelle, and B. Perozzi. "DDGK: Learning Graph Representations for Deep Divergence Graph Kernels". In: *Proceedings of the 2019 World Wide Web Conference on World Wide Web* (2019).

[AS17] A. Achille and S. Soatto. "On the Emergence of Invariance and Disentangling in Deep Representations". In: (2017). arXiv: 1706.01350 [cs.LG].

[AS19] A. Achille and S. Soatto. "Where is the Information in a Deep Neural Network?" In: (2019). arXiv: 1905.12213 [cs.LG].

[Ash18] J. Asher. "A Rise in Murder? Let's Talk About the Weather". In: *The New York Times* (2018).

[ASR15] A. Ali, S. M. Shamsuddin, and A. L. Ralescu. "Classification with class imbalance problem: A Review". In: *Int. J. Advance Soft Compu. Appl* 7.3 (2015).

[Ath+19] B. Athiwaratkun, M. Finzi, P. Izmailov, and A. G. Wilson. "There Are Many Consistent Explanations of Unlabeled Data: Why You Should Average". In: *ICLR*. 2019.

[AV07] D. Arthur and S. Vassilvitskii. "k-means++: the advantages of careful seeding". In: *Proc. 18th ACM-SIAM symp. on Discrete algorithms*. 2007, 1027–1035.

[AWS19] E. Amid, M. K. Warmuth, and S. Srinivasan. "Two-temperature logistic regression based on the Tsallis divergence". In: *AISTATS*. 2019.

[Axl15] S. Axler. *Linear algebra done right*. 2015.

[BA10] R. Bailey and J. Addison. *A Smoothed-Distribution Form of Nadaraya-Watson Estimation*. Tech. rep. 10-30. Univ. Birmingham, 2010.

[BA97a] A. Bowman and A. Azzalini. *Applied Smoothing Techniques for Data Analysis*. Oxford, 1997.

[BA97b] L. A. Breslow and D. W. Aha. "Simplifying decision trees: A survey". In: *Knowl. Eng. Rev.* 12.1 (1997), pp. 1–40.

[Bab19] S. Babu. *A 2019 guide to Human Pose Estimation with Deep Learning*. 2019.

[Bac+16] O. Bachem, M. Lucic, H. Hassani, and A. Krause. "Fast and Provably Good Seedings for k-Means". In: *NIPS*. 2016, pp. 55–63.

[Bah+12] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii. "Scalable k-Means++". In: *VLDB*. 2012.

[Bah+20] Y. Bahri, J. Kadmon, J. Pennington, S. Schoenholz, J. Sohl-Dickstein, and S. Ganguli. "Statistical Mechanics of Deep Learning". In: *Annu. Rev. Condens. Matter Phys.* (2020).

[BAP14] P. Bachman, O. Alsharif, and D. Precup. "Learning with pseudo-ensembles". In: *Advances in neural information processing systems*. 2014, pp. 3365–3373.

[Bar09] M. Bar. "The proactive brain: memory for predictions". en. In: *Philos. Trans. R. Soc. Lond. B Biol. Sci.* 364.1521 (2009), pp. 1235–1243.

[Bar19] J. T. Barron. "A General and Adaptive Robust Loss Function". In: *CVPR*. 2019.

[Bat+18] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, et al. "Relational inductive biases, deep learning, and graph networks". In: *arXiv preprint arXiv:1806.01261* (2018).

[BB08] O. Bousquet and L. Bottou. "The Tradeoffs of Large Scale Learning". In: *NIPS*. 2008, pp. 161–168.

[BB11] L. Bottou and O. Bousquet. "The Tradeoffs of Large Scale Learning". In: *Optimization for Machine Learning*. Ed. by S. Sra, S. Nowozin, and S. J. Wright. MIT Press, 2011, pp. 351–368.

[BBV11] R. Benassi, J. Bect, and E. Vazquez. "Bayesian optimization using sequential Monte Carlo". In: (2011). arXiv: 1111.4802 [math.OC].

[BC17] D. Beck and T. Cohn. "Learning Kernels over Strings using Gaussian Processes". In: *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*. Vol. 2. 2017, pp. 67–73.

[BCB15] D. Bahdanau, K. Cho, and Y. Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate". In: *ICLR*. 2015.

[BCD01] L. Brown, T. Cai, and A. DasGupta. "Interval Estimation for a Binomial Proportion". In: *Statistical Science* 16.2 (2001), pp. 101–133.

[BCN18] L. Bottou, F. E. Curtis, and J. Nocedal. "Optimization Methods for Large-Scale Machine Learning". In: *SIAM Rev.* 60.2 (2018), pp. 223–311.

[BCV13] Y. Bengio, A. Courville, and P. Vincent. "Representation learning: a review and new perspectives". en. In: *IEEE PAMI* 35.8 (2013), pp. 1798–1828.

[BD20] B. Barz and J. Denzler. "Do We Train on Test Data? Purging CIFAR of Near-Duplicates". In: *J. of Imaging* 6.6 (2020).

[BD21] D. G. T. Barrett and B. Dherin. "Implicit Gradient Regularization". In: *ICLR*. 2021.

[BD87] G. Box and N. Draper. *Empirical Model-Building and Response Surfaces*. Wiley, 1987.

[BDEL03] S. Ben-David, N. Eiron, and P. M. Long. "On the difficulty of approximately maximizing agreements". In: *J. Comput. System Sci.* 66.3 (2003), pp. 496–514.

[Bel+19] M. Belkin, D. Hsu, S. Ma, and S. Mandal. "Reconciling modern machine-learning practice and the classical bias-variance trade-off". en. In: *PNAS* 116.32 (2019), pp. 15849–15854.

[Ben+04a] Y. Bengio, O. Delalleau, N. Roux, J. Paiement, P. Vincent, and M. Ouimet. "Learning eigenfunctions links spectral embedding and kernel PCA". In: *Neural Computation* 16 (2004), pp. 2197–2219.

[Ben+04b] Y. Bengio, J.-F. Paiement, P. Vincent, O. Delalleau, N. L. Roux, and M. Ouimet. "Out-of-Sample Extensions for LLE, Isomap, MDS, Eigenmaps, and Spectral Clustering". In: *NIPS*. MIT Press, 2004, pp. 177–184.

[Ben+15a] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer. "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks". In: *NIPS*. 2015.

[Ben+15b] Y. Bengio, D.-H. Lee, J. Bornschein, T. Mesnard, and Z. Lin. "Towards Biologically Plausible Deep Learning". In: (2015). arXiv: 1502.04156 [cs.LG].

[Ben+17] A. Benavoli, G. Corani, J. Demsar, and M. Zaffalon. "Time for a change: a tutorial for comparing multiple classifiers through Bayesian analysis". In: *JMLR* (2017).

[Ber15] D. Bertsekas. *Convex Optimization Algorithms*. Athena Scientific, 2015.

[Ber16] D. Bertsekas. *Nonlinear Programming*. Third. Athena Scientific, 2016.

[Ber+19a] D. Berthelot, N. Carlini, E. D. Cubuk, A. Kurakin, K. Sohn, H. Zhang, and C. Raffel. "Remixmatch: Semi-supervised learning with distribution alignment and augmentation anchoring". In: *arXiv preprint arXiv:1911.09785* (2019).

[Ber+19b] D. Berthelot, N. Carlini, I. Goodfellow, N. Papernot, A. Oliver, and C. Raffel. "Mixmatch: A holistic approach to semi-supervised learning". In: *Advances in Neural Information Processing Systems*. 2019, pp. 5049–5059.

[Ber+21] J. Berner, P. Grohs, G. Kutyniok, and P. Petersen. "The Modern Mathematics of Deep Learning". In: (2021). arXiv: 2105.04026 [cs.LG].

[Ber85] J. Berger. "Bayesian Salesmanship". In: *Bayesian Inference and Decision Techniques with Applications: Essays in Honor of Bruno deFinetti*. Ed. by P. K. Goel and A. Zellner. North-Holland, 1985.

[Ber99] D. Bertsekas. *Nonlinear Programming*. Second. Athena Scientific, 1999.

[Bey+19] M. Beyeler, E. L. Rounds, K. D. Carlson, N. Dutt, and J. L. Krichmar. "Neural correlates of sparse coding and dimensionality reduction". en. In: *PLoS Comput. Biol.* 15.6 (2019), e1006908.

[Bey+20] L. Beyer, O. J. Hénaff, A. Kolesnikov, X. Zhai, and A. van den Oord. "Are we done with ImageNet?" In: (2020). arXiv: 2006.07159 [cs.CV].

[BFO84] L. Breiman, J. Friedman, and R. Olshen. *Classification and regression trees*. Wadsworth, 1984.

[BG11] P. Buhlmann and S. van de Geer. *Statistics for High-Dimensional Data: Methodology, Theory and Applications*. Springer, 2011.

[BH07] P. Buhlmann and T. Hothorn. "Boosting Algorithms: Regularization, Prediction and Model Fitting". In: *Statistical Science* 22.4 (2007), pp. 477–505.

[BH69] A. Bryson and Y.-C. Ho. *Applied optimal control: optimization, estimation, and control*. Blaisdell Publishing Company, 1969.

[BH86] J. Barnes and P. Hut. "A hierarchical O(N log N) force-calculation algorithm". In: *Nature* 324.6096 (1986), pp. 446–449.

[BH89] P. Baldi and K. Hornik. "Neural networks and principal components analysis: Learning from examples without local minima". In: *Neural Networks* 2 (1989), pp. 53–58.

[Bha+19] A. Bhadra, J. Datta, N. G. Polson, and B. T. Willard. "Lasso Meets Horseshoe: a survey". In: *Bayesian Anal.* 34.3 (2019), pp. 405–427.

[Bha+20] A. Bhadra, J. Datta, Y. Li, and N. Polson. "Horseshoe regularisation for machine learning

in complex and deep models". en. In: *Int. Stat. Rev.* 88.2 (2020), pp. 302–320.

[BHM92] J. S. Bridle, A. J. Heading, and D. J. MacKay. "Unsupervised Classifiers, Mutual Information and'Phantom Targets". In: *Advances in neural information processing systems*. 1992, pp. 1096–1101.

[BI19] P. Barham and M. Isard. "Machine Learning Systems are Stuck in a Rut". In: *Proceedings of the Workshop on Hot Topics in Operating Systems*. HotOS '19. Association for Computing Machinery, 2019, pp. 177–183.

[Bis06] C. Bishop. *Pattern recognition and machine learning*. Springer, 2006.

[Bis94] C. M. Bishop. *Mixture Density Networks*. Tech. rep. NCRG 4288. Neural Computing Research Group, Department of Computer Science, Aston University, 1994.

[Bis99] C. Bishop. "Bayesian PCA". In: *NIPS*. 1999.

[BJ05] F. Bach and M. Jordan. *A probabilistic interpretation of canonical correlation analysis*. Tech. rep. 688. U. C. Berkeley, 2005.

[BJM06] P. Bartlett, M. Jordan, and J. McAuliffe. "Convexity, Classification, and Risk Bounds". In: *JASA* 101.473 (2006), pp. 138–156.

[BK07] R. M. Bell and Y. Koren. "Lessons from the Netflix Prize Challenge". In: *SIGKDD Explor. Newsl.* 9.2 (2007), pp. 75–79.

[BKC17] V. Badrinarayanan, A. Kendall, and R. Cipolla. "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation". In: *IEEE PAMI* 39.12 (2017).

[BKH16] J. L. Ba, J. R. Kiros, and G. E. Hinton. "Layer Normalization". In: (2016). arXiv: 1607.06450 [stat.ML].

[BKL10] S. Bird, E. Klein, and E. Loper. *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. 2010.

[BL04] P. Bickel and E. Levina. "Some theory for Fisher's linear discriminant function, "Naive Bayes", and some alternatives when there are many more variables than observations". In: *Bernoulli* 10 (2004), pp. 989–1010.

[BL07a] C. M. Bishop and J. Lasserre. "Generative or discriminative? Getting the best of both worlds". In: *Bayesian Statistics 8*. 2007.

[BL07b] J. A. Bullinaria and J. P. Levy. "Extracting semantic representations from word co-occurrence statistics: a computational study". en. In: *Behav. Res. Methods* 39.3 (2007), pp. 510–526.

[BL12] J. A. Bullinaria and J. P. Levy. "Extracting semantic representations from word co-occurrence statistics: stop-lists, stemming, and SVD". en. In: *Behav. Res. Methods* 44.3 (2012), pp. 890–907.

[BL88] D. S. Broomhead and D Lowe. "Multivariable Functional Interpolation and Adaptive Networks". In: *Complex Systems* (1988).

[BLK17] O. Bachem, M. Lucic, and A. Krause. "Distributed and provably good seedings for k-means in constant rounds". In: *ICML*. 2017, pp. 292–300.

[Blo20] M. Blondel. *Automatic differentiation*. 2020.

[BLV19] X. Bouthillier, C. Laurent, and P. Vincent. "Unreproducible Research is Reproducible". In: *ICML*. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 725–734.

[BM98] A. Blum and T. Mitchell. "Combining labeled and unlabeled data with co-training". In: *Proceedings of the eleventh annual conference on Computational learning theory*. 1998, pp. 92–100.

[BN01] M. Belkin and P. Niyogi. "Laplacian Eigenmaps and Spectral Techniques for Embedding and Clustering". In: *NIPS*. 2001, pp. 585–591.

[BNJ03] D. Blei, A. Ng, and M. Jordan. "Latent Dirichlet allocation". In: *JMLR* 3 (2003), pp. 993–1022.

[Bo+08] L. Bo, C. Sminchisescu, A. Kanaujia, and D. Metaxas. "Fast Algorithms for Large Scale Conditional 3D Prediction". In: *CVPR*. 2008.

[Boh92] D. Bohning. "Multinomial logistic regression algorithm". In: *Annals of the Inst. of Statistical Math.* 44 (1992), pp. 197–200.

[Bon13] S. Bonnabel. "Stochastic gradient descent on Riemannian manifolds". In: *IEEE Transactions on Automatic Control* 58.9 (2013), pp. 2217–2229.

[Bos+16] D. Boscaini, J. Masci, E. Rodolà, and M. Bronstein. "Learning shape correspondence with anisotropic convolutional neural networks". In: *Advances in Neural Information Processing Systems*. 2016, pp. 3189–3197.

[Bot+13] L. Bottou, J. Peters, J. Quiñonero-Candela, D. X. Charles, D. M. Chickering, E. Portugaly, D. Ray, P. Simard, and E. Snelson. "Counterfactual Reasoning and Learning Systems: The Example of Computational Advertising". In: *JMLR* 14 (2013), pp. 3207–3260.

[Bow+15] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning. "A large annotated corpus for learning natural language inference". In: *EMNLP*. Association for Computational Linguistics, 2015, pp. 632–642.

[BPC20] I. Beltagy, M. E. Peters, and A. Cohan. "Longformer: The Long-Document Transformer". In: *CoRR* abs/2004.05150 (2020). arXiv: 2004.05150.

[Bre01] L. Breiman. "Random Forests". In: *Machine Learning* 45.1 (2001), pp. 5–32.

[Bre96] L. Breiman. "Bagging predictors". In: *Machine Learning* 24 (1996), pp. 123–140.

[Bri50] G. W. Brier. "Verification of forecasts expressed in terms of probability". In: *Monthly Weather Review* 78.1 (1950), pp. 1–3.

[Bri90] J. Bridle. "Probabilistic Interpretation of Feedforward Classification Network Outputs, with Relationships to Statistical Pattern Recognition". In: *Neurocomputing: Algorithms, Architectures and Applications*. Ed. by F. F. Soulie and J. Herault. Springer Verlag, 1990, pp. 227–236.

[Bro+17a] M. M. Bronstein, J Bruna, Y LeCun, A Szlam, and P Vandergheynst. "Geometric Deep Learning: Going beyond Euclidean data". In: *IEEE Signal Process. Mag.* 34.4 (2017), pp. 18–42.

[Bro+17b] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst. "Geometric deep learning: going beyond euclidean data". In: *IEEE Signal Processing Magazine* 34.4 (2017), pp. 18–42.

[Bro19] J. Brownlee. *Deep Learning for Computer Vision - Machine Learning Mastery*. Accessed: 2020-6-30. Machine Learning Mastery, 2019.

[Bro+20] T. B. Brown et al. "Language Models are Few-Shot Learners". In: (2020). arXiv: 2005.14165 [cs.CL].

[Bro+21] A. Brock, S. De, S. L. Smith, and K. Simonyan. "High-Performance Large-Scale Image Recognition Without Normalization". In: (2021). arXiv: 2102.06171 [cs.CV].

[BRR18] T. D. Bui, S. Ravi, and V. Ramavajjala. "Neural Graph Machines: Learning Neural Networks Using Graphs". In: *WSDM*. 2018.

[Bru+14] J. Bruna, W. Zaremba, A. Szlam, and Y. Lecun. "Spectral networks and locally connected networks on graphs International Conference on Learning Representations (ICLR2014)". In: *CBLS, April* (2014).

[Bru+19] G. Brunner, Y. Liu, D. Pascual, O. Richter, and R. Wattenhofer. "On the Validity of Self-Attention as Explanation in Transformer Models". In: (2019). arXiv: 1908.04211 [cs.CL].

[BS02] M. Balasubramanian and E. L. Schwartz. "The isomap algorithm and topological stability". en. In: *Science* 295.5552 (2002), p. 7.

[BS16] P. Baldi and P. Sadowski. "A Theory of Local Learning, the Learning Channel, and the Optimality of Backpropagation". In: *Neural Netw.* 83 (2016), pp. 51–74.

[BS17] D. M. Blei and P. Smyth. "Science and data science". en. In: *Proc. Natl. Acad. Sci. U. S. A.* (2017).

[BS21] S. Bubeck and M. Sellke. "A Universal Law of Robustness via Isoperimetry". In: *NIPS* 34 (Dec. 2021), pp. 28811–28822.

[BS94] J. Bernardo and A. Smith. *Bayesian Theory*. John Wiley, 1994.

[BS97] A. J. Bell and T. J. Sejnowski. "The "independent components" of natural scenes are edge filters". en. In: *Vision Res.* 37.23 (1997), pp. 3327–3338.

[BT04] G. Bouchard and B. Triggs. "The tradeoff between generative and discriminative classifiers". In: *IASC International Symposium on Computational Statistics (COMPSTAT '04)*. 2004.

[BT08] D. Bertsekas and J. Tsitsiklis. *Introduction to Probability*. 2nd Edition. Athena Scientific, 2008.

[BT09] A Beck and M Teboulle. "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems". In: *SIAM J. Imaging Sci.* 2.1 (2009), pp. 183–202.

[BT73] G. Box and G. Tiao. *Bayesian inference in statistical analysis*. Addison-Wesley, 1973.

[Bul11] A. D. Bull. "Convergence rates of efficient global optimization algorithms". In: *JMLR* 12 (2011), 2879–2904.

[Bur10] C. J. C. Burges. "Dimension Reduction: A Guided Tour". en. In: *Foundations and Trends in Machine Learning* (2010).

[Bur25] A. Burkov. *The Hundred-Page Language Models Book*. 2025.

[BV04] S. Boyd and L. Vandenberghe. *Convex optimization*. Cambridge, 2004.

[BW08] P. L. Bartlett and M. H. Wegkamp. "Classification with a Reject Option using a Hinge Loss". In: *JMLR* 9.Aug (2008), pp. 1823–1840.

[BW88] J. Berger and R. Wolpert. *The Likelihood Principle*. 2nd edition. The Institute of Mathematical Statistics, 1988.

[BWL19] Y. Bai, Y.-X. Wang, and E. Liberty. "ProxQuant: Quantized Neural Networks via Proximal Operators". In: *ICLR*. 2019.

[BY03] P. Buhlmann and B. Yu. "Boosting with the L2 loss: Regression and classification". In: *JASA* 98.462 (2003), pp. 324–339.

[Byr+16] R Byrd, S Hansen, J Nocedal, and Y Singer. "A Stochastic Quasi-Newton Method for Large-Scale Optimization". In: *SIAM J. Optim.* 26.2 (2016), pp. 1008–1031.

[BZ20] A. Barbu and S.-C. Zhu. *Monte Carlo Methods*. en. Springer, 2020.

[Cal20] O. Calin. *Deep Learning Architectures: A Mathematical Approach*. en. 1st ed. Springer, 2020.

[Cao+18] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh. "OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields". In: (2018). arXiv: 1812.08008 [cs.CV].

[CAS16] P. Covington, J. Adams, and E. Sargin. "Deep Neural Networks for YouTube Recommendations". In: *Proceedings of the 10th ACM Conference on Recommender Systems*. RecSys '16. Association for Computing Machinery, 2016, pp. 191–198.

[CB02] G. Casella and R. Berger. *Statistical inference*. 2nd edition. Duxbury, 2002.

[CBD15] M. Courbariaux, Y. Bengio, and J.-P. David. "BinaryConnect: Training Deep Neural Networks with binary weights during propagations". In: *NIPS*. 2015.

[CC07] H. Choi and S. Choi. "Robust kernel Isomap". In: *Pattern Recognit.* 40.3 (2007), pp. 853–862.

[CCD17] B. P. Chamberlain, J. Clough, and M. P. Deisenroth. "Neural embeddings of graphs in hyperbolic space". In: *arXiv preprint arXiv:1705.10359* (2017).

[CD14] K. Chaudhuri and S. Dasgupta. "Rates of Convergence for Nearest Neighbor Classification". In: *NIPS*. 2014.

[CD88] W. Cleveland and S. Devlin. "Locally-Weighted Regression: An Approach to Regression Analysis by Local Fitting". In: *JASA* 83.403 (1988), pp. 596–610.

[CDL16] J. Cheng, L. Dong, and M. Lapata. "Long Short-Term Memory-Networks for Machine Reading". In: *EMNLP*. Association for Computational Linguistics, 2016, pp. 551–561.

[CDL19] S. Chen, E. Dobriban, and J. H. Lee. "Invariance reduces Variance: Understanding Data Augmentation in Deep Learning and Beyond". In: (2019). arXiv: 1907.10905 [stat.ML].

[CDS02] M. Collins, S. Dasgupta, and R. E. Schapire. "A Generalization of Principal Components Analysis to the Exponential Family". In: *NIPS-14*. 2002.

[CEL19] Z. Chen, J. B. Estrach, and L. Li. "Supervised community detection with line graph neural networks". In: *7th International Conference on Learning Representations, ICLR 2019*. 2019.

[Cer+17] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia. "SemEval-2017 Task 1: Semantic Textual Similarity Multilingual and Crosslingual Focused Evaluation". In: *Proc. 11th Intl. Workshop on Semantic Evaluation (SemEval-2017)*. Association for Computational Linguistics, 2017, pp. 1–14.

[CFD10] Y. Cui, X. Z. Fern, and J. G. Dy. "Learning Multiple Nonredundant Clusterings". In: *ACM Transactions on Knowledge Discovery from Data* 4.3 (2010).

[CG16] T. Chen and C. Guestrin. "XGBoost: A Scalable Tree Boosting System". In: *KDD*. ACM, 2016, pp. 785–794.

[CG18] J. Chen and Q. Gu. "Closing the Generalization Gap of Adaptive Gradient Methods in Training Deep Neural Networks". In: (2018). arXiv: 1806.06763 [cs.LG].

[CGG17] S. E. Chazan, J. Goldberger, and S. Gannot. "Speech Enhancement using a Deep Mixture of Experts". In: (2017). arXiv: 1703.09302 [cs.SD].

[CGW21] W. Chen, X. Gong, and Z. Wang. "Neural Architecture Search on ImageNet in Four GPU Hours: A Theoretically Inspired Perspective". In: *ICLR*. 2021.

[CH67] T. Cover and P. Hart. " Nearest neighbor pattern classification". In: *IEEE Trans. Inform. Theory* 13.1 (1967), pp. 21–27.

[CH90] K. W. Church and P. Hanks. "Word Association Norms, Mutual Information, and Lexicography". In: *Computational Linguistics* (1990).

[Cha+01] O. Chapelle, J. Weston, L. Bottou, and V. Vapnik. "Vicinal Risk Minimization". In: *NIPS*. MIT Press, 2001, pp. 416–422.

[Cha+17] P. Chaudhari, A. Choromanska, S. Soatto, Y. LeCun, C. Baldassi, C. Borgs, J. Chayes, L. Sagun, and R. Zecchina. "Entropy-SGD: Biasing Gradient Descent Into Wide Valleys". In: *ICLR*. 2017.

[Cha+19a] I. Chami, Z. Ying, C. Ré, and J. Leskovec. "Hyperbolic graph convolutional neural networks". In: *Advances in Neural Information Processing Systems*. 2019, pp. 4869–4880.

[Cha+19b] J. J. Chandler, I. Martinez, M. M. Finucane, J. G. Terziev, and A. M. Resch. "Speaking on Data's Behalf: What Researchers Say and How Audiences Choose". en. In: *Eval. Rev.* (2019), p. 193841X19834968.

[Cha+21] I. Chami, S. Abu-El-Haija, B. Perozzi, C. Ré, and K. Murphy. "Machine Learning on Graphs: A Model and Comprehensive Taxonomy". In: *JMLR* (2021).

[Cha21] S. H. Chan. *Introduction to Probability for Data Science*. Michigan Publishing, 2021.

[Che+16] H.-T. Cheng et al. "Wide & Deep Learning for Recommender Systems". In: (2016). arXiv: 1606.07792 [cs.LG].

[Che+20a] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. "A Simple Framework for Contrastive Learning of Visual Representations". In: *ICML*. 2020.

[Che+20b] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. "A simple framework for contrastive learning of visual representations". In: *ICML*. 2020.

[Che+20c] T. Chen, S. Kornblith, K. Swersky, M. Norouzi, and G. Hinton. "Big Self-Supervised Models are Strong Semi-Supervised Learners". In: *NIPS*. 2020.

[Chi+19a] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh. "Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks". In: *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 2019.

[Chi+19b] R. Child, S. Gray, A. Radford, and I. Sutskever. "Generating Long Sequences with Sparse Transformers". In: *CoRR* abs/1904.10509 (2019). arXiv: 1904.10509.

[CHL05] S. Chopra, R. Hadsell, and Y. LeCun. "Learning a Similarity Metric Discriminatively, with Application to Face Verification". en. In: *CVPR*. 2005.

[Cho+14a] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation". In: *EMNLP*. 2014.

[Cho+14b] K. Cho, B. Van Merriënboer, D. Bahdanau, and Y. Bengio. "On the properties of neural machine translation: Encoder-decoder approaches". In: *arXiv preprint arXiv:1409.1259* (2014).

[Cho+15] Y. Chow, A. Tamar, S. Mannor, and M. Pavone. "Risk-Sensitive and Robust Decision-Making: a CVaR Optimization Approach". In: *NIPS*. 2015, pp. 1522–1530.

[Cho17] F. Chollet. *Deep learning with Python*. Manning, 2017.

[Cho+19] K. Choromanski, M. Rowland, W. Chen, and A. Weller. "Unifying Orthogonal Monte Carlo Methods". In: *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach,*

California, USA. Ed. by K. Chaudhuri and R. Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 1203–1212.

[Cho+20a] K. Choromanski et al. "Masked Language Modeling for Proteins via Linearly Scalable Long-Context Transformers". In: (2020). arXiv: 2006.03555 [cs.LG].

[Cho+20b] K. Choromanski et al. "Rethinking Attention with Performers". In: *CoRR* abs/2009.14794 (2020). arXiv: 2009.14794.

[Cho21] F. Chollet. *Deep learning with Python (second edition)*. Manning, 2021.

[Cho70] C Chow. "On optimum recognition error and reject tradeoff". en. In: *IEEE Trans. Inf. Theory* 16.1 (Jan. 1970), pp. 41–46.

[Chr20] B. Christian. *The Alignment Problem: Machine Learning and Human Values*. en. 1st ed. W. W. Norton & Company, 2020.

[Chu+15] J. Chung, K. Kastner, L. Dinh, K. Goel, A. Courville, and Y. Bengio. "A Recurrent Latent Variable Model for Sequential Data". In: *NIPS*. 2015.

[Chu+22] H. W. Chung et al. "Scaling Instruction-Finetuned Language Models". In: (Oct. 2022). arXiv: 2210.11416 [cs.LG].

[Chu97] F. Chung. *Spectral Graph Theory*. AMS, 1997.

[Cir+10] D. C. Ciresan, U. Meier, L. M. Gambardella, and J. Schmidhuber. "Deep Big Simple Neural Nets For Handwritten Digit Recognition". In: *Neural Computation* 22.12 (2010), pp. 3207–3220.

[Cir+11] D. C. Ciresan, U. Meier, J. Masci, L. M. Gambardella, and J. Schmidhuber. "Flexible, High Performance Convolutional Neural Networks for Image Classification". In: *IJCAI*. 2011.

[CL96] B. P. Carlin and T. A. Louis. *Bayes and Empirical Bayes Methods for Data Analysis*. Chapman and Hall, 1996.

[Cla21] A. Clayton. *Bernoulli's Fallacy: Statistical Illogic and the Crisis of Modern Science*. en. Columbia University Press, 2021.

[CLX15] S. Cao, W. Lu, and Q. Xu. "Grarep: Learning graph representations with global structural information". In: *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*. ACM. 2015, pp. 891–900.

[CNB17] C. Chelba, M. Norouzi, and S. Bengio. "N-gram Language Modeling using Recurrent Neural Network Estimation". In: (2017). arXiv: 1703.10724 [cs.CL].

[Coh+17] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik. "EMNIST: an extension of MNIST to handwritten letters". In: (2017). arXiv: 1702.05373 [cs.CV].

[Coh94] J. Cohen. "The earth is round ($p < .05$)". In: *American Psychologist* 49.12 (1994), pp. 997–1003.

[Con+17] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, and A. Bordes. "Supervised learning of universal sentence representations from natural language inference data". In: *arXiv preprint arXiv:1705.02364* (2017).

[Coo05] J. Cook. *Exact Calculation of Beta Inequalities*. Tech. rep. M. D. Anderson Cancer Center, Dept. Biostatistics, 2005.

[CP10] M. A. Carreira-Perpinan. "The Elastic Embedding Algorithm for Dimensionality Reduction". In: *ICML*. 2010.

[CP19] A. Coenen and A. Pearce. *Understanding UMAP*. 2019.

[CPS06] K. Chellapilla, S. Puri, and P. Simard. "High Performance Convolutional Neural Networks for Document Processing". In: *10th Intl. Workshop on Frontiers in Handwriting Recognition*. 2006.

[CRW17] K. Choromanski, M. Rowland, and A. Weller. "The Unreasonable Effectiveness of Structured Random Orthogonal Embeddings". In: *NIPS*. 2017.

[CS20] F. E. Curtis and K Scheinberg. "Adaptive Stochastic Optimization: A Framework for Analyzing Stochastic Optimization Algorithms". In: *IEEE Signal Process. Mag.* 37.5 (2020), pp. 32–42.

[Csu17] G. Csurka. "Domain Adaptation for Visual Applications: A Comprehensive Survey". In: *Domain Adaptation in Computer Vision Applications*. Ed. by G. Csurka. 2017.

[CT06] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. 2nd edition. John Wiley, 2006.

[CT91] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. John Wiley, 1991.

[Cub+19] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le. "AutoAugment: Learning Augmentation Policies from Data". In: *CVPR*. 2019.

[CUH16] D.-A. Clevert, T. Unterthiner, and S. Hochreiter. "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)". In: *ICLR*. 2016.

[Cui+19] X. Cui, K. Zheng, L. Gao, B. Zhang, D. Yang, and J. Ren. "Multiscale Spatial-Spectral Convolutional Network with Image-Based Framework for Hyperspectral Imagery Classification". en. In: *Remote Sensing* 11.19 (2019), p. 2220.

[Cur+17] J. D. Curtó, I. C. Zarza, F Yang, A Smola, F Torre, C. W. Ngo, and L Gool. "McKernel: A Library for Approximate Kernel Expansions in Log-linear Time". In: (2017). arXiv: 1702.08159v14 [cs.LG].

[Cyb89] G. Cybenko. "Approximation by superpositions of a sigmoidal function". In: *Mathematics of Control, Signals, and Systems* 2 (1989), 303–331.

[D'A+20] A. D'Amour et al. "Underspecification Presents Challenges for Credibility in Modern Machine Learning". In: (2020). arXiv: 2011.03395 [cs.LG].

[Dah+11] G. E. Dahl, D. Yu, L. Deng, and A. Acero. "Large vocabulary continuous speech recognition with context-dependent DBN-HMMS". In: *ICASSP*. IEEE, 2011, pp. 4688–4691.

[Dai+19] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. V. Le, and R. Salakhutdinov. "Transformer-XL: Attentive Language Models beyond a Fixed-Length Context". In: *Proc. ACL*. 2019, pp. 2978–2988.

[Dao+19] T. Dao, A. Gu, A. J. Ratner, V. Smith, C. De Sa, and C. Re. "A Kernel Theory of Modern Data Augmentation". In: *ICML*. 2019.

[Dau17] J. Daunizeau. "Semi-analytical approximations to statistical moments of sigmoid and softmax mappings of normal variables". In: (2017). arXiv: 1703.00091 [stat.ML].

[Day+95] P. Dayan, G. Hinton, R. Neal, and R. Zemel. "The Helmholtz machine". In: *Neural Networks* 9.8 (1995).

[DB18] A. Defazio and L. Bottou. "On the Ineffectiveness of Variance Reduced Optimization for Deep Learning". In: (2018). arXiv: 1812.04529 [cs.LG].

[DBLJ14] A. Defazio, F. Bach, and S. Lacoste-Julien. "SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives". In: *NIPS*. Curran Associates, Inc., 2014, pp. 1646–1654.

[DDDM04] I Daubechies, M Defrise, and C De Mol. "An iterative thresholding algorithm for linear inverse problems with a sparsity constraint". In: *Commun. Pure Appl. Math.* Advances in E 57.11 (2004), pp. 1413–1457.

[Dee+90] S. Deerwester, S. Dumais, G. Furnas, T. Landauer, and R. Harshman. "Indexing by Latent Semantic Analysis". In: *J. of the American Society for Information Science* 41.6 (1990), pp. 391–407.

[DeG70] M. DeGroot. *Optimal Statistical Decisions*. McGraw-Hill, 1970.

[Den+12] J. Deng, J Krause, A. C. Berg, and L. Fei-Fei. "Hedging your bets: Optimizing accuracy-specificity trade-offs in large scale visual recognition". In: *CVPR*. 2012, pp. 3450–3457.

[Den+14] J. Deng, N. Ding, Y. Jia, A. Frome, K. Murphy, S. Bengio, Y. Li, H. Neven, and H. Adam. "Large-Scale Object Classification using Label Relation Graphs". In: *ECCV*. 2014.

[Dev+19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *NAACL*. 2019.

[DG06] J. Davis and M. Goadrich. "The Relationship Between Precision-Recall and ROC Curves". In: *ICML*. 2006, pp. 233–240.

[DHM07] P. Diaconis, S. Holmes, and R. Montgomery. "Dynamical Bias in the Coin Toss". In: *SIAM Review* 49.2 (2007), pp. 211–235.

[DHS01] R. O. Duda, P. E. Hart, and D. G. Stork. *Pattern Classification*. 2nd edition. Wiley Interscience, 2001.

[DHS11] J. Duchi, E. Hazan, and Y. Singer. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization". In: *JMLR* 12 (2011), pp. 2121–2159.

[Die98] T. G. Dietterich. "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms". In: *Neural Computation*. 10.7 (1998), pp. 1895–1923.

[Din+15] N. Ding, J. Deng, K. Murphy, and H. Neven. "Probabilistic Label Relation Graphs with Ising Models". In: *ICCV*. 2015.

[DJ15] S. Dray and J. Josse. "Principal component analysis with missing values: a comparative survey of methods". In: *Plant Ecol.* 216.5 (2015), pp. 657–667.

[DKK12] G Dror, N Koenigstein, and Y Koren. "Web-Scale Media Recommendation Systems". In: *Proc. IEEE* 100.9 (2012), pp. 2722–2736.

[DKS95] J. Dougherty, R. Kohavi, and M. Sahami. "Supervised and Unsupervised Discretization of Continuous Features". In: *ICML*. 1995.

[DLLP97] T. Dietterich, R. Lathrop, and T. Lozano-Perez. "Solving the multiple instance problem with axis-parallel rectangles". In: *Artificial Intelligence* 89 (1997), pp. 31–71.

[DLR77] A. P. Dempster, N. M. Laird, and D. B. Rubin. "Maximum likelihood from incomplete data via the EM algorithm". In: *J. of the Royal Statistical Society, Series B* 34 (1977), pp. 1–38.

[DM01] D. van Dyk and X.-L. Meng. "The Art of Data Augmentation". In: *J. Computational and Graphical Statistics* 10.1 (2001), pp. 1–50.

[DM16] P. Drineas and M. W. Mahoney. "RandNLA: Randomized Numerical Linear Algebra". In: *CACM* (2016).

[DMB21] Y. Dar, V. Muthukumar, and R. G. Baraniuk. "A Farewell to the Bias-Variance Tradeoff? An Overview of the Theory of Overparameterized Machine Learning". In: (Sept. 2021). arXiv: 2109.02355 [stat.ML].

[Do+19] T.-T. Do, T. Tran, I. Reid, V. Kumar, T. Hoang, and G. Carneiro. "A Theoretically Sound Upper Bound on the Triplet Loss for Improving the Efficiency of Deep Distance Metric Learning". In: *CVPR*. 2019, pp. 10404–10413.

[Doe16] C. Doersch. "Tutorial on Variational Autoencoders". In: (2016). arXiv: 1606.05908 [stat.ML].

[Don95] D. L. Donoho. "De-noising by soft-thresholding". In: *IEEE Trans. Inf. Theory* 41.3 (1995), pp. 613–627.

[Dos+21] A. Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *ICLR*. 2021.

[Doy+07] K. Doya, S. Ishii, A. Pouget, and R. P. N. Rao, eds. *Bayesian Brain: Probabilistic Approaches to Neural Coding*. MIT Press, 2007.

[DP97] P. Domingos and M. Pazzani. "On the Optimality of the Simple Bayesian Classifier under Zero-One Loss". In: *Machine Learning* 29 (1997), pp. 103–130.

[DR21] H. Duanmu and D. M. Roy. "On extended admissibale procedures and their nonstandard Bayes risk". In: *Annals of Statistics* (2021).

[Dri+04] P. Drineas, A. Frieze, R. Kannan, S. Vempala, and V. Vinay. "Clustering Large Graphs via the Singular Value Decomposition". In: *Machine Learning* 56 (2004), pp. 9–33.

[DS12] M. Der and L. K. Saul. "Latent Coincidence Analysis: A Hidden Variable Model for Distance Metric Learning". In: *NIPS*. Curran Associates, Inc., 2012, pp. 3230–3238.

[DSK16] V. Dumoulin, J. Shlens, and M. Kudlur. "A Learned Representation For Artistic Style". In: (2016). arXiv: 1610.07629 [cs.CV].

[Dum+18] A. Dumitrache, O. Inel, B. Timmermans, C. Ortiz, R.-J. Sips, L. Aroyo, and C. Welty. "Empirical Methodology for Crowdsourcing Ground Truth". In: *Semantic Web Journal* (2018).

[Duv14] D. Duvenaud. "Automatic Model Construction with Gaussian Processes". PhD thesis. Computational and Biological Learning Laboratory, University of Cambridge, 2014.

[DV16] V. Dumoulin and F. Visin. "A guide to convolution arithmetic for deep learning". In: (2016). arXiv: 1603.07285 [stat.ML].

[Dwi+23] R. Dwivedi, C. Singh, B. Yu, and M. J. Wainwright. "Revisiting minimum description length complexity in overparameterized models". In: *J. Mach. Learn. Res.* (2023).

[Dzi+24] N. Dziri et al. "Faith and Fate: Limits of Transformers on Compositionality". In: *NIPS*. 2024.

[EDH19] K. Ethayarajh, D. Duvenaud, and G. Hirst. "Towards Understanding Linear Word Analogies". In: *Proc. ACL*. Association for Computational Linguistics, 2019, pp. 3253–3262.

[EF15] D. Eigen and R. Fergus. "Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture". In: *ICCV*. 2015.

[Efr+04] B. Efron, I. Johnstone, T. Hastie, and R. Tibshirani. "Least angle regression". In: *Annals of Statistics* 32.2 (2004), pp. 407–499.

[Efr86] B. Efron. "Why Isn't Everyone a Bayesian?" In: *The American Statistician* 40.1 (1986).

[Efr87] B. Efron. *The Jackknife, the Bootstrap, and Other Resampling Plans (CBMS-NSF Regional Conference Series in Applied Mathematics)*. en. Society for Industrial and Applied Mathematics, 1987.

[EH16] B. Efron and T. Hastie. *Computer Age Statistical Inference: Algorithms, Evidence, and Data Science*. en. Cambridge University Press, June 2016.

[Ein16] A Einstein. "Die Grundlage der allgemeinen Relativitätstheorie". In: *Ann. Phys.* 354.7 (1916), pp. 769–822.

[Eis19] J. Eisenstein. *Introduction to Natural Language Processing*. 2019.

[Elk03] C. Elkan. "Using the triangle inequality to accelerate k-means". In: *ICML*. 2003.

[EMH19] T. Elsken, J. H. Metzen, and F. Hutter. "Neural Architecture Search: A Survey". In: *JMLR* 20 (2019), pp. 1–21.

[Erh+10] D. Erhan, Y. Bengio, A. Courville, P.-A. Manzagol, P. Vincent, and S. Bengio. "Why Does Unsupervised Pre-training Help Deep Learning?" In: *JMLR* 11 (2010), pp. 625–660.

[FAL17] C. Finn, P. Abbeel, and S. Levine. "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks". In: *ICML*. 2017.

[FB81] M. A. Fischler and R. Bolles. "Random sample concensus: A paradigm for model fitting with applications to image analysis and automated cartography". In: *Comm. ACM* 24.6 (1981), pp. 381–395.

[Fen+21] S. Y. Feng, V. Gangal, J. Wei, S. Chandar, S. Vosoughi, T. Mitamura, and E. Hovy. "A Survey of Data Augmentation Approaches for NLP". In: (2021). arXiv: 2105.03075 [cs.CL].

[Fer+10] D. Ferrucci et al. "Building Watson: An Overview of the DeepQA Project". In: *AI Magazine* (2010), pp. 59–79.

[FH20] E. Fong and C. Holmes. "On the marginal likelihood and cross-validation". In: *Biometrika* 107.2 (2020).

[FHK12] A. Feuerverger, Y. He, and S. Khatri. "Statistical Significance of the Netflix Challenge". In: *Stat. Sci.* 27.2 (2012), pp. 202–231.

[FHT00] J. Friedman, T. Hastie, and R. Tibshirani. "Additive logistic regression: a statistical view of boosting". In: *Annals of statistics* 28.2 (2000), pp. 337–374.

[FHT10] J. Friedman, T. Hastie, and R. Tibshirani. "Regularization Paths for Generalized Linear Models via Coordinate Descent". In: *J. of Statistical Software* 33.1 (2010).

[Fir57] J. Firth. "A synopsis of linguistic theory 1930-1955". In: *Studies in Linguistic Analysis*. Ed. by F. Palmer. 1957.

[FJ02] M. A. T. Figueiredo and A. K. Jain. "Unsupervised Learning of Finite Mixture Models". In: *IEEE PAMI* 24.3 (2002), pp. 381–396.

[FM03] J. H. Friedman and J. J. Meulman. "Multiple additive regression trees with application in epidemiology". en. In: *Stat. Med.* 22.9 (2003), pp. 1365–1381.

[FMN16] C. Fefferman, S. Mitter, and H. Narayanan. "Testing the manifold hypothesis". In: *J. Amer. Math. Soc.* 29.4 (2016), pp. 983–1049.

[FNW07] M. Figueiredo, R. Nowak, and S. Wright. "Gradient projection for sparse reconstruction: application to compressed sensing and other inverse problems". In: *IEEE. J. on Selected Topics in Signal Processing* (2007).

[For+21] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur. "Sharpness-aware Minimization for Efficiently Improving Generalization". In: *ICLR*. 2021.

[Fos19] D. Foster. *Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play*. 1 edition. O'Reilly Media, 2019.

[FR07] C. Fraley and A. Raftery. "Bayesian Regularization for Normal Mixture Estimation and Model-Based Clustering". In: *J. of Classification* 24 (2007), pp. 155–181.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

[Fra+17] L. Franceschi, M. Donini, P. Frasconi, and M. Pontil. "Forward and Reverse Gradient-Based Hyperparameter Optimization". In: *ICML*. 2017.

[Fre98] B. Frey. *Graphical Models for Machine Learning and Digital Communication*. MIT Press, 1998.

[Fri01] J. Friedman. "Greedy Function Approximation: a Gradient Boosting Machine". In: *Annals of Statistics* 29 (2001), pp. 1189–1232.

[Fri97a] J. Friedman. "On bias, variance, 0-1 loss and the curse of dimensionality". In: *J. Data Mining and Knowledge Discovery* 1 (1997), pp. 55–77.

[Fri97b] J. H. Friedman. "Data mining and statistics: What's the connection". In: *Proceedings of the 29th Symposium on the Interface Between Computer Science and Statistics*. 1997.

[Fri99] J. Friedman. *Stochastic Gradient Boosting*. Tech. rep. 1999.

[FS96] Y. Freund and R. R. Schapire. "Experiments with a new boosting algorithm". In: *ICML*. 1996.

[FT05] M. Fashing and C. Tomasi. "Mean shift is a bound optimization". en. In: *IEEE Trans. Pattern Anal. Mach. Intell.* 27.3 (2005), pp. 471–474.

[Fu98] W. Fu. "Penalized regressions: the bridge versus the lasso". In: *J. Computational and graphical statistics* 7 (1998), 397– 416.

[Fuk75] K. Fukushima. "Cognitron: a self-organizing multilayered neural network". In: *Biological Cybernetics* 20.6 (1975), pp. 121–136.

[Fuk80] K Fukushima. "Neocognitron: a self organizing neural network model for a mechanism of pattern recognition unaffected by shift in position". en. In: *Biol. Cybern.* 36.4 (1980), pp. 193–202.

[Fuk90] K. Fukunaga. *Introduction to Statistical Pattern Recognition*. 2nd edition. Academic Press, 1990.

[Gag94] P. Gage. "A New Algorithm for Data Compression". In: *Dr Dobbs Journal* (1994).

[Gan+16] Y Ganin, E Ustinova, H Ajakan, P Germain, and others. "Domain-adversarial training of neural networks". In: *JMLR* (2016).

[Gär03] T. Gärtner. "A Survey of Kernels for Structured Data". In: *SIGKDD Explor. Newsl.* 5.1 (2003), pp. 49–58.

[Gar+18] J. Gardner, G. Pleiss, K. Q. Weinberger, D. Bindel, and A. G. Wilson. "GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration". In: *NIPS*. Ed. by S Bengio, H Wallach, H Larochelle, K Grauman, N Cesa-Bianchi, and R Garnett. Curran Associates, Inc., 2018, pp. 7576–7586.

[GASG18] D. G. A. Smith and J. Gray. "opt-einsum - A Python package for optimizing contraction order for einsum-like expressions". In: *JOSS* 3.26 (2018), p. 753.

[GB05] Y. Grandvalet and Y. Bengio. "Semi-supervised learning by entropy minimization". In: *Advances in neural information processing systems*. 2005, pp. 529–536.

[GB10] X. Glorot and Y. Bengio. "Understanding the difficulty of training deep feedforward neural networks". In: *AISTATS*. 2010, pp. 249–256.

[GB18] V. Garcia and J. Bruna. "Few-shot Learning with Graph Neural Networks". In: *International Conference on Learning Representations (ICLR)*. 2018.

[GBB11] X. Glorot, A. Bordes, and Y. Bengio. "Deep Sparse Rectifer Neural Networks". In: *AISTATS*. 2011.

[GBC16] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[GBD92] S. Geman, E. Bienenstock, and R. Doursat. "Neural networks and the bias-variance dilemma". In: *Neural Computing* 4 (1992), pp. 1–58.

[GC20] A. Gelman and B. Carpenter. "Bayesian analysis of tests with unknown specificity and sensitivity". In: *J. of Royal Stat. Soc. Series C* medrxiv;2020.05.22.20108944v2 (2020).

[GEB16] L. A. Gatys, A. S. Ecker, and M. Bethge. "Image style transfer using convolutional neural networks". In: *CVPR*. 2016, pp. 2414–2423.

[GEH19] T. Gale, E. Elsen, and S. Hooker. "The State of Sparsity in Deep Neural Networks". In: (2019). arXiv: 1902.09574 [cs.LG].

[Gel+04] A. Gelman, J. Carlin, H. Stern, and D. Rubin. *Bayesian data analysis*. 2nd edition. Chapman and Hall, 2004.

[Gel+14] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin. *Bayesian Data Analysis, Third Edition*. Third edition. Chapman and Hall/CRC, 2014.

[Gel16] A. Gelman. "The problems with p-values are not just with p-values". In: *American Statistician* (2016).

[Gér17] A. Géron. *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques for Building Intelligent Systems*. en. O'Reilly Media, Incorporated, 2017.

[Gér19] A. Géron. *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques for Building Intelligent Systems (2nd edition)*. en. O'Reilly Media, Incorporated, 2019.

[GEY19] Y. Geifman and R. El-Yaniv. "SelectiveNet: A Deep Neural Network with an Integrated Reject Option". In: *ICML*. 2019.

[GG16] Y. Gal and Z. Ghahramani. "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning". In: *ICML*. 2016.

[GH96] Z. Ghahramani and G. Hinton. *The EM Algorithm for Mixtures of Factor Analyzers*. Tech. rep. Dept. of Comp. Sci., Uni. Toronto, 1996.

[GHK17] Y. Gal, J. Hron, and A. Kendall. "Concrete Dropout". In: (2017). arXiv: 1705.07832 [stat.ML].

[GHV14] A. Gelman, J. Hwang, and A. Vehtari. "Understanding predictive information criteria for Bayesian models". In: *Statistics and Computing* 24.6 (2014), pp. 997–1016.

[Gib97] M. Gibbs. "Bayesian Gaussian Processes for Regression and Classification". PhD thesis. U. Cambridge, 1997.

[Gil+17] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. "Neural message passing for quantum chemistry". In: *ICML*. 2017, pp. 1263–1272.

[Gil+21] J. Gilmer, B. Ghorbani, A. Garg, S. Kudugunta, B. Neyshabur, D. Cardoze, G. Dahl, Z. Nado, and O. Firat. "A Loss Curvature Perspective on Training Instability in Deep Learning". In: (2021). arXiv: 2110.04369 [cs.LG].

[GIM99] A. Gionis, P. Indyk, and R. Motwani. "Similarity Search in High Dimensions via Hashing". In: *Proc. 25th Intl. Conf. on Very Large Data Bases*. VLDB '99. 1999, pp. 518–529.

[GKS18] V. Gupta, T. Koren, and Y. Singer. "Shampoo: Preconditioned Stochastic Tensor Optimization". In: *ICML*. 2018.

[GL15] B. Gu and C. Ling. "A New Generalized Error Path Algorithm for Model Selection". In: *ICML*. 2015.

[GL16] A. Grover and J. Leskovec. "node2vec: Scalable feature learning for networks". In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM. 2016, pp. 855–864.

[GMS05] M. Gori, G. Monfardini, and F. Scarselli. "A new model for learning in graph domains". In: *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.* Vol. 2. IEEE. 2005, pp. 729–734.

[GNK18] R. A. Güler, N. Neverova, and I. Kokkinos. "Densepose: Dense human pose estimation in the wild". In: *CVPR*. 2018, pp. 7297–7306.

[God18] P. Godec. *Graph Embeddings; The Summary*. https://towardsdatascience.com/graph-embeddings-the-summary-cc6075aba007. 2018.

[GOF18] O. Gouvert, T. Oberlin, and C. Févotte. "Negative Binomial Matrix Factorization for Recommender Systems". In: (2018). arXiv: 1801.01708 [cs.LG].

[Gol+01] K. Goldberg, T. Roeder, D. Gupta, and C. Perkins. "Eigentaste: A Constant Time Collaborative Filtering Algorithm". In: *Information Retrieval* 4.2 (2001), pp. 133–151.

[Gol+05] J. Goldberger, S. Roweis, G. Hinton, and R. Salakhutdinov. "Neighbourhood Components Analysis". In: *NIPS*. 2005.

[Gol+92] D. Goldberg, D. Nichols, B. M. Oki, and D. Terry. "Using collaborative filtering to weave an information tapestry". In: *Commun. ACM* 35.12 (1992), pp. 61–70.

[Gon85] T. Gonzales. "Clustering to minimize the maximum intercluster distance". In: *Theor. Comp. Sci.* 38 (1985), pp. 293–306.

[Goo01] N. Goodman. "Classes for fast maximum entropy training". In: *ICASSP*. 2001.

[Goo+14] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. "Generative Adversarial Networks". In: *NIPS*. 2014.

[Gor06] P. F. Gorder. "Neural Networks Show New Promise for Machine Vision". In: *Computing in science & engineering* 8.6 (2006), pp. 4–8.

[Got+19] A. Gotmare, N. S. Keskar, C. Xiong, and R. Socher. "A Closer Look at Deep Learning Heuristics: Learning rate restarts, Warmup and Distillation". In: *ICLR*. 2019.

[GOV18] W Gao, S Oh, and P Viswanath. "Demystifying Fixed k -Nearest Neighbor Information Estimators". In: *IEEE Trans. Inf. Theory* 64.8 (2018), pp. 5629–5661.

[GR07] T. Gneiting and A. E. Raftery. "Strictly Proper Scoring Rules, Prediction, and Estimation". In: *JASA* 102.477 (2007), pp. 359–378.

[GR18] A. Graves and M.-A. Ranzato. "Tutorial on unsupervised deep learning: part 2". In: *NIPS*. 2018.

[GR19] P. Grünwald and T. Roos. "Minimum description length revisited". In: *Int. J. Math. Ind.* 11.01 (Dec. 2019), p. 1930001.

[Gra04] Y. Grandvalet. "Bagging Equalizes Influence". In: *Mach. Learn.* 55 (2004), pp. 251–270.

[Gra11] A. Graves. "Practical variational inference for neural networks". In: *Advances in neural information processing systems*. 2011, pp. 2348–2356.

[Gra13] A. Graves. "Generating Sequences With Recurrent Neural Networks". In: (2013). arXiv: 1308.0850 [cs.NE].

[Gra+17] E. Grave, A. Joulin, M. Cissé, D. Grangier, and H. Jégou. "Efficient softmax approximation for GPUs". In: *ICML*. 2017.

[Gra+18] E. Grant, C. Finn, S. Levine, T. Darrell, and T. Griffiths. "Recasting Gradient-Based Meta-Learning as Hierarchical Bayes". In: *ICLR*. 2018.

[Gra+20] W. Grathwohl, K.-C. Wang, J.-H. Jacobsen, D. Duvenaud, M. Norouzi, and K. Swersky. "Your classifier is secretly an energy based model and you should treat it like one". In: *ICLR*. 2020.

[Gre+17] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber. "LSTM: A Search Space Odyssey". In: *IEEE Transactions on Neural Networks and Learning Systems* 28.10 (2017).

[Gri20] T. L. Griffiths. "Understanding Human Intelligence through Human Limitations". en. In: *Trends Cogn. Sci.* 24.11 (2020), pp. 873–883.

[Gru07] P. Grunwald. *The Minimum Description Length Principle*. MIT Press, 2007.

[GS08] Y Guo and D Schuurmans. "Efficient global optimization for exponential family PCA and low-rank matrix factorization". In: *2008 46th Annual Allerton Conference on Communication, Control, and Computing*. 2008, pp. 1100–1107.

[GS97] C. M. Grinstead and J. L. Snell. *Introduction to probability (2nd edition)*. American Mathematical Society, 1997.

[GSK18] S. Gidaris, P. Singh, and N. Komodakis. "Unsupervised Representation Learning by Predicting Image Rotations". In: *ICLR*. 2018.

[GT07] L. Getoor and B. Taskar, eds. *Introduction to Relational Statistical Learning*. MIT Press, 2007.

[GTA00] G. Gigerenzer, P. M. Todd, and ABC Research Group. *Simple Heuristics That Make Us Smart*. en. Illustrated edition. Oxford University Press, 2000.

[Gu+18] A. Gu, F. Sala, B. Gunel, and C. Ré. "Learning Mixed-Curvature Representations in Product Spaces". In: *International Conference on Learning Representations* (2018).

[Gua+10] Y. Guan, J. Dy, D. Niu, and Z. Ghahramani. "Variational Inference for Nonparametric Multiple Clustering". In: *1st Intl. Workshop on Discovering, Summarizing and Using Multiple Clustering (MultiClust)*. 2010.

[Gua+17] S. Guadarrama, R. Dahl, D. Bieber, M. Norouzi, J. Shlens, and K. Murphy. "PixColor: Pixel Recursive Colorization". In: *BMVC*. 2017.

[Gul+20] A. Gulati et al. "Conformer: Convolution-augmented Transformer for Speech Recognition". In: (2020). arXiv: 2005.08100 [eess.AS].

[Guo09] Y. Guo. "Supervised exponential family principal component analysis via convex optimization". In: *NIPS*. 2009.

[Guo+17] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He. "DeepFM: a factorization-machine based neural network for CTR prediction". In: *IJCAI*. IJCAI'17. AAAI Press, 2017, pp. 1725–1731.

[Gus01] M. Gustafsson. "A probabilistic derivation of the partial least-squares algorithm". In: *Journal of Chemical Information and Modeling* 41 (2001), pp. 288–294.

[GVZ16] A. Gupta, A. Vedaldi, and A. Zisserman. "Synthetic Data for Text Localisation in Natural Images". In: *CVPR*. 2016.

[GZE19] A. Grover, A. Zweig, and S. Ermon. "Graphite: Iterative Generative Modeling of Graphs". In: *International Conference on Machine Learning*. 2019, pp. 2434–2444.

[HA85] L. Hubert and P. Arabie. "Comparing Partitions". In: *J. of Classification* 2 (1985), pp. 193–218.

[HAB19] M. Hein, M. Andriushchenko, and J. Bitterwolf. "Why ReLU networks yield high-confidence predictions far away from the training data and how to mitigate the problem". In: *CVPR*. 2019.

[Hac75] I. Hacking. *The Emergence of Probability: A Philosophical Study of Early Ideas about Probability, Induction and Statistical Inference*. Cambridge University Press, 1975.

[Háj08] A. Hájek. "Dutch Book Arguments". In: *The Oxford Handbook of Rational and Social Choice*. Ed. by P. Anand, P. Pattanaik, and C. Puppe. Oxford University Press, 2008.

[Han+20] B. Han, Q. Yao, T. Liu, G. Niu, I. W. Tsang, J. T. Kwok, and M. Sugiyama. "A Survey of Label-noise Representation Learning: Past, Present and Future". In: (2020). arXiv: 2011.04406 [cs.LG].

[Har54] Z. Harris. "Distributional structure". In: *Word* 10.23 (1954), pp. 146–162.

[Has+04] T. Hastie, S. Rosset, R. Tibshirani, and J. Zhu. "The entire regularization path for the support vector machine". In: *JMLR* 5 (2004), pp. 1391–1415.

[Has+09] T. Hastie, S. Rosset, J. Zhu, and H. Zou. "Multi-class AdaBoost". In: *Statistics and tis Interface* 2.3 (2009), pp. 349–360.

[Has+17] D. Hassabis, D. Kumaran, C. Summerfield, and M. Botvinick. "Neuroscience-Inspired Artificial Intelligence". en. In: *Neuron* 95.2 (2017), pp. 245–258.

[Has87] J. Hastad. *Computational limits of small-depth circuits*. MIT Press, 1987.

[HB17] X. Huang and S. Belongie. "Arbitrary style transfer in real-time with adaptive instance normalization". In: *ICCV*. 2017.

[HBK23] P. Haluptzok, M. Bowers, and A. T. Kalai. "Language Models Can Teach Themselves to Program Better". In: *ICLR*. Feb. 2023.

[HCD12] D. Hoiem, Y. Chodpathumwan, and Q. Dai. "Diagnosing Error in Object Detectors". In: *ECCV*. 2012.

[HCL03] C.-W. Hsu, C.-C. Chang, and C.-J. Lin. *A Practical Guide to Support Vector Classification*. Tech. rep. Dept. Comp. Sci., National Taiwan University, 2003.

[HDR19] S. Hayou, A. Doucet, and J. Rousseau. "On the Impact of the Activation Function on Deep Neural Networks Training". In: (2019). arXiv: 1902.06853 [stat.ML].

[He+15] K. He, X. Zhang, S. Ren, and J. Sun. "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". In: *ICCV*. 2015.

[He+16a] K. He, X. Zhang, S. Ren, and J. Sun. "Deep Residual Learning for Image Recognition". In: *CVPR*. 2016.

[He+16b] K. He, X. Zhang, S. Ren, and J. Sun. "Identity Mappings in Deep Residual Networks". In: *ECCV*. 2016.

[He+17] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua. "Neural Collaborative Filtering". In: *WWW*. 2017.

[HE18] D. Ha and D. Eck. "A Neural Representation of Sketch Drawings". In: *ICLR*. 2018.

[He+20] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. "Momentum contrast for unsupervised visual representation learning". In: *CVPR*. 2020, pp. 9729–9738.

[Hen+15] J. Hensman, A. Matthews, M. Filippone, and Z. Ghahramani. "MCMC for Variationally Sparse Gaussian Processes". In: *NIPS*. 2015, pp. 1648–1656.

[HG16] D. Hendrycks and K. Gimpel. "Gaussian Error Linear Units (GELUs)". In: *arXiv [cs.LG]* (2016).

[HG20] J. Howard and S. Gugger. *Deep Learning for Coders with Fastai and PyTorch: AI Applications Without a PhD*. en. 1st ed. O'Reilly Media, 2020.

[HG21] M. K. Ho and T. L. Griffiths. "Cognitive science as a source of forward and inverse models of human decisions for robotics and control". In: *Annual Review of Control, Robotics, and Autonomous Systems*. 2021.

[HGD19] K. He, R. Girshick, and P. Dollár. "Rethinking ImageNet Pre-training". In: *CVPR*. 2019.

[Hin+12] G. E. Hinton et al. "Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups". In: *IEEE Signal Process. Mag.* 29.6 (2012), pp. 82–97.

[Hin13] G. Hinton. *CSC 2535 Lecture 11: Non-linear dimensionality reduction*. 2013.

[Hin14] G. Hinton. *Lecture 6e on neural networks (RMSprop: Divide the gradient by a running average of its recent magnitude)*. 2014.

[HK15] F. M. Harper and J. A. Konstan. "The MovieLens Datasets: History and Context". In: *ACM Trans. Interact. Intell. Syst.* 5.4 (2015), pp. 1–19.

[HK92] A Hertz and J Krogh. "Generalization in a linear perceptron in the presence of noise". In: *J. Physics A* (1992).

[HKV19] F. Hutter, L. Kotthoff, and J. Vanschoren, eds. *Automated Machine Learning - Methods, Systems, Challenges*. Springer, 2019.

[HL04] D. R. Hunter and K. Lange. "A Tutorial on MM Algorithms". In: *The American Statistician* 58 (2004), pp. 30–37.

[HMT11] N. Halko, P.-G. Martinsson, and J. A. Tropp. "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions". In: *SIAM Rev., Survey and Review section* 53.2 (2011), pp. 217–288.

[HN19] C. M. Holmes and I. Nemenman. "Estimation of mutual information for real-valued data with error bars and controlled bias". en. In: *Phys Rev E* 100.2-1 (2019), p. 022404.

[Hoc+01] S. Hochreiter, Y. Bengio, P. Frasconi, and J. Schmidhuber. "Gradient flow in recurrent nets: the difficulty of learning long-term dependencies". In: *A Field Guide to Dynamical Recurrent Neural Networks*. Ed. by S. C. Kremer and J. F. Kolen. 2001.

[Hoe+14] R. Hoekstra, R. D. Morey, J. N. Rouder, and E.-J. Wagenmakers. "Robust misinterpretation of confidence intervals". en. In: *Psychon. Bull. Rev.* 21.5 (2014), pp. 1157–1164.

[Hoe+21] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste. "Sparsity in Deep Learning: Pruning and growth for efficient inference and training in neural networks". In: (2021). arXiv: 2102.00554 [cs.LG].

[Hof09] P. D. Hoff. *A First Course in Bayesian Statistical Methods*. Springer, 2009.

[Hor61] P Horst. "Generalized canonical correlations and their applications to experimental data". en. In: *J. Clin. Psychol.* 17 (1961), pp. 331–347.

[Hor91] K. Hornik. "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 4.2 (1991), pp. 251–257.

[Hos+19] M. Z. Hossain, F. Sohel, M. F. Shiratuddin, and H. Laga. "A Comprehensive Survey of Deep Learning for Image Captioning". In: *ACM Computing Surveys* (2019).

[HOT06] G. Hinton, S. Osindero, and Y. Teh. "A fast learning algorithm for deep belief nets". In: *Neural Computation* 18 (2006), pp. 1527–1554.

[Hot36] H. Hotelling. "Relations Between Two Sets of Variates". In: *Biometrika* 28.3/4 (1936), pp. 321–377.

[Hou+12] N. Houlsby, F. Huszar, Z. Ghahramani, and J. M. Hernández-lobato. "Collaborative Gaussian Processes for Preference Learning". In: *NIPS*. 2012, pp. 2096–2104.

[Hou+19] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. "Parameter-Efficient Transfer Learning for NLP". In: *ICML*. 2019.

[How+17] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications". In: *CVPR*. 2017.

[HR03] G. E. Hinton and S. T. Roweis. "Stochastic Neighbor Embedding". In: *NIPS*. 2003, pp. 857–864.

[HR76] L. Hyafil and R. Rivest. "Constructing Optimal Binary Decision Trees is NP-complete". In: *Information Processing Letters* 5.1 (1976), pp. 15–17.

[HRP21] M. Huisman, J. N. van Rijn, and A. Plaat. "A Survey of Deep Meta-Learning". In: *AI Review* (2021).

[HS19] J. Haochen and S. Sra. "Random Shuffling Beats SGD after Finite Epochs". In: *ICML*. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 2624–2633.

[HS97a] S Hochreiter and J Schmidhuber. "Flat minima". en. In: *Neural Comput.* 9.1 (1997), pp. 1–42.

[HS97b] S. Hochreiter and J. Schmidhuber. "Long short-term memory". In: *Neural Computation* 9.8 (1997), 1735–1780.

[HSW89] K. Hornik, M. Stinchcombe, and H. White. "Multilayer feedforward networks are universal approximators". In: *Neural Networks* 2.5 (1989), pp. 359–366.

[HT90] T. Hastie and R. Tibshirani. *Generalized additive models*. Chapman and Hall, 1990.

[HTF01] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. Springer, 2001.

[HTF09] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. 2nd edition. Springer, 2009.

[HTW15] T. Hastie, R. Tibshirani, and M. Wainwright. *Statistical Learning with Sparsity: The Lasso and Generalizations*. CRC Press, 2015.

[Hua14] G.-B. Huang. "An Insight into Extreme Learning Machines: Random Neurons, Random Features and Kernels". In: *Cognit. Comput.* 6.3 (2014), pp. 376–390.

[Hua+17a] G. Huang, Z. Liu, K. Q. Weinberger, and L. van der Maaten. "Densely Connected Convolutional Networks". In: *CVPR*. 2017.

[Hua+17b] J. Huang et al. "Speed/accuracy trade-offs for modern convolutional object detectors". In: *CVPR*. 2017.

[Hua+18] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. Shazeer, I. Simon, C. Hawthorne, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck. "Music Transformer". In: (2018). arXiv: 1809.04281 [cs.LG].

[Hub+08] M. F. Huber, T Bailey, H Durrant-Whyte, and U. D. Hanebeck. "On entropy approximation for Gaussian mixture random vectors". In: *2008 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems*. 2008, pp. 181–188.

[Hub64] P. Huber. "Robust Estimation of a Location Parameter". In: *Annals of Statistics* 53 (1964), 73–101.

[Hut90] M. F. Hutchinson. "A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines". In: *Communications in Statistics - Simulation and Computation* 19.2 (1990), pp. 433–450.

[HVD14] G. Hinton, O. Vinyals, and J. Dean. "Distilling the Knowledge in a Neural Network". In: *NIPS Deep Learning Workshop*. 2014.

[HW62] D. Hubel and T. Wiesel. "Receptive fields, binocular interaction, and functional architecture in the cat's visual cortex". In: *J. Physiology* 160 (1962), pp. 106–154.

[HY01a] D. J. Hand and K. Yu. "Idiot's Bayes: Not So Stupid after All?" In: *Int. Stat. Rev.* 69.3 (2001), pp. 385–398.

[HY01b] M. Hansen and B. Yu. "Model selection and the principle of minimum description length". In: *JASA* (2001).

[HYL17] W. Hamilton, Z. Ying, and J. Leskovec. "Inductive representation learning on large graphs". In: *Advances in Neural Information Processing Systems*. 2017, pp. 1024–1034.

[Idr+17] H. Idrees, A. R. Zamir, Y.-G. Jiang, A. Gorban, I. Laptev, R. Sukthankar, and M. Shah. "The THUMOS challenge on action recognition for videos "in the wild"". In: *Comput. Vis. Image Underst.* 155 (2017), pp. 1–23.

[Ie+19] E. Ie, V. Jain, J. Wang, S. Narvekar, R. Agarwal, R. Wu, H.-T. Cheng, T. Chandra, and C. Boutilier. "SlateQ: A tractable decomposition for reinforcement learning with recommendation sets". In: *IJCAI*. International Joint Conferences on Artificial Intelligence Organization, 2019.

[Iof17] S. Ioffe. "Batch Renormalization: Towards Reducing Minibatch Dependence in Batch-Normalized Models". In: (2017). arXiv: 1702.03275 [cs.LG].

[Ips09] I. Ipsen. *Numerical matrix analysis: Linear systems and least squares*. SIAM, 2009.

[IR10] A. Ilin and T. Raiko. "Practical Approaches to Principal Component Analysis in the Presence of Missing Values". In: *JMLR* 11 (2010), pp. 1957–2000.

[IS15] S. Ioffe and C. Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *ICML*. 2015, pp. 448–456.

[Isc+19] A. Iscen, G. Tolias, Y. Avrithis, and O. Chum. "Label Propagation for Deep Semi-supervised Learning". In: *CVPR*. 2019.

[Izm+18] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, and A. G. Wilson. "Averaging Weights Leads to Wider Optima and Better Generalization". In: *UAI*. 2018.

[Izm+20] P. Izmailov, P. Kirichenko, M. Finzi, and A. G. Wilson. "Semi-supervised learning with normalizing flows". In: *ICML*. 2020, pp. 4615–4630.

[Jac+91] R. Jacobs, M. Jordan, S. Nowlan, and G. Hinton. "Adaptive mixtures of local experts". In: *Neural Computation* (1991).

[JAFF16] J. Johnson, A. Alahi, and L. Fei-Fei. "Perceptual Losses for Real-Time Style Transfer and Super-Resolution". In: *ECCV*. 2016.

[Jan18] E. Jang. *Normalizing Flows Tutorial*. https://blog.evjang.com/2018/01/nf1.html. 2018.

[Jay03] E. T. Jaynes. *Probability theory: the logic of science*. Cambridge university press, 2003.

[Jay76] E. T. Jaynes. "Confidence intervals vs Bayesian intervals". In: *Foundations of Probability Theory, Statistical Inference, and Statistical Theories of Science, vol II*. Ed. by W. L. Harper and C. A. Hooker. Reidel Publishing Co., 1976.

[JD88] A. Jain and R. Dubes. *Algorithms for Clustering Data*. Prentice Hall, 1988.

[JDJ17] J. Johnson, M. Douze, and H. Jégou. "Billion-scale similarity search with GPUs". In: (2017). arXiv: 1702.08734 [cs.CV].

[Jef61] H. Jeffreys. *Theory of Probability*. Oxford, 1961.

[Jef73] H. Jeffreys. *Scientific Inference*. Third edition. Cambridge, 1973.

[JGH18] A. Jacot, F. Gabriel, and C. Hongler. "Neural Tangent Kernel: Convergence and Generalization in Neural Networks". In: *NIPS*. 2018.

[JH04] H. Jaeger and H. Haas. "Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication". In: *Science* 304.5667 (2004).

[JHG00] N. Japkowicz, S. Hanson, and M. Gluck. "Nonlinear autoassociation is not equivalent to PCA". In: *Neural Computation* 12 (2000), pp. 531–545.

[Jia+20] Y. Jiang, B. Neyshabur, H. Mobahi, D. Krishnan, and S. Bengio. "Fantastic Generalization Measures and Where to Find Them". In: *ICLR*. 2020.

[Jin+17] Y. Jing, Y. Yang, Z. Feng, J. Ye, Y. Yu, and M. Song. "Neural Style Transfer: A Review". In: *arXiv [cs.CV]* (2017).

[JJ94] M. I. Jordan and R. A. Jacobs. "Hierarchical mixtures of experts and the EM algorithm". In: *Neural Computation* 6 (1994), pp. 181–214.

[JK13] A. Jern and C. Kemp. "A probabilistic account of exemplar and category generation". en. In: *Cogn. Psychol.* 66.1 (2013), pp. 85–125.

[JM08] D. Jurafsky and J. H. Martin. *Speech and language processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. 2nd edition. Prentice-Hall, 2008.

[JM20] D. Jurafsky and J. H. Martin. *Speech and language processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition (Third Edition)*. Draft of 3rd edition. 2020.

[Jor19] M. Jordan. "Artificial Intelligence — The Revolution Hasn't Happened Yet". In: *Harvard Data Science Review* 1.1 (2019).

[JT19] L. Jing and Y. Tian. "Self-supervised Visual Feature Learning with Deep Neural Networks: A Survey". In: (2019). arXiv: 1902.06162 [cs.CV].

[Jun+19] W. Jung, D. Jung, B. Kim, S. Lee, W. Rhee, and J. Anh. "Restructuring Batch Normalization to Accelerate CNN Training". In: *SysML*. 2019.

[JW19] S. Jain and B. C. Wallace. "Attention is not Explanation". In: *NAACL*. 2019.

[JZ13] R. Johnson and T. Zhang. "Accelerating Stochastic Gradient Descent using Predictive Variance Reduction". In: *NIPS*. Curran Associates, Inc., 2013, pp. 315–323.

[JZS15] R. Jozefowicz, W. Zaremba, and I. Sutskever. "An Empirical Exploration of Recurrent Network Architectures". In: *ICML*. 2015, pp. 2342–2350.

[KAG19] A. Kirsch, J. van Amersfoort, and Y. Gal. "BatchBALD: Efficient and Diverse Batch Acquisition for Deep Bayesian Active Learning". In: *NIPS*. 2019.

[Kai58] H. Kaiser. "The varimax criterion for analytic rotation in factor analysis". In: *Psychometrika* 23.3 (1958).

[Kan+12] E. Kandel, J. Schwartz, T. Jessell, S. Siegelbaum, and A. Hudspeth, eds. *Principles of Neural Science*. Fifth Edition. 2012.

[Kan+20] B. Kang, S. Xie, M. Rohrbach, Z. Yan, A. Gordo, J. Feng, and Y. Kalantidis. "Decoupling Representation and Classifier for Long-Tailed Recognition". In: *ICLR*. 2020.

[Kap16] J. Kaplan. *Artificial Intelligence: What Everyone Needs to Know*. en. 1st ed. Oxford University Press, 2016.

[Kap+20] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. "Scaling laws for neural language models". In: *arXiv [cs.LG]* (Jan. 2020).

[Kat+20] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. "Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention". In: *ICML*. 2020.

[KB15] D. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization". In: *ICLR*. 2015.

[KB19] M. Kaya and H. S. Bilge. "Deep Metric Learning: A Survey". en. In: *Symmetry* 11.9 (2019), p. 1066.

[KBV09] Y. Koren, R. Bell, and C. Volinsky. "Matrix factorization techniques for recommender systems". In: *IEEE Computer* 42.8 (2009), pp. 30–37.

[KD09] A. D. Kiureghian and O. Ditlevsen. "Aleatory or epistemic? Does it matter?" In: *Structural Safety* 31.2 (2009), pp. 105–112.

[Kem+06] C. Kemp, J. Tenenbaum, T. Y. T. Griffiths and, and N. Ueda. "Learning systems of concepts with an infinite relational model". In: *AAAI*. 2006.

[KF05] H. Kuck and N. de Freitas. "Learning about individuals from group statistics". In: *UAI*. 2005.

[KG05] A. Krause and C. Guestrin. "Near-optimal value of information in graphical models". In: *UAI*. 2005.

[KG17] A. Kendall and Y. Gal. "What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?" In: *NIPS*. Curran Associates, Inc., 2017, pp. 5574–5584.

[KGS20] J. von Kügelgen, L. Gresele, and B. Schölkopf. "Simpson's paradox in Covid-19 case fatality rates: a mediation analysis of age-related causal effects". In: (2020). arXiv: 2005.07180 [stat.AP].

[KH09] A Krizhevsky and G Hinton. *Learning multiple layers of features from tiny images*. Tech. rep. U. Toronto, 2009.

[KH19] D. Krotov and J. J. Hopfield. "Unsupervised learning by competing hidden units". en. In: *PNAS* 116.16 (2019), pp. 7723–7731.

[Kha+10] M. E. Khan, B. Marlin, G. Bouchard, and K. P. Murphy. "Variational bounds for mixed-data factor analysis". In: *NIPS*. 2010.

[Kha+20] A. Khan, A. Sohail, U. Zahoora, and A. S. Qureshi. "A Survey of the Recent Architectures of Deep Convolutional Neural Networks". In: *AI Review* (2020).

[KHB07] A. Kapoor, E. Horvitz, and S. Basu. "Selective Supervision: Guiding Supervised Learning with Decision-Theoretic Active Learning". In: *IJCAI*. 2007.

[KHW19] W. Kool, H. van Hoof, and M. Welling. "Stochastic Beams and Where to Find Them: The Gumbel-Top-k Trick for Sampling Sequences Without Replacement". In: *ICML*. 2019.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

[Kim14] Y. Kim. "Convolutional Neural Networks for Sentence Classification". In: *EMNLP*. 2014.

[Kim19] D. H. Kim. *Survey of Deep Metric Learning*. 2019.

[Kin+14] D. P. Kingma, D. J. Rezende, S. Mohamed, and M. Welling. "Semi-Supervised Learning with Deep Generative Models". In: *NIPS*. 2014.

[Kir+19] A. Kirillov, K. He, R. Girshick, C. Rother, and P. Dollár. "Panoptic Segmentation". In: *CVPR*. 2019.

[KJ16] L Kang and V Joseph. "Kernel Approximation: From Regression to Interpolation". In: *SIAM/ASA J. Uncertainty Quantification* 4.1 (2016), pp. 112–129.

[KJ95] J. Karhunen and J. Joutsensalo. "Generalizations of principal component analysis, optimization problems, and neural networks". In: *Neural Networks* 8.4 (1995), pp. 549–562.

[KJM19] N. M. Kriege, F. D. Johansson, and C. Morris. "A Survey on Graph Kernels". In: (2019). arXiv: 1903.11835 [cs.LG].

[KK06] S. Kotsiantis and D. Kanellopoulos. "Discretization Techniques: A recent survey". In: *GESTS Intl. Trans. on Computer Science and Engineering* 31.1 (2006), pp. 47–58.

[KKH20] I. Khemakhem, D. P. Kingma, and A. Hyvärinen. "Variational Autoencoders and Nonlinear ICA: A Unifying Framework". In: *AISTATS*. 2020.

[KKL20] N. Kitaev, L. Kaiser, and A. Levskaya. "Reformer: The Efficient Transformer". In: *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.

[KKS20] F. Kunstner, R. Kumar, and M. Schmidt. "Homeomorphic-Invariance of EM: Non-Asymptotic Convergence in KL Divergence for Exponential Families via Mirror Descent". In: (2020). arXiv: 2011.01170 [cs.LG].

[KL17] J. K. Kruschke and T. M. Liddell. "The Bayesian New Statistics: Hypothesis testing, estimation, meta-analysis, and power analysis from a Bayesian perspective". In: *Psychon. Bull. Rev.* (2017).

[KL21] W. M. Kouw and M. Loog. "A review of domain adaptation without target labels". en. In: *IEEE PAMI* (2021).

[Kla+17] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter. "Self-Normalizing Neural Networks". In: *NIPS*. 2017.

[Kle02] J. Kleinberg. "An Impossibility Theorem for Clustering". In: *NIPS*. 2002.

[Kle+11] A. Kleiner, A. Talwalkar, P. Sarkar, and M. I. Jordan. *A scalable bootstrap for massive data*. Tech. rep. UC Berkeley, 2011.

[Kle13] P. N. Klein. *Coding the Matrix: Linear Algebra through Applications to Computer Science*. en. 1 edition. Newtonian Press, 2013.

[KLQ95] C. Ko, J. Lee, and M. Queyranne. "An exact algorithm for maximum entropy sampling". In: *Operations Research* 43 (1995), 684–691.

[Kok17] I. Kokkinos. "UberNet: Training a Universal Convolutional Neural Network for Low-, Mid-, and High-Level Vision Using Diverse Datasets and Limited Memory". In: *CVPR*. Vol. 2. 2017, p. 8.

[Kol+19] A. Kolesnikov, L. Beyer, X. Zhai, J. Puigcerver, J. Yung, S. Gelly, and N. Houlsby. "Large Scale Learning of General Visual Representations for Transfer". In: (2019). arXiv: 1912.11370 [cs.CV].

[Kol+20] A. Kolesnikov, L. Beyer, X. Zhai, J. Puigcerver, J. Yung, S. Gelly, and N. Houlsby. "Large Scale Learning of General Visual Representations for Transfer". In: *ECCV*. 2020.

[Kon20] M. Konnikova. *The Biggest Bluff: How I Learned to Pay Attention, Master Myself, and Win*. en. Penguin Press, 2020.

[Kor09] Y. Koren. *The BellKor Solution to the Netflix Grand Prize*. Tech. rep. Yahoo! Research, 2009.

[KR19] M. Kearns and A. Roth. *The Ethical Algorithm: The Science of Socially Aware Algorithm Design*. en. Oxford University Press, 2019.

[KR87] L. Kaufman and P. Rousseeuw. "Clustering by means of Medoids". In: *Statistical Data Analysis Based on the L1-norm and Related Methods*. Ed. by Y. Dodge. North-Holland, 1987, 405–416.

[KR90] L. Kaufman and P. Rousseeuw. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley, 1990.

[Kri+05] B. Krishnapuram, L. Carin, M. Figueiredo, and A. Hartemink. "Learning sparse Bayesian classifiers: multi-class formulation, fast algorithms, and generalization bounds". In: *IEEE Transaction on Pattern Analysis and Machine Intelligence* (2005).

[Kru13] J. K. Kruschke. "Bayesian estimation supersedes the t test". In: *J. Experimental Psychology: General* 142.2 (2013), pp. 573–603.

[Kru15] J. Kruschke. *Doing Bayesian Data Analysis: A Tutorial with R, JAGS and STAN*. Second edition. Academic Press, 2015.

[KS15] H. Kaya and A. A. Salah. "Adaptive Mixtures of Factor Analyzers". In: (2015). arXiv: 1507.02801 [stat.ML].

[KSG04] A. Kraskov, H. Stögbauer, and P. Grassberger. "Estimating mutual information". en. In: *Phys. Rev. E Stat. Nonlin. Soft Matter Phys.* 69.6 Pt 2 (2004), p. 066138.

[KSH12] A. Krizhevsky, I. Sutskever, and G. Hinton. "Imagenet classification with deep convolutional neural networks". In: *NIPS*. 2012.

[KSJ09] I. Konstas, V. Stathopoulos, and J. M. Jose. "On social networks and collaborative recommendation". In: *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 2009, pp. 195–202.

[KST82] D. Kahneman, P. Slovic, and A. Tversky, eds. *Judgment under uncertainty: Heuristics and biases*. Cambridge, 1982.

[KTB11] D. P. Kroese, T. Taimre, and Z. I. Botev. *Handbook of Monte Carlo Methods*. en. 1 edition. Wiley, 2011.

[Kua+09] P. Kuan, G. Pan, J. A. Thomson, R. Stewart, and S. Keles. *A hierarchical semi-Markov model for detecting enrichment with application to ChIP-Seq experiments*. Tech. rep. U. Wisconsin, 2009.

[Kul13] B. Kulis. "Metric Learning: A Survey". In: *Foundations and Trends in Machine Learning* 5.4 (2013), pp. 287–364.

[KV94] M. J. Kearns and U. V. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.

[KVK10] A. Klami, S. Virtanen, and S. Kaski. "Bayesian exponential family projections for coupled data sources". In: *UAI*. 2010.

[KW14] D. P. Kingma and M. Welling. "Auto-encoding variational Bayes". In: *ICLR*. 2014.

[KW16a] T. N. Kipf and M. Welling. "Semi-supervised classification with graph convolutional networks". In: *arXiv preprint arXiv:1609.02907* (2016).

[KW16b] T. N. Kipf and M. Welling. "Variational graph auto-encoders". In: *arXiv preprint arXiv:1611.07308* (2016).

[KW19a] D. P. Kingma and M. Welling. "An Introduction to Variational Autoencoders". In: *Foundations and Trends in Machine Learning* 12.4 (2019), pp. 307–392.

[KW19b] M. J. Kochenderfer and T. A. Wheeler. *Algorithms for Optimization*. en. The MIT Press, 2019.

[KWW22] M. J. Kochenderfer, T. A. Wheeler, and K. Wray. *Algorithms for Decision Making*. The MIT Press, 2022.

[Kyu+10] M. Kyung, J. Gill, M. Ghosh, and G. Casella. "Penalized Regression, Standard Errors and Bayesian Lassos". In: *Bayesian Analysis* 5.2 (2010), pp. 369–412.

[LA16] S. Laine and T. Aila. "Temporal ensembling for semi-supervised learning". In: *arXiv preprint arXiv:1610.02242* (2016).

[Lak+17] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman. "Building Machines That Learn and Think Like People". en. In: *Behav. Brain Sci.* (2017), pp. 1–101.

[Lam18] B. Lambert. *A Student's Guide to Bayesian Statistics*. en. 1st ed. SAGE Publications Ltd, 2018.

[Lam25] N. Lambert. *Reinforcement Learning from Human Feedback*. 2025.

[Law12] N. D. Lawrence. "A Unifying Probabilistic Perspective for Spectral Dimensionality Reduction: Insights and New Models". In: *JMLR* 13.May (2012), pp. 1609–1638.

[LBM06] J. A. Lasserre, C. M. Bishop, and T. P. Minka. "Principled Hybrids of Generative and Discriminative Models". In: *CVPR*. Vol. 1. June 2006, pp. 87–94.

[LBS19] Y. Li, J. Bradshaw, and Y. Sharma. "Are Generative Classifiers More Robust to Adversarial Attacks?" In: *ICML*. Ed. by K. Chaudhuri and R. Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 3804–3814.

[LeC18] Y. LeCun. *Self-supervised learning: could machines learn like humans?* 2018.

[LeC+98] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[Lee13] D.-H. Lee. "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks". In: *ICML Workshop on Challenges in Representation Learning*. 2013.

[Lee+13] J. Lee, S. Kim, G. Lebanon, and Y. Singer. "Local Low-Rank Matrix Approximation". In: *ICML*. Vol. 28. Proceedings of Machine Learning Research. PMLR, 2013, pp. 82–90.

[Lee+19] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh. "Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks". In: *ICML*. 2019.

[Lee77] J. de Leeuw. "Applications of Convex Analysis to Multidimensional Scaling". In: *Recent Developments in Statistics*. Ed. by J. R. Barra, F Brodeau, G Romier, and B Van Cutsem. 1977.

[Lep+21] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding". In: *ICLR*. 2021.

[LG14] O. Levy and Y. Goldberg. "Neural Word Embedding as Implicit Matrix Factorization". In: *NIPS*. 2014.

[LH17] I. Loshchilov and F. Hutter. "SGDR: Stochastic Gradient Descent with Warm Restarts". In: *ICLR*. 2017.

[Li+15] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel. "Gated graph sequence neural networks". In: *arXiv preprint arXiv:1511.05493* (2015).

[Li+17] A. Li, A. Jabri, A. Joulin, and L. van der Maaten. "Learning Visual N-Grams from Web Data". In: *ICCV*. 2017.

[Lia20] S. M. Liao, ed. *Ethics of Artificial Intelligence*. en. 1st ed. Oxford University Press, 2020.

[Lim+19] S. Lim, I. Kim, T. Kim, C. Kim, and S. Kim. "Fast AutoAugment". In: (2019). arXiv: 1905.00397 [cs.LG].

[Lin06] D. Lindley. *Understanding Uncertainty*. Wiley, 2006.

[Lin+21] T. Lin, Y. Wang, X. Liu, and X. Qiu. "A Survey of Transformers". In: (2021). arXiv: 2106.04554 [cs.LG].

[Lin56] D. Lindley. "On a measure of the information provided by an experiment". In: *The Annals of Math. Stat.* (1956), 986–1005.

[Liu01] J. Liu. *Monte Carlo Strategies in Scientific Computation*. Springer, 2001.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

[Liu+16] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, and S. Reed. "SSD: Single Shot MultiBox Detector". In: *ECCV*. 2016.

[Liu+18a] H. Liu, Y.-S. Ong, X. Shen, and J. Cai. "When Gaussian Process Meets Big Data: A Review of Scalable GPs". In: (2018). arXiv: 1807.01065 [stat.ML].

[Liu+18b] L. Liu, X. Liu, C.-J. Hsieh, and D. Tao. "Stochastic Second-order Methods for Non-convex Optimization with Inexact Hessian and Gradient". In: (2018). arXiv: 1809.09853 [math.OC].

[Liu+20] F. Liu, X. Huang, Y. Chen, and J. A. K. Suykens. "Random Features for Kernel Approximation: A Survey on Algorithms, Theory, and Beyond". In: (2020). arXiv: 2004.11154 [stat.ML].

[Liu+22] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. "A ConvNet for the 2020s". In: (2022). arXiv: 2201.03545 [cs.CV].

[LJ09] H. Lukosevicius and H. Jaeger. "Reservoir computing approaches to recurrent neural network training". In: *Computer Science Review* 3.3 (2009), 127–149.

[LKB20] Q. Liu, M. J. Kusner, and P. Blunsom. "A Survey on Contextual Embeddings". In: (2020). arXiv: 2003.07278 [cs.CL].

[LLM14] D. Lay, S. Lay, and J. McConald. *Linear algebra and its applications*. Pearson, 2014.

[Llo82] S Lloyd. "Least squares quantization in PCM". In: *IEEE Trans. Inf. Theory* 28.2 (1982), pp. 129–137.

[LLT89] K. Lange, R. Little, and J. Taylor. "Robust Statistical Modeling Using the T Distribution". In: *JASA* 84.408 (1989), pp. 881–896.

[LM04] E. Learned-Miller. *Hyperspacings and the estimation of information theoretic quantities*. Tech. rep. 04-104. U. Mass. Amherst Comp. Sci. Dept, 2004.

[LM86] R. Larsen and M. Marx. *An introduction to mathematical statistics and its applications*. Prentice Hall, 1986.

[LN81] D. V. Lindley and M. R. Novick. "The Role of Exchangeability in Inference". en. In: *Annals of Statistics* 9.1 (1981), pp. 45–58.

[LNK19] Q. Liu, M. Nickel, and D. Kiela. "Hyperbolic graph neural networks". In: *Advances in Neural Information Processing Systems*. 2019, pp. 8228–8239.

[Loa00] C. F. V. Loan. "The ubiquitous Kronecker product". In: *J. Comput. Appl. Math.* 123.1 (2000), pp. 85–100.

[Lod+02] H. Lodhi, C. Saunders, J. Shawe-Taylor, N. Cristianini, and C. Watkins. "Text classification using string kernels". en. In: *J. Mach. Learn. Res.* (2002).

[LPM15] M.-T. Luong, H. Pham, and C. D. Manning. "Effective Approaches to Attention-based Neural Machine Translation". In: *EMNLP*. 2015.

[LR87] R. J. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. Wiley and Son, 1987.

[LRU14] J. Leskovec, A. Rajaraman, and J. Ullman. *Mining of massive datasets*. Cambridge, 2014.

[LS10] P. Long and R. Servedio. "Random classification noise beats all convex potential boosters". In: *JMLR* 78.3 (2010), pp. 287–304.

[LS19a] S. Lattanzi and C. Sohler. "A Better k-means++ Algorithm via Local Search". In: *ICML*. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 3662–3671.

[LS19b] Z. C. Lipton and J. Steinhardt. "Troubling Trends in Machine Learning Scholarship: Some ML papers suffer from flaws that could mislead the public and stymie future research". In: *The Queue* 17.1 (2019), pp. 45–77.

[LSS13] Q. Le, T. Sarlos, and A. Smola. "Fastfood - Computing Hilbert Space Expansions in log-linear time". In: *ICML*. Vol. 28. Proceedings of Machine Learning Research. PMLR, 2013, pp. 244–252.

[LSY19] H. Liu, K. Simonyan, and Y. Yang. "DARTS: Differentiable Architecture Search". In: *ICLR*. 2019.

[Lu+19] L. Lu, Y. Shin, Y. Su, and G. E. Karniadakis. "Dying ReLU and Initialization: Theory and Numerical Examples". In: (2019). arXiv: 1903.06733 [stat.ML].

[Luo16] M.-T. Luong. "Neural machine translation". PhD thesis. Stanford Dept. Comp. Sci., 2016.

[Luo+19] P. Luo, X. Wang, W. Shao, and Z. Peng. "Towards Understanding Regularization in Batch Normalization". In: *ICLR*. 2019.

[LUW17] C. Louizos, K. Ullrich, and M. Welling. "Bayesian Compression for Deep Learning". In: *NIPS*. 2017.

[Lux07] U. von Luxburg. "A tutorial on spectral clustering". In: *Statistics and Computing* 17.4 (2007), pp. 395–416.

[LW04a] O. Ledoit and M. Wolf. "A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices". In: *J. of Multivariate Analysis* 88.2 (2004), pp. 365–411.

[LW04b] O. Ledoit and M. Wolf. "Honey, I Shrunk the Sample Covariance Matrix". In: *J. of Portfolio Management* 31.1 (2004).

[LW04c] H. Lopes and M. West. "Bayesian model assessment in factor analysis". In: *Statisica Sinica* 14 (2004), pp. 41–67.

[LW16] C. Li and M. Wand. "Precomputed Real-Time Texture Synthesis with Markovian Generative Adversarial Networks". In: *ECCV*. 2016.

[LWG12] U. von Luxburg, R. Williamson, and I. Guyon. "Clustering: science or art?" In: *Workshop on Unsupervised and Transfer Learning*. 2012.

[LXW19] X. Liu, Q. Xu, and N. Wang. "A survey on deep neural network-based image captioning". In: *The Visual Computer* 35.3 (2019), pp. 445–470.

[Lyu+20] X.-K. Lyu, Y. Xu, X.-F. Zhao, X.-N. Zuo, and C.-P. Hu. "Beyond psychology: prevalence of p value and confidence interval misinterpretation across different fields". In: *Journal of Pacific Rim Psychology* 14 (2020).

[MA10] I. Murray and R. P. Adams. "Slice sampling covariance hyperparameters of latent Gaussian models". In: *NIPS*. 2010, pp. 1732–1740.

[MA+17] Y. Movshovitz-Attias, A. Toshev, T. K. Leung, S. Ioffe, and S. Singh. "No Fuss Distance Metric Learning using Proxies". In: *ICCV*. 2017.

[Maa+11] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts. "Learning Word Vectors for Sentiment Analysis". In: *Proc. ACL*. 2011, pp. 142–150.

[Maa14] L. van der Maaten. "Accelerating t-SNE using Tree-Based Algorithms". In: *JMLR* (2014).

[Mac03] D. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[Mac09] L. W. Mackey. "Deflation Methods for Sparse PCA". In: *NIPS*. 2009.

[Mac67] J MacQueen. "Some methods for classification and analysis of multivariate observations". en. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*. The Regents of the University of California, 1967.

[Mac95] D. MacKay. "Probable networks and plausible predictions — a review of practical Bayesian methods for supervised neural networks". In: *Network: Computation in Neural Systems* 6.3 (1995), pp. 469–505.

[Mad+20] A. Madani, B. McCann, N. Naik, N. S. Keskar, N. Anand, R. R. Eguchi, P.-S. Huang, and R. Socher. "ProGen: Language Modeling for Protein Generation". en. 2020.

[Mah07] R. P. S. Mahler. *Statistical Multisource-Multitarget Information Fusion*. Artech House, Inc., 2007.

[Mah13] R Mahler. "Statistics 102 for Multisource-Multitarget Detection and Tracking". In: *IEEE J. Sel. Top. Signal Process.* 7.3 (2013), pp. 376–389.

[Mah+18] D. Mahajan, R. Girshick, V. Ramanathan, K. He, M. Paluri, Y. Li, A. Bharambe, and L. van der Maaten. "Exploring the Limits of Weakly Supervised Pretraining". In: (2018). arXiv: 1805.00932 [cs.CV].

[Mai15] J Mairal. "Incremental Majorization-Minimization Optimization with Application to Large-Scale Machine Learning". In: *SIAM J. Optim.* 25.2 (2015), pp. 829–855.

[Mak+19] D. Makowski, M. S. Ben-Shachar, S. H. A. Chen, and D. Lüdecke. "Indices of Effect Existence and Significance in the Bayesian Framework". en. In: *Front. Psychol.* 10 (2019), p. 2767.

[Mal99] S. Mallat. *A Wavelet Tour of Signal Processing*. Academic Press, 1999.

[Man+16] V. Mansinghka, P. Shafto, E. Jonas, C. Petschulat, M. Gasner, and J. Tenenbaum. "Crosscat: A Fully Bayesian, Nonparametric Method For Analyzing Heterogeneous, High-dimensional Data." In: *JMLR* 17 (2016).

[Mar06] H. Markram. "The blue brain project". en. In: *Nat. Rev. Neurosci.* 7.2 (2006), pp. 153–160.

[Mar08] B. Marlin. "Missing Data Problems in Machine Learning". PhD thesis. U. Toronto, 2008.

[Mar+11] B. M. Marlin, R. S. Zemel, S. T. Roweis, and M. Slaney. "Recommender Systems, Missing Data and Statistical Model Estimation". In: *IJCAI*. 2011.

[Mar18] O. Martin. *Bayesian analysis with Python*. Packt, 2018.

[Mar72] G. Marsaglia. "Choosing a Point from the Surface of a Sphere". en. In: *Ann. Math. Stat.* 43.2 (1972), pp. 645–646.

[Mas+00] L. Mason, J. Baxter, P. L. Bartlett, and M. R. Frean. "Boosting Algorithms as Gradient Descent". In: *NIPS*. 2000, pp. 512–518.

[Mas+15] J. Masci, D. Boscaini, M. Bronstein, and P. Vandergheynst. "Geodesic convolutional neural networks on riemannian manifolds". In: *Proceedings of the IEEE international conference on computer vision workshops*. 2015, pp. 37–45.

[Mat00] R. Matthews. "Storks Deliver Babies (p = 0.008)". In: *Teach. Stat.* 22.2 (2000), pp. 36–38.

[Mat98] R. Matthews. *Bayesian Critique of Statistics in Health: The Great Health Hoax*. 1998.

[MAV17] D. Molchanov, A. Ashukha, and D. Vetrov. "Variational Dropout Sparsifies Deep Neural Networks". In: *ICML*. 2017.

[MB05] F. Morin and Y. Bengio. "Hierarchical Probabilistic Neural Network Language Model". In: *AISTATS*. 2005.

[MB06] N. Meinshausen and P. Buhlmann. "High dimensional graphs and variable selection with the lasso". In: *The Annals of Statistics* 34 (2006), pp. 1436–1462.

[MBL20] K. Musgrave, S. Belongie, and S.-N. Lim. "A Metric Learning Reality Check". In: *ECCV*. 2020.

[McE20] R. McElreath. *Statistical Rethinking: A Bayesian Course with Examples in R and Stan (2nd edition)*. en. Chapman and Hall/CRC, 2020.

[McL75] G. J. McLachlan. "Iterative reclassification procedure for constructing an asymptotically optimal rule of allocation in discriminant analysis". In: *Journal of the American Statistical Association* 70.350 (1975), pp. 365–369.

[MD97] X. L. Meng and D. van Dyk. "The EM algorithm — an old folk song sung to a fast new tune (with Discussion)". In: *J. Royal Stat. Soc. B* 59 (1997), pp. 511–567.

[ME14] S. Masoudnia and R. Ebrahimpour. "Mixture of experts: a literature survey". In: *Artificial Intelligence Review* 42.2 (2014), pp. 275–293.

[Mei01] M. Meila. "A random walks view of spectral segmentation". In: *AISTATS*. 2001.

[Mei05] M. Meila. "Comparing clusterings: an axiomatic view". In: *ICML*. 2005.

[Men+12] T. Mensink, J. Verbeek, F. Perronnin, and G. Csurka. "Metric Learning for Large Scale Image Classification: Generalizing to

New Classes at Near-Zero Cost". In: *ECCV*. Springer Berlin Heidelberg, 2012, pp. 488–501.

[Men+21] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar. "Long-tail learning via logit adjustment". In: *ICLR*. 2021.

[Men+22] K. Meng, D. Bau, A. Andonian, and Y. Belinkov. "Locating and Editing Factual Associations in GPT". In: (Feb. 2022). arXiv: 2202.05262 [cs.CL].

[Met21] C. Metz. *Genius Makers: The Mavericks Who Brought AI to Google, Facebook, and the World*. en. Dutton, 2021.

[MF17] J. Matejka and G. Fitzmaurice. "Same Stats, Different Graphs: Generating Datasets with Varied Appearance and Identical Statistics through Simulated Annealing". In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2017, pp. 1290–1294.

[MFR20] G. M. Martin, D. T. Frazier, and C. P. Robert. "Computing Bayes: Bayesian Computation from 1763 to the 21st Century". In: (2020). arXiv: 2004.06425 [stat.CO].

[MG05] I. Murray and Z. Ghahramani. *A note on the evidence and Bayesian Occam's razor*. Tech. rep. Gatsby, 2005.

[MH07] A. Mnih and G. Hinton. "Three new graphical models for statistical language modelling". en. In: *ICML*. 2007.

[MH08] L. v. d. Maaten and G. Hinton. "Visualizing Data using t-SNE". In: *JMLR* 9.Nov (2008), pp. 2579–2605.

[MHM18] L. McInnes, J. Healy, and J. Melville. "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction". In: (2018). arXiv: 1802.03426 [stat.ML].

[MHN13] A. L. Maas, A. Y. Hannun, and A. Y. Ng. "Rectifier Nonlinearities Improve Neural Network Acoustic Models". In: *ICML*. Vol. 28. 2013.

[Mik+13a] T. Mikolov, K. Chen, G. Corrado, and J. Dean. "Efficient Estimation of Word Representations in Vector Space". In: *ICLR*. 2013.

[Mik+13b] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean. "Distributed Representations of Words and Phrases and their Compositionality". In: *NIPS*. 2013.

[Mik+13c] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. "Distributed representations of words and phrases and their compositionality". In: *NIPS*. 2013, pp. 3111–3119.

[Min00] T. Minka. *Bayesian model averaging is not model combination*. Tech. rep. MIT Media Lab, 2000.

[Min+09] M. Mintz, S. Bills, R. Snow, and D. Jurafksy. "Distant supervision for relation extraction without labeled data". In: *Prof. Conf. Recent Advances in NLP*. 2009.

[Mit97] T. Mitchell. *Machine Learning*. McGraw Hill, 1997.

[Miy+18] T. Miyato, S.-I. Maeda, M. Koyama, and S. Ishii. "Virtual Adversarial Training: A Regularization Method for Supervised and Semi-Supervised Learning". In: *IEEE PAMI* (2018).

[MK97] G. J. McLachlan and T. Krishnan. *The EM Algorithm and Extensions*. Wiley, 1997.

[MKH19] R. Müller, S. Kornblith, and G. E. Hinton. "When does label smoothing help?" In: *NIPS*. 2019, pp. 4694–4703.

[MKL11] O. Martin, R. Kumar, and J. Lao. *Bayesian Modeling and Computation in Python*. CRC Press, 2011.

[MKL21] O. A. Martin, R. Kumar, and J. Lao. *Bayesian Modeling and Computation in Python*. CRC Press, 2021.

[MKS21] K. Murphy, A. Kumar, and S. Serghiou. "Risk score learning for COVID-19 contact tracing apps". In: *Machine Learning for Healthcare*. 2021.

[MM16] D. Mishkin and J. Matas. "All you need is a good init". In: *ICLR*. 2016.

[MN89] P. McCullagh and J. Nelder. *Generalized linear models*. 2nd edition. Chapman and Hall, 1989.

[MNM02] W. Maass, T. Natschlaeger, and H. Markram. "Real-time computing without stable states: A new framework for neural computation based on perturbations". In: *Neural Computation* 14.11 (2002), 2531—2560.

[MO04] S. C. Madeira and A. L. Oliveira. "Biclustering Algorithms for Biological Data Analysis: A Survey". In: *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 1.1 (2004), pp. 24–45.

[Mol04] C. Moler. *Numerical Computing with MATLAB*. SIAM, 2004.

[Mon+14] G. F. Montufar, R. Pascanu, K. Cho, and Y. Bengio. "On the Number of Linear Regions of Deep Neural Networks". In: *NIPS*. 2014.

[Mon+17] F. Monti, D. Boscaini, J. Masci, E. Rodola, J. Svoboda, and M. M. Bronstein. "Geometric deep learning on graphs and manifolds using mixture model cnns". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 5115–5124.

[Mon+19] N. Monath, A. Kobren, A. Krishnamurthy, M. R. Glass, and A. McCallum. "Scalable Hierarchical Clustering with Tree Grafting". In: *KDD*. KDD '19. Association for Computing Machinery, 2019, pp. 1438–1448.

[Mon+21] N. Monath et al. "Scalable Bottom-Up Hierarchical Clustering". In: *KDD*. 2021.

[Mor+16] R. D. Morey, R. Hoekstra, J. N. Rouder, M. D. Lee, and E.-J. Wagenmakers. "The fallacy of placing confidence in confidence intervals". en. In: *Psychon. Bull. Rev.* 23.1 (2016), pp. 103–123.

[MOT15] A. Mordvintsev, C. Olah, and M. Tyka. *Inceptionism: Going Deeper into Neural Networks*. https://ai.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html. Accessed: NA-NA-NA. 2015.

[MP43] W. McCulloch and W. Pitts. "A logical calculus of the ideas immanent in nervous activity".

In: *Bulletin of Mathematical Biophysics* 5 (1943), pp. 115–137.

[MP69] M. Minsky and S. Papert. *Perceptrons*. MIT Press, 1969.

[MRS08] C. Manning, P. Raghavan, and H. Schuetze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[MS11] D. Mayo and A. Spanos. "Error Statistics". In: *Handbook of Philosophy of Science*. Ed. by P. S. Bandyopadhyay and M. R. Forster. 2011.

[Muk+19] B. Mukhoty, G. Gopakumar, P. Jain, and P. Kar. "Globally-convergent Iteratively Reweighted Least Squares for Robust Regression Problems". In: *AISTATS*. 2019, pp. 313–322.

[Mur23] K. P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023.

[MV15] A Mahendran and A Vedaldi. "Understanding deep image representations by inverting them". In: *CVPR*. 2015, pp. 5188–5196.

[MV16] A. Mahendran and A. Vedaldi. "Visualizing Deep Convolutional Neural Networks Using Natural Pre-images". In: *Intl. J. Computer Vision* (2016), pp. 1–23.

[MWK16] A. H. Marblestone, G. Wayne, and K. P. Kording. "Toward an Integration of Deep Learning and Neuroscience". en. In: *Front. Comput. Neurosci.* 10 (2016), p. 94.

[MWP98] B Moghaddam, W Wahid, and A Pentland. "Beyond eigenfaces: probabilistic matching for face recognition". In: *Proceedings Third IEEE International Conference on Automatic Face and Gesture Recognition*. 1998, pp. 30–35.

[Nad+19] S. Naderi, K. He, R. Aghajani, S. Sclaroff, and P. Felzenszwalb. "Generalized Majorization-Minimization". In: *ICML*. 2019.

[Nak+19] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, and I. Sutskever. "Deep Double Descent: Where Bigger Models and More Data Hurt". In: (2019). arXiv: 1912.02292 [cs.LG].

[NAM21] C. G. Northcutt, A. Athalye, and J. Mueller. "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks". In: *NeurIPS Track on Datasets and Benchmarks*. Mar. 2021.

[Nar+23] H. Narasimhan, A. K. Menon, W. Jitkrittum, and S. Kumar. "Plugin estimators for selective classification with out-of-distribution detection". In: *arXiv [cs.LG]* (Jan. 2023).

[Nea96] R. Neal. *Bayesian learning for neural networks*. Springer, 1996.

[Nes04] Y. Nesterov. *Introductory Lectures on Convex Optimization. A basic course*. Kluwer, 2004.

[Neu04] A. Neumaier. "Complete search in continuous global optimization and constraint satisfaction". In: *Acta Numer.* 13 (2004), pp. 271–369.

[Neu17] G. Neubig. "Neural Machine Translation and Sequence-to-sequence Models: A Tutorial". In: (2017). arXiv: 1703.01619 [cs.CL].

[Ngu+17] A. Nguyen, J. Yosinski, Y. Bengio, A. Dosovitskiy, and J. Clune. "Plug & Play Generative Networks: Conditional Iterative Generation of Images in Latent Space". In: *CVPR*. 2017.

[NH98] R. M. Neal and G. E. Hinton. "A View of the EM Algorithm that Justifies Incremental, Sparse, and other Variants". In: *Learning in Graphical Models*. Ed. by M. I. Jordan. Springer Netherlands, 1998, pp. 355–368.

[NHLS19] E. Nalisnick, J. M. Hernández-Lobato, and P. Smyth. "Dropout as a Structured Shrinkage Prior". In: *ICML*. 2019.

[Nic+15] M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich. "A Review of Relational Machine Learning for Knowledge Graphs". In: *Proc. IEEE* (2015).

[Niu+11] F. Niu, B. Recht, C. Re, and S. J. Wright. "HOGWILD!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent". In: *NIPS*. 2011.

[NJ02] A. Y. Ng and M. I. Jordan. "On Discriminative vs. Generative Classifiers: A comparison of logistic regression and Naive Bayes". In: *NIPS-14*. 2002.

[NJW01] A. Ng, M. Jordan, and Y. Weiss. "On Spectral Clustering: Analysis and an algorithm". In: *NIPS*. 2001.

[NK17] M. Nickel and D. Kiela. "Poincaré embeddings for learning hierarchical representations". In: *Advances in neural information processing systems*. 2017, pp. 6338–6347.

[NK18] M. Nickel and D. Kiela. "Learning Continuous Hierarchies in the Lorentz Model of Hyperbolic Geometry". In: *International Conference on Machine Learning*. 2018, pp. 3779–3788.

[NK24] A. Narayanan and S. Kapoor. *AI Snake Oil: What Artificial Intelligence Can Do, What It Can't, and How to Tell the Difference*. en. 2024.

[NMC05] A. Niculescu-Mizil and R. Caruana. "Predicting Good Probabilities with Supervised Learning". In: *ICML*. 2005.

[Nou+02] M. N. Nounou, B. R. Bakshi, P. K. Goel, and X. Shen. "Process modeling by Bayesian latent variable regression". In: *Am. Inst. Chemical Engineers Journal* 48.8 (2002), pp. 1775–1793.

[Nov62] A. Novikoff. "On convergence proofs on perceptrons". In: *Symp. on the Mathematical Theory of Automata* 12 (1962), pp. 615–622.

[NR18] G. Neu and L. Rosasco. "Iterate Averaging as Regularization for Stochastic Gradient Descent". In: *COLT*. 2018.

[NTL20] J. Nixon, D. Tran, and B. Lakshminarayanan. "Why aren't bootstrapped neural networks better?" In: *NIPS Workshop on "I can't believe it's not better"*. 2020.

[NW06] J. Nocedal and S. Wright. *Numerical Optimization*. Springer, 2006.

[Ode16] A. Odena. "Semi-supervised learning with generative adversarial networks". In: *arXiv preprint arXiv:1606.01583* (2016).

[OLV18] A. van den Oord, Y. Li, and O. Vinyals. "Representation Learning with Contrastive Predictive Coding". In: (2018). arXiv: 1807.03748 [cs.LG].

[OMS17] C. Olah, A. Mordvintsev, and L. Schubert. "Feature Visualization". In: *Distill* (2017).

[Oor+16] A. Van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu. "WaveNet: A Generative Model for Raw Audio". In: (2016). arXiv: 1609.03499 [cs.SD].

[Oor+18] A. van den Oord et al. "Parallel WaveNet: Fast High-Fidelity Speech Synthesis". In: *ICML*. Ed. by J. Dy and A. Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 3918–3926.

[Ope] OpenAI. *ChatGPT: Optimizing Language Models for Dialogue*. Blog.

[Ope23] OpenAI. *GPT4*. Tech. rep. 2023.

[OPK12] G. Ohloff, W. Pickenhagen, and P. Kraft. *Scent and Chemistry*. en. Wiley, 2012.

[OPT00a] M. R. Osborne, B. Presnell, and B. A. Turlach. "A new approach to variable selection in least squares problems". In: *IMA Journal of Numerical Analysis* 20.3 (2000), pp. 389–403.

[OPT00b] M. R. Osborne, B. Presnell, and B. A. Turlach. "On the lasso and its dual". In: *J. Computational and graphical statistics* 9 (2000), pp. 319–337.

[Ort+19] P. A. Ortega et al. "Meta-learning of Sequential Strategies". In: (2019). arXiv: 1905.03030 [cs.LG].

[Osb16] I. Osband. "Risk versus Uncertainty in Deep Learning: Bayes, Bootstrap and the Dangers of Dropout". In: *NIPS workshop on Bayesian deep learning*. 2016.

[OTJ07] G. Obozinski, B. Taskar, and M. I. Jordan. *Joint covariate selection for grouped classification*. Tech. rep. UC Berkeley, 2007.

[Ouy+22] L. Ouyang et al. "Training language models to follow instructions with human feedback". In: (Mar. 2022). arXiv: 2203.02155 [cs.CL].

[Pai05] A. Pais. *Subtle Is the Lord: The Science and the Life of Albert Einstein*. en. Oxford University Press, 2005.

[Pan+15] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur. "Librispeech: an asr corpus based on public domain audio books". In: *ICASSP*. IEEE. 2015, pp. 5206–5210.

[Pap+18] G. Papandreou, T. Zhu, L.-C. Chen, S. Gidaris, J. Tompson, and K. Murphy. "PersonLab: Person Pose Estimation and Instance Segmentation with a Bottom-Up, Part-Based, Geometric Embedding Model". In: *ECCV*. 2018, pp. 269–286.

[Par+16a] A. Parikh, O. Täckström, D. Das, and J. Uszkoreit. "A Decomposable Attention Model for Natural Language Inference". In: *EMNLP*. Association for Computational Linguistics, 2016, pp. 2249–2255.

[Par+16b] A. Parikh, O. Täckström, D. Das, and J. Uszkoreit. "A Decomposable Attention Model for Natural Language Inference". In: *EMNLP*. Association for Computational Linguistics, 2016, pp. 2249–2255.

[Par+18] N. Parmar, A. Vaswani, J. Uszkoreit, Ł. Kaiser, N. Shazeer, A. Ku, and D. Tran. "Image Transformer". In: *ICLR*. 2018.

[PARS14] B. Perozzi, R. Al-Rfou, and S. Skiena. "Deepwalk: Online learning of social representations". In: *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM. 2014, pp. 701–710.

[Pas14] R. Pascanu. "On Recurrent and Deep Neural Networks". PhD thesis. U. Montreal, 2014.

[Pat12] A. Paterek. *Predicting movie ratings and recommender systems*. 2012.

[Pat+16] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. "Context Encoders: Feature Learning by Inpainting". In: *CVPR*. 2016.

[Pau+20] A. Paullada, I. D. Raji, E. M. Bender, E. Denton, and A. Hanna. "Data and its (dis)contents: A survey of dataset development and use in machine learning research". In: *NeurIPS 2020 Workshop: ML Retrospectives, Surveys & Meta-analyses (ML-RSA)*. 2020.

[PB+14] N. Parikh, S. Boyd, et al. "Proximal algorithms". In: *Foundations and Trends in Optimization* 1.3 (2014), pp. 127–239.

[Pea18] J. Pearl. *Theoretical Impediments to Machine Learning With Seven Sparks from the Causal Revolution*. Tech. rep. UCLA, 2018.

[Pen+20] Z. Peng, W. Huang, M. Luo, Q. Zheng, Y. Rong, T. Xu, and J. Huang. "Graph Representation Learning via Graphical Mutual Information Maximization". In: *Proceedings of The Web Conference*. 2020.

[Per+17] B. Perozzi, V. Kulkarni, H. Chen, and S. Skiena. "Don't Walk, Skip! Online Learning of Multi-Scale Network Embeddings". In: *Proceedings of the 2017 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining 2017*. ASONAM '17. Association for Computing Machinery, 2017, 258–265.

[Pet13] J. Peters. *When Ice Cream Sales Rise, So Do Homicides. Coincidence, or Will Your Next Cone Murder You?* https://slate.com/news-and-politics/2013/07/warm-weather-homicide-rates-when-ice-cream-sales-rise-homicides-rise-coincidence.html. Accessed: 2020-5-20. 2013.

[Pet+18] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer. "Deep contextualized word representations". In: *NAACL*. 2018.

[Pey20] G. Peyre. "Course notes on Optimization for Machine Learning". 2020.

[PH18] T. Parr and J. Howard. "The Matrix Calculus You Need For Deep Learning". In: (2018). arXiv: 1802.01528 [cs.LG].

[Pin88] F. J. Pineda. "Generalization of back propagation to recurrent and higher order neural

networks". In: *Neural information processing systems*. 1988, pp. 602–611.

[Piz01] Z Pizlo. "Perception viewed as an inverse problem". en. In: *Vision Res.* 41.24 (2001), pp. 3145–3161.

[PJ09] H.-S. Park and C.-H. Jun. "A simple and fast algorithm for K-medoids clustering". In: *Expert Systems with Applciations* 36.2, Part 2 (2009), pp. 3336–3341.

[PJ92] B Polyak and A Juditsky. "Acceleration of Stochastic Approximation by Averaging". In: *SIAM J. Control Optim.* 30.4 (1992), pp. 838–855.

[Pla00] J. Platt. "Probabilities for SV machines". In: *Advances in Large Margin Classifiers*. Ed. by A. Smola, P. Bartlett, B. Schoelkopf, and D. Schuurmans. MIT Press, 2000.

[Pla98] J. Platt. "Using analytic QP and sparseness to speed training of support vector machines". In: *NIPS*. 1998.

[PM17] D. L. Poole and A. K. Mackworth. *Artificial intelligenceL foundations computational agents 2nd edition*. Cambridge University Press, 2017.

[PM18] J. Pearl and D. Mackenzie. *The book of why: the new science of cause and effect*. 2018.

[PMB19] J. Pérez, J. Marinkovic, and P. Barcelo. "On the Turing Completeness of Modern Neural Network Architectures". In: *ICLR*. 2019.

[Pog+17] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao. "Why and when can deep-but not shallow-networks avoid the curse of dimensionality: A review". en. In: *Int. J. Autom. Comput.* (2017), pp. 1–17.

[PP+20] M. Papadatou-Pastou, E. Ntolka, J. Schmitz, M. Martin, M. R. Munafò, S. Ocklenburg, and S. Paracchini. "Human handedness: A meta-analysis". en. In: *Psychol. Bull.* 146.6 (2020), pp. 481–524.

[PPS18] T. Pierrot, N. Perrin, and O. Sigaud. "First-order and second-order variants of the gradient descent in a unified framework". In: (2018). arXiv: 1810.08102 [cs.LG].

[Pre21] K. Pretz. "Stop Calling Everything AI, Machine-Learning Pioneer Says". In: *IEEE Spectrum* (2021).

[Pri+23] I. Price, A. Sanchez-Gonzalez, F. Alet, T. Ewalds, A. El-Kadi, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. "GenCast: Diffusion-based ensemble forecasting for medium-range weather". In: (Dec. 2023). arXiv: 2312.15796 [cs.LG].

[PSM14a] J. Pennington, R. Socher, and C. Manning. "GloVe: Global vectors for word representation". In: *EMNLP*. 2014, pp. 1532–1543.

[PSM14b] J. Pennington, R. Socher, and C. Manning. "Glove: Global vectors for word representation". In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 2014, pp. 1532–1543.

[PSW15] N. G. Polson, J. G. Scott, and B. T. Willard. "Proximal Algorithms in Statistics and Machine Learning". en. In: *Stat. Sci.* 30.4 (2015), pp. 559–581.

[QC+06] J. Quiñonero-Candela, C. E. Rasmussen, F. Sinz, O. Bousquet, and B. Schölkopf. "Evaluating Predictive Uncertainty Challenge". In: *Machine Learning Challenges. Evaluating Predictive Uncertainty, Visual Object Classification, and Recognising Tectual Entailment*. Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2006, pp. 1–27.

[Qia+19] Q. Qian, L. Shang, B. Sun, J. Hu, H. Li, and R. Jin. "SoftTriple Loss: Deep Metric Learning Without Triplet Sampling". In: *ICCV*. 2019.

[Qiu+18] J. Qiu, Y. Dong, H. Ma, J. Li, K. Wang, and J. Tang. "Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec". In: *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*. 2018, pp. 459–467.

[Qiu+19a] J. Qiu, Y. Dong, H. Ma, J. Li, C. Wang, K. Wang, and J. Tang. "NetSMF: Large-Scale Network Embedding as Sparse Matrix Factorization". In: *The World Wide Web Conference*. WWW '19. Association for Computing Machinery, 2019, 1509–1520.

[Qiu+19b] J. Qiu, H. Ma, O. Levy, S. W. Yih, S. Wang, and J. Tang. "Blockwise Self-Attention for Long Document Understanding". In: *CoRR* abs/1911.02972 (2019). arXiv: 1911.02972.

[Qui86] J. R. Quinlan. "Induction of decision trees". In: *Machine Learning* 1 (1986), pp. 81–106.

[Qui93] J. R. Quinlan. *C4.5 Programs for Machine Learning*. Morgan Kauffman, 1993.

[Rad+] A. Radford et al. *Learning transferable visual models from natural language supervision*. Tech. rep. OpenAI.

[Rad+18] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. *Improving Language Understanding by Generative Pre-Training*. Tech. rep. OpenAI, 2018.

[Rad+19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[Raf+20] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *JMLR* (2020).

[Raf22] E. Raff. *Inside Deep Learning: Math, Algorithms, Models*. en. Annotated edition. Manning, May 2022.

[Rag+17] M. Raghu, B. Poole, J. Kleinberg, S. Ganguli, and J. Sohl-Dickstein. "On the Expressive Power of Deep Neural Networks". In: *ICML*. 2017.

[Rag+19] M. Raghu, C. Zhang, J. Kleinberg, and S. Bengio. "Transfusion: Understanding transfer learning for medical imaging". In: *NIPS*. 2019, pp. 3347–3357.

[Rag+21] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. "Do Vision Transformers See Like Convolutional Neural Networks?" In: *NIPS*. 2021.

[Raj+16] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. "SQuAD: 100,000+ Questions for Ma-

chine Comprehension of Text". In: *EMNLP*. 2016.

[Raj+18] A. Rajkomar et al. "Scalable and accurate deep learning with electronic health records". en. In: *NPJ Digit Med* 1 (2018), p. 18.

[Rat+09] M. Rattray, O. Stegle, K. Sharp, and J. Winn. "Inference algorithms and learning theory for Bayesian sparse factor analysis". In: *Proc. Intl. Workshop on Statistical-Mechanical Informatics*. 2009.

[RB93] M. Riedmiller and H. Braun. "A direct adaptive method for faster backpropagation learning: The RPROP algorithm". In: *ICNN*. IEEE. 1993, pp. 586–591.

[RBV17] S.-A. Rebuffi, H. Bilen, and A. Vedaldi. "Learning multiple visual domains with residual adapters". In: *NIPS*. 2017.

[RBV18] S.-A. Rebuffi, H. Bilen, and A. Vedaldi. "Efficient parametrization of multi-domain deep neural networks". In: *CVPR*. 2018.

[RC04] C. Robert and G. Casella. *Monte Carlo Statisical Methods*. 2nd edition. Springer, 2004.

[Rec+19] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar. "Do Image Net Classifiers Generalize to Image Net?" In: *ICML*. 2019.

[Red+16] J Redmon, S Divvala, R Girshick, and A Farhadi. "You Only Look Once: Unified, Real-Time Object Detection". In: *CVPR*. 2016, pp. 779–788.

[Ren+09] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme. "BPR: Bayesian Personalized Ranking from Implicit Feedback". In: *UAI*. 2009.

[Ren12] S. Rendle. "Factorization Machines with libFM". In: *ACM Trans. Intell. Syst. Technol.* 3.3 (2012), pp. 1–22.

[Ren19] Z. Ren. *List of papers on self-supervised learning*. 2019.

[Res+11] D. Reshef, Y. Reshef, H. Finucane, S. Grossman, G. McVean, P. Turnbaugh, E. Lander, M. Mitzenmacher, and P. Sabeti. "Detecting Novel Associations in Large Data Sets". In: *Science* 334 (2011), pp. 1518–1524.

[Res+16] Y. A. Reshef, D. N. Reshef, H. K. Finucane, P. C. Sabeti, and M. Mitzenmacher. "Measuring Dependence Powerfully and Equitably". In: *J. Mach. Learn. Res.* 17.211 (2016), pp. 1–63.

[RF17] J. Redmon and A. Farhadi. "YOLO9000: Better, Faster, Stronger". In: *CVPR*. 2017.

[RFB15] O. Ronneberger, P. Fischer, and T. Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *MICCAI (Intl. Conf. on Medical Image Computing and Computer Assisted Interventions)*. 2015.

[RG11] A. Rodriguez and K. Ghosh. *Modeling relational data through nested partition models*. Tech. rep. UC Santa Cruz, 2011.

[RHS05] C. Rosenberg, M. Hebert, and H. Schneiderman. "Semi-Supervised Self-Training of Object Detection Models". In: *Proceedings of the Seventh IEEE Workshops on Application of Computer Vision (WACV/MOTION'05)-Volume 1-Volume 01*. 2005, pp. 29–36.

[RHW86] D. Rumelhart, G. Hinton, and R. Williams. "Learning internal representations by error propagation". In: *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*. Ed. by D. Rumelhart, J. McClelland, and the PDD Research Group. MIT Press, 1986.

[Ric95] J. Rice. *Mathematical statistics and data analysis*. 2nd edition. Duxbury, 1995.

[Rif+11] S. Rifai, P. Vincent, X. Muller, X. Glorot, and Y. Bengio. "Contractive Auto-Encoders: Explicit Invariance During Feature Extraction". In: *ICML*. 2011.

[Ris+08] I. Rish, G. Grabarnik, G. Cecchi, F. Pereira, and G. Gordon. "Closed-form supervised dimensionality reduction with generalized linear models". In: *ICML*. 2008.

[RKK18] S. J. Reddi, S. Kale, and S. Kumar. "On the Convergence of Adam and Beyond". In: *ICLR*. 2018.

[RM01] N. Roy and A. McCallum. "Toward optimal active learning through Monte Carlo estimation of error reduction". In: *ICML*. 2001.

[RMC09] H. Rue, S. Martino, and N. Chopin. "Approximate Bayesian Inference for Latent Gaussian Models Using Integrated Nested Laplace Approximations". In: *J. of Royal Stat. Soc. Series B* 71 (2009), pp. 319–392.

[RML22] S. Ramasinghe, L. Macdonald, and S. Lucey. "On the frequency-bias of coordinate-MLPs". In: *NIPS*. 2022.

[RMW14] D. J. Rezende, S. Mohamed, and D. Wierstra. "Stochastic Backpropagation and Approximate Inference in Deep Generative Models". In: *ICML*. Ed. by E. P. Xing and T. Jebara. Vol. 32. Proceedings of Machine Learning Research. PMLR, 2014, pp. 1278–1286.

[RN10] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 3rd edition. Prentice Hall, 2010.

[Roo+21] F. de Roos, C. Jidling, A. Wills, T. Schön, and P. Hennig. "A Probabilistically Motivated Learning Rate Adaptation for Stochastic Optimization". In: (2021). arXiv: 2102.10880 [cs.LG].

[Ros58] F. Rosenblatt. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6 (1958), pp. 386–408.

[Ros98] K. Rose. "Deterministic Annealing for Clustering, Compression, Classification, Regression, and Related Optimization Problems". In: *Proc. IEEE* 80 (1998), pp. 2210–2239.

[Rot+18] W. Roth, R. Peharz, S. Tschiatschek, and F. Pernkopf. "Hybrid generative-discriminative training of Gaussian mixture models". In: *Pattern Recognit. Lett.* 112 (Sept. 2018), pp. 131–137.

[Rot+20] K. Roth, T. Milbich, S. Sinha, P. Gupta, B. Ommer, and J. P. Cohen. "Revisiting Training Strategies and Generalization Performance in Deep Metric Learning". In: *ICML*. 2020.

[Rou+09] J. Rouder, P. Speckman, D. Sun, and R. Morey. "Bayesian t tests for accepting and rejecting the null hypothesis". In: *Psychonomic Bulletin & Review* 16.2 (2009), pp. 225–237.

[Row97] S. Roweis. "EM algorithms for PCA and SPCA". In: *NIPS*. 1997.

[Roy+20] A. Roy, M. Saffar, A. Vaswani, and D. Grangier. "Efficient Content-Based Sparse Attention with Routing Transformers". In: *CoRR* abs/2003.05997 (2020). arXiv: 2003.05997.

[Roz+19] B. Rozemberczki, R. Davies, R. Sarkar, and C. Sutton. "GEMSEC: Graph Embedding with Self Clustering". In: *Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. ASONAM '19. Association for Computing Machinery, 2019, 65–72.

[RP99] M. Riesenhuber and T. Poggio. "Hierarchical Models of Object Recognition in Cortex". In: *Nature Neuroscience* 2 (1999), pp. 1019–1025.

[RR08] A. Rahimi and B. Recht. "Random Features for Large-Scale Kernel Machines". In: *NIPS*. Curran Associates, Inc., 2008, pp. 1177–1184.

[RR09] A. Rahimi and B. Recht. "Weighted Sums of Random Kitchen Sinks: Replacing minimization with randomization in learning". In: *NIPS*. Curran Associates, Inc., 2009, pp. 1313–1320.

[RS00] S. T. Roweis and L. K. Saul. "Nonlinear dimensionality reduction by locally linear embedding". en. In: *Science* 290.5500 (2000), pp. 2323–2326.

[RT82] D. B. Rubin and D. T. Thayer. "EM algorithms for ML factor analysis". In: *Psychometrika* 47.1 (1982), pp. 69–76.

[RTA18] H. G. Ramaswamy, A. Tewari, and S. Agarwal. "Consistent algorithms for multiclass classification with an abstain option". en. In: *Electron. J. Stat.* 12.1 (Jan. 2018), pp. 530–554.

[Rub84] D. B. Rubin. "Bayesianly Justifiable and Relevant Frequency Calculations for the Applied Statistician". In: *Ann. Stat.* 12.4 (1984), pp. 1151–1172.

[Rup88] D Ruppert. *Efficient Estimations from a Slowly Convergent Robbins-Monro Process*. Tech. rep. 1988.

[Rus+15] O. Russakovsky et al. "ImageNet Large Scale Visual Recognition Challenge". In: *Intl. J. Computer Vision* (2015), pp. 1–42.

[Rus15] S. Russell. "Unifying Logic and Probability". In: *Commun. ACM* 58.7 (2015), pp. 88–97.

[Rus18] A. M. Rush. "The Annotated Transformer". In: *Proceedings of ACL Workshop on Open Source Software for NLP*. 2018.

[Rus19] S. Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. en. Kindle. Viking, 2019.

[RW06] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[RY21] D. Roberts and S. Yaida. *The Principles of Deep Learning Theory: An Effective Theory Approach to Understanding Neural Network*. 2021.

[RZL17] P. Ramachandran, B. Zoph, and Q. V. Le. "Searching for Activation Functions". In: (2017). arXiv: 1710.05941 [cs.NE].

[SA93] P Sinha and E Adelson. "Recovering reflectance and illumination in a world of painted polyhedra". In: *ICCV*. 1993, pp. 156–163.

[Sab21] W. Saba. "Machine Learning Won't Solve Natural Language Understanding". In: (2021).

[Sal+16] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen. "Improved Techniques for Training GANs". In: (2016). arXiv: 1606.03498 [cs.LG].

[SAM04] D. J. Spiegelhalter, K. R. Abrams, and J. P. Myles. *Bayesian Approaches to Clinical Trials and Health-Care Evaluation*. Wiley, 2004.

[San+18a] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. "Inverted Residuals and Linear Bottlenecks: Mobile Networks for Classification, Detection and Segmentation". In: (2018). arXiv: 1801.04381 [cs.CV].

[San+18b] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry. "How Does Batch Normalization Help Optimization? (No, It Is Not About Internal Covariate Shift)". In: *NIPS*. 2018.

[San96] R. Santos. "Equivalence of regularization and truncated iteration for general ill-posed problems". In: *Linear Algebra and its Applications* 236.15 (1996), pp. 25–33.

[Sar11] R. Sarkar. "Low distortion delaunay embedding of trees in hyperbolic plane". In: *International Symposium on Graph Drawing*. Springer. 2011, pp. 355–366.

[SAV20] E. Stevens, L. Antiga, and T. Viehmann. *Deep Learning with PyTorch*. Manning, 2020.

[SBB01] T. Sellke, M. J. Bayarri, and J. Berger. "Calibration of p Values for Testing Precise Null Hypotheses". In: *The American Statistician* 55.1 (2001), pp. 62–71.

[SBP17] Y Sun, P Babu, and D. P. Palomar. "Majorization-Minimization Algorithms in Signal Processing, Communications, and Machine Learning". In: *IEEE Trans. Signal Process.* 65.3 (2017), pp. 794–816.

[SBS20] K. Shi, D. Bieber, and C. Sutton. "Incremental sampling without replacement for sequence models". In: *ICML*. 2020.

[SC16] A. N. Sanborn and N. Chater. "Bayesian brains without probabilities". en. In: *Trends Cogn. Sci.* 20.12 (Dec. 2016), pp. 883–893.

[Sca+09] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. "The graph neural network model". In: *IEEE Transactions on Neural Networks* 20.1 (2009), pp. 61–80.

[Sca+17] S. Scardapane, D. Comminiello, A. Hussain, and A. Uncini. "Group Sparse Regularization for Deep Neural Networks". In: *Neurocomputing* 241 (2017).

[Sch+00] B Scholkopf, A. J. Smola, R. C. Williamson, and P. L. Bartlett. "New support vector algo-

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

rithms". en. In: *Neural Comput.* 12.5 (2000), pp. 1207–1245.

[Sch19] B. Schölkopf. "Causality for Machine Learning". In: (2019). arXiv: 1911.10500 [cs.LG].

[Sch78] G. Schwarz. "Estimating the dimension of a model". In: *Annals of Statistics* 6.2 (1978), pp. 461–464.

[Sch90] R. E. Schapire. "The strength of weak learnability". In: *Mach. Learn.* 5.2 (1990), pp. 197–227.

[Sco79] D. Scott. "On optimal and data-based histograms". In: *Biometrika* 66.3 (1979), pp. 605–610.

[Scu10] D Sculley. "Web-scale k-means clustering". In: *WWW*. WWW '10. Association for Computing Machinery, 2010, pp. 1177–1178.

[Scu65] H. Scudder. "Probability of error of some adaptive pattern-recognition machines". In: *IEEE Transactions on Information Theory* 11.3 (1965), pp. 363–371.

[Sed+15] S. Sedhain, A. K. Menon, S. Sanner, and L. Xie. "AutoRec: Autoencoders Meet Collaborative Filtering". In: *WWW*. WWW '15 Companion. Association for Computing Machinery, 2015, pp. 111–112.

[Sej18] T. J. Sejnowski. *The Deep Learning Revolution*. en. Kindle. The MIT Press, 2018.

[Set12] B. Settles. "Active learning". In: *Synthesis Lectures on Artificial Intelligence and Machine Learning* 6 (2012), 1–114.

[SF12] R. Schapire and Y. Freund. *Boosting: Foundations and Algorithms*. MIT Press, 2012.

[SGJ11] D. Sontag, A. Globerson, and T. Jaakkola. "Introduction to Dual Decomposition for Inference". In: *Optimization for Machine Learning*. Ed. by S. Sra, S. Nowozin, and S. J. Wright. MIT Press, 2011.

[Sha+06] P. Shafto, C. Kemp, V. Mansinghka, M. Gordon, and J. B. Tenenbaum. "Learning cross-cutting systems of categories". In: *Cognitive Science Conference*. 2006.

[Sha+17] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *ICLR*. 2017.

[Sha88] T. Shallice. *From Neuropsychology to Mental Structure*. 1988.

[SHB16] R. Sennrich, B. Haddow, and A. Birch. "Neural Machine Translation of Rare Words with Subword Units". In: *Proc. ACL*. 2016.

[She+18] Z. Shen, M. Zhang, S. Yi, J. Yan, and H. Zhao. "Factorized Attention: Self-Attention with Linear Complexities". In: *CoRR* abs/1812.01243 (2018). arXiv: 1812.01243.

[She94] J. R. Shewchuk. *An introduction to the conjugate gradient method without the agonizing pain*. Tech. rep. CMU, 1994.

[SHF15] R. Steorts, R. Hall, and S. Fienberg. "A Bayesian Approach to Graphical Record Linkage and De-duplication". In: *JASA* (2015).

[Shu+13] D. I. Shuman, S. K. Narang, P Frossard, A Ortega, and P Vandergheynst. "The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains". In: *IEEE Signal Process. Mag.* 30.3 (2013), pp. 83–98.

[Sin+20] S. Sinha, H. Zhang, A. Goyal, Y. Bengio, H. Larochelle, and A. Odena. "Small-GAN: Speeding up GAN Training using Core-Sets". In: *ICML*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 9005–9015.

[Sit+20] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein. "Implicit Neural Representations with Periodic Activation Functions". In: *NIPS*. https://www.vincentsitzmann.com/siren/. June 2020.

[SIV17] C. Szegedy, S. Ioffe, and V. Vanhoucke. "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning". In: *AAAI*. 2017.

[SJ03] N. Srebro and T. Jaakkola. "Weighted low-rank approximations". In: *ICML*. 2003.

[SJT16] M. Sajjadi, M. Javanmardi, and T. Tasdizen. "Regularization with stochastic transformations and perturbations for deep semi-supervised learning". In: *Advances in neural information processing systems*. 2016, pp. 1163–1171.

[SK20] S. Singh and S. Krishnan. "Filter Response Normalization Layer: Eliminating Batch Dependence in the Training of Deep Neural Networks". In: *CVPR*. 2020.

[SKP15] F. Schroff, D. Kalenichenko, and J. Philbin. "FaceNet: A Unified Embedding for Face Recognition and Clustering". In: *CVPR*. 2015.

[SKT14] A. Szlam, Y. Kluger, and M. Tygert. "An implementation of a randomized algorithm for principal component analysis". In: (2014). arXiv: 1412.3510 [stat.CO].

[SKTF18] H. Shao, A. Kumar, and P Thomas Fletcher. "The Riemannian Geometry of Deep Generative Models". In: *CVPR*. 2018, pp. 315–323.

[SL18] S. L. Smith and Q. V. Le. "A Bayesian Perspective on Generalization and Stochastic Gradient Descent". In: *ICLR*. 2018.

[SL+19] B. Sanchez-Lengeling, J. N. Wei, B. K. Lee, R. C. Gerkin, A. Aspuru-Guzik, and A. B. Wiltschko. "Machine Learning for Scent: Learning Generalizable Perceptual Representations of Small Molecules". In: (2019). arXiv: 1910.10685 [stat.ML].

[SL90] D. J. Spiegelhalter and S. L. Lauritzen. "Sequential updating of conditional probabilities on directed graphical structures". In: *Networks* 20 (1990).

[SL99] A Suarez and J. F. Lutsko. "Globally optimal fuzzy decision trees for classification and regression". en. In: *IEEE Trans. Pattern Anal. Mach. Intell.* 21.12 (1999), pp. 1297–1311.

[SLC20] G. I. Shamir, D. Lin, and L. Coviello. "Smooth activations and reproducibility in deep networks". In: *arXiv [cs.LG]* (Oct. 2020).

[SLRB17] M. Schmidt, N. Le Roux, and F. Bach. "Minimizing finite sums with the stochastic average gradient". In: *Mathematical Programming* 162.1-2 (2017), pp. 83–112.

[SM00] J. Shi and J. Malik. "Normalized Cuts and Image Segmentation". In: *IEEE PAMI* (2000).

[SM08] R. Salakhutdinov and A. Mnih. "Probabilistic Matrix Factorization". In: *NIPS*. Vol. 20. 2008.

[SMG14] A. M. Saxe, J. L. McClelland, and S. Ganguli. "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks". In: *ICLR*. 2014.

[SMH07] R. Salakhutdinov, A. Mnih, and G. Hinton. "Restricted Boltzmann machines for collaborative filtering". In: *ICML*. ICML '07. Association for Computing Machinery, 2007, pp. 791–798.

[Smi18] L. Smith. "A disciplined approach to neural network hyper-parameters: Part 1 – learning rate, batch size, momentum, and weight decay". In: (2018). arXiv: 1803.09820.

[Smi+21] S. L. Smith, B. Dherin, D. Barrett, and S. De. "On the Origin of Implicit Regularization in Stochastic Gradient Descent". In: *ICLR*. 2021.

[SMM03] Q. Sheng, Y. Moreau, and B. D. Moor. "Biclustering Microarray data by Gibbs sampling". In: *Bioinformatics* 19 (2003), pp. ii196–ii205.

[SNM16] M. Suzuki, K. Nakayama, and Y. Matsuo. "Joint Multimodal Learning with Deep Generative Models". In: (2016). arXiv: 1611.01891 [stat.ML].

[Soh16] K. Sohn. "Improved Deep Metric Learning with Multi-class N-pair Loss Objective". In: *NIPS*. Curran Associates, Inc., 2016, pp. 1857–1865.

[Soh+20] K. Sohn, D. Berthelot, C.-L. Li, Z. Zhang, N. Carlini, E. D. Cubuk, A. Kurakin, H. Zhang, and C. Raffel. "FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence". In: (2020). arXiv: 2001.07685 [cs.LG].

[SP97] M Schuster and K. K. Paliwal. "Bidirectional recurrent neural networks". In: *IEEE. Trans on Signal Processing* 45.11 (1997), pp. 2673–2681.

[Spe11] T. Speed. "A correlation for the 21st century". In: *Science* 334 (2011), pp. 1502–1503.

[Spe+22] A. Z. Spector, P. Norvig, C. Wiggins, and J. M. Wing. *Data Science in Context: Foundations, Challenges, Opportunities*. en. New edition. Cambridge University Press, Oct. 2022.

[SR15] T. Saito and M. Rehmsmeier. "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets". en. In: *PLoS One* 10.3 (2015), e0118432.

[SRG03] R. Salakhutdinov, S. T. Roweis, and Z. Ghahramani. "Optimization with EM and Expectation-Conjugate-Gradient". In: *ICML*. 2003.

[Sri+14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". In: *JMLR* (2014).

[SS01] B. Schlkopf and A. J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond (Adaptive Computation and Machine Learning)*. en. 1st edition. The MIT Press, 2001.

[SS02] B. Schoelkopf and A. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.

[SS05] J. Schaefer and K. Strimmer. "A shrinkage approach to large-scale covariance matrix estimation and implications for functional genomics". In: *Statist. Appl. Genet. Mol. Biol* 4.32 (2005).

[SS19] S. Serrano and N. A. Smith. "Is Attention Interpretable?" In: *Proc. ACL*. 2019.

[SS95] H. T. Siegelmann and E. D. Sontag. "On the Computational Power of Neural Nets". In: *J. Comput. System Sci.* 50.1 (1995), pp. 132–150.

[SSM98] B. Schoelkopf, A. Smola, and K.-R. Mueller. "Nonlinear component analysis as a kernel Eigenvalue problem". In: *Neural Computation* 10 (5 1998), pp. 1299 –1319.

[Sta+06] C. Stark, B.-J. Breitkreutz, T. Reguly, L. Boucher, A. Breitkreutz, and M. Tyers. "BioGRID: a general repository for interaction datasets". In: *Nucleic acids research* 34.suppl_1 (2006), pp. D535–D539.

[Ste56] C. Stein. "Inadmissibility of the usual estimator for the mean of a multivariate distribution". In: *Proc. 3rd Berkeley Symposium on Mathematical Statistics and Probability* (1956), 197–206.

[Str09] G. Strang. *Introduction to linear algebra*. 4th edition. SIAM Press, 2009.

[Sug+19] A. S. Suggala, K. Bhatia, P. Ravikumar, and P. Jain. "Adaptive Hard Thresholding for Near-optimal Consistent Robust Regression". In: *Proceedings of the Annual Conference On Learning Theory (COLT)*. 2019, pp. 2892–2897.

[Sun+09] L. Sun, S. Ji, S. Yu, and J. Ye. "On the Equivalence Between Canonical Correlation Analysis and Orthonormalized Partial Least Squares". In: *IJCAI*. 2009.

[Sun+19a] C. Sun, A. Myers, C. Vondrick, K. Murphy, and C. Schmid. "VideoBERT: A Joint Model for Video and Language Representation Learning". In: *ICCV*. 2019.

[Sun+19b] S. Sun, Z. Cao, H. Zhu, and J. Zhao. "A Survey of Optimization Methods from a Machine Learning Perspective". In: (2019). arXiv: 1906.06821 [cs.LG].

[SVL14] I. Sutskever, O. Vinyals, and Q. V. V. Le. "Sequence to Sequence Learning with Neural Networks". In: *NIPS*. 2014.

[SVZ14] K. Simonyan, A. Vedaldi, and A. Zisserman. "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps". In: *ICLR*. 2014.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

[SW87] M. Shewry and H. Wynn. "Maximum entropy sampling". In: *J. Applied Statistics* 14 (1987), 165–170.

[SWY75] G Salton, A Wong, and C. S. Yang. "A vector space model for automatic indexing". In: *Commun. ACM* 18.11 (1975), pp. 613–620.

[Sze+15a] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. "Going Deeper with Convolutions". In: *CVPR*. 2015.

[Sze+15b] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. "Rethinking the Inception Architecture for Computer Vision". In: (2015). arXiv: 1512.00567 [cs.CV].

[Tal07] N. Taleb. *The Black Swan: The Impact of the Highly Improbable*. Random House, 2007.

[Tan+15] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei. "Line: Large-scale information network embedding". In: *Proceedings of the 24th International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee. 2015, pp. 1067–1077.

[Tan+18] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, and C. Liu. "A Survey on Deep Transfer Learning". In: *ICANN*. 2018.

[Tan+20] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. "Fourier features let networks learn high frequency functions in low dimensional domains". In: *NIPS*. June 2020.

[Tan+24] S. Taniguchi, K. Harada, G. Minegishi, Y. Oshima, S. C. Jeong, G. Nagahara, T. Iiyama, M. Suzuki, Y. Iwasawa, and Y. Matsuo. "ADOPT: Modified Adam can converge with any β_2 with the optimal rate". In: *NIPS*. Nov. 2024.

[TAS18] M. Teye, H. Azizpour, and K. Smith. "Bayesian Uncertainty Estimation for Batch Normalized Deep Networks". In: *ICML*. 2018.

[Tay+20a] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, and D. Metzler. "Long Range Arena: A Benchmark for efficient Transformers". In: *CoRR* (2020).

[Tay+20b] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler. "Efficient Transformers: A Survey". In: (2020). arXiv: 2009.06732 [cs.LG].

[TB97] L. Trefethen and D. Bau. *Numerical Linear Algebra*. SIAM, 1997.

[TB99] M. Tipping and C. Bishop. "Probabilistic principal component analysis". In: *J. of Royal Stat. Soc. Series B* 21.3 (1999), pp. 611–622.

[TDP19] I. Tenney, D. Das, and E. Pavlick. "BERT Rediscovers the Classical NLP Pipeline". In: *Proc. ACL*. 2019.

[TF03] M. Tipping and A. Faul. "Fast marginal likelihood maximisation for sparse Bayesian models". In: *AI/Stats*. 2003.

[Tho16] M. Thoma. "Creativity in Machine Learning". In: (2016). arXiv: 1601.03642 [cs.CV].

[Tho17] R. Thomas. *Computational Linear Algebra for Coders*. 2017.

[Tib96] R. Tibshirani. "Regression shrinkage and selection via the lasso". In: *J. Royal. Statist. Soc B* 58.1 (1996), pp. 267–288.

[Tip01] M. Tipping. "Sparse Bayesian learning and the relevance vector machine". In: *JMLR* 1 (2001), pp. 211–244.

[Tip98] M. Tipping. "Probabilistic visualization of high-dimensional binary data". In: *NIPS*. 1998.

[Tit16] M. Titsias. "One-vs-Each Approximation to Softmax for Scalable Estimation of Probabilities". In: *NIPS*. 2016, pp. 4161–4169.

[TK86] L. Tierney and J. Kadane. "Accurate approximations for posterior moments and marginal densities". In: *JASA* 81.393 (1986).

[TL21] M. Tan and Q. V. Le. "EfficientNetV2: Smaller Models and Faster Training". In: (2021). arXiv: 2104.00298 [cs.CV].

[TM15] D. Trafimow and M. Marks. "Editorial". In: *Basic Appl. Soc. Psych.* 37.1 (2015), pp. 1–2.

[TMP20] A. Tsitsulin, M. Munkhoeva, and B. Perozzi. "Just SLaQ When You Approximate: Accurate Spectral Distances for Web-Scale Graphs". In: *Proceedings of The Web Conference 2020*. WWW '20. 2020, 2697–2703.

[TOB16] L. Theis, A. van den Oord, and M. Bethge. "A note on the evaluation of generative models". In: *ICLR*. 2016.

[Tol+21] I. Tolstikhin et al. "MLP-Mixer: An all-MLP Architecture for Vision". In: (2021). arXiv: 2105.01601 [cs.CV].

[TP10] P. D. Turney and P. Pantel. "From Frequency to Meaning: Vector Space Models of Semantics". In: *JAIR* 37 (2010), pp. 141–188.

[TP97] S. Thrun and L. Pratt, eds. *Learning to learn*. Kluwer, 1997.

[TS92] D. G. Terrell and D. W. Scott. "Variable kernel density estimation". In: *Annals of Statistics* 20.3 (1992), 1236–1265.

[Tsi+18] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, and E. Müller. "NetLSD: Hearing the Shape of a Graph". In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '18. 2018, 2347–2356.

[TSL00] J. Tenenbaum, V. de Silva, and J. Langford. "A global geometric framework for nonlinear dimensionality reduction". In: *Science* 290.550 (2000), pp. 2319–2323.

[Tur13] M. Turk. "Over Twenty Years of Eigenfaces". In: *ACM Trans. Multimedia Comput. Commun. Appl.* 9.1s (2013), 45:1–45:5.

[Tur23] R. E. Turner. "An Introduction to Transformers". In: (Apr. 2023). arXiv: 2304.10557 [cs.LG].

[TV17] A. Tarvainen and H. Valpola. "Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results". In: *Advances in neural information processing systems*. 2017, pp. 1195–1204.

[TVW05] B. Turlach, W. Venables, and S. Wright. "Simultaneous Variable Selection". In: *Technometrics* 47.3 (2005), pp. 349–363.

[TW18] J. Tang and K. Wang. "Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding". In: *WSDM*. WSDM '18. Association for Computing Machinery, 2018, pp. 565–573.

[TXT19] V. Tjeng, K. Xiao, and R. Tedrake. "Evaluating Robustness of Neural Networks with Mixed Integer Programming". In: *ICLR*. 2019.

[UB05] I. Ulusoy and C. Bishop. "Generative Versus Discriminative Methods for Object Recognition". In: *CVPR*. 2005.

[Ude+16] M. Udell, C. Horn, R. Zadeh, and S. Boyd. "Generalized Low Rank Models". In: *Foundations and Trends in Machine Learning* 9.1 (2016), pp. 1–118.

[Uly+16] D. Ulyanov, V. Lebedev, Andrea, and V. Lempitsky. "Texture Networks: Feed-forward Synthesis of Textures and Stylized Images". In: *ICML*. 2016, pp. 1349–1357.

[Uur+17] V. Uurtio, J. M. Monteiro, J. Kandola, J. Shawe-Taylor, D. Fernandez-Reyes, and J. Rousu. "A Tutorial on Canonical Correlation Methods". In: *ACM Computing Surveys* (2017).

[UVL16] D. Ulyanov, A. Vedaldi, and V. Lempitsky. "Instance Normalization: The Missing Ingredient for Fast Stylization". In: (2016). arXiv: 1607.08022 [cs.CV].

[Van06] L. Vandenberghe. *Applied Numerical Computing: Lecture notes*. 2006.

[Van14] J. VanderPlas. *Frequentism and Bayesianism III: Confidence, Credibility, and why Frequentism and Science do not Mix*. Blog post. 2014.

[Van18] J. Vanschoren. "Meta-Learning: A Survey". In: (2018). arXiv: 1810.03548 [cs.LG].

[Vap98] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

[Vas+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. "Attention Is All You Need". In: *NIPS*. 2017.

[Vas+19] S. Vaswani, A. Mishkin, I. Laradji, M. Schmidt, G. Gidel, and S. Lacoste-Julien. "Painless Stochastic Gradient: Interpolation, Line-Search, and Convergence Rates". In: *NIPS*. Curran Associates, Inc., 2019, pp. 3727–3740.

[VD99] S. Vaithyanathan and B. Dom. "Model Selection in Unsupervised Learning With Applications To Document Clustering". In: *ICML*. 1999.

[VEB09] N. Vinh, J. Epps, and J. Bailey. "Information Theoretic Measures for Clusterings Comparison: Is a Correction for Chance Necessary?" In: *ICML*. 2009.

[Vel+18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. "Graph attention networks". In: *ICLR*. 2018.

[Vel+19] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm. "Deep Graph Infomax". In: *International Conference on Learning Representations*. 2019.

[VGG17] A. Vehtari, A. Gelman, and J. Gabry. "Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC". In: *Stat. Comput.* 27.5 (2017), pp. 1413–1432.

[VGS97] V. Vapnik, S. Golowich, and A. Smola. "Support vector method for function approximation, regression estimation, and signal processing". In: *NIPS*. 1997.

[Vig15] T. Vigen. *Spurious Correlations*. en. Gift edition. Hachette Books, 2015.

[Vij+18] A. K. Vijayakumar, M. Cogswell, R. R. Selvaraju, Q. Sun, S. Lee, D. Crandall, and D. Batra. "Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models". In: *IJCAI*. 2018.

[Vin+10a] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol. "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion". In: *JMLR* 11 (2010), pp. 3371–3408.

[Vin+10b] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol. "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion". In: *Journal of machine learning research* 11.Dec (2010), pp. 3371–3408.

[Vin+16] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra. "Matching Networks for One Shot Learning". In: *NIPS*. 2016.

[Vir10] S. Virtanen. "Bayesian exponential family projections". MA thesis. Aalto University, 2010.

[Vis+10] S. V. N. Vishwanathan, N. N. Schraudolph, R. Kondor, and K. M. Borgward. "Graph Kernels". In: *JMLR* 11 (2010), pp. 1201–1242.

[Vo+15] B.-N. Vo, M. Mallick, Y. Bar-Shalom, S. Coraluppi, R. Osborne, R. Mahler, B. t Vo, and J. Webster. *Multitarget tracking*. John Wiley and Sons, 2015.

[Vor+17] E. Vorontsov, C. Trabelsi, S. Kadoury, and C. Pal. "On orthogonality and learning recurrent networks with long term dependencies". In: *ICML*. 2017.

[VT17] C. Vondrick and A. Torralba. "Generating the Future with Adversarial Transformers". In: *CVPR*. 2017.

[VV13] G. Valiant and P. Valiant. "Estimating the unseen: improved estimators for entropy and other properties". In: *NIPS*. 2013.

[Wah+22] O. Wahltinez, A. Cheung, R. Alcantara, D. Cheung, M. Daswani, A. Erlinger, M. Lee, P. Yawalkar, M. P. Brenner, and K. Murphy. "COVID-19 Open-Data: a global-scale, spatially granular meta-dataset for SARS-CoV-2". In: *Nature Scientific Data* 9.162 (2022).

[Wal05] C. S. Wallace. *Statistical and Inductive Inference by Minimum Message Length (Information Science and Statistics)*. en. 2005th ed. Springer, May 2005.

[Wal+20] M. Walmsley et al. "Galaxy Zoo: probabilistic morphology through Bayesian CNNs and active learning". In: *Monthly Notices Royal*

Astronomial Society 491.2 (2020), pp. 1554–1574.

[Wal47] A. Wald. "An Essentially Complete Class of Admissible Decision Functions". en. In: *Ann. Math. Stat.* 18.4 (1947), pp. 549–555.

[Wan+15] J. Wang, W. Liu, S. Kumar, and S.-F. Chang. "Learning to Hash for Indexing Big Data - A Survey". In: *Proc. IEEE* (2015).

[Wan+17] Y. Wang et al. "Tacotron: Towards End-to-End Speech Synthesis". In: *Interspeech*. 2017.

[Wan+20a] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. "Linformer: Self-Attention with Linear Complexity". In: *CoRR* abs/2006.04768 (2020). arXiv: 2006.04768.

[Wan+20b] Y. Wang, Q. Yao, J. Kwok, and L. M. Ni. "Generalizing from a Few Examples: A Survey on Few-Shot Learning". In: *ACM Computing Surveys* 1.1 (2020).

[Wan+21] R. Wang, M. Cheng, X. Chen, X. Tang, and C.-J. Hsieh. "Rethinking Architecture Selection in Differentiable NAS". In: *ICLR*. 2021.

[Was04] L. Wasserman. *All of statistics. A concise course in statistical inference*. Springer, 2004.

[Wat10] S. Watanabe. "Asymptotic Equivalence of Bayes Cross Validation and Widely Applicable Information Criterion in Singular Learning Theory". In: *JMLR* 11 (2010), pp. 3571–3594.

[Wat13] S. Watanabe. "A Widely Applicable Bayesian Information Criterion". In: *JMLR* 14 (2013), pp. 867–897.

[WCS08] M. Welling, C. Chemudugunta, and N. Sutter. "Deterministic Latent Variable Models and their Pitfalls". In: *ICDM*. 2008.

[WCZ16] D. Wang, P. Cui, and W. Zhu. "Structural deep network embedding". In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM. 2016, pp. 1225–1234.

[Wei76] J. Weizenbaum. *Computer Power and Human Reason: From Judgment to Calculation*. en. 1st ed. W H Freeman & Co, 1976.

[Wen+16] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li. "Learning Structured Sparsity in Deep Neural Networks". In: (2016). arXiv: 1608.03665 [cs.NE].

[Wen18] L. Weng. "Attention? Attention!" In: *lilianweng.github.io/lil-log* (2018).

[Wen19] L. Weng. "Generalized Language Models". In: *lilianweng.github.io/lil-log* (2019).

[Wer74] P. Werbos. "Beyond regression: New Tools for Prediction and Analysis in the Behavioral Sciences". PhD thesis. Harvard, 1974.

[Wer90] P. J. Werbos. "Backpropagation Through Time: What It Does and How to Do It". In: *Proc. IEEE* 78.10 (1990), pp. 1550–1560.

[Wes03] M. West. "Bayesian Factor Regression Models in the "Large p, Small n" Paradigm". In: *Bayesian Statistics 7* (2003).

[WF14] Z. Wang and N. de Freitas. "Theoretical Analysis of Bayesian Optimisation with Unknown Gaussian Process Hyper-Parameters". In: (2014). arXiv: 1406.7758 [stat.ML].

[WF20] T. Wu and I. Fischer. "Phase Transitions for the Information Bottleneck in Representation Learning". In: *ICLR*. 2020.

[WH18] Y. Wu and K. He. "Group Normalization". In: *ECCV*. 2018.

[WH60] B. Widrow and M. E. Hoff. "Adaptive Switching Circuits". In: *1960 IRE WESCON Convention Record, Part 4*. IRE, 1960, pp. 96–104.

[WI20] A. G. Wilson and P. Izmailov. "Bayesian Deep Learning and a Probabilistic Perspective of Generalization". In: *NIPS*. 2020.

[Wil14] A. G. Wilson. "Covariance kernels for fast automatic pattern discovery and extrapolation with Gaussian processes". PhD thesis. University of Cambridge, 2014.

[Wil20] C. K. I. Williams. "The Effect of Class Imbalance on Precision-Recall Curves". In: *Neural Comput.* (2020).

[WL08] T. T. Wu and K. Lange. "Coordinate descent algorithms for lasso penalized regression". In: *Ann. Appl. Stat* 2.1 (2008), pp. 224–244.

[WLL16] W. Wang, H. Lee, and K. Livescu. "Deep Variational Canonical Correlation Analysis". In: *arXiv* (2016).

[WM00] D. R. Wilson and T. R. Martinez. "Reduction Techniques for Instance-Based Learning Algorithms". In: *Mach. Learn.* 38.3 (2000), pp. 257–286.

[WNF09] S. Wright, R. Nowak, and M. Figueiredo. "Sparse reconstruction by separable approximation". In: *IEEE Trans. on Signal Processing* 57.7 (2009), pp. 2479–2493.

[WNS19] C. White, W. Neiswanger, and Y. Savani. "BANANAS: Bayesian Optimization with Neural Architectures for Neural Architecture Search". In: (2019). arXiv: 1910.11858 [cs.LG].

[Wol92] D. Wolpert. "Stacked Generalization". In: *Neural Networks* 5.2 (1992), pp. 241–259.

[Wol96] D. Wolpert. "The lack of a priori distinctions between learning algorithms". In: *Neural Computation* 8.7 (1996), pp. 1341–1390.

[WP19] S. Wiegreffe and Y. Pinter. "Attention is not not Explanation". In: *EMNLP*. 2019.

[WRC08] J. Weston, F. Ratle, and R. Collobert. "Deep learning via semi-supervised embedding". In: *Proceedings of the 25th international conference on Machine learning*. ACM. 2008, pp. 1168–1175.

[WS09] K. Weinberger and L. Saul. "Distance Metric Learning for Large Margin Classification". In: *JMLR* 10 (2009), pp. 207–244.

[WSH16] L. Wu, C. Shen, and A. van den Hengel. "PersonNet: Person Re-identification with Deep Convolutional Neural Networks". In: (2016). arXiv: 1601.07255 [cs.CV].

[WSL19] R. L. Wasserstein, A. L. Schirm, and N. A. Lazar. "Moving to a World Beyond "$p < 0.05$"".

In: *The American Statistician* 73.sup1 (2019), pp. 1–19.

[WSS04] K. Q. Weinberger, F. Sha, and L. K. Saul. "Learning a kernel matrix for nonlinear dimensionality reduction". In: *ICML*. 2004.

[WTN19] Y. Wu, G. Tucker, and O. Nachum. "The Laplacian in RL: Learning Representations with Efficient Approximations". In: *ICLR*. 2019.

[Wu+16] Y. Wu et al. "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation". In: (2016). arXiv: 1609.08144 [cs.CL].

[Wu+19] Y. Wu, E. Winston, D. Kaushik, and Z. Lipton. "Domain Adaptation with Asymmetrically-Relaxed Distribution Alignment". In: *ICML*. 2019.

[WVJ16] M. Wattenberg, F. Viégas, and I. Johnson. "How to Use t-SNE Effectively". In: *Distill* 1.10 (2016).

[WW93] D. Wagner and F. Wagner. "Between min cut and graph bisection". In: *Proc. 18th Intl. Symp. on Math. Found. of Comp. Sci.* 1993, pp. 744–750.

[Xie+19] Q. Xie, Z. Dai, E. Hovy, M.-T. Luong, and Q. V. Le. "Unsupervised data augmentation for consistency training". In: *arXiv preprint arXiv:1904.12848* (2019).

[Xie+20] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le. "Self-training with noisy student improves imagenet classification". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020, pp. 10687–10698.

[XJ96] L. Xu and M. I. Jordan. "On Convergence Properties of the EM Algorithm for Gaussian Mixtures". In: *Neural Computation* 8 (1996), pp. 129–151.

[XRV17] H. Xiao, K. Rasul, and R. Vollgraf. "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms". In: (2017). arXiv: 1708.07747 [stat.ML].

[Xu+15] K. Xu, J. L. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. S. Zemel, and Y. Bengio. "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention". In: *ICML*. 2015.

[Yal+19] I. Z. Yalniz, H. Jégou, K. Chen, M. Paluri, and D. Mahajan. "Billion-scale semi-supervised learning for image classification". In: *arXiv preprint arXiv:1905.00546* (2019).

[Yan+14] X. Yang, Y. Guo, Y. Liu, and H. Steck. "A Survey of Collaborative Filtering Based Social Recommender Systems". In: *Comput. Commun.* 41 (2014), pp. 1–10.

[Yar95] D. Yarowsky. "Unsupervised word sense disambiguation rivaling supervised methods". In: *33rd annual meeting of the association for computational linguistics*. 1995, pp. 189–196.

[YB19] C. Yadav and L. Bottou. "Cold Case: The Lost MNIST Digits". In: *arXiv* (2019).

[YCS16] Z. Yang, W. W. Cohen, and R. Salakhutdinov. "Revisiting semi-supervised learning with graph embeddings". In: *Proceedings of the 33rd International Conference on International Conference on Machine Learning-Volume 48*. JMLR. org. 2016, pp. 40–48.

[Yeu91] R. W. Yeung. "A new outlook on Shannon's information measures". In: *IEEE Trans. Inf. Theory* 37.3 (1991), pp. 466–474.

[YHJ09] D. Yan, L. Huang, and M. I. Jordan. "Fast approximate spectral clustering". In: *15th ACM Conf. on Knowledge Discovery and Data Mining*. 2009.

[Yin+19] P. Yin, J. Lyu, S. Zhang, S. Osher, Y. Qi, and J. Xin. "Understanding Straight-Through Estimator in Training Activation Quantized Neural Nets". In: *ICLR*. 2019.

[YK16] F. Yu and V. Koltun. "Multi-Scale Context Aggregation by Dilated Convolutions". In: *ICLR*. 2016.

[YL06] M. Yuan and Y. Lin. "Model Selection and Estimation in Regression with Grouped Variables". In: *J. Royal Statistical Society, Series B* 68.1 (2006), pp. 49–67.

[YL21] A. L. Yuille and C. Liu. "Deep Nets: What have they ever done for Vision?" In: *Intl. J. Computer Vision* 129 (2021), pp. 781–802.

[Yon19] E. Yong. "The Human Brain Project Hasn't Lived Up to Its Promise". In: *The Atlantic* (2019).

[Yos+15] J. Yosinski, J. Clune, A. Nguyen, T. Fuchs, and H. Lipson. "Understanding Neural Networks Through Deep Visualization". In: *ICML Workshop on Deep Learning*. 2015.

[Yu+06] S. Yu, K. Yu, V. Tresp, K. H-P., and M. Wu. "Supervised probabilistic principal component analysis". In: *KDD*. 2006.

[Yu+16] F. X. X. Yu, A. T. Suresh, K. M. Choromanski, D. N. Holtmann-Rice, and S. Kumar. "Orthogonal Random Features". In: *NIPS*. Curran Associates, Inc., 2016, pp. 1975–1983.

[YWG12] S. E. Yuksel, J. N. Wilson, and P. D. Gader. "Twenty Years of Mixture of Experts". In: *IEEE Trans. on neural networks and learning systems* (2012).

[Zah+18] M. Zaheer, S. Reddi, D. Sachan, S. Kale, and S. Kumar. "Adaptive Methods for Nonconvex Optimization". In: *NIPS*. Curran Associates, Inc., 2018, pp. 9815–9825.

[Zah+20] M. Zaheer et al. "Big Bird: Transformers for Longer Sequences". In: *CoRR* abs/2007.14062 (2020). arXiv: 2007.14062.

[Zei12] M. D. Zeiler. "ADADELTA: An Adaptive Learning Rate Method". In: (2012). arXiv: 1212.5701 [cs.LG].

[Zel76] A. Zellner. "Bayesian and non-Bayesian analysis of the regression model with multivariate Student-t error terms". In: *JASA* 71.354 (1976), pp. 400–405.

[ZG02] X. Zhu and Z. Ghahramani. *Learning from labeled and unlabeled data with label propagation*. Tech. rep. CALD tech report CMU-CALD-02-107. CMU, 2002.

[ZG06] M. Zhu and A. Ghodsi. "Automatic dimensionality selection from the scree plot via the

use of profile likelihood". In: *Computational Statistics & Data Analysis* 51 (2006), pp. 918–930.

[ZH05] H. Zou and T. Hastie. "Regularization and Variable Selection via the Elastic Net". In: *J. of Royal Stat. Soc. Series B* 67.2 (2005), pp. 301–320.

[Zha+17a] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. "Understanding deep learning requires rethinking generalization". In: *ICLR*. 2017.

[Zha+17b] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. "mixup: Beyond Empirical Risk Minimization". In: *ICLR*. 2017.

[Zha+18] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu. "Object Detection with Deep Learning: A Review". In: (2018). arXiv: 1807.05511 [cs.CV].

[Zha+19a] J. Zhang, Y. Zhao, M. Saleh, and P. J. Liu. "PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization". In: (2019). arXiv: 1912.08777 [cs.CL].

[Zha+19b] S. Zhang, L. Yao, A. Sun, and Y. Tay. "Deep Learning Based Recommender System: A Survey and New Perspectives". In: *ACM Comput. Surv.* 52.1 (2019), pp. 1–38.

[Zha+20] A. Zhang, Z. Lipton, M. Li, and A. Smola. *Dive into deep learning*. 2020.

[Zha+22] Y. Zhang, C. Chen, N. Shi, R. Sun, and Z.-Q. Luo. "Adam Can Converge Without Any Modification On Update Rules". In: (Aug. 2022). arXiv: 2208.09632 [cs.LG].

[Zho+04] D. Zhou, O. Bousquet, T. N. Lal, J. Weston, and B. Schölkopf. "Learning with local and global consistency". In: *Advances in neural information processing systems*. 2004, pp. 321–328.

[Zho+18] D. Zhou, Y. Tang, Z. Yang, Y. Cao, and Q. Gu. "On the Convergence of Adaptive Gradient Methods for Nonconvex Optimization". In: (2018). arXiv: 1808.05671 [cs.LG].

[Zho+21] C. Zhou, X. Ma, P. Michel, and G. Neubig. "Examining and Combating Spurious Features under Distribution Shift". In: *ICML*. 2021.

[ZHT06] H. Zou, T. Hastie, and R. Tibshirani. "Sparse principal component analysis". In: *JCGS* 15.2 (2006), pp. 262–286.

[Zhu05] X. Zhu. "Semi-supervised learning with graphs". PhD thesis. Carnegie Mellon University, 2005.

[Zhu+21] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He. "A Comprehensive Survey on Transfer Learning". In: *Proc. IEEE* 109.1 (2021).

[Zie+05] C.-N. Ziegler, S. M. McNee, J. A. Konstan, and G. Lausen. "Improving recommendation lists through topic diversification". In: *WWW*. WWW '05. Association for Computing Machinery, 2005, pp. 22–32.

[ZK16] S. Zagoruyko and N. Komodakis. "Wide Residual Networks". In: *BMVC*. 2016.

[ZL05] Z.-H. Zhou and M. Li. "Tri-training: Exploiting unlabeled data using three classifiers". In: *IEEE Transactions on knowledge and Data Engineering* 17.11 (2005), pp. 1529–1541.

[ZL17] B. Zoph and Q. V. Le. "Neural Architecture Search with Reinforcement Learning". In: *ICLR*. 2017.

[ZLZ20] D. Zhang, Y. Li, and Z. Zhang. "Deep metric learning with spherical embedding". In: *NIPS*. 2020.

[ZMY19] D. Zabihzadeh, R. Monsefi, and H. S. Yazdi. "Sparse Bayesian approach for metric learning in latent space". In: *Knowledge-Based Systems* 178 (2019), pp. 11–24.

[ZRY05] P. Zhao, G. Rocha, and B. Yu. *Grouped and Hierarchical Model Selection through Composite Absolute Penalties*. Tech. rep. UC Berkeley, 2005.

[ZS14] H. Zen and A Senior. "Deep mixture density networks for acoustic modeling in statistical parametric speech synthesis". In: *ICASSP*. 2014, pp. 3844–3848.

[ZY08] J.-H. Zhao and P. L. H. Yu. "Fast ML Estimation for the Mixture of Factor Analyzers via an ECM Algorithm". In: *IEEE. Trans. on Neural Networks* 19.11 (2008).